# 爱丽丝漫游可微奇境

神经网络设计入门

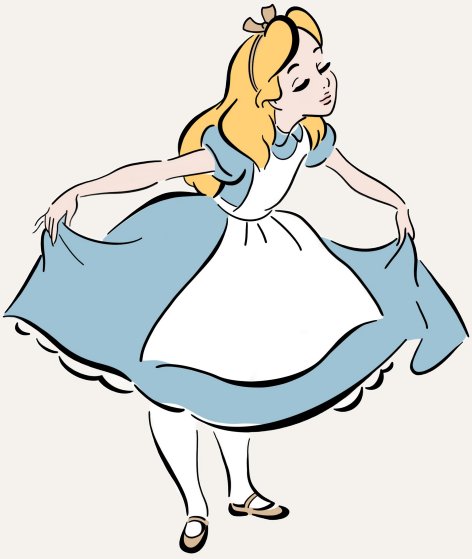
**第 I 卷 - 领域概览**

[意] **西莫内 · 斯卡尔达帕内**著
**刘医**译



“你想，阿丽思（爱丽丝）近来遇见
这么些出奇的事，简直觉得天下没
有做不到的事情。”

**—第一章，钻进兔子洞**



# 前言

本书是对（深度）**神经网络**（neural networks）这一主题的系统阐述，它是大语言模型、生成式人工智能，以及其他众多应用的核心基石。由于“神经”一词承载过多历史渊源，而神经网络本质上是可微原语（primitive）的组合，在可行情况下，我将采用更简单的术语——**可微模型**（differentiable model）。

2009 年，时值诸如 Theano [ARAA+16]等自动微分库兴起之际，我碰巧读到 Yoshua Bengio 的一篇关于“深度”神经网络强大效用的文章 [Ben09]。就像爱丽丝钻进了兔子洞，我无意中闯入了一个奇特的编程领域——一个可微奇境。在这里，诸如元素选择这般的简单操作，变得无比困难，而其他事情，比如猫脸识别，却异常简单。

我花了十多年的时间阅读、实现和教授这些知识。本书试图凝练我在以上过程中的认知精要，着重介绍神经网络设计和最常见的组件。由于本领域发展迅速，笔者尽力在理论与代码、历史沿革和前沿动态之间寻求平衡。笔者假设读者对机器学习和线性代数有一定了解，但在必要处仍将补充基础知识。



来吧，朋友们：
是时候开启我们挚爱的
爱丽丝漫游可微奇境
之旅了！

# 目录


**前言** **i**

**1 绪论** **1**

**I 罗盘与磁针** **13**

**2 数学预备知识** **15**
2.1 线性代数 . . . . . 16
2.2 梯度和雅可比矩阵 . . . . . 27
2.3 梯度下降 . . . . . 35

**3 数据集和损失** **45**
3.1 什么是数据集? . . . . . 45
3.2 损失函数 . . . . . 51
3.3 贝叶斯学习 . . . . . 58

**4 线性模型** **63**
4.1 最小二乘回归 . . . . . 63
4.2 线性分类模型 . . . . . 73
4.3 更多相关内容 . . . . . 79

**5 全连接模型** **89**
5.1 线性模型的局限 . . . . . 89

5.2 复合与隐藏层 . . . . . 90
5.3 随机优化 . . . . . 97
5.4 激活函数 . . . . . 100

**6 自动微分 109**
6.1 问题设定 . . . . . 109
6.2 前向模式微分 . . . . . 114
6.3 反向模式微分 . . . . . 116
6.4 实践注意事项 . . . . . 118

**II 陌生的土地 131**

**7 卷积层 133**
7.1 走进卷积层 . . . . . 134
7.2 卷积模型 . . . . . 143

**8 图像之外的卷积 153**
8.1 面向一维与三维数据的卷积 . . . . . 153
8.2 一维与三维卷积模型 . . . . . 158
8.3 预测与因果模型 . . . . . 165
8.4 生成模型 . . . . . 172

**9 扩展模型规模 179**
9.1 ImageNet 挑战赛 . . . . . 179
9.2 数据与训练策略 . . . . . 181
9.3 Dropout 与归一化 . . . . . 189
9.4 残差连接 . . . . . 200

**III 钻进兔子洞 207**

**10 transformer 模型 209**
10.1 长卷积与非局部模型 . . . . . 209
10.2 位置嵌入 . . . . . 219
10.3 构建 transformer 模型 . . . . . 226

**11 实践中的 transformer** **233**
11.1 编码器-解码器 transformer . . . . . 233
11.2 计算方面的考虑 . . . . . 238
11.3 transformer 变体 . . . . . 246

**12 图模型** **249**
12.1 基于图数据的学习 . . . . . 249
12.2 图卷积层 . . . . . 258
12.3 图卷积层之外 . . . . . 268

**13 循环模型** **277**
13.1 线性化注意力模型 . . . . . 277
13.2 经典循环层 . . . . . 281
13.3 结构化状态空间模型 . . . . . 288
13.4 其他变体 . . . . . 293

**A 概率论** **301**
A.1 概率的基本法则 . . . . . 301
A.2 实值分布 . . . . . 304
A.3 常见分布 . . . . . 304
A.4 矩与期望值 . . . . . 306
A.5 分布之间的距离 . . . . . 307
A.6 最大似然估计 . . . . . 307

**B 通用逼近定理** **311**
B.1 逼近阶跃函数 . . . . . 312
B.2 逼近常值函数 . . . . . 313
B.3 逼近一般函数 . . . . . 314

# 1 | 绪论

神经网络已成为日常生活中不可或缺的组成部分，或显性存在（例如**大型语言模型**, LLMs），或隐性赋能无数技术和科学发现，包括无人机、汽车、搜索引擎、分子设计和推荐系统 [WFD+23]。本书将揭示，这一切成就均源于一小套指导原则和组件，它构成了本书的核心内容，而研究重点已转向将其规模扩展至物理可能的极限。

扩展的力量体现在一种新兴概念——**神经扩展定律**，这个概念推动了对人工智能（artificial intelligence, AI）的巨额投资 [KMH+20, HBE+24]：简言之，对几乎任何任务，同时增加数据量、算力和模型规模——几乎总能带来可预测的性能提高。换句话说，达到特定任务精度所需的算力，正随着时间推移，以常数因子减少 [HBE+24]。在 AI 中结合简单、通用工具与指数级增加的计算能力所带来的惊人力量，被 R. Sutton 称为*苦涩的教训*。[1]

如果将扩展定律视为既定前提，那么我们便拥有了一种神奇的工具。简单来说，神经网络被优化为：在给定从某个概率分布中抽取的数据的情况下，对该概率分布进行近似。原则上，这种近似有可能失败：例如，现代神经网络规模之大足以完全记忆训练数据 [ZBH+21]并将其转换为简单的查表。然而，训练后的模型表现出良好的泛化能力，甚至适用于训练数据中未明确考虑的任务

[1] http://www.incompleteideas.net/IncIdeas/BitterLesson.html.

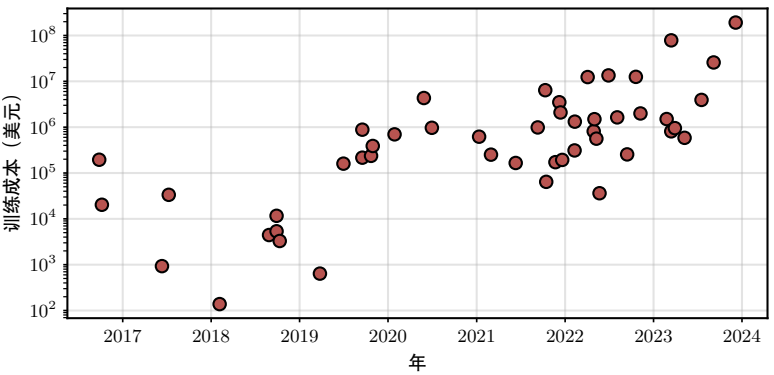



**图 F.1.1:** *2016 年以来知名 AI 模型的训练成本（以美元计）。训练成本与扩展定律的三要素相关：数据规模、算力以及模型规模。随着性能的稳步提升，模型架构差异的影响渐趋微弱 [HBE+24]。数据来自《斯坦福 AI 指数报告 2024》。*[2]

[ASA+23]。事实上，随着数据集大小增长，分布内（in-distribution）和分布外（out-of-distribution）的界限变得模糊，大模型显示出强大的泛化能力，并且对纯粹记忆（即**过拟合**）的依赖较低 [PBE+22]，这一点令人着迷。

可用于各种下游任务的超大型模型（有时称为**基础模型**，foundation models），结合活跃的开源社区，[3]也改变了我们与这些模型的交互方式。许多任务只需对网上的预训练模型进行提示（prompting，即通过文本或视觉指令交互）来完成 [ASA+23]，而模型内部仍是完全的黑盒。从高层次的角度看，这类似于从自行编写程序库（如 C++），转向依赖开源或闭源的商业软件。这一比喻并非毫无根据：如今，全球只有少数团队具备设计和发布真正大模型（如 LLaMA LLM[TLI+23]）所需的算力和技术能力，就像只有少数公司有资源构建企业级客户关系管理（CRM）软件一样。

[2] https://hai.stanford.edu/research/ai-index-report

[3] https://huggingface.co/

同样地，正如开源软件提供了从头开始定制或设计程序的无限可能，消费级硬件和创造力也为您提供了可微模型的广泛选项，从**微调**模型以适应您的任务 [LTM+22]，到合并不同的模型 [AHS23]，量化它们以适应低功耗硬件，测试它们的稳健性，甚至设计全新的变体和创意。为此，您需要“抽丝剥茧”，理解这些模型内部如何处理和操纵数据，包括所有源自经验和调试的技巧与特质。本书是进入这个世界的入口：如果您像爱丽丝一样天生好奇，希望您能享受这段旅程。

# 关于本书

我们假设读者已熟悉**机器学习**（machine learning, ML）的基础知识，特别是**监督学习**（supervised learning, SL）。监督学习可以收集关于目标行为的数据，并“训练”（优化）系统以近似该行为，来解决复杂的任务。这个想法看似简单，却蕴含强大的力量：例如，图像生成可转化为收集足够大的带文字描述的图像集合的问题；模拟英语变成了收集大量文本并学习从前文预测下一句的任务；诊断 X 光片则等同于拥有大量扫描图像和对应医生决策的数据库（图F.1.2）。

广义上，学习是一个**搜索**问题。我们首先定义具有大量自由度（称为参数, parameter）的程序，然后不断调整参数，直到模型性能达到满意的水平。为使该想法可行，我们需要在拥有数百万（甚至数十亿或数万亿）参数的情况下，找到高效方式来搜索最佳配置。如其名所示，**可微模型**通过将模型选择限制在可微的组件上来实现这一点，即可进行微分的数学函数。能够计算高维函数的导数（即梯度, gradient）意味着能了解参数微小扰动时会发生什么，这衍生出自动优化的流程（最著名的有**自动微分**，automatic differentiation，和**梯度下降**，gradient descent）。本书第一部分将详细阐述这一体系（第I部分，**罗盘与磁针**），涵盖第2章到第6章。

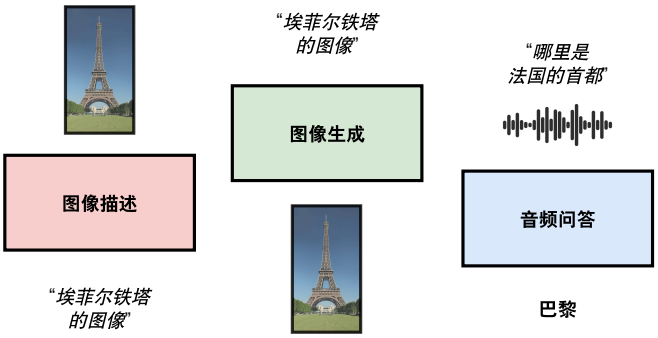



**图 F.1.2:** 大多数任务可以根据所需输入-输出类型进行分类：图像生成（*image generation*）需要从文本（字符序列）生成图像（像素的有序网格），而相反的任务（图像描述，*image captioning*）则是从图像生成说明文字。再比如，音频问答（*audio query answering*）需要从音频（另一种有序序列，它是数值序列）中获取文本。有趣的是，在以上所有情况下，模型设计都遵循了相似的规格。

将神经网络视为简单的可微原语的组合，我们可以提出两个基本问题（图F.1.3）：第一，我们可以处理哪些**数据类型**（data types）的输入或输出？第二，我们可以使用哪些类型的原语？可微性是一个强要求，它不允许我们直接处理许多标准数据类型，例如字符或整数，因为它们本质上是离散的，不连续的。相对而言，我们将看到可微模型能轻松处理以大型数组（我们称之为**张量**，tensor）表示的，更复杂的数据，例如图像，这些数据可以通过线性和非线性变换的基本组合来进行代数操作。

在本书第二部分中，我们将着重介绍可微组件的典型示例——**卷积**算子（convolutional operator）（第II部分，第7章到第9章）。只要数据能被表示为有序的元素序列，就可以应用卷积运算：这样的数据包括音频、图像、文本和视频等。在此过程中，我们还会介绍许多用于设计深度（即，由多个步骤顺序组成）模型的实用技术，以及一些重要的概念，例如**文本词元化**（text tokenization）、

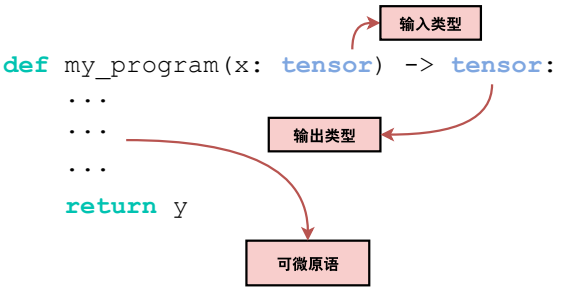



**图 F.1.3:** 神经网络是可微**原语**组成的序列，对结构化数组（**张量**）进行操作：每个原语可以根据其输入/输出特征来分类，这反过来定义了它们的组合规则。

序列的**自回归**（autoregressive）生成和 **因果建模**（casual modeling），这些概念构成了当代最先进的大型语言模型的基础。

本书的第三部分（第III部分，**钻进兔子洞**）将继续探索可微模型，探讨用于集合的替代设计（最重要的是第10和第11章中的**注意力**层和 **transformer** 模型），图结构（第12章），最后是用于时间序列的循环层（第13章）。

本书配有网站，[4]我希望在那里陆续收录不局限于特定数据类型的有趣主题，例如**生成式建模**（generative modelling）、**条件计算**（conditional computation）、**迁移学习**（transfer learning）和**可解释性**（explainability）。这些章节更偏向研究性质，可以按任何顺序阅读。此外，我还提供了一系列笔记本形式的指导实验，涵盖了本书的大部分内容以及对比学习和模型合并等高级主题。[5]

[4] https://sscardapane.it/alice-book

[5] http://tinyurl.com/guided-labs

## 在可微的领域

神经网络的历史悠久而丰富。其名称本身可以追溯到 20 世纪早期对（生物）神经元的建模尝试，类似的术语仍然普遍存在：为了与现有框架保持一致，在接下来的章节中，我们可能会提到*神经元*（neuron）、*层*（layer），或者*激活值*（activation）等。在多次兴趣热潮之后，网络的复杂性在 2012 年至 2017 年间迎来了前所未有的增长，这一增长受大规模的基准测试和竞赛推动，最著名的就是将在第9章介绍的 **ImageNet 大规模视觉识别挑战赛**（ILSVRC）。第二波主要的兴趣浪潮来自 2017 年 **transformer** 的引入（第10章）：就像几年前计算机视觉领域被卷积模型主导，自然语言处理在短时间内也被 transformer 所取代。在接下来的几年中，视频、图（第12章），和音频领域也取得了进一步的进展，最终引发了当前对于大型语言模型、多模态网络和生成模型的热情。[6]

这一时期的术语也经历了快速演化，从 80 年代的**连接主义**（connectionism）[RHM86]，到后来用**深度学习**（deep learning）来指代现代网络，以区别于过去那些较小、较浅的模型 [Ben09, LBH15]。尽管如此，这些术语依然显得模糊，因为现代（人工）神经网络与生物神经网络及神经内科几乎没有任何相似之处 [ZER+23]。观察现代神经网络，其本质特征是由可微模块组成：基于这一特性，在本书中，我在适当情形下更倾向于使用**可微模型**这一术语。将神经网络视为可微模型，将直接引向一个更广阔的领域——**可微编程**（differentiable programming），这是一门新兴学科，融合了计算机科学与优化理论，旨

[6]此处不会全面概述现代神经网络发展史；对于感兴趣的读者，推荐阅读 [Met22]作为起点。


The New York Times

NEW NAVY DEVICE LEARNS BY DOING; Psychologist Shows Embryo of Computer Designed to Read and Grow Wiser

July 8, 1958

**图 F.1.4:** 人工智能热——但是在 *1958* 年。美国心理学家 *Frank Rosenblatt* 通过他对“感知机”的研究引起了大量媒体关注，感知机（*perceptron*）是最早的神经网络工作原型之一。

在对更广义上的可微计算机程序进行研究 [BR24]。[7]

当我们探索可微模型的领域时，我们也穿越了历史：通过梯度下降法对线性模型数值优化的基本概念（第4章中涉及）至少自 19 世纪就已为人所知 [Sti81]；稍后将使用的所谓“全连接网络”可以追溯到 20 世纪 80 年代 [RHM86]；卷积模型在 90 年代末已经被了解并使用 [LBBH98]。[8] 然而，花费了数十年时间，我们才拥有充足的数据和算力，来真正实现其卓越性能。

尽管本书无法涵盖所有可能的主题（也由于快速迭代的研究进展），但我希望本书能提供足够的基础，帮助读者轻松浏览前沿文献。

[7]和许多人一样，我受到 Y.LeCun 于 2018 年在 Facebook 发表的“宣言”的启发：https://www.facebook.com/yann.lecun/posts/10155003011462143。关于神经网络与开源编程（及开发）之间的联系，我也感谢 C.Raffel 于 2021 年发布的另一份宣言：https://colinraffel.com/blog/a-call-to-build-models-like-we-build-open-source-software.html.

[8]可以参考 [AR00]，通过一些关键人物的采访记录来了解这一时期之前 NN 的历史；J.Schmidhuber 也撰写了一份带有个人观点的详尽历史《*Annotated History of Neural Networks*》：https://people.idsia.ch/~juergen/deep-learning-history.html.

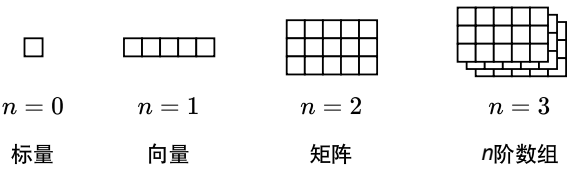



**图 F.1.5:** 基本数据类型：标量、向量、矩阵和通用的 $n$ 阶数组。我们将它们统称为**张量**。$n$ 被称为张量的**阶**。为了方便阅读，我们将向量展示为行向量，但在文中，我们假定所有向量都是列向量。

## 术语与符号[9]

在处理可微模型时，最基本的数据类型是**张量**,[10]本书将其定义为包含对象的 $n$ 阶数组，对象通常是实数。在此，对任何正在阅读的数学研究者表示必要的歉意，我们将 $n$ 称为张量的**阶**（rank）。本书中的表示法会根据 $n$ 的不同有所变化：

1. 只有单独元素的张量（$n = 0$）是单个数值（**标量**，scalar）。对于标量，本书采用小写字母表示，例如 $x$ 或 $y$。[11]

2. 一列数值构成的张量（$n = 1$）是**向量**（或**矢量**，vector）。对于向量，我们使用加粗的小写字体，例如 $\mathbf{x}$。需要区分时，相应的行向量表示为 $\mathbf{x}^\top$。如果

[9]译者注：本书中大部分术语的译法与《深度学习》（即“花书”，Ian Goodfellow、Yoshua Bengio、Aaron Courville 著）和《动手学深度学习》（Aston Zhang、Zachary Lipton、Mu Li、Alex Smola 著）的中文版保持一致。

[10]在科学文献中，张量被更严格地定义为多重线性算子 [Lim21]，而本书中涉及的对象只是相对简单的多阶数组。尽管这种用法存在误差，但由于张量一词的使用极为普遍，本书将沿用这一用法。

[11]之所以如此命名，是因为任何标量均可写为 1 的倍数。另外，本书后续章节将适当精简脚注的数量。

上下文清晰，亦可省略转置符号，以提高可读性。

3. 按矩形排列的数值数组（$n = 2$）是**矩阵**（matrix）。本书使用大写加粗字体表示矩阵，例如 $\mathbf{X}$ 或 $\mathbf{Y}$。

4. 对于 $n > 2$ 的张量，不使用特定的符号表示。本书避免使用花体符号（如 $\mathcal{X}$），此类符号被预留用于表示集合或概率分布。

处理张量时，我们将采用多种索引策略，详细介绍见第2.1节。在大多数情况下，理解一个算法或操作，可以归结为理解参与运算的每个张量的形状。为了简洁地表示形状，我们使用以下符号：

$$X \sim (b, h, w, 3)$$

这是一个 4 阶张量，形状为 $(b, h, w, 3)$。一些维度可以预先指定（例如 3），其余维度可以用变量来表示。需要指出，本书使用相同的符号表示从概率分布中抽样，例如 $\varepsilon \sim \mathcal{N}(0, 1)$，但这种情况较少出现，并且符号含义通常可以根据上下文明确区分。因此，$\mathbf{x} \sim (d)$ 将替代更常用的 $\mathbf{x} \in \mathbb{R}^d$，类似地，$\mathbf{X} \sim (n, d)$ 将替代 $\mathbf{X} \in \mathbb{R}^{n \times d}$。最后，当需要对张量的元素施加特定约束条件时，将采用如下特殊标记法：

1. $\mathbf{x} \sim \text{Binary}(c)$ 表示只包含二元值的张量，即其元素仅限于集合 $\{0, 1\}$。

2. $\mathbf{x} \sim \Delta(a)$ 表示属于**单纯形**的向量，即 $x_i \geq 0$ 且 $\sum_i x_i = 1$。对于更高阶的张量，例如 $\mathbf{X} \sim \Delta(n, c)$，我们假设其归一化是沿着最后一个维度进行的。（例如，$\mathbf{X}_i$ 的每一行都属于单纯形的情况）。

在后续各章节中，会根据需要引入其他的补充符号。此外，本书还使用了一些特殊符号来提示读者：




- **瓶子**符号用于强调某些重要定义。本书包含较多定义，特别是在前几章中，我们使用这个符号在视觉上区分最重要的定义。




- **时钟**符号用于标记我们认为对理解后续内容至关重要的章节——请不要跳过！




- 相反地，**茶杯**符号用于标记比较轻松的部分，这些部分通常是阐述性的，与其他内容相比大多可选择性阅览。

## 启程前的最后想法

本书源于一个想法：希望能将我多年来在罗马大学的数据科学硕士专业中讲授《**面向数据科学应用的神经网络**》课程的授课内容，系统地整理成册。本书核心章节构成该课程主体框架，其余拓展章节则根据每年的情况动态调整。此外，本书部分内容由我曾经（或计划）讲授的其他课程材料补充，包括计算机工程专业《**神经网络**》课程的部分内容、面向电信工程专业的机器学习导论，以及这些年间的一些教程、博士课程和暑期学校的内容。

当前，关于现代深度神经网络这一领域，已有不少优秀的（和最新的）书籍，包括 [Pri23, ZLLS23, BB23, Fle23, HR22]。本书在开头部分涵盖了与这些书籍相似的内容，而在阐述方式和某些拓展部分（或进阶章节中的若干内容）则与它们交集较少，这些部分主要源于笔者个人的研究兴趣。笔者希望能够为现有的材料提供额外（且互补的）视角。

如书名所暗示，对可微程序的理解既来自理论，也来自编程实践：模型设计与实现之间始终存在相互作用，这种关联性在自动微分等技术中体现得尤为显著。当前的神经网络复兴（大约始于 2012 年）很大程度上可以归功于强大软件库的涌现，从 Theano[ARAA+16]到 Caffe、

Chainer，再到如今的 TensorFlow、PyTorch 和 JAX 等。本书在论述中将适时关联现有编程框架的概念，尤其着重于 PyTorch 和 JAX。然而，本书并非编程手册，推荐读者参考这些库的官方文档，以获得它们的完整介绍。

在继续深入探讨之前，需特别指出本书不能涵盖或侧重的几个方面。第一，本书尽力选取了一些既 (a) 在当前普遍使用，又 (b) 足够通用、在不久的将来依然有用的概念。然而，我无法预见未来，也不强求内容面面俱到，因此，在您阅读时，某些章节的内容可能不完整或已过时。第二，对于每个概念，我会提供一些文献中的变体示例（例如从批量归一化到层归一化）。但请注意，实际存在的变体可能有成百上千种：对此，鼓励您探索Papers With Code网站上的众多页面。最后，本书主要聚焦于可微模型的基本组件，但要大规模实现这些组件（并使其运行），需要精湛的工程技术和（一定程度的）实践直觉。本书对硬件方面内容涉猎较少，而对于后者，没有什么能比实践经验和有见解的博客文章更具价值。[12]

## 致谢

公式着色得益于 ST John 提供的精美 LaTeX 包。[13] 《爱丽丝梦游仙境》的彩色图片以及页边的黑白符号均从 Shutterstock.com 获得授权。正文中的《爱丽丝梦游仙境》插图均为约翰 · 坦尼尔原版插图的复刻，感谢 Wikimedia。感谢 Roberto Alma 对本书早期草稿的大量反馈，并鼓励我出版这本书。同时感谢 Corrado Zoccolo，Emanuele Rodolà，Marcin Słaby，Konstantin Burlachenko，Shahed Akter 和 Diego Sandoval 对当前版本提供的大量更正和建议，以及所有通过电子邮件向

[12] 例如，请参阅 A.Karpathy 的博客文章: http://karpathy.github.io/2019/04/25/recipe/，或他近期发布的 **Zero to Hero** 视频系列: https://karpathy.ai/zero-to-hero.html.

[13] https://github.com/st--/annotate-equations/tree/main

我提供反馈的读者。

## 许可协议

本书以 CC BY-SA 许可协议发布。[14]该许可协议允许“再使用者以任何媒介或格式分发、重混、改编本材料并以其为基础进行创作，只要注明原作者即可。该许可协议允许商业性使用。如果您重混、改编本材料或以其为基础进行创作，必须以相同的条款对修改后的材料进行许可”。




[14] https://creativecommons.org/licenses/by-sa/4.0/

# Part I
# 罗盘与磁针



“请您告诉我，从这儿我应该
走哪条路？”
那猫说：“那要看你要到哪里去。”
阿丽思道：“我倒不拘上哪儿去——”
“那么你就不拘走哪条路。”

**—第六章，胡椒厨房和猪孩子**

# 2 | 数学预备知识

**本章导读**

本章简要总结了阅读本书所需的数学概念。我们默认读者已掌握相关基础知识，因此将侧重阐述特定符号表示法，并提供整体概述。在可能的情况下，会强调部分内容（例如张量）与其在实际应用中的实现方式之间的关联。

本章由三个逻辑连贯的部分组成，首先是**线性代数**，然后是对 $n$ 阶对象的**梯度**定义，最后是如何利用这类梯度来**优化**函数。关于**概率论**的独立概述见附录A，其中重点探讨**最大似然**原理。

本章内容丰富，定义众多：请耐心跟我一起学习！

## 2.1 线性代数

本书将回顾线性代数的若干基本概念，为后续内容奠基（也为了统一符号表示）。本书大部分内容都围绕**张量**这一核心概念展开。




**定义 D.2.1 (张量)**

**张量** $X$ 是一个由相同类型元素构成的 $n$ 阶数组。在本书中，使用以下符号：

$$X \sim (s_1, s_2, \dots, s_n)$$

来表示张量的**形状**。

当 $n = 0$ 时，对应**标量**（单个数值）；$n = 1$ 时为**向量**，$n = 2$ 时为**矩阵**；对于更高的 $n$，则得到更高阶的数组。回忆一下，本书使用小写 $x$ 表示标量,，加粗小写 $\mathbf{x}$ 表示向量，加粗大写 $\mathbf{X}$ 表示矩阵。此类张量在深度学习中具有基础性地位，因其特性适用于大规模并行计算的实现，例如使用 GPU 或专用硬件（如 TPU、IPU）计算。

张量由其元素类型和形状来描述。本书主要讨论浮点数类型的张量（具体格式将在后续部分详述），但张量也可以定义为整数（例如在分类任务中）或字符串（例如文本）。张量可以通过**索引**操作来获取其值的**切片**（子集），并且大多数情况下本书遵循 NumPy 的索引惯例[1]约定。简单公式使用下标符号表示：例如，对于一个三阶张量 $X \sim (a, b, c)$ 我们可以写作 $X_i$ 来表示大小为 $(b, c)$ 的切片，或者用 $X_{ijk}$ 表示一个标量。对于更复杂的表达式，使用逗号分隔索引，例如 $X_{i,:,j:k}$ 表示一个大小为

[1]如需复习，请参阅：https://numpy.org/doc/stable/user/basics.indexing.html。为提升本书可读性，采用 1 为起始索引，而非编程常规的从 0 起始。另请参阅章末的练习。

```python
import torch
x = torch.randn((4, 1))  # " 列 "
y = torch.randn((4,))    # 1D 张量
print((x + y).shape)
# [输出]: (4,4) (由于广播机制!)
```

**框 C.2.1:** 一个（可能不正确）广播的示例，由于两个向量的形状，导致逐元素运算输出为矩阵。此错误广泛存在于主流框架中（*NumPy*、*TensorFlow*、*JAX* 等）。

$(b, k-j)$ 的切片。为避免视觉上的杂乱，必要时采用浅灰色标注：

$$[X]_{ijk}$$

直观地分开索引与其余部分，其中 $[\bullet]$ 的参数也可以是一个表达式。

### 2.1.1 常见向量运算

本书主要关注可表示为可微运算组合的模型。具体而言，大多数模型将由基本加法、乘法和一些额外的非线性函数组成，比如指数函数 $\exp(x)$、正弦和余弦，以及平方根。

向量 $\mathbf{x} \sim (d)$ 是一阶张量的典型实例。线性代数教科书严格区分列 (column) 向量 $\mathbf{x}$ 和行 (row) 向量 $\mathbf{x}^\top$，本书将尽量遵循此规范。在代码中，这一点更为复杂，因为行向量和列向量分别对应于形状为 $(1, d)$ 或 $(d, 1)$ 的 2 阶张量，这与形状为 $(d)$ 的 1 阶张量不同。切记，此差异至关重要，因为大多数框架实施的广播规则[2]与 NumPy 一致，这可能导致一些反直觉的结果。有关隐式广播张量形状引发的典型错误示例，请参考框C.2.1。

[2]简而言之，广播机制从右对齐张量的形状，并在可能的情况下复制数据，以匹配两者的形状: https://numpy.org/doc/stable/user/basics.broadcasting.html.

向量具备独特的代数结构（称为**向量空间**，vector space），意味着任何两个形状相同的向量 $\mathbf{x}$ 和 $\mathbf{y}$ 都可以通过线性组合 $\mathbf{z} = a\mathbf{x} + b\mathbf{y}$ 来生成新向量：

$$z_i = ax_i + by_i$$

若将向量理解为 $d$ 维欧氏空间中的一个点，则可以通过平行四边形法则来解释向量加法，而向量到原点的距离由欧几里得范数（$\ell_2$）给出：

$$\|\mathbf{x}\| = \sqrt{\sum_i x_i^2}$$

平方范数 $\|\mathbf{x}\|^2$ 具有特殊重要性，因其等于向量各元素平方和。向量的核心基本运算是**内积**（或**点积**），它通过逐元素相乘并将结果求和来计算。




**定义 D.2.2 (内积)**

两个向量 $\mathbf{x}, \mathbf{y} \sim (d)$ 之间的内积由下式给出：

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\top \mathbf{y} = \sum_i x_i y_i \tag{E.2.1}$$

物理学中常用符号 $\langle \bullet, \bullet \rangle$ 表示内积，本书在需明确表达式结构时亦采用此记法。需着重强调，两个向量之间的点积运算结果为标量。例如，当 $\mathbf{x} = [0.1, 0, -0.3]$ 且 $\mathbf{y} = [-4.0, 0.05, 0.1]$：

$$\langle \mathbf{x}, \mathbf{y} \rangle = -0.4 + 0 - 0.03 = -0.43$$

点积的几何意义可通过其与两向量夹角 $\alpha$ 的关系表示：

$$\mathbf{x}^\top \mathbf{y} = \|\mathbf{x}\|\|\mathbf{y}\| \cos(\alpha) \tag{E.2.2}$$

因此，对于两个满足 $\|\bullet\| = 1$ 的归一化向量，点积等同于它们夹角的余弦，此时称点积为**余弦相似度**。余弦相似度 $\cos(\alpha)$ 在 1（两个向量方向相同）和 $-1$（两个向量方向相反）之间波动，特殊情况 $\langle \mathbf{x}, \mathbf{y} \rangle = 0$ 表示两向量**正交**（方向垂直）。对于两个归一化向量（具有单位范数），如果固定第一个参数 $\mathbf{x}$，可得：

$$\mathbf{y}^* = \arg\max \ \langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x} \tag{E.2.3}$$

其中 $\arg\max$ 表示查找与最高可能输出值对应的 $\mathbf{y}$ 值的操作。通过(E.2.3)可知，为了最大化点积，第二个向量必须与第一个向量相等。该结论在后续章节中至关重要：当 $\mathbf{x}$ 表示输入，而 $\mathbf{w}$ 表示（可适应的）参数时，若 $\mathbf{x}$ 与 $\mathbf{w}$ “共振”，点积被最大化（**模板匹配**）。

最后，补充两点有用的观察结果。第一，可以将一个向量的元素之和写成它与一个完全由 1 组成的向量 $\mathbf{1} = [1, 1, \ldots, 1]^\top$ 的点积：

$$\langle \mathbf{x}, \mathbf{1} \rangle = \sum_{i=1}^{d} x_i$$

其次，两个向量之间的距离也可以用它们的点积来表示：

$$\|\mathbf{x} - \mathbf{y}\|^2 = \langle \mathbf{x}, \mathbf{x} \rangle + \langle \mathbf{y}, \mathbf{y} \rangle - 2\langle \mathbf{x}, \mathbf{y} \rangle$$

当 $\mathbf{y} = \mathbf{0}$ 时，可得 $\|\mathbf{x}\|^2 = \langle \mathbf{x}, \mathbf{x} \rangle$。这两个方程在编写方程或代码时都很有用。

### 2.1.2 常见矩阵运算

在 2 阶情况下，假设有矩阵：

$$\mathbf{X} = \begin{bmatrix} X_{11} & \cdots & X_{1d} \\ \vdots & \ddots & \vdots \\ X_{n1} & \cdots & X_{nd} \end{bmatrix} \sim (n, d)$$

在这种情况下，可定义一个具有 $n$ 行和 $d$ 列的矩阵。对于后续内容尤其重要的是，矩阵可以理解为由 $n$ 个向量的**堆叠** (stack) $(\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n)$，其中堆叠按行方向组织:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_n^\top \end{bmatrix}$$

我们称 $\mathbf{X}$ 表示一**批** (batch) 数据向量。正如我们将看到的，通常会定义模型（无论数学还是代码）以处理这种批量数据。矩阵的一种核心运算是乘法：




**定义 D.2.3 (矩阵乘法)**

对于任意两个形状兼容的矩阵 $\mathbf{X} \sim (a,b)$ 和 $\mathbf{Y} \sim (b,c)$，矩阵乘法 $\mathbf{Z} = \mathbf{XY}$，其中 $\mathbf{Z} \sim (a,c)$ 的元素级定义为：

$$Z_{ij} = \langle \mathbf{X}_i, \mathbf{Y}_j^\top \rangle \tag{E.2.4}$$

即，乘积第 $(i,j)$ 元素是矩阵 $\mathbf{X}$ 的第 $i$ 行与矩阵 $\mathbf{Y}$ 的第 $j$ 列之间的点积。

作为特例，如果第二项是向量，则会得到矩阵-向量乘积如下：

$$\mathbf{z} = \mathbf{W}\mathbf{x} \tag{E.2.5}$$

如果将 $\mathbf{X}$ 解释为一批向量，则矩阵乘法 $\mathbf{X}\mathbf{W}^\top$ 本质上实现了公式(E.2.5)中计算 $n$ 个点积的**向量化**（vectorized）方法，每个点积对应 $\mathbf{X}$ 的一行，进行一次线性代数运算。同理，矩阵与其转置相乘，$\mathbf{X}\mathbf{X}^\top \sim (n, n)$，这种向量化方法可以同时计算 $\mathbf{X}$ 各行两两之间的所有可能点积。

最后列举若干关键矩阵运算。

**定义 D.2.4 (阿达玛乘法)**

对两个形状相同的矩阵，其**阿达玛乘法**（*Hadamard multiplication*）逐元素定义为：

$$[\mathbf{X} \odot \mathbf{Y}]_{ij} = X_{ij} Y_{ij}$$

虽然阿达玛乘法不具备标准矩阵乘法的完备代数性质，但在可微模型中，它常用于执行掩码（masking）操作（例如，将某些元素设为零）或缩放操作。如下文所述，乘法交互也在一些新兴模型家族中受到重视。

对于形如 $\exp(\mathbf{X})$ 的表达式，应理解为在矩阵中逐元素指数运算：

$$[\exp(\mathbf{X})]_{ij} = \exp(X_{ij}) \tag{E.2.6}$$

作为对比，对于方阵，“真正的”矩阵指数运算表示为：

$$\text{mat-exp}(\mathbf{X}) = \sum_{k=0}^{\infty} \frac{1}{k!} \mathbf{X}^k \tag{E.2.7}$$

```
X = torch.randn((5, 5))
# 逐元素指数运算
X = torch.exp(X)
# 矩阵指数运算
X = torch.linalg.matrix_exp(X)
```

**框 C.2.2:** 矩阵的逐元素指数运算与线性代数教材中定义的矩阵指数运算之间的区别。专用的线性代数操作通常被封装在它们各自的子包中。

核心差异在于：(E.2.6)可以对推广至任意维度张量，而(E.2.7)仅适用于（方形）矩阵。因此，所有框架（如PyTorch）都有专门的矩阵运算子模块，例如求逆和行列式。请参阅框C.2.2中的示例。

最后，我们可以编写跨轴进行的*归约*（reduction）操作(如求和、求均值等)，而无需指定上下标，此时假设求和沿全轴范围进行：

$$\sum_i \mathbf{X}_i = \sum_{i=1}^{n} \mathbf{X}_i$$

在 PyTorch 和其他框架中，对于具有`axis`参数的方法的归约操作：

```
r = X.sum(axis=1)
```

**关于矩阵乘法的定义**

为什么矩阵乘法定义为(E.2.4) 而不是阿达玛乘法? 考虑一个向量 $\mathbf{x}$ 以及在其上定义的某个通用函数 $f$。如果满足 $f(\alpha\mathbf{x}_1 + \beta\mathbf{x}_2) = \alpha f(\mathbf{x}_1) + \beta f(\mathbf{x}_2)$，则该函数为**线性**。任何这样的函数都可以表示为一个矩阵 $\mathbf{A}$（通过将两个向量作为基向量进行扩展，可以看到这一点）。那么，矩阵-向量乘积 $\mathbf{Ax}$ 对应于函数应用（function application），即 $f(\mathbf{x}) = \mathbf{Ax}$，矩阵乘法 $\mathbf{AB}$ 对应于复合函数（function composition）$f \circ g$，其中 $(f \circ g)(\mathbf{x}) = f(g(\mathbf{x}))$ 且 $g(\mathbf{x}) = \mathbf{Bx}$。

## 计算复杂度




我们将以矩阵乘法为例，介绍运算复杂度（complexity）的概念。观察(E.2.4)可得知，如果直接根据定义计算，从输入参数 $\mathbf{X} \sim (a, b)$ 和 $\mathbf{Y} \sim (b, c)$ 生成的矩阵 $\mathbf{Z} \sim (a, c)$，需要进行 $ac$ 次维度为 $b$ 的内积运算（我们称之为时间复杂度）。而在顺序实现过程中，内存需求仅为输出矩阵的大小（称为空间复杂度）。

为了抽象化具体硬件差异，计算机科学主要关注所谓的大 $\mathcal{O}$ 表示法，其源于德语“*ordnung*”（代表逼近阶，order of approximation）。当函数 $f(x)$ 复杂度为 $\mathcal{O}(g(x))$ 时，假设其输入和输出均为非负，存在一个常数 $c$ 和一个值 $x_0$，使得：

$$\text{对于任意} x \geq x_0, \quad f(x) \leq cg(x) \tag{E.2.8}$$

这意味着，一旦 $x$ 规模足够大，在分析中可以忽略 $g(x)$ 以外的所有因素。这种分析方法被称为**渐近**分析（asymptotic analysis）。因此，我们可以说，朴素实现的矩阵乘法（时间）复杂度是 $\mathcal{O}(abc)$，随三个输入参数呈线性增长。对于两个尺寸为 $(n, n)$ 的方阵，我们称矩阵

乘法在输入维度上是立方级（cubic）的。

基于渐近复杂度的理论分析重要且优雅，但仅根据大 $\mathcal{O}$ 复杂度选择算法，不一定能转化为实际的性能提升。实际性能取决于许多细节因素，例如所使用的硬件、其支持的并行性等等。[3] 举例来说，目前已知的两个大小为 $(n, n)$ 的方阵相乘的最佳渐近算法复杂度为 $\mathcal{O}(n^c)$，其中常数 $c < 2.4$[CW82]，这显著优于朴素实现的立方级 $\mathcal{O}(n^3)$ 复杂度。然而，这类算法很难在高度并行的硬件（如 GPU）上有效并行化，因此实践中较少见。

请注意，从渐近复杂度的角度来看，在拥有具有 $k$ 个处理器的并行环境中，其理论复杂度并不会发生改变。因为与非并行实现相比，它最多只能提供 $\frac{1}{k}$ 倍的常数级加速。此外，渐近复杂度并未考虑数据在不同存储单元间传输的时间开销，而这种传输延迟在许多情况下可能成为主要性能瓶颈。[4]在这些情况下，我们称该实现为*内存受限*（memory-bound）而不是*计算受限*（compute-bound）。实际上，这只能通过分析器来检查代码性能。可以看出，由于理论（渐进）复杂度和实测复杂度间的相互作用，分析算法复杂度绝非易事。

### 2.1.3 高阶张量运算

向量和矩阵之所以重要，是因为它们允许我们定义许多在更高阶张量中未定义或较复杂的运算（例如，矩阵指数、矩阵乘法、行列式等）。拓展至高阶情形时，我们关

[3] 当您在线性代数框架中调用一个特定的原语，例如 PyTorch 中的矩阵乘法`A @ B`，最终执行的底层实现（即**内核**，kernel）取决于运行时的硬件，通过称为**分发**（dispatching）的过程。因此，相同的代码可以通过 GPU 内核、CPU 内核、TPU 内核等运行。像 XLA(https://openxla.org/xla) 这样的编译器使此过程变得更加复杂，XLA 可以通过融合和优化操作，针对特定目标硬件，来优化代码。

[4] https://docs.nvidia.com/deeplearning/performance/dl-performance-gpu-background/index.html

```
X = torch.randn((4, 5, 2))
Y = torch.randn((4, 2, 3))
(torch.matmul(X, Y)).shape # 或 X @ Y
# [输出]: (4, 5, 3)
```

**框 C.2.3:** *PyTorch* 中的 *BMM* 等同于标准矩阵乘法。大部分操作都可以处理批量输入。

注的大部分运算要么是矩阵操作的批处理变体，要么是矩阵运算与归约操作的特定组合。

作为前者的例子，考虑两个张量 $X \sim (n, a, b)$ 和 $Y \sim (n, b, c)$。**批量矩阵乘法**（batched matrix multiplication，BMM）定义为：

$$[\mathrm{BMM}(X, Y)]_i = \mathbf{X}_i \mathbf{Y}_i \sim (a, c) \tag{E.2.9}$$

其中公式给出的是结果的第 $i$ 个 $(a, c)$ 切片，完整的 BMM 运算结果形状为 $(n, a, c)$。

大多数框架能直接支持参数的批处理版本运算，此时批处理维度通常被假设为张量的主维度（leading dimension，即第一维度）。例如，PyTorch 中的批处理矩阵乘法与标准矩阵乘法调用方式相同，详见框C.2.3。

作为归约操作的例子，考虑两个张量 $X, Y \sim (a, b, c)$。泛化版点积（GDT）可以写成：

$$\mathrm{GDT}(X, Y) = \sum_{i,j,k} [X \odot Y]_{ijk} \tag{E.2.10}$$

这仅仅是对其输入的‘扁平化’版本进行的点积。本简要概述涵盖了后续将涉及的大部分张量运算，其余必要内容将在相关章节适时补充。

```
M = torch.einsum('ijz,izk->ijk', A, B) # 分批矩阵乘法
M = torch.einsum('ijk,ijk->', A, B)    # 泛化的点积
```

**框 C.2.4:** *在 PyTorch 中使用 einsum 的示例。*

### 2.1.4 爱因斯坦标记法




本节为选读内容，主要介绍einsum,[5] ，这套约定规则允许用户通过基于文本字符串的简洁语法，来指定几乎所有类型的张量运算（包括规约、求和、乘法等）。

为了介绍这种标记法，再次考虑前文(E.2.9)和(E.2.10)中展示的两个例子，明确地写出所有的轴：

$$M_{ijk} = \sum_z A_{ijz} B_{izk} \tag{E.2.11}$$

$$M = \sum_i \sum_j \sum_k X_{ijk} Y_{ijk} \tag{E.2.12}$$

按照**爱因斯坦标记法**，[6] 我们可以通过去掉求和符号来简化这两个等式，约定是：任何在右边出现，但未出现在左边的指标都被自动求和。

$$M_{ijk} = A_{ijz} B_{izk} \triangleq \sum_z A_{ijz} B_{izk} \tag{E.2.13}$$

$$M = X_{ijk} Y_{ijk} \triangleq \sum_i \sum_j \sum_k X_{ijk} Y_{ijk} \tag{E.2.14}$$

然后，我们可以将索引集中到唯一的字符串中，来合并这两个定义（此时操作数位于左侧）：

[5] https://numpy.org/doc/stable/reference/generated/numpy.einsum.html

[6] 我们使用的标记法是一种简化版本，忽略了上标和下标之间的区别： https://en.wikipedia.org/wiki/Einstein_notation.

```
M = jax.numpy.einsum('ijz,izk->ijk', A, B)
```

**框 C.2.5:** *在 JAX 中使用 einsum 的示例——与C.2.4对比。*

- '*ijz,izk→ijk*'（分批矩阵乘法）；
- '*ijk,ijk→*'（泛化版点积）。

在(E.2.13)-(E.2.14)的定义与其简化的字符串定义之间存在直接的一一对应关系。这在大多数框架的`einsum`操作中都能实现，见框C.2.4。

该标记法的优势在于：无需记忆特定框架的 API，即可实现指定运算；由于 einsum 语法是通用的，从不同框架间迁移代码时能够实现无缝转换。例如，PyTorch 有多个矩阵乘法方法，包括`matmul`和`bmm`，它们具有不同的广播规则和形状约束，而 einsum 为所有这类运算提供了统一的语法。此外，批量矩阵乘法的 einsum 定义与 JAX 等框架中的定义完全相同，见框C.2.5。

处理转置轴也很简单。例如，对于 $A \sim (n, a, b)$ 和 $B \sim (n, c, b)$，$[\mathbf{A}]_i$ 与 $[\mathbf{B}^\top]_i$ 的批量乘法可以通过在 einsum 定义中交换相应的轴来实现：

```
M = torch.einsum('ijz,ikz->ijk', A, B)
```

基于以上原因，einsum 及其泛化版本（如流行的`einops`[7]包）获得了广泛的欢迎。

## 2.2 梯度和雅可比矩阵

正如名称可微所暗示，梯度在本书中扮演核心角色，通过源自梯度下降的半自动机制为模型优化提供了关键方

[7] http://einops.rocks

法。我们在此复习一些关于多值函数微分的基本定义和概念，重点阐述后续章节必需的数学性质，在表述严谨性方面会有所取舍。

### 2.2.1 标量函数的导数

从输入和输出均为标量的简单函数 $y = f(x)$ 开始，其导数的定义如下.

**定义 D.2.5 (导数)**

函数 $f(x)$ 的**导数** (*derivative*) 被定义为:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h} \tag{E.2.15}$$

我们使用多种符号来表示导数: $\partial$ 可泛指任意维度（向量、矩阵）的导数和梯度；需要指明求导所针对的输入参数时，使用 $\partial_x$ 或 $\frac{\partial}{\partial x}$；而 $f'(x)$ 特指标量函数的导数，有时称为拉格朗日表示法 (Lagrange's notation)。

我们在此处不关注函数导数的存在性（即使对于连续函数，导数也不保证处处存在），假设它是已知的。我们仅会在后续第6章讨论非光滑函数的求导问题（例如在 0 处的 $f(x) = |x|$）时，简单涉及这一点。

简单函数的导数可以直接应用定义来得到，例如，熟悉的多项式、对数或正弦函数的导数:

$$\begin{aligned} \partial x^p &= p x^{p-1} \\ \partial \log(x) &= \frac{1}{x} \\ \partial \sin(x) &= \cos(x) \end{aligned}$$

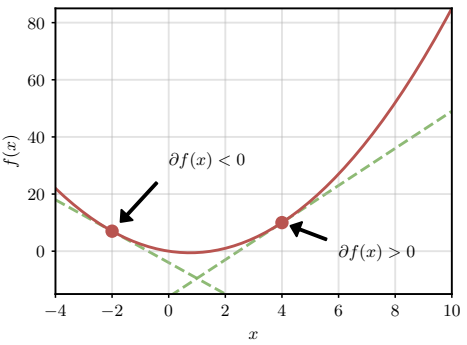



**图 F.2.1:** 函数 $f(x) = x^2 - 1.5x$ 的图像，与在两个不同点上的导数一起展示。

从几何角度来看，导数可以理解为经过某点的切线斜率，或者等价地，视为函数在该点处的最佳一阶逼近，如图F.2.1所示。这是一个基本的观点，因为切线斜率揭示了函数在邻近区域的变化趋势：对于正斜率，函数在右侧增加，左侧减少（同样，在足够小的区间内成立）；而对于负斜率，则相反。后续我们将看到，这一发现可以扩展到向量值函数。

回顾一些重要的导数性质，这些性质可以被扩展到多维情况：

- **线性**：导数是线性的，因此多项之和的导数等于各项导数的和：

$$\partial\Big[f(x) + g(x)\Big] = f'(x) + g'(x)$$

- **乘法法则**：

$$\partial\Big[f(x)g(x)\Big] = f'(x)g(x) + f(x)g'(x)$$

- **链式法则**：复合函数的导数可以通过相应的导数

相乘来得到：

$$\partial\Big[f(g(x))\Big] = f'(g(x))g'(x) \tag{E.2.16}$$

### 2.2.2 梯度和方向导数

现在考虑输入为向量 $\mathbf{x} \sim (d)$ 的函数 $y = f(\mathbf{x})$。在此讨论无穷小扰动没有意义，除非明确扰动的方向（在标量情况下，只存在“左”和“右”，而在欧几里得空间中有无穷多个可能方向）。在最简单的情况下，考虑只沿第 $i$ 轴移动，同时保持所有其他值不变：

$$\partial_{x_i} f(\mathbf{x}) = \frac{\partial y}{\partial x_i} = \lim_{h\to 0} \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h}, \tag{E.2.17}$$

其中 $\mathbf{e}_i \sim (d)$ 是第 $i$ 个基向量（单位矩阵的第 $i$ 行）：

$$[\mathbf{e}_i]_j = \begin{cases} 1 & \text{若 } i = j \\ 0 & \text{否则} \end{cases} \tag{E.2.18}$$

(E.2.17)称为**偏导数**（partial derivative）。将所有偏导数堆叠在一起，得到一个 $d$ 维向量，称为函数的**梯度**(gradient)。




**定义 D.2.6 (梯度)**

一个函数 $y = f(\mathbf{x})$ 的**梯度**被定义为：

$$\nabla f(\mathbf{x}) = \partial f(\mathbf{x}) = \begin{bmatrix} \partial_{x_1} f(\mathbf{x}) \\ \vdots \\ \partial_{x_d} f(\mathbf{x}) \end{bmatrix} \tag{E.2.19}$$

由于梯度的基础性地位，使用特殊符号 $\nabla f(\mathbf{x})$ 予以区

分。那么，沿任意方向 $\mathbf{v}$ 的位移又该如何分析？此时，可以得到**方向导数**（directional derivative）：

$$\mathrm{D}_{\mathbf{v}} f(\mathbf{x}) = \lim_{h \to 0} \frac{f(\mathbf{x} + h\mathbf{v}) - f(\mathbf{x})}{h}, \tag{E.2.20}$$

空间中的运动可以分解为沿各个坐标轴的独立位移，因此很容易证明，方向导数等于梯度与位移向量 $\mathbf{v}$ 的点积：

$$\mathrm{D}_{\mathbf{v}} f(\mathbf{x}) = \langle \nabla f(\mathbf{x}), \mathbf{v} \rangle = \sum_i \partial_{x_i} f(\mathbf{x}) v_i \tag{E.2.21}$$

第 $i$ 轴上的位移

因此，了解如何计算函数的梯度，就足以计算所有可能的方向导数。

### 2.2.3 雅可比矩阵




现在考虑函数 $\mathbf{y} = f(\mathbf{x})$ 的一般情形，其输入向量 $\mathbf{x} \sim (d)$ 与前文相同，但输出向量为 $\mathbf{y} \sim (o)$。正如我们将看到的，这是最具普遍性的情形。由于函数有多个输出，我们为每个输出计算梯度，并将梯度向量堆叠起来，构成一个形状为 $(o, d)$ 的矩阵，称之为函数 $f$ 的**雅可比**（Jacobian）矩阵。

**定义 D.2.7 (雅可比矩阵)**

函数 $\mathbf{y} = f(\mathbf{x})$, $\mathbf{x} \sim (d)$, $\mathbf{y} \sim (o)$ 的**雅可比**矩阵由下式得出：

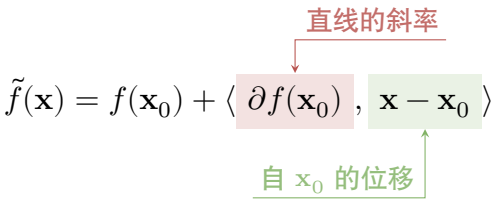
$$\partial f(\mathbf{x}) = \begin{pmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_o}{\partial x_1} & \cdots & \frac{\partial y_o}{\partial x_d} \end{pmatrix} \sim (o, d) \tag{E.2.22}$$

当 $o = 1$ 时，我们得到梯度；当 $d = o = 1$ 时，雅可比

矩阵退化为标准导数。雅可比矩阵继承了导数的所有性质：重要的是，复合函数的雅可比矩阵是等于各函数对应雅可比矩阵的矩阵乘积：

$$\partial\left[f(g(\mathbf{x}))\right] = \left[\partial f(\bullet)\right]\partial g(\mathbf{x}) \tag{E.2.23}$$

其中，第一导数在 $g(\mathbf{x}) \sim (h)$ 处被计算。有关梯度和雅可比矩阵的数值计算示例，请参考 [PP08, 第 2 章]。与标量情况类似，梯度和雅可比矩阵可以被理解为与函数上特定点相切的线性函数。特别地，梯度是以下意义上的最佳“一阶逼近”。对于点 $\mathbf{x}_0$，在 $f(\mathbf{x}_0)$ 无穷小邻域内的最佳线性逼近由下式给出：




这叫做**泰勒定理**（Taylor's theorem）。请参考框C.2.6和图F.2.2，了解标量情况 $f(x) = x^2 - 1.5x$ 的可视化。

```python
# 通用函数
f = lambda x: x**2-1.5*x

# 导数（目前暂时手动计算）
df = lambda x: 2*x-1.5

# 在 0.5 线性化
x = 0.5
f_lin = lambda h: f(x) + df(x)*(h-x)

# 数值检查
print(f(x + 0.01))      # -0.5049
print(f_lin(x + 0.01)) # -0.5050
```

**框 C.2.6:** 计算一阶逼近的示例（标量情况）。结果绘制在图*F.2.2*中。

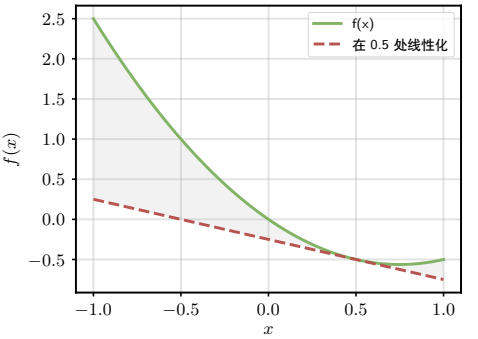



**图 F.2.2:** 函数 $f(x) = x^2 - 1.5x$ 及其在 0.5 的一阶逼近。

## 关于雅可比矩阵的维度




最后，我们以一个关于维度的注解作为结束，这将在后续内容中派上用场。考虑以下函数：

$$\mathbf{y} = \mathbf{W}\mathbf{x}$$

将其视为 $\mathbf{x}$ 的函数时，导数如前所述，是 $(o, d)$ 矩阵，可证明：

$$\partial_{\mathbf{x}} [\mathbf{W}\mathbf{x}] = \mathbf{W}$$

然而，将其视为 $\mathbf{W}$ 的函数时，输入本身是一个 $(o, d)$ 矩阵，此时的“雅可比矩阵”形状为 $(o, o, d)$（参考下一页的框）。然而，我们总可以设想一个等价（同构的）函数，其输入是 $\mathbf{W}$ 的向量化版本,$\text{vect}(\mathbf{W}) \sim (od)$，在这种情况下，雅可比矩阵将是形状为 $(o, od)$ 的矩阵。

**计算雅可比矩阵**

为了计算雅可比矩阵 $\partial_{\mathbf{W}}\mathbf{W}\mathbf{x}$，可以将表达式逐元素重写为：

$$y_i = \sum_j W_{ij}x_j$$

由此，可以直接得出：

$$\frac{\partial y_i}{\partial W_{ij}} = x_j \tag{E.2.24}$$

注意，要显式构造[a]雅可比矩阵（即在内存中存储），会产生大量重复的值。正如第6章将阐明的，这种显式存储可以被避免。因为在实践中，我们只关心将雅可比矩阵作用于另一个张量的结果。

[a]译者注："显式构造"对应英文术语 materialize，在数据库、编译器等领域亦称"物化"。

这个简单例子阐明了我们的观点，从符号角度来看，处理向量输入和输出已经足够。然而，在第6章中需要特别注意：为了简化符号（尤其是避免索引过多），我们会将雅可比表示为矩阵，但这些矩阵的具体尺寸仍需根据输入和输出的实际形状来确定，尤其要注意批量大小（batch size）。我们将在第6章中看到，通过所谓的**向量-雅可比乘积**，可以避免显式计算雅可比矩阵。这也可以通过将雅可比矩阵视为抽象的线性映射来形式化——有关该主题的正式概述，请参考 [BR24]。

## 2.3 梯度下降




为了理解获得梯度的作用，考虑最小化通用函数 $f(\mathbf{x})$，其中 $\mathbf{x} \sim (d)$：

$$\mathbf{x}^* = \arg\min_{\mathbf{x}} f(\mathbf{x}) \tag{E.2.25}$$

其中，与 arg max 相似，arg min $f(\mathbf{x})$ 表示寻找 $f(\mathbf{x})$ 最低值对应的 $\mathbf{x}$ 值的操作。假设该函数具有单一输出（**单目标优化**，single-objective optimization），且优化的 $\mathbf{x}$ 定义域是不受约束的。

在后续章节中，$\mathbf{x}$ 将代表模型参数，$f$ 用于描述模型在数据上的性能表现。这种框架被称为**监督学习**（supervised learning），将在下一章中介绍。我们可以不失一般性地考虑最小化，而非最大化。因为最小化 $f(\mathbf{x})$ 等于最大化 $-f(\mathbf{x})$，反之亦然（为了可视化这一点，可以想象一维函数绕 $x$ 轴旋转，观察其低点如何变化）。

在极少数情况下，也许可以得到问题的闭式解（将在第4.1.2节以最小二乘优化为例）。但一般情况下，我们不得不依赖迭代方法。假设从一个随机猜测 $\mathbf{x}_0$ 开始，在每次迭代中前进一步，这一步可以分解为其大小 $\eta_t$（步长的长度）和方向 $\mathbf{p}_t$：

第 $t$ 次迭代的猜测

$$\mathbf{x}_t = \mathbf{x}_{t-1} + \eta_t \mathbf{p}_t \tag{E.2.26}$$

第 $t$ 次迭代的位移

我们将 $\eta_t$ 称为**步长**（step size，或在机器学习术语中称为**学习率**，learning rate，原因将在下一章中阐明）。对于某个方向 $\mathbf{p}_t$,，如果存在 $\eta_t$ 使 $f(\mathbf{x}_t) \leq f(\mathbf{x}_{t-1})$，则该方向称为**下降方向**（descent direction）。如果能够在每次迭代中选择一个下降方向，并且谨慎选择步长，迭代算法(E.2.26)将在下文说明的意义下收敛到一个最小值。

对于可微函数，可以使用(E.2.20)中的方向导数来精确量化所有的下降方向，因为这些方向可以被定义为相对于之前的猜测 $\mathbf{x}_{t-1}$ 引起负变化的方向：

$$\mathbf{p}_t\text{为下降方向} \quad \Rightarrow \quad \mathrm{D}_{\mathbf{p}_t} f(\mathbf{x}_{t-1}) \leq 0$$

根据第2.2节所学内容，以及(E.2.2)基于余弦相似度的点

积定义，得到：

$$\mathrm{D}_{\mathbf{p}_t} f(\mathbf{x}_{t-1}) = \langle \nabla f(\mathbf{x}_{t-1}), \mathbf{p}_t \rangle = \|\nabla f(\mathbf{x}_{t-1})\| \|\mathbf{p}_t\| \cos(\alpha)$$

其中 $\alpha$ 是 $\mathbf{p}_t$ 和 $\nabla f(\mathbf{x}_{t-1})$ 之间的夹角。观察等式右侧，第一项相对于 $\mathbf{p}_t$ 是一个常数。由于我们假设 $\mathbf{p}_t$ 仅表示移动的方向，我们也可以安全地约束 $\|\mathbf{p}_t\| = 1$，使第二项也成为一个常数。因此，根据余弦的性质，可以推断出任意与 $\nabla f(\mathbf{x}_{t-1})$ 夹角在 $\pi/2$ 和 $3\pi/2$ 之间的方向 $\mathbf{p}_t$ 都是下降方向。在这些方向中，方向 $\mathbf{p}_t = -\nabla f(\mathbf{x}_{t-1})$（夹角为 $\pi$）具有最低的方向导数，我们将其称为**最速下降方向**（steepest descent direction）。

将该发现与(E.2.26)的迭代过程结合，可得到用于最小化任何可微函数的算法，我们称之为**（最速）梯度下降法**（steepest gradient descent）。

**定义 D.2.8 (最速梯度下降法)**

给定一个可微函数 $f(\mathbf{x})$、一个起始点 $\mathbf{x}_0$ 和一个步长序列 $\eta_t$，**梯度下降法**过程如下：

$$\mathbf{x}_t = \mathbf{x}_{t-1} - \eta_t \nabla f(\mathbf{x}_{t-1}) \tag{E.2.27}$$




我们暂不讨论寻找合适步长的问题，而是假设步长“足够小”，以确保梯度下降每次迭代都能使函数值 $f$ 下降。在下一节中，我们将讨论从通用初始化开始运行梯度下降所得到的点。请注意，梯度下降的效率取决于我们计算梯度的过程：为此，我们将在第6章介绍一种高效的算法。

### 2.3.1 梯度下降的收敛性

在讨论梯度下降的收敛性时，我们需要澄清函数的“极小值点（minimizer）”指什么。如果你不关心收敛性且

信任梯度下降，请毫不犹豫地阅读下一节。

**定义 D.2.9 (极小值)**

对于 $f(\mathbf{x})$ 的**局部极小值** *(local minimum)* 点 $\mathbf{x}^+$，满足对于某个 $\varepsilon > 0$ 时，以下条件成立：

$$f(\mathbf{x}^+) \leq f(\mathbf{x}) \quad \forall \mathbf{x} \ : \ \|\mathbf{x} - \mathbf{x}^+\| < \varepsilon$$

尺寸为 $\varepsilon$，中心为 $\mathbf{x}^+$ 的球

换句话说，在 $\mathbf{x}^+$ 的足够小的邻域内，$f(\mathbf{x}^+)$ 的值是一个最小值。直观上，在这样的点上，切线斜率为 0，并且 $\mathbf{x}^+$ 邻域内其他地方的梯度均指向上方。我们可以通过**驻点** (stationary points) 的概念来形式化第一种观点。

**定义 D.2.10 (驻点)**

如果某个点 $\mathbf{x}^+$ 满足 $\nabla f(\mathbf{x}^+) = 0$，则该点被称为 $f(\mathbf{x})$ 的**驻点**。

驻点不仅限于极小值：还可能是极大值（函数 $-f(\mathbf{x})$ 的极小值）或**鞍点**（saddle points），鞍点是曲率变化的拐点（可参考图F.2.3中的示例）。一般来说，如果没有 $f$ 施加任何约束，只能证明梯度下降会收敛到一个通用的静止点，这取决于其初始化状态。

能否获得更优的解? 想象一条抛物线：在这种情况下，函数没有任何鞍点，且仅有一个最小值。这个最小值具有特殊性质，因为函数在该点达到了其整个定义域内的最低值：我们称其为**全局最小值**（global minimum）。

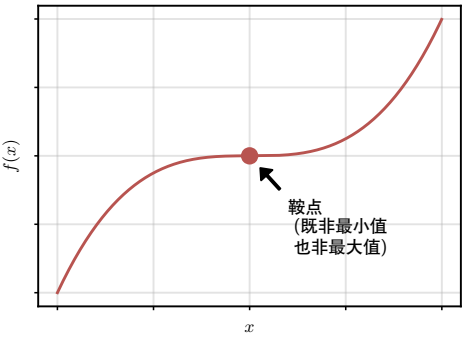



**图 F.2.3:** 鞍点的简单示例（尝试在该点可视化切线以确认它确实是静止的）。

**定义 D.2.11 (全局最小值)**

假如存在点 $\mathbf{x}^*$，对于任何可能的输入 $\mathbf{x}$，都满足 $f(\mathbf{x}^*) \leq f(\mathbf{x})$，则 $\mathbf{x}^*$ 为 $f(\mathbf{x})$ 的**全局最小值**。

直观地说，如果在抛物线上运行梯度下降算法（从任意初始化开始），都会收敛到这个全局最小值，因为所有的梯度都会指向该点。我们可以通过函数**凸性** (convexity) 概念来概括这一思想。凸性有多种等价定义，为了便于阐述，我们选择以下定义。

**定义 D.2.12 (凸函数)**

如果对于任意两点 $\mathbf{x}_1$ 和 $\mathbf{x}_2$ 且 $\alpha \in [0, 1]$ 满足以下条件，则函数 $f(\mathbf{x})$ 为凸 (*convex*)：

从 $f(\mathbf{x}_1)$ 到 $f(\mathbf{x}_2)$ 的线段

$$f(\alpha\mathbf{x}_1 + (1-\alpha)\mathbf{x}_2) \leq \alpha f(\mathbf{x}_1) + (1-\alpha)f(\mathbf{x}_2) \tag{E.2.28}$$

从 $\mathbf{x}_1$ 到 $\mathbf{x}_2$ 的间隔

等式(E.2.28)的左侧代表 $f$ 在 $\mathbf{x}_1$ 到 $\mathbf{x}_2$ 的区间内任意一点的值，右侧表示连接点 $f(\mathbf{x}_1)$ 和 $f(\mathbf{x}_2)$ 的线段上相对应点的值。如果函数始终位于任意两点连线的下方，则该函数为凸（例如，开口向上的抛物线是凸函数）。

凸性在以下意义上定义了优化函数的难易程度 [JK+17]：

1. 对于一般非凸函数，梯度下降仅能收敛至驻点。除非考察更高阶导数（导数的导数），否则无法获得更多信息。

2. 对于凸函数，梯度下降会收敛到全局最小值，不受初始化的影响。

3. 如果不等式(E.2.28)严格成立（**严格凸性**，strict convexity），则全局最小值点也是**唯一**的。

这是一个严苛实现的性质：要在非凸问题中使用梯度下降找到全局最小值，唯一方法是遍历所有可能的初始点，无限次运行优化器，使这个问题变成了 NP 难（NP-hard）任务 [JK+17]。

这一讨论具有重要的历史意义。正如第5章将阐述的，任何非平凡（non-trivial）的模型都是非凸的，这意味其优化问题可能有多个驻点。这与一些监督学习的传统替代算法（如支持向量机）形成对比，它们在保持非线性的同时允许凸优化。有趣的是，即使面临这样的限制，复杂的可微模型仍然表现良好。这意味着，假如从合理的初始化开始，模型总能收敛到具有良好经验性能的点。

### 2.3.2 加速梯度下降

负梯度描述了最速下降的方向，但仅适用于该点的无穷小邻域内。如我们将在第5章（引入随机优化）看到的，这些方向可能包含显著噪声，特别是在大模型中。为了加速优化算法收敛，人们开发了多种技术来选择更好的

下降方向。出于计算效率考虑，我们特别关注无需高阶导数（例如黑塞矩阵，Hessian matrix）或多次调用函数的方法。

此处重点介绍**动量**（momentum）法这一典型技术，并推荐阅读 [ZLLS23, 第 12 章]获取系统概述。[8] 如果你将梯度下降想象为一个“滚下山坡的球”，其运动相对不稳定，，因为每一步迭代的梯度都可能指向不同方向（事实上，对于凸损失函数，如果步长选择得当，任何两次连续迭代的梯度都会保持正交）。通过引入“动量”项，保留部分上一次迭代的方向信息，可有效平滑优化轨迹：

最速下降　　动量项

$$\mathbf{g}_t = -\eta_t \nabla f(\mathbf{x}_{t-1}) + \lambda \mathbf{g}_{t-1}$$
$$\mathbf{x}_t = \mathbf{x}_{t-1} + \mathbf{g}_t$$

初始化 $\mathbf{g}_0 = \mathbf{0}$。见图F.2.4中的示例。系数 $\lambda$ 决定了之前动量项的衰减程度。展开两项可得：

$$\begin{aligned}\mathbf{g}_t &= -\eta_t \nabla f(\mathbf{x}_{t-1}) + \lambda(-\eta_{t-1} \nabla f(\mathbf{x}_{t-2}) + \lambda \mathbf{g}_{t-2}) \\ &= -\eta_t \nabla f(\mathbf{x}_{t-1}) - \lambda \eta_{t-1} \nabla f(\mathbf{x}_{t-2}) + \lambda^2 \mathbf{g}_{t-2}\end{aligned}$$

以此类推，在 $t-n$ 时刻的迭代衰减因子为 $\lambda^{n-1}$。可以证明，动量法通过平滑优化路径来加速训练 [SMDH13]. 另一种常见技术是根据梯度大小，为每个参数自适应地调整步长 [ZLLS23]。结合了这些思想的一种经典优化算法是 Adam 算法 [KB15]。Adam 的优势在于，它对**超参数**（hyper-parameters）的选择相对稳健，[9]在大多数框

[8] 另请参阅 S.Ruder 于 2016 年发表的博客文章：https://www.ruder.io/optimizing-gradient-descent/.

[9] 超参数是由用户选择的参数，不是经梯度下降学习得到的参数。

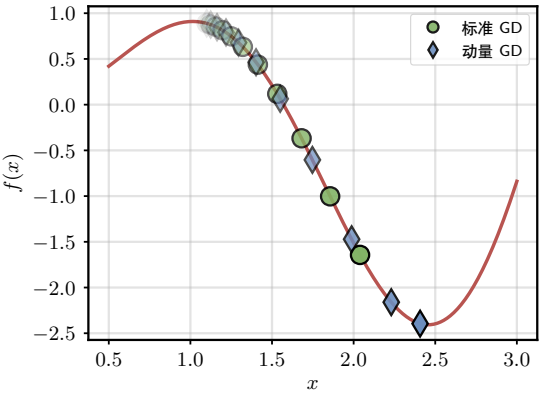



**图 F.2.4:** 最小化函数 $x\sin(2x)$ 时，从 $x = 1 + \varepsilon$ 开始，$\lambda = 0.3$，使用标准 *GD* 和动量 *GD*。

架中，其默认选项可提供良好的初始配置。然而，设计能够取代 Adam（及其变体如 AdamW[LH19]）的新型优化器，仍是当前开放的研究方向，例如近期研究 [BN24] 尝试从第一性原理出发，为神经网络设计定制化优化器。

使用加速优化算法的缺点是可能增加存储需求：例如，动量方法需要在内存中存储上一次的梯度迭代，这使得优化算法的存储开销翻倍（尽管在多数情况下，计算梯度所需的内存才是最主要因素，详见第6.3节）。

# 从理论到实践

**关于练习**

本书没有传统的章末习题，因为已有许多教科书涵盖了这部分内容。取而代之的是贯穿全书的自学路径，旨在帮助您在阅读过程中同步探索两种框架（JAX 和 PyTorch）。练习答案将在本书的官方网站上公布。[a]这些章节中包含了大量指向在线资料的 URL 链接——当您查找它们时，这些链接可能已失效或被移动。

[a] https://www.sscardapane.it/alice-book

## 打好基础



对于任何可微模型的设计者，起点在于对 NumPy 的认真学习。NumPy 提供了操作多维数组（本书中称为张量）的通用函数集，以及索引与变换功能。您可以通过阅读该库的官方入门教程深化理解。[10]重点需要熟练运用 NumPy 数组操作（特别是索引机制）：`rougier/numpy-100`[11]代码库提供了一套由浅入深的练习体系，可有效巩固相关知识。

## 迈向实用框架

尽管 NumPy 影响巨大，但它对并行硬件（如 GPU）的支持有限（除非使用额外扩展库），并且缺乏自动微分功能（将在第6章介绍）。JAX 复刻了 NumPy 的接口，同时新增了硬件扩展支持、自动梯度计算，以及一些额外

[10] https://numpy.org/doc/stable/user/quickstart.html

[11] https://github.com/rougier/numpy-100

的变换功能，比如**向量化映射**（`jax.vmap`）。PyTorch 等框架同样以类 NumPy 接口为核心，但在命名规范与功能设计上略有调整，也添加了用于构建可微模型的高级工具。请花些时间浏览`jax.numpy.array`和`torch.tensor`文档，以理解它们与 NumPy 之间的相似之处。您当前可以暂时跳过诸如`torch.nn`这样的高级模块。在第6章介绍梯度计算机制后，我们会进一步讨论这些框架的设计理念。

## 实现梯度下降算法

为了熟练掌握这三个框架（NumPy、JAX、PyTorch），建议分别用三者完成以下实践——如果熟悉语法，每种实现仅需几分钟。考虑一个二维函数 $f(\mathbf{x})$，其中 $\mathbf{x} \sim (2)$，取定义域为 $[0, 10]$：[12]

$$f(\mathbf{x}) = \sin(x_1)\cos(x_2) + \sin(0.5x_1)\cos(0.5x_2)$$

在继续阅读之前，请用每个框架分别完成以下练习：

1. 以**向量化**的方式实现该函数，即：给定一个由 $n$ 个输入组成的矩阵 $\mathbf{X} \sim (n, 2)$，函数返回向量 $f(\mathbf{X}) \sim (n)$，其中 $[f(\mathbf{X})]_i = f(\mathbf{X}_i)$。

2. 实现另一个函数来计算其梯度（使用硬编码——我们尚未涉及自动微分）。

3. 实现基础的梯度下降算法，并可视化优化过程从不同起始点出发的路径。

4. 尝试添加动量项，并可视化梯度的范数，它应该在算法接近驻点时逐渐收敛到零。

如果使用 JAX 或 PyTorch 练习，第 (3) 步是体验使用`vmap`向量化函数的好机会。

[12] 我咨询 ChatGPT 生成了一个包含多个极小值和极大值的函数。书中其他内容均非 LLM 生成，我认为这是一个重要声明。

# 3 | 数据集和损失

**本章导读**

本章对监督学习的场景做了形式化定义。我们将引入数据集（dataset）、损失函数（loss）、经验风险最小化（empirical risk minimization）的概念，以及监督学习中的基本假设。最后，我们基于最大似然的概念，提供监督学习的概率建模方法。本短章为全书的其余部分奠定了核心基础。

## 3.1 什么是数据集？

我们考虑这样一种情景：手动编码实现某个功能（例如，从真实世界的图像中识别物体）不可行，但收集符合期望行为的**样例**（example）相对容易。从语音识别到机器人导航，这种情况十分常见。我们用以下定义对这一想法进行形式化描述。




**定义 D.3.1 (数据集)**

一个大小为 $n$ 的**监督数据集** (*supervised dataset*) $\mathcal{S}_n$ 是由 $n$ 对数据组成的集合 $\mathcal{S}_n = \{(x_i, y_i)\}_{i=1}^n$，每个 $(x_i, y_i)$ 是我们想要建模的输入-输出关系的一个样例。我们进一步假设每个样例是从某个未知 (且不可知) 的概率分布 $p(x, y)$ 中**独立同分布** (*identically and independently distributed*, *i.i.d.*) 抽取的。

如果您希望在阅读定义后复习概率论，可以参考附录A。最后的假设看起来技术性较强，但其目的是确保我们建模的关系具有实际意义。具体而言，样本 (sample) **同分布** (identically distributed) 意味着我们试图近似的是某种在时间上相对稳定和不变的东西。举一个例子，假设我们需要收集一个数据集来识别照片中的汽车型号。如果我们收集短时间跨度内的图片，该假设成立；但如果图片来自过去几十年内的数据，这一假设就会失效，因为常见的车型会随时间推移发生变化。在后一种情况下，用该数据集训练并部署模型将会失败，因为模型可能无法识别新型号，或者实际表现不佳。

类似地，样本的**独立分布** (independently distributed) 意味着我们的数据集在收集过程中没有偏差，且能充分代表整体分布。回到前面的例子，如果在特斯拉汽车经销店附近收集图像，这样的数据集就是无效的，因为我们会收集到大量特定类型的图像，而缺少其他品牌和型号的图像。请注意，这些假设的有效性取决于具体情况：在意大利收集的汽车数据集，用于罗马或米兰地区的模型部署时可能有效，但若用于日本东京或中国台湾等地区，可能会失效。i.i.d. 假设应始终被仔细检查，以确保我们将监督学习工具应用于有效场景。有趣的是，现代 LLM 在海量数据分布上进行训练，以至于很难区分哪些任务是真正属于分布内，哪些是分布外 (以及模型能泛化到什么程度)，其界限已变得模糊不清。[YCC+24]。

**i.i.d. 属性的更多信息**

需要注意的是，确保 i.i.d. 特性不是一次性的过程，必须在模型的整个生命周期中经常检查。以汽车分类为例，若不加以控制，汽车型号分布随时间的微妙变化将会导致机器学习模型性能下降，这是**域偏移**（domain shift）的一个例子。另一个例子是推荐系统，它会改变用户与某个应用程序的交互方式，因为用户会对推荐系统本身的建议做出反应。这产生了**反馈循环**（feedback loop）[CMMB22]，需要持续重新评估系统和应用程序的性能。

### 3.1.1 监督学习的变体

虽然大多数成功的应用在某种形式上都利用了监督学习，但标准的监督学习场景中存在许多变体。例如，有些数据集可能没有可用的**目标**（target）$y_i$，这种情况下，我们称之为**无监督**学习（unsupervised learning）。无监督学习的典型应用是**聚类**（clustering）算法，其中我们希望将输入数据聚集成，簇（cluster），使得同一簇中的点相似，不同簇之间的点则不相似 [HTF09]。另一个例子是在**检索**（retrieval）系统中，我们希望在一个大型数据库中搜索与用户给定查询最相似的前 $k$ 个元素。

在处理图像等复杂数据时，这并非简单的任务，因为如果我们直接操作像素（即使是微小的扰动也能影响数百万个像素），图像上的距离是没有明确定义的。然而，假设我们已经有可用的可微模型，该模型已经为其他任务进行了优化，我们假设这个任务具有足够的通用性，比如图像分类。我们称之为**预训练**模型。正如我们将看到的，这个模型的内部状态可以被解释为高维空间中的向量。在许多情况下，这些向量展示了有用的几何特性，即语义上相似的物体在这些表示中被发送（**嵌入**）到接近的点。因此，我们可以将这些潜在表示与标准聚类模型一

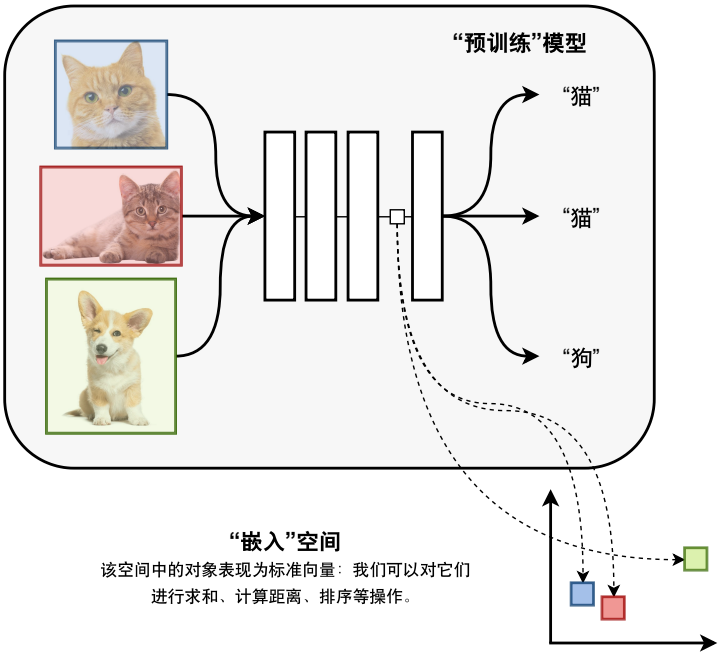



**图 F.3.1:** 可微模型通过线性代数操作，以顺序方式处理数据。在许多情况下，在我们优化这些程序之后，模型的输入数据的内部表示（我们称之为**预训练**（*pre-trained*）模型）具有一些几何特性：例如，语义上相似的图像被投射到“潜在”空间中相近的点。将数据从非度量空间（原始输入图像）转换到度量空间（右下方）被称为对数据进行**嵌入**（*embedding*）。

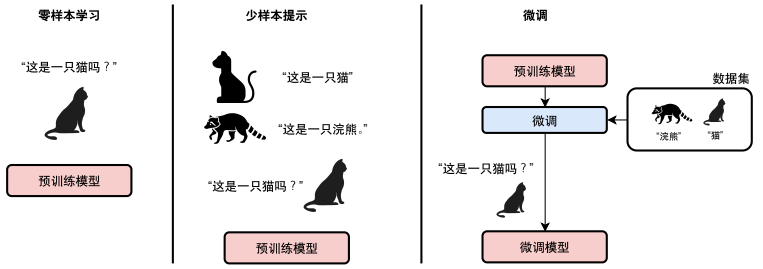



**图 F.3.2:** 使用训练模型的三种方法。**零样本**：直接向模型提出问题。这种方式可以通过生成式语言模型实现（第8章介绍）。**少样本提示**与前者相似，但会提供一些样例作为输入。这两种技术的前提是基础模型具有强泛化能力。**微调**：在一个小型样例数据集上使用梯度下降，对模型优化。这与从头训练模型的方法类似。

起使用，例如高斯混合模型 [HHWW14]。参考图F.3.1了解这一思想的高层概述。

如果没有预训练模型，应该怎么办？无监督学习的一种常见变体称为**自监督**学习（self-supervised learning，SSL，[ZJM+21]）。SSL 的目标是从一个通用的无监督数据集中自动找到某种监督目标，从而优化一个可以用于多种下游任务的模型。例如，如果我们可以访问一个大型文本语料库，就可以优化程序来预测一小段文本的可能续写方式 [RWC+19]。在以自监督方式进行预训练时，神经网络也可以高效地对文本进行嵌入，这一发现对整个研究领域产生了深远影响 [MSC+13]。[1]

正如我们将在第8章和第10章中看到的，LLM 被视为这一基本理念的现代迭代，因为优化诸如 GPT 或 LLaMA[TLI+23]之类模型的过程通常按照基本的自监

[1]大规模网络数据集也充满了偏见、不雅语言和低俗内容。认识到基于这些数据训练的模型会内化这些偏见是另一个重要发现 [BCZ+16]，也是闭源基础模型受到批评的主要原因之一 [BGMMS21].

督训练开始，即下一个词元预测（next-token prediction）。这些模型有时被称为**基础模型**（foundation models）。在最简单的情况下，它们可以直接用于新的任务，比如回答查询，我们称其为**零样本**（zero-shot）方式。对于 LLM，还可以通过提供少量新任务的样例作为输入提示，这被称为**少样本提示**（few-shot prompting）。在最通用的情况下，我们可以在新任务上使用梯度下降，优化预训练的基础模型参数，这叫做**微调**（fine-tuning）模型。有关这三种方法的对比，参考图F.3.2。本书着重讲解从头构建模型，但微调也可以通过类似方法完成。

在网上存在大量开源模型库，使微调变得特别简单。[2]微调可以针对起始模型的全部参数进行，也可以只针对较小的参数子集或新增的少量参数进行，这种方法被称为**参数高效微调**（parameter-efficient fine-tuning, PEFT）。[LDR23].[3]我们将在下一卷中讨论 PEFT 技术。

监督学习还有许多其他变体，由于篇幅限制无法在此处详细列举，仅提供一些一般提示。如果只有部分数据集带有标签，就属于**半监督**（semi-supervised）学习[BNS06]。我们将在第12章看到一些半监督学习的示例。此外，还可能出现多个数据集属于“相似”分布的场景，或者相同分布随时间变化的场景，根据任务或数据提供的顺序，会产生无数任务类型，例如**域自适应**（domain adaptation），**元学习**（meta-learning）[FAL17]，**持续学习**（continual learning）[PKP+19, BBCJ20]，**度量学习**（metric learning），**机器遗忘**（unlearning）等。一些主题将在下一卷中讨论。

[2] https://huggingface.co/models

[3] 少样本学习也可以通过微调模型来实现。在某些情况下，如果不需要微调，则称模型在进行**上下文学习**（in-context learning）[ASA+23]。

## 3.2 损失函数




收集到数据后，需要形式化“近似”目标行为的概念。我们通过引入**损失函数**（loss function）的概念来实现。




**定义 D.3.2 (损失函数)**

给定目标值 $y$ 和模型 $f$ 预测的值 $\hat{y} = f(x)$，**损失函数**$l(y, \hat{y}) \in \mathbb{R}$ 是可微的标量函数，其值与模型性能相关。例如，若 $l(y, \hat{y}_1) < l(y, \hat{y}_2)$，则意味着在考虑参考值（目标）$y$ 的情况下，预测值 $\hat{y}_1$ 的表现优于预测值 $\hat{y}_2$。

损失函数将我们对任务的理解和偏好以一个实值标度嵌入到解决方案空间，使其可以被优化算法利用。由于损失函数是可微的，它使我们能够将学习问题转化为一个数学优化问题。该问题可以使用梯度下降法，通过最小化数据集上的平均损失来求解。

为此，给定一个数据集 $\mathcal{S}_n = \{(x_i, y_i)\}$ 和损失函数 $l(\cdot, \cdot)$，一个合理的优化任务是找到任意可微模型 $f$ 在数据集上能实现的最小平均损失：

在数据集上的平均损失

$$f^* = \arg\min_f \frac{1}{n} \sum_{i=1}^{n} l(y_i, f(x_i)) \tag{E.3.1}$$

对第 $i$ 个样本的预测

出于历史原因，公式(E.3.1)被称为**经验风险最小化**(ERM)，其中风险被用作损失的泛义同义词。请参阅下文的框注以了解有关该术语来源的更多信息。

在(E.3.1)中，隐含地假设我们是在最小化定义在输入 $x$

上的所有可能函数空间。我们稍后会看到，模型总是可以用一组张量 $w$（称为模型的**参数**）进行参数化，最小化是通过数值优化在参数的可能值中搜索最佳值来完成的，表示为 $f(x, w)$。因此，给定数据集 $\mathcal{S}_n$、损失函数 $l$，和模型空间 $f$，我们可以通过梯度下降(E.2.27)来优化经验风险(E.3.1)，以此**训练**我们的模型：

$$w^* = \arg\min_w \frac{1}{n} \sum_{i=1}^{n} l(y_i, f(x_i, w)) \tag{E.3.2}$$

其中，最小化现在是相对于参数张量 $w$ 进行的。

## 损失的可微性




在继续之前，我们对 ERM 框架提出两点观察。首先，损失函数 $l$ 的可微性是至关重要的。考虑一个简单的**二分类**任务（将在下一章中详细介绍），其中 $y \in \{-1, +1\}$ 只能取两个值：$-1$ 或 $1$。给定一个实值模型 $f(x) \in \mathbb{R}$，我们可以将这两种决策对应 $f$ 的符号——表示为 $\text{sign}(f(x))$，并定义 **0/1 损失**（zero-one loss）函数如下：

$$l(y, \hat{y}) = \begin{cases} 0 & \text{若 } \text{sign}(\hat{y}) = y \\ 1 & \text{否则} \end{cases} \tag{E.3.3}$$

虽然它符合我们直觉上的“正确”概念，但它作为损失函数几乎没有实际用处，因为其梯度几乎总是为零（除非 $f$ 的符号发生变化），这导致任何基于梯度下降的算法都将停滞在初始化状态。这种情况下，一个不太直观但更有用的量是**间隔**（margin）$y\hat{y}$，其正负取决于模型预测的符号与目标是否一致。间隔是一个连续变化的量，与(E.3.3)中的 0/1 损失函数不同。这种情况下，一种可行的损失函数是**铰链损失**（hinge loss）函数 $l(y, \hat{y}) = \max(0, 1 - y\hat{y})$，它被用于训练支持向量机模型。撇开细节不谈，这一例

子表现了设计损失函数时的内在张力：既要表示性能概念，又要对数值优化有用。

**风险和损失**

以此方式构建的经验和期望风险最小化通常与俄罗斯计算机科学家 V.Vapnik 的研究成果有关 [Vap13]，他的研究催生了*统计学习理论*（statistical learning theory，SLT）领域。SLT 特别关注在某些受限函数类 $f$ 和底层复杂性度量条件下，公式(E.3.1)作为(E.3.5)的有限样本近似的行为 [PS+03, SSBD14, MRT18]。现代神经网络的反直觉性质（例如，过拟合后很长一段时间表现出强泛化能力）为 SLT 开辟了许多新的研究方向 [PBL20]。另请参阅第9章介绍。

### 3.2.1 期望风险和过拟合

作为第二点观察，请注意，经验风险总是可以通过以下方式轻松最小化：

$$f(x) = \begin{cases} y & \text{若 } (x,y) \in \mathcal{S}_n \\ \bar{y} & \text{否则} \end{cases} . \qquad \text{(E.3.4)}$$

$x$ 在训练集中

默认值，例如 0

这是一个查找表，它在数据集中包含输入输出对 $(x, y)$ 时返回预测值 $y$，否则默认返回某个常量预测值 $\bar{y}$（例如 0）。假设 $y = \hat{y}$ 时损失为下界，那么这种模型总能实现最低的经验风险值，但并没有任何实际的应用价值。

这显示了**记忆**与**学习**（优化）的不同。虽然我们通过优化训练数据上的平均损失来寻找模型，如(E.3.1)，但我们的真正目标是最小化在未知的未来输入上的损失。训

练集中的元素仅仅是达到这一目标的一个代理工具。我们可以通过定义**期望风险最小化**（expected risk minimization）问题来形式化这个想法。

**定义 D.3.3 (期望风险)**

给定概率分布 $p(x, y)$ 和损失函数 $l$，**期望风险**（*expected risk*，*ER*）定义为：

$$\mathrm{ER}[f] = \mathbb{E}_{p(x,y)}\left[l(y, f(x))\right] \tag{E.3.5}$$

最小化(E.3.5)可以解释为最小化模型在所有可能的输入输出对（例如，所有可能的电子邮件）上的平均（期望）损失。显然，具有低期望风险的模型可以保证正确地工作。然而，公式(E.3.5)中的量在实际中无法被计算，因为枚举并标注所有数据点是不可能的。经验风险为期望风险提供了一种基于给定数据集选择的估计，并可以看作是 ER 的蒙特卡洛近似。

期望风险与经验风险之间的损失差异被称为**泛化差距**：类似公式(E.3.4)的纯记忆算法具有较差的泛化能力，换句话说，它会**过拟合**（overfit 我们提供的特定训练数据。在实践中，可以通过保留一个从未在训练中使用的独立**测试数据集**（test dataset）$\mathcal{T}_m$ 来测试泛化能力，其中包括 $m$ 个数据点，且 $\mathcal{S}_n \cap \mathcal{T}_m = \emptyset$。然后，$\mathcal{S}_n$ 和 $\mathcal{T}_m$ 之间的经验损失差异可用作过拟合的近似度量。

### 3.2.2 如何选择有效的损失函数？

> 如果您还没有这样做，现在是学习（或浏览）附录A内容的好时机，尤其是关于概率分布、充分统计量和最大似然估计的部分。

正如我们将在接下来的章节中看到的，损失函数包含了我们对待解决任务的先验知识，并对性能产生重大影响。在某些情况下，仅依赖对问题的简单考虑就足以设计出有效的损失函数（例如，第3.2节的铰链损失函数）

但是，将整个训练过程完全重新表述为纯概率的形式，可以使我们更系统地开展工作，如我们接下来所展示的。这种表述提供了学习的一种替代视角，在某些场景中可能更直观或更有用。这也是许多教科书的首选视角 [BB23]。本节将介绍基本思想，具体应用将在后续章节中讨论。

关键观察点如下。在3.1中，我们假设示例来自分布 $p(x, y)$。根据概率的乘法法则，可以将 $p(x, y)$ 分解为 $p(x, y) = p(x)p(y \mid x)$，其中 $p(x)$ 取决于观察到每个输入 $x$ 的概率，条件项 $p(y \mid x)$ 描述给定输入 $x$ 观察到某个输出 $y$ 的概率。[4] 假设 $p(y \mid x)$ 的概率质量主要集中在一个点 $y$ 附近（即 $p(y \mid x)$ 接近所谓的狄拉克 $\delta$ 函数），那么用一个函数 $f(x)$ 来近似 $p(y \mid x)$ 是有意义的。这一假设可以极大地简化问题的整体表述。

然而，我们可以放宽这一假设，可以假设模型 $f(x)$ 不直接给出预测，而是用来参数化条件概率分布 $p(y \mid f(x))$ 的充分统计量，用以描述可能输出的分布。例如，考虑一个分类问题，其中 $y \in \{1, 2, 3\}$ 可以取三个可能的值。我们可以假设模型具有三个输出，用于参数化这些类别

[4] 还可以将其分解为 $p(x, y) = p(x \mid y)p(y)$。需要估计 $p(x)$ 或 $p(x \mid y)$ 的方法被称为**生成式**（generative）方法，而估计 $p(y \mid x)$ 的方法被称为**判别式**（discriminative）方法。除语言建模之外，本书主要关注后者。我们将在下一卷中更广泛地探讨生成式建模。

的一个类别分布（categorical distribution），使得：

$$p(\mathbf{y} \mid f(x)) = \prod_{i=1}^{3} f_i(x)^{y_i}$$

其中，$\mathbf{y} \sim \text{Binary}(3)$ 是类别 $y$ 的独热（one-hot）编码，[5] $f(x) \sim \Delta(3)$ 表示每个类别的预测概率分布。举另一个例子，假设我们要预测一个标量值 $y \in \mathbb{R}$（**回归**，regression）。我们可以通过一个二值函数 $f(x) \sim (2)$ 建模，使预测是有适当均值和方差的高斯分布：

$$p(y \mid f(x)) = \mathcal{N}(y \mid f_1(x), \; f_2^2(x)\;) \tag{E.3.6}$$

平方以确保正值

其中 $f(x)$ 的第二个输出被平方以确保预测的方差始终为正。可以看出，这是一个非常通用的设置，不仅涵盖了我们之前的讨论，还为设计者提供了更大的灵活性，因为选择 $p(y \mid x)$ 的特定参数化形式通常比直接选择一个特定的损失函数 $l(y, \hat{y})$ 更简单。此外，这一框架还提供了一种更直接的方式来建模不确定性，例如(E.3.6)中的方差。

### 3.2.3 最大似然法

如何训练概率模型？我们假设数据集 $\mathcal{S}_n$ 中的样本是来自概率分布 $p(x, y)$ 的 i.i.d. 样本。因此，给定一个模型 $f(x)$，特定函数选择 $f$ 分配给数据集本身的概率可以由

[5] 一个整数 $i$ 的独热表示是一个向量，除了第 $i$ 个元素为 1，其他元素均为 0. 将在第4.2节正式介绍。

数据集中每个样本概率的乘积给出：

$$p(\mathcal{S}_n \mid f) = \prod_{i=1}^{n} p(y_i \mid f(x_i))$$

$p(\mathcal{S}_n \mid f)$ 称为数据集的**似然**（likelihood）。对于随机选择的 $f(x)$，模型会在所有可能的输入和输出中随机分配概率，我们的特定数据集的似然值会很小。因此，一个明智的策略是选择使数据集的似然值最大化的模型。这就是最大似然法的直接应用（见附录A中的第A.6节）。




**定义 D.3.4 (最大似然法)**

给定数据集 $\mathcal{S}_n = \{(x_i, y_i)\}$ 和由 $f(x)$ 参数化的概率分布族 $p(y \mid f(x))$，其**最大似然**解由下式给出：

$$f^* = \arg\max_f \prod_{i=1}^{n} p(y_i \mid f(x_i)) \,.$$

虽然我们再次面临优化问题，但是，一旦所有概率分布被选定，它将直接遵循概率定律，这与之前不同。在之前的情况中，特定的损失函数是设计空间的一部分。然而，这两种观点之间是紧密相连的。在对数空间中处理，并将问题转换为最小化问题，可以得到：

$$\arg\max_f \left\{ \log \prod_{i=1}^{n} p(y_i \mid f(x_i)) \right\} = \arg\min_f \left\{ \sum_{i=1}^{n} -\log(p(y_i \mid f(x_i)) \right\} \tag{E.3.7}$$

因此，如果我们将 $-\log(p(y \mid f(x))$ 当作一个需要优化的“伪损失”（pseudo-loss），那么这两种形式是相同的。正如我们将看到的，在实践中使用的所有损失函数都可以通过对该项的特定选择，，根据最大似然原则推导出来。这两种观点都具有重要意义，我们强烈建议读者在阅读后续内容时牢记它们。

## 3.3 贝叶斯学习




我们在此讨论概率形式化的一个进一步推广，称为**贝叶斯神经网络**（Bayesian Neural Networks, BNNs），这是文献关注的一个主题。我们仅提供这一概念的基本思路，并建议读者参考众多详尽教程之一，例如 [JLB+22]，获取更多细节。

通过设计概率函数 $p(y \mid f(x))$ 而不是直接设计 $f(x)$，我们可以处理多个预测值得关注的情况（即概率函数具有多个模态）。然而，该方法仍然只从所有可能的函数空间中返回单一函数 $f(x)$，而整个模型空间内实际上可能存在多个有效的参数化方案。在这种情况下，访问所有的方案可能更有助于实现更准确的预测。

再一次地，我们可以通过设计另一个概率分布来实现这个目标，然后让概率规则来指导我们。由于我们现在计划获得在所有可能函数上的分布，首先定义一个所有可能函数的**先验概率分布**（prior probability distribution）

$p(f)$（回想一下，在实践中 $f$ 由一组有限的参数描述，此时先验 $p(f)$ 就成为这些权重上的先验）。例如，在许多情况下，范数较小的函数更受青睐（因为它们更稳定），在这种情况下，我们可以定义先验 $p(f) \propto \frac{1}{\|f\|}$，其中 $\|f\|$ 是 $f$ 的某种范数。

一旦观测到一个数据集，$f$ 的概率分布将根据先验（prior）和似然发生变化，更新由**贝叶斯定理**（Bayes' theorem）给出：

先验（观测数据集之前）

$$p(f \mid \mathcal{S}_n) = \frac{p(\mathcal{S}_n \mid f)\, p(f)}{p(\mathcal{S}_n)} \tag{E.3.8}$$

后验（观测数据集之后）

项 $p(f \mid \mathcal{S}_n)$ 被称为**后验分布函数**（posterior distribution function），分母中的 $p(\mathcal{S}_n)$ 被称为**证据**（evidence），用于确保后验被正确归一化。假设我们能够获取后验。与之前不同的是，这种分布能够表达对多个函数 $f$ 的偏好，从而可能提供更强的预测能力。给定输入 $x$，我们可以基于后验权重，对所有可能的模型进行加权平均，来进行预测：

$f(x)$ 的预测　　分配给 $f$ 的权重

$$p(y \mid x) = \int_f p(y \mid f(x))\; p(f \mid \mathcal{S}_n) \tag{E.3.9}$$

$$\approx \frac{1}{k} \sum_{i=1}^{k} p(y \mid f_i(x)) p(f_i \mid \mathcal{S}_n) \tag{E.3.10}$$

蒙特卡洛近似

在(E.3.10)中，我们用来自后验分布 $f_k \sim p(f \mid \mathcal{S}_n)$ 的 $k$ 个随机样本的蒙特卡洛平均来近似积分。这种设置的整体美感被这样一个事实所破坏：后验通常无法以闭式计算，除非在某些特定的先验和似然选择下 [Bis06]。在无法求得闭式解的情况下，只能被迫使用**马尔可夫链蒙特卡洛**（Markov chain Monte Carlo）或**变分推断**（variational inference）取得近似解 [JLB+22]。我们将在第9.3.1节中看到一个贝叶斯处理模型参数的例子，称为**蒙特卡洛 Dropout**。

在结束本节之前，我们来谈谈关于后验的两个有趣事实。首先，假设我们仅对后验密度最高的函数感兴趣。在这种情况下，可以忽略证据项，此时的解可以分为两个独立的部分：

$$f^* = \arg\max_f p(\mathcal{S}_n \mid f)p(f) = \tag{E.3.11}$$

$$\arg\max_f \left\{ \log p(\mathcal{S}_n \mid f) + \log p(f) \right\} \tag{E.3.12}$$

似然项　　正则化项

这被称为**最大后验概率**（Maximum A Posteriori, MAP）解。如果所有函数具有相同的先验权重（即，$p(f)$ 在函数空间中均匀分布），那么第二项是一个常数，问题将简化为最大似然解。然而，一般情况下，MAP 解会对偏离先验分布太多的函数施加惩罚。对于缓解过拟合并对函数 $f$ 施加特定约束，这是一种有用的想法。$\log p(f)$ 通常被称为函数空间上的**正则化项**（regularizer），它将解推向由先验分布定义的吸引区域。[6]

[6]最大似然解和最大后验解之间的差异可以大致对应于**频率学派**（frequentist）和**贝叶斯学派**对概率解释的差异 [Bis06]，即，概率是事件的频率，还是不确定性的度量。从高层次的角度来看：最大似然法将参数视为未知的固定项，将数据视为随机样本；而贝叶斯方法将数据视为固定的，而将参数视为随机变量。

第二，完整的贝叶斯方法提供了一个简单的机制来合并新数据，例如来自同一分布的新数据集 $\mathcal{S}'_n$。为了实现这一点，我们用基于第一部分数据集计算得到的后验分布代替公式(E.3.8)中原来的先验函数，该分布现在代表了对 $f$ 可能值的初始假设，通过观察新数据进行更新。[7]这种方法可以缓解在线训练模型时的问题，尤其是所谓的灾难性遗忘 (catastrophic forgetting) 旧信息。[KPR+17].

[7]将原始的先验函数视为在观察初始空数据集之后，对函数 $f$ 的分布。

# 4 | 线性模型

**本章导读**

编程是通过选择适当的原语操作序列来解决任务。同样地，构建模型是通过选择正确的可微模块序列来完成的。在本章中，我们将介绍最简单的模块——线性模型，它假设输入通过加权平均对输出产生加性作用。从某种意义上说，所有的可微模型都是线性模块的智能变体和组合。

## 4.1 最小二乘回归

总结前一章内容，监督学习问题可以通过选择输入类型 $x$、输出类型 $y$、模型 $f$ 和损失函数 $l$ 来定义。在本章中，我们将探讨它们的最简单选择，即：

- 输入是一个对应一组特征的向量 $\mathbf{x} \sim (c)$（例如，一名银行客户的 $c$ 个个人特征）。为了与后续内容保持一致，我们用标量 $c$（通道，channels 的缩写）表示特征的数量。
- 输出是一个实数 $y \in \mathbb{R}$。在无约束的情况下，我们称之为**回归**（regression）任务。如果 $y$ 只能在 $m$

**表 T.4.1:** 本章需要记住的基本形状。为了统一，我们将在整本书中尽可能使用相同的字母表示。

| | |
|---|---|
| $n$ | **数据库大小** |
| $c$ | **特征** |
| $m$ | **类** |

个可能值当中取一个，即 $y \in \{1, \dots, m\}$，则称之为**分类**（classification）任务。在 $m = 2$ 的特殊情况下，我们称之为**二分类**（binary classification）任务。

- 将 $f$ 视为线性模型，在某些情况下能为我们提供简单的闭式解，最典型的例子是**最小二乘回归**（least-squares regression，见4.1.2节）。

表T.4.1总结了需要记住的基本形状。首先讨论回归情况下损失函数的选择，我们从回归开始，因为分类问题可以通过对回归问题进行微小修改来解决，正如后文所述。

### 4.1.1 平方损失及其变体

为回归问题找到合适的损失函数相对简单，因为模型预测输出 $\hat{y} = f(x)$ 与真实输出 $y$ 之间的预测误差 $e = (\hat{y} - y)$ 是一个明确定义的目标。预测误差是关于模型输出的连续函数，随着误差的减小而单调递减。由于我们通常不关心预测误差的符号，一个常见的选择是**平方损失**（squared loss）函数：

$$l(\hat{y}, y) = (\hat{y} - y)^2 \tag{E.4.1}$$

在此处以及后续内容中，我们使用符号 $\hat{y}$ 表示通用模型的预测值。正如我们将看到的，使用公式(E.4.1)提供的平方损失函数带来了许多有趣的优势。其中之一是，平

方损失的梯度是关于模型输出的线性函数，使我们可以通过闭式求解来获得最优解。

回顾最大似然原理（见第3.2.2节），平方损失可以通过以下假设推导出来：模型的输出服从以 $f(\mathbf{x})$ 为中心，方差为常数 $\sigma^2$ 的高斯分布：

$$p(y \mid f(\mathbf{x})) = \mathcal{N}(y \mid f(\mathbf{x}), \sigma^2)$$

在这种情况下，对于单个点的对数似然可以写为：[1]

$$\log(p(y \mid f(\mathbf{x}), \sigma^2)) = \\ -\log(\sigma) - \frac{1}{2}\log(2\pi) - \frac{1}{2\sigma^2}(y - f(\mathbf{x}))^2 \qquad \text{(E.4.2)}$$

在对公式(E.4.2)中的 $f$ 最大化时，我们可以发现，公式右侧的前两项是常数项，第三项则恢复为平方损失。$\sigma^2$ 的最小化可以独立于优化 $f$ 进行，并且可以通过简单的闭式解完成（详见下文公式(E.4.9)）。

提出平方损失的变体也很容易。例如，平方损失的缺点之一是，对高误差的惩罚强度会随着误差的平方增长，这可能会导致**异常值**（outliers，即被严重错误标注的点）对模型产生过大的影响。为了减少异常值的影响，可以选择其他损失函数，例如：**绝对值损失**（absolute value loss）$l(\hat{y}, y) = |\hat{y} - y|$ 或 Huber 损失（平方损失和绝对值损失的组合）：

$$L(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{若 } |y - \hat{y}| \leq 1 \\ (|y - \hat{y}| - \frac{1}{2}) & \text{否则} \end{cases} \qquad \text{(E.4.3)}$$

[1] 回想一下，$\log(ab) = \log(a) + \log(b)$ and $\log(a^b) = b\log(a)$。

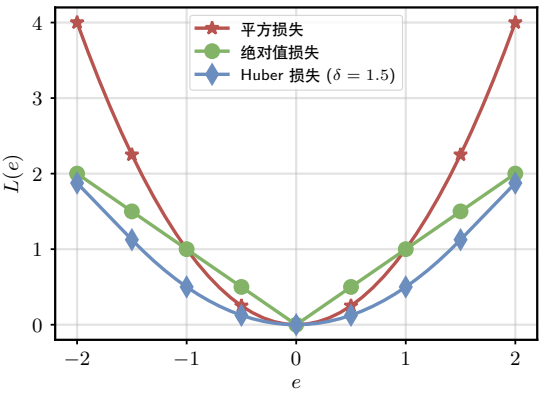



**图 F.4.1:** 预测误差 $e = (\hat{y} - y)$ 的平方损失、绝对值损失和 *Huber* 损失可视化。

误差接近 0 时，Huber 损失函数是二次的，否则是线性的（添加了 $-\frac{1}{2}$ 项以确保连续性）。有关这些损失函数随预测误差变化的可视化，请参考图F.4.1。

在我们的上下文中，绝对值损失看起来是一个无效的选择，因为它在 0 处由于绝对值的性质存在不可微点。我们稍后将看到，函数具有一个或少数几个不可微点的情况并不是什么大问题。这些点可以在数学上通过**次梯度**（subgradient，导数的轻微推广）来处理。从实践的角度，可以想象，如果从随机初始化开始，梯度下降几乎不可能以完美精度到达这些不可微点。并且对于任意 $\varepsilon > 0$，$|\varepsilon|$ 的导数总是被定义的。

### 4.1.2 最小二乘模型




确定了损失函数之后，我们考虑以下线性模型来完成我们第一个监督学习问题的定义。

**定义 D.4.1 (线性模型)**

输入为 $\mathbf{x}$ 的**线性模型**（*linear model*）定义为：

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$

其中 $\mathbf{w} \sim (c)$ 和 $b \in \mathbb{R}$（偏置，*bias*）是可训练的参数。

直观上，该模型为每个输入特征 $x_i$ 分配一个固定权重 $w_i$，通过对给定输入 $\mathbf{x}$ 的所有影响进行线性求和来预测。当 $\mathbf{x} = \mathbf{0}$ 时，默认预测等于偏置 $b$。从几何角度，$d = 1$ 时，模型定义了一条直线；$d = 2$ 时，模型定义了一个平面；$d > 1$ 时，模型定义了一个通用的超平面。从符号角度来看，有时我们可以通过假设 $\mathbf{x}$ 的最后一个特征为常数项 1 来省略偏置项。

$$f\left(\begin{bmatrix}\mathbf{x}\\1\end{bmatrix}\right) = \mathbf{w}^\top \begin{bmatrix}\mathbf{x}\\1\end{bmatrix} = \mathbf{w}_{1:c}^\top \mathbf{x} + w_{c+1}$$

结合线性模型、平方损失和经验风险最小化问题，我们得到**最小二乘优化问题**（least-squares optimization problem）。

**定义 D.4.2 (最小二乘)**




**最小二乘**优化问题由以下公式给出：

$$\mathbf{w}^*, b^* = \arg\min_{\mathbf{w},b} \frac{1}{n}\sum_{i=1}^{n} \left(y_i - \mathbf{w}^\top \mathbf{x}_i - b\right)^2 \quad \text{(E.4.4)}$$

继续分析该问题前，我们将最小二乘优化问题改写为仅

```python
def linear_model(w: Float[Tensor, "c"],
                 b: Float,
                 X: Float[Tensor, "n c"])
                 -> Float[Tensor, "n"]:
  return X @ w + b
```

**框 C.4.1:** 计算像公式(E.4.5)中的分批线性模型为清楚起见，我们使用 *jaxtyping* (https://docs.kidger.site/jaxtyping/) 将数组维度显示为类型提示。

涉及矩阵运算（矩阵乘法和范数）的**向量化**形式。这种形式非常实用，因为如前文所述，现代可微模型的训练代码基于 $n$ 阶数组构建，利用优化的硬件高效执行矩阵运算。为此，我们首先将训练集的所有输入和输出堆叠到**输入矩阵**（input matrix）中：

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_n^\top \end{bmatrix} \sim (n, c)$$

以及相似的**输出向量**（output vector）$y = [y_1, \dots, y_n]^\top$。分批模型输出（小批量输入值的模型输出）可以写为：

$$f(\mathbf{X}) = \mathbf{X}\mathbf{w} + \mathbf{1}b \tag{E.4.5}$$

所有 $n$ 个预测的偏置 $b$ 相同

像(E.4.5)这样的公式几乎可以逐行对应地实现为代码——参考框C.4.1中 PyTorch 的示例。

目前仅具有一定的边缘意义，但在后续将更为重要的一点是，我们注意到，输入矩阵和输出向量的行顺序本质上是任意的。置换它们的行只会导致 $f(\mathbf{X})$ 行顺序的对应置换。这是被称为**置换等变性**（permutation

equivariance）现象的一个简单例子。

最小二乘优化问题以向量化形式写为：

$$\mathrm{LS}(\mathbf{w}, b) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\mathbf{w} - \mathbf{1}b\|^2 \tag{E.4.6}$$

回忆一下，其中向量的范数定义为 $\|\mathbf{e}\|^2 = \sum_i e_i^2$。

### 4.1.3 求解最小二乘问题

要通过梯度下降法解决最小二乘问题，我们需要计算梯度方程。尽管稍后我们将开发一个通用的算法框架（第6章）来自动计算梯度，但在这个简单场景中分析梯度本身是很有意义的。忽略偏置（如前文所述，可以将其合并到权重向量中）以及其他常数项，可得：

$$\nabla \mathrm{LS}(\mathbf{w}) = \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$$

LS 问题在模型权重当中为凸，它可以通俗地理解为在权重空间中描述了一个抛物面（二次函数）。然后，全局最小值由以下方程描述：

$$\mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) = 0 \quad \Rightarrow \quad \mathbf{X}^\top \mathbf{X}\mathbf{w} = \mathbf{X}^\top \mathbf{y}$$

这些方程被称为**正规方程** (normal equation)。重要的是，正规方程描述了一个关于 $\mathbf{w}$ 的线性方程组，[2]这意味着在适当条件下（对应于矩阵 $\mathbf{X}^\top\mathbf{X}$ 的可逆性），我们可以求最优解：

$$\mathbf{w}_* = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y} \tag{E.4.7}$$

[2]也就是说，可以将它们写为 $\mathbf{A}\mathbf{w} = \mathbf{b}$，其中 $\mathbf{A} = \mathbf{X}^\top\mathbf{X}$，$\mathbf{b} = \mathbf{X}^\top\mathbf{y}$。

```python
from torch import linalg
def ls_solve(X: Float[Tensor, "n c"],
             y: Float[Tensor, "n"],
             numerically_stable = True) \
             -> Float[Tensor, "c"]:
  # 显式解
  if not numerically_stable:
    return linalg.inv(X.T @ X) @ X.T @ y
  else:
    return linalg.solve(X.T @ X, X.T @ y)
```

**框 C.4.2:** 使用闭式解求解最小二乘问题。数值稳定变体会调用专门用于线性方程组的求解器。

**零碎信息**

矩阵 $\mathbf{X}^\dagger = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$ 被称作非方阵 $\mathbf{X}$ 的**伪逆**（pseudoinverse，或**穆尔-彭罗斯逆**，Moore-Penrose inverse），因为它满足关系 $\mathbf{X}^\dagger\mathbf{X} = \mathbf{I}$。然而，在一些情况下执行(E.4.7)中的矩阵求逆并非总是可行：例如，假如一个特征是另一个特征的标量倍数，矩阵 $\mathbf{X}$ 不满秩（称为**共线性**，collinearity）。最后，最小二乘模型的预测可以表示为 $\hat{\mathbf{y}} = \mathbf{M}\mathbf{y}$，其中 $\mathbf{M} = \mathbf{X}\mathbf{X}^\dagger$。因此，最小二乘也可以被解释为对训练标签的加权平均，其中权重通过 $\mathbf{X}$ 在列空间上的投影给出。这被称为最小二乘的**对偶**（dual）形式。对偶形式为模型提供了一种内在的调试方法，因为它们允许通过检查相应的对偶权重，来检查哪些输入与预测最相关 [ICS22]。

这是能以闭式表达最优解的唯一情况，将这种解与梯度下降法的解对比具有启发意义。为此，我们在框C.4.2展示了使用公式(E.4.7)闭式求解最小二乘问题的示例，并在框C.4.3中给出了等效的梯度下降形式。后者的典型损失变化趋势如图F.4.2所示。由于我们选择了非常小的学习率，梯度下降的每一步都能使损失稳定下降，直到收

```python
def ls_gd(X: Float[Tensor, "n c"],
          y: Float[Tensor, "n 1"],
          lr=1e-3) \
          -> Float[Tensor, "c"]:
    # 初始化参数
    w = torch.randn((X.shape[1], 1))

    # 固定迭代次数
    for i in range(15000):
      # 注意符号（为什么？）
      w = w + lr * X.T @ (y - X @ w)

    return w
```

**框 C.4.3:** 与框*C.4.2*的任务相同，通过梯度下降的朴素实现来解决，固定学习率默认为 $\eta = 0.001$。

敛为止。在实践中，可以通过数值检查收敛。例如，评估两次迭代间的范数差是否小于某个阈值 $\varepsilon > 0$：

$$\|\mathbf{w}_{t+1} - \mathbf{w}_t\|^2 < \varepsilon \tag{E.4.8}$$

我们会发现，判断更复杂的模型是否已收敛将是更为微妙的任务。

再次考虑(E.4.2)中的高斯对数似然，我们还可以在权重训练完成后针对 $\sigma^2$ 优化该项，可得：

$$\sigma_*^2 = \frac{1}{n}\sum_{i=1}^{n}(y_i - \mathbf{w}_*^\top \mathbf{x}_i)^2 \,. \tag{E.4.9}$$

这直观表明模型的方差是常量（按照定义），由训练数据上的平均平方预测误差给出。更复杂的概率模型可以通过假设方差本身由模型预测来实现（**异方差**模型，heteroscedastic model），见 [Bis06]。

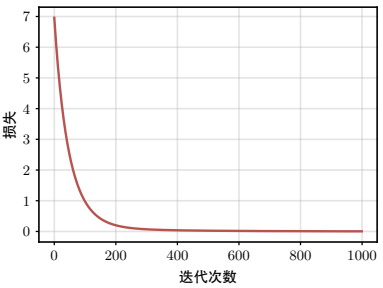



**图 F.4.2:** 基于框*C.4.2*的代码运行示例，其中数据由线性模型 $\mathbf{w}^\top\mathbf{x}+\varepsilon$ 提取的 $n=10$ 个点组成，其中 $w_i\sim\mathcal{N}(0,1)$ 且 $\varepsilon\sim\mathcal{N}(0,0.01)$。暂时抛开细节，请注意曲线的平滑下降：每一步都使损失减少。

### 4.1.4 计算方面的考虑




即使能进行矩阵求逆运算，求解结果的质量仍将取决于 $\mathbf{X}^\top\mathbf{X}$ 的条件数，并且非良置的矩阵会导致显著的数值误差。[3] 此外，求解(E.4.7)的计算成本可能过高。矩阵求逆的计算复杂度大约为 $\mathcal{O}(c^3)$。至于矩阵乘法，该算法需要计算一个 $c\times n$ 矩阵与 $n\times c$ 矩阵相乘，以及一个 $c\times c$ 矩阵与 $c\times n$ 矩阵相乘。这两个矩阵乘法操作的复杂度均为 $\mathcal{O}(c^2n)$。

一般而言，我们始终倾向选择在特征维度 $c$ 和批量大小 $n$ 上都呈线性增长的算法，因为超线性算法很快就会变得不切实际（例如，一批 32 张大小为 $1024\times1024$ 的 RGB 图像，其中 $c$ 的量级为 $10^6$，而 $nc\approx10^7$）。为避免在梯度方程计算中出现二次复杂度，可以通过正确的运算顺序来降低计算量，即首先计算矩阵-向量积 $\mathbf{Xw}$。因此，如果在纯梯度下降法实施过程中采取适当的措施，其在 $c$ 和 $n$ 上都会是线性的：推广这一思想是发展**反向模式自动微分**（reverse-mode automatic differentiation，即**反向传播**，back-propagation，见第6.3节）的重要见解。

[3]矩阵 $\mathbf{A}$ 的条件数被定义为 $\kappa(\mathbf{A})=\|\mathbf{A}\|\|\mathbf{A}^{-1}\|$，其中 $\|\bullet\|$ 是某种选择的矩阵范数。条件数过大会使逆矩阵运算变得困难，尤其是在浮点精度不足的情况下。

### 4.1.5 正则化最小二乘解

再次考虑求逆运算的潜在不稳定性。假设我们有一个数据集，其对应的矩阵接近奇异（singular），我们仍希望使用闭式解法来求解。在这种情况下，我们可以对问题适当微调，使获得的解“尽可能接近”原始解，同时又易于计算。例如，一个常用的技巧是，在待求逆的矩阵上加一个很小的正数倍 $\lambda > 0$ 的单位矩阵：

$$\mathbf{w}^* = \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}\right)^{-1} \mathbf{X}^\top \mathbf{y}$$

这使矩阵“更加对角化”，而且改善了它的条件数。回到原来的问题，我们注意到这是一个修正的优化问题的闭式解：

$$\text{LS-Reg}(\mathbf{w}) = \frac{2}{n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

这类问题叫做**正则化最小二乘**（regularized least-squares，或**岭回归**，ridge regression），损失函数中的红色部分是 $\ell_2$ 正则化项（或更一般地称为正则化项）。需要注意的是，正则化项与数据集本身无关，因为它仅体现了对某类解的偏好（本例中为低范数权重），这种偏好程度由超参数 $\lambda$ 定义。从贝叶斯角度来看（见第3.3节），当对权重赋予中心为零、方差恒定的高斯先验时，正则化最小二乘对应于 MAP 方法。

## 4.2 线性分类模型

现在，我们来讨论分类问题，其中 $y_i \in \{1, \ldots, m\}$，$m$ 定义了**类别**（class）的数量。正如我们稍后将看到的，这是一个影响广泛的问题，涵盖了计算机视觉（如，图像分类）和自然语言处理（例如，下一个词元预测）中的一系列任务。我们可以通过稍微改变回归情况来解决这个问题。

虽然可以通过直接对整数值 $y_i$ 回归来解决这个任务，但考虑为什么这不是一个好主意，会对我们有启发。首先，对于模型来说，直接预测一个整数值是困难的，因为这需要一些阈值处理，几乎会使其梯度在大多数地方为零。相反，我们可以对在范围 1 到 $m$ 区间的实数值 $\tilde{y}_i \in [1, m]$ 进行回归（正如下文将展示的，可以很容易地将模型输出限制在一个区间内）。在推理过程中，给定输出 $\hat{y}_i = f(\mathbf{x}_i)$，通过四舍五入将其映射回原始域：

$$\text{预测类别} = \text{round}(\hat{y}_i)$$

例如，$\hat{y}_i = 1.3$ 会被映射到类别 1，而 $\hat{y}_i = 3.7$ 被映射到类别 4。注意，这是一种仅在推理时可行的事后处理方法。这种建模不够好的原因在于，我们引入了类别之间的虚假顺序，这一点可能会被模型利用，例如类别 2 比类别 4 更接近类别 3。我们可以通过转向经典的**独热编码**版本 $y$ 来避免这种情况，将其表示为 $\mathbf{y}^{\text{oh}} \sim \text{Binary}(m)$：

$$[\mathbf{y}^{\text{oh}}]_j = \begin{cases} 1 & \text{若 } y = j \\ 0 & \text{否则} \end{cases}$$

例如，在三个类别的情况下，将有 $\mathbf{y}^{\text{oh}} = [1 \;\; 0 \;\; 0]^\top$ 对应类别 1，$\mathbf{y}^{\text{oh}} = [0 \;\; 1 \;\; 0]^\top$ 对应类别 2，$\mathbf{y}^{\text{oh}} = [0 \;\; 0 \;\; 1]^\top$ 对应类别 3（这种表示方式对于有机器学习背景的读者来说应该很熟悉，因为它是分类变量的标准表示）。

独热向量是无序的，这意味着对于两个通用输出 $\mathbf{y}_1^{\text{oh}}$ 和 $\mathbf{y}_2^{\text{oh}}$，它们的欧几里得距离要么为 0（相同类别），要么为 $\sqrt{2}$（不同类别）。虽然我们可以直接对独热编码的输出执行多值回归，其均方误差被称为**布里尔分数**（Brier score），但我们即将展示一种更好更优雅的解决方案，即**逻辑回归**（logistic regression）。

### 4.2.1 softmax 函数

我们不能训练模型来直接预测独热编码向量（原因如前文所述），但可以轻微放宽标准，来实现类似的目标。为此，我们重新引入**概率单纯形**（probability simplex）。

**定义 D.4.3 (概率单纯形)**

**概率单纯形**$\Delta_n$ 是向量 $\mathbf{x} \sim \Delta(n)$ 的集合，满足：

$$x_i \geq 0, \sum_i x_i = 1$$

从几何角度，可以把一组独热向量看做一个 $n$ 维多面体的顶点,，而单纯形则是其凸包（convex hull）：例如 $[0.2, 0.05, 0.75]$ 等位于单纯形内部的值，虽然不完全对应某个顶点，但它们允许使用梯度下降法进行优化，因为我们可以平滑地在多面体内移动。给定一个值 $\mathbf{x} \in \Delta_n$，我们可以将其投影到最接近的顶点（即预测类别），形式为：

$$\arg\max_i \{\mathbf{x}_i\}$$

顾名思义，我们可以将位于单纯形内部的值解释为概率分布，而投影到最近的顶点则相当于在该分布中找到众数（最可能的类别）。在这种解释下，独热编码向量是一个“特例”，其中所有的概率质量都集中在一个类别上（这个类别是正确的）。

为了预测这个单纯形中的值，我们需要对(E.4.4)中的线性模型作两项修改：一，我们需要同时预测整个向量；二，我们需要将输出约束在单纯形内。第一步，我们将线性模型修改为预测一个 $m$ 维向量：

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b} \tag{E.4.10}$$

其中 $\mathbf{W} \sim (m, c)$ 可以解释为 $m$ 个并行运行的线性回归模型，而 $\mathbf{b} \sim (m)$。这个输出是无约束的，而且不能保证它位于单纯形内。逻辑回归的核心思想是将公式(E.4.10)中的线性模型与一个简单的、无参数的变换结合。该变换将输出投影到单纯形内部，被称为**softmax** 函数。




**定义 D.4.4 (softmax 函数)**

对于通用向量 $\mathbf{x} \sim (m)$，***softmax*** 被定义为：

$$[\text{softmax}(\mathbf{x})]_i = \frac{\exp(x_i)}{\sum_j \exp(x_j)} \tag{E.4.11}$$

为了理解其中的机制，我们引入两个中间项，对公式(E.4.11)中的各项进行分解。首先，softmax 函数的分子通过指数运算将每个数值转换为一个正值 $h_i$：

$$h_i = \exp(x_i) \tag{E.4.12}$$

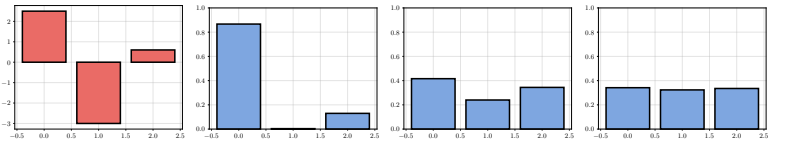
然后，我们计算一个归一化因子 $Z$ 作为这些新（非负）值的总和：

$$Z = \sum_j h_j \tag{E.4.13}$$

softmax 函数的输出通过将 $h_i$ 除以 $Z$ 得到，确保新的值的总和为 1：

$$y_i = \frac{h_i}{Z} \tag{E.4.14}$$

另一种视角来自对更通用版本的 softmax 函数的考虑，其中我们添加了一个额外的超参数 $\tau > 0$，称为**温度**(temperature)：



**(a)** 输入 **(b)** $\tau = 1$ **(c)** $\tau = 10$ **(d)** $\tau = 100$

**图 F.4.3:** *softmax* 应用于三维向量 *(a)* 的示例，温度分别设置为 *1 (b)*，*10 (c)* 和 *100 (d)*。随着温度的增加，输出逐渐趋近于均匀分布。请注意，输入值可以是正数或负数，但 *softmax* 输出值始终限制在 $[0, 1]$ 之间。

$$\text{softmax}(\mathbf{x}; \tau) = \text{softmax}(\mathbf{x}/\tau)$$

对于所有 $\tau$ 值，softmax 函数能保持 $x_i$ 值之间的相对顺序，但它们的绝对距离会根据温度 $\tau$ 的变化而增加或减小。具体来说，有以下两种极限情况：

$$\lim_{\tau \to \infty} \text{softmax}(\mathbf{x}; \tau) = \frac{1}{m}\mathbf{1} \tag{E.4.15}$$

$$\lim_{\tau \to 0} \text{softmax}(\mathbf{x}; \tau) = \arg\max_i \ \mathbf{x} \tag{E.4.16}$$

当温度趋于无穷大时，值之间的相对距离将消失，输出将回归为均匀分布。相反，温度为 0 时，softmax 函数将会还原为（难以微分的）argmax 运算。因此，可以将 softmax 看作是对 argmax 的一个简单可微逼近，更恰当的名字应该是 **softargmax**。然而，我们在本书中保留最常用的名称。对于 softmax 应用于不同温度值的通用三维向量的可视化效果，见图F.4.3。

### 4.2.2 逻辑回归模型

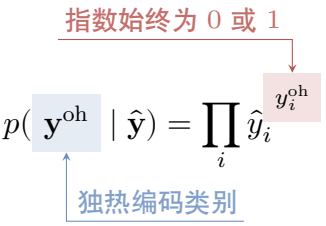



总结之前的讨论，我们可以将(E.4.11)中的 softmax 函数与(E.4.10)中的线性模型结合，得到一个用于分类的线性模型：

$$\hat{\mathbf{y}} = \text{softmax}\left(\mathbf{W}\mathbf{x} + \mathbf{b}\right)$$

预归一化的输出 $\mathbf{h} = \mathbf{W}\mathbf{x} + \mathbf{b}$ 被称为模型的**对数几率**（logit），这个名称将在下一节中进行更详细的讨论。

现在需要一个损失函数。考虑第3.2.2节中的概率视角，由于我们的输出被限制在概率单纯形内，因此我们将它们用作类别分布的参数：




在这种情况下，计算最大似然估计（请试一试）会得到**交叉熵损失**（cross-entropy loss）函数。




**定义 D.4.5 (交叉熵损失)**

$\mathbf{y}^{oh}$ 和 $\hat{\mathbf{y}}$ 之间的**交叉熵**损失函数如下：

$$\text{CE}(\mathbf{y}^{\text{oh}}, \hat{\mathbf{y}}) = -\sum_i y_i^{\text{oh}} \log(\hat{y}_i) \tag{E.4.17}$$

损失也可以作为两个概率分布之间的 KL 散度推导出来。虽然看起来不太直观，但它有非常简单的解释。请注意 $\mathbf{y}^{\text{oh}}$ 中只有一个值为非零，它对应于真实类别 $y = \arg\max_i \{y_i^{\text{oh}}\}$。我们可以将损失函数简化为：

$$\mathrm{CE}(y, \hat{\mathbf{y}}) = -\log(\hat{y}_y) \tag{E.4.18}$$

分配给真实类别的概率

从公式(E.4.18)中可以看出，最小化交叉熵损失的效果是，最大化真实类别对应的输出概率。这之所以有效，是因为在 softmax 的分母中，任意一个输出项的增加都会自动导致其他项的减少。结合所有内容，可以得到逻辑回归的优化问题：

$$\mathrm{LR}(\mathbf{W}, \mathbf{b}) = \frac{1}{n} \sum_{i=1}^{n} \mathrm{CE}\left(\mathbf{y}_i^{\mathrm{oh}}, \mathrm{softmax}(\mathbf{W}\mathbf{x}_i + \mathbf{b})\right) .$$

与最小二乘法不同，我们无法再计算出闭式解，但仍然可以继续使用梯度下降法。下一节将展示此情况下的梯度示例，并且第6.3节会介绍一种在这类情况下计算梯度的通用技术。

## 4.3 更多相关内容

### 4.3.1 二分类

现在考虑 $m = 2$ 的特定情况。在这种情况下，有 $y \in \{0, 1\}$，问题简化为**二分类**（binary classification）问题，有时也被称为**概念学习**（concept learning，因为需要学习某个二元“概念”是否在输入中存在）。在标准逻辑回归中，这将由一个具有两个输出的函数来建模。然而，由于 softmax 分母的存在，逻辑回归的最后一个输出总是冗余的，因为已知输出和必须为 1 时，可以推断出它：

$$f_m(\mathbf{x}) = \sum_{i=1}^{m-1} f_i(\mathbf{x})$$

基于这一点，我们可以稍微简化模型的形式。考虑一个标量模型，具有一个输出 $f(\mathbf{x}) \in [0,1]$，使得：

$$\text{预测类别} = \text{round}(f(\mathbf{x})) = \begin{cases} 0 & \text{若 } f(\mathbf{x}) \leq 0.5 \\ 1 & \text{否则} \end{cases}$$

为了实现所需的 $[0,1]$ 归一化，二值 softmax 的第一个输出可以被重写为 $\frac{\exp(x_1)}{1+\exp(x_1)}$，并且可以通过将两边同时除以 $\exp(x_1)$ 进一步简化。得到的结果就是 **sigmoid** 函数。




**定义 D.4.6 (sigmoid 函数)**

***sigmoid*** 函数 $\sigma(s) : \mathbb{R} \to [0,1]$ 由以下公式给出：

$$\sigma(s) = \frac{1}{1+\exp(-s)}$$

sigmoid 函数提供了一种将任何实数值映射到 $[0,1]$ 区间的通用变换（两个极值只能渐近到达）。其图像如图F.4.4所示。

**二元逻辑回归** (binary logistic regression) 模型是通过将一维线性模型与输出的 sigmoid 函数重新缩放相结合得到的：

$$f(\mathbf{x}) = \sigma\left(\mathbf{w}^\top \mathbf{x} + b\right)$$

交叉熵同样简化为：

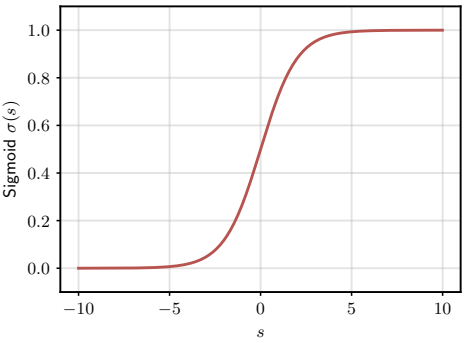



**图 F.4.4:** *sigmoid* 函数图，请注意 $\sigma(0) = 0.5$。

类别 1 的损失　　类别 2 的损失

$$\mathrm{CE}(\hat{y}, y) = -y \log(\hat{y}) - (1-y) \log(1-\hat{y}) \tag{E.4.19}$$

因此，在二分类问题中，可以通过两种等效方法来解决问题：(a) 使用标准 softmax 的二值模型，或 (b) 使用 sigmoid 输出变换的简化单值输出。

作为一个有趣的旁注，考虑二元逻辑回归模型相对于 $\mathbf{w}$ 的梯度（对于标准的多分类情况，也可以写出类似的梯度）：

$$\nabla \mathrm{CE}(f(\mathbf{x}), y) = (f(\mathbf{x}) - y)\mathbf{x}$$

请注意，这与标准线性回归模型的梯度相似。通过将我们的模型重写为以下形式，这种相似性可以被进一步理解：

对数几率　　sigmoid 反函数：$\sigma^{-1}(y)$

$$\mathbf{w}^\top \mathbf{x} + b = \log\left(\frac{y}{1-y}\right) \tag{E.4.20}$$

这也解释了为什么将该模型称为分类的“线性模型”：我们总是可以按照输出的非线性变换，将其重写为一个纯

```
from torch.nn import functional as F
# 二元交叉熵
F.binary_cross_entropy
# 接受对数几率的二元交叉熵损失
F.binary_cross_entropy_with_logits
# 从对数几率计算的交叉熵
F.cross_entropy
# 以 log f(x) 为输入的交叉熵
F.nll_loss
```

**框 C.4.4:** *PyTorch* 中的交叉熵损失。一些损失函数仅在模型的对数几率基础上定义，而不是基于 *softmax* 的输出。这些是损失函数的函数式变体——大多数框架中也有等效的面向对象变体。

粹的线性模型（在此情况下，是 sigmoid 的反函数，也称为**对数几率**，log-odd）。事实上，逻辑回归模型是一个更广泛的模型家族的一部分，这个**广义线性模型**（generalized linear models）家族扩展了这一思想。对于有兴趣的读者，可以通过与 probit 函数的关系来理解对数几率的名字。[4]

### 4.3.2 logsumexp 技巧




这是一个更为技术性的子节，阐明了我们至此所描述内容的实现方面之一。查看像 TensorFlow 或 PyTorch 这样的框架，我们可以找到多种现有的交叉熵损失函数实现，它们取决于输出是作为整数还是作为独热编码向量来描述的。这个差异很容易理解，因为我们已经看到在这两种情况下都可以表示交叉熵损失。然而，还可以找到一些变体，这些变体接受对数几率而不是 softmax 归一化的输出，如框C.4.4所示。

为了理解为什么我们需要这些，考虑交叉熵中第 $i$ 项的

[4] https://en.wikipedia.org/wiki/Probit

对数几率表示 $\mathbf{p}$：

$$-\log\left(\frac{\exp p_i}{\sum_j \exp p_j}\right).$$

该项可能引起一些数值问题，特别是由于（可能无界的）对数几率与指数运算间的相互作用。为了解决这个问题，首先将其重写为：

$$-\log\left(\frac{\exp p_i}{\sum_j \exp p_j}\right) = -p_i + \underbrace{\log\left(\sum_j \exp p_j\right)}_{\triangleq \operatorname{logsumexp}(\mathbf{p})}$$

第一个项不会受不稳定性的影响，而第二项（对数几率的 **logsumexp** 运算）是关于整个对数几率向量的函数，并且可以证明，对于给定的标量 $c \geq 0$，它在以下意义下是不变的：[5]

$$\operatorname{logsumexp}(\mathbf{p}) = \operatorname{logsumexp}(\mathbf{p} - c) + c$$

请注意 $\nabla \operatorname{logsumexp}(\bullet) = \operatorname{softmax}(\bullet)$。通过取 $c = \max(\mathbf{p})$，我们可以通过将最大对数几率值限制为 0 来防止数值问题。然而，这只有在能够访问原始对数几率时才可行，这也是交叉熵的数值稳定变体要求将对数几率作为输入的原因。这会造成一些歧义，因为 softmax 现在可以作为模型的一部分，也可以作为损失函数的一部分。

### 4.3.3 校准和分类

在本章的最后，我们简单讨论分类器**校准**（calibration）这一重要主题。为了理解这一点，请考虑以下事实：尽管我们的模型提供了关于所有可能类别的完整分布，但

[5] https://gregorygundersen.com/blog/2020/02/09/log-sum-exp/

我们的训练标准仅针对真实类别的最大化。因此，以下陈述是有依据的：

$f(\mathbf{x})$ 的预测类别是 $\arg\max_i [f(\mathbf{x})]_i$。

相反，下面这个更笼统的陈述可能不正确：

$\mathbf{x}$ 属于 $i$ 类的概率是 $[f(\mathbf{x})]_i$。

当网络的置信度分数（confidence score）与给定预测正确的概率相匹配时，我们称网络的输出**经过校准**（calibrated）。

**定义 D.4.7 (校准)**

如果一个输出类别概率的分类模型 $f(\mathbf{x})$ 对于任何可能的预测满足以下条件，则称其为已校准的模型：

$$[f(\mathbf{x})]_i = p(y = i \mid \mathbf{x})$$

尽管在无限数据的极限下，交叉熵应当恢复不受限制类别模型的条件概率分布 [HTF09]，但在实际上，两者之间的差异可能会很大 [BGHN24]，尤其是对于稍后将介绍的复杂模型。

为了理解准确率和校准之间的区别，请考虑以下两种情景。第一，考虑一个二分类模型，具有完美准确率，但总是以 0.8 的置信度预测真实类别。在这种情况下，模型对其预测是欠自信（under-confident）的，因为通过观察置信度，我们可能假设 20% 的预测会是错误的。第二，考虑一个 4 分类问题，其类别完全平衡，模型总是预测 $[0.25, 025, 0.25, 0.25]$。这种情况下，模型是被完美校准的，但从准确率的角度来看毫无用处。

在不同预测可能成本不同的情况下，获得经过校准的模

型非常重要。这可以通过定义所谓的成本矩阵（cost matrix）来形式化，该矩阵为被预测为 $j$ 类的输入类 $i$ 分配一个成本 $C_{ij}$。一个标准例子是二分类问题，成本矩阵如表T.4.2所示。

**表 T.4.2:** 具有不对称误分类成本的分类问题的成本矩阵示例。

| | 真实类别 0 | 真实类别 1 |
|---|---|---|
| 预测类别 0 | 0 | 10 |
| 预测类别 1 | 1 | 0 |

对表T.4.2的解释如下：正确预测不会产生成本，而犯下假阴性错误（预测 0 而不是 1）的成本是犯下假阳性错误的 10 倍。例如，在医学诊断中假阴性错误的后果远比假阳性错误更严重，因为后者可以通过进一步检查来纠正。经过校准的模型可以帮助我们更好地估计其部署的平均风险，并调整假阳性与假阴性错误之间的平衡。

为了理解这一点，假设 $\mathbf{C} \sim (m, m)$ 为一个多分类问题的通用成本矩阵（类似表T.4.2中的 $2 \times 2$ 矩阵）。合理的选择是，根据模型分配的得分，选择一个能够最小化期望成本的类别：

$$\arg\min_i \sum_{j=1}^{m} C_{ij}[f(\mathbf{x})]_j$$

如果当 $i \neq j$ 时 $C_{ij} = 1$，否则为 0，则可以简化为选择 $f$ 的 argmax，但是对于一般的成本矩阵，预测类别的选择将受到特定错误的相对成本的影响。这是**决策理论**（decision theory）的一个简单例子 [Bis06]。

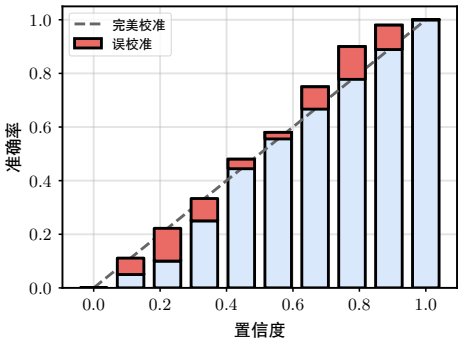



**图 F.4.5:** 可靠性图的示例，使用了 $b = 10$ 个箱。蓝色条形表示模型在该箱子中的平均准确率，而红色条形表示该箱子的误校准程度，误校准可以表现为欠自信（位于对角线以下）或过自信（位于对角线以上）。红色条形的加权和就是(E.4.21)中的 *ECE*。

### 4.3.4 估计校准误差

为了估计一个模型是否经过校准，我们可以将其预测值进行分箱，然后比较每个箱中的校准情况与准确率。为此，假设我们把区间 $[0, 1]$ 划分为 $b$ 个等间距的箱，每个箱大小为 $1/b$。取大小为 $n$ 的验证集，并用 $\mathcal{B}_i$ 表示置信度落在第 $i$ 个箱中的元素。对于每个箱，还可以进一步计算模型的平均置信度 $p_i$（大约位于该箱子的中间位置），和平均准确率 $a_i$。在直方图上绘制的成对 $(a_i, p_i)$ 的集合被叫做**可靠性图**（reliability diagram），如图F.4.5所示。为了得到一个单一的标量校准指标，我们可以使用例如**期望校准误差**（expected calibration error，ECE）：

$$\mathrm{ECE} = \sum_i \frac{|\mathcal{B}_i|}{n} |a_i - p_i| \tag{E.4.21}$$

箱 $i$ 的校准误差

落入箱 $i$ 的样本比例

也可以使用其他评估指标，比如在所有箱中选择最大

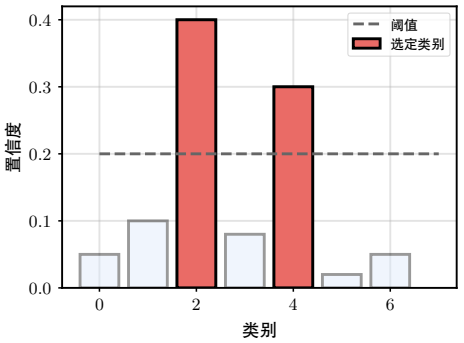



**图 F.4.6:** 将模型输出转换为一个集合进行校准：返回所有预测概率超过给定阈值的类别。通过适当选择阈值，可以限定真实类别出现在输出集合中的概率。

值。如果发现模型未经校准，需要进行一些修改。示例包括通过温度缩放（temperature scaling）重新调整预测 [GPSW17]，或者使用不同的损失函数进行优化，如焦点损失（focal loss）[MKS+20]。

最后介绍一种替代直接校准模型的方法，叫做**共形预测**（conformal prediction），这种方法近年来很流行 [AB21]。假设我们固定一个阈值 $\gamma$，然后选取模型预测的相应概率高于 $\gamma$ 的类别集合：

$$\mathcal{C}(\mathbf{x}) = \{i \mid [f(\mathbf{x})]_i > \gamma\} \tag{E.4.22}$$

即，模型的输出现在是一个潜在类别的集合 *set* $\mathcal{C}(\mathbf{x})$。图F.4.6展示了一个例子。共形预测的核心思想是选择最小的阈值 $\gamma$，在该阈值下，在集合中找到正确类别 $y$ 的概率大于用户定义的误差 $\alpha$：[6]

$$p(y \in \mathcal{C}(\mathbf{x})) \geq 1 - \alpha \tag{E.4.23}$$

[6]请注意，通过选择 $\gamma = 0$，可以始终满足该性质。即将所有类别包含在输出集合中。

$\gamma$ 和 $\alpha$ 之间存在直观的反比关系。共形预测通过自动化算法保证公式(E.4.23)成立，但输出不再是单一类别。

## 从理论到实践



在第2章的学习中，您应该已经熟练掌握了 NumPy，JAX 和 PyTorch 的`torch.tensor`。本章所需的只是它们的基本接口，不需要额外内容。从下一章开始，我们将逐步深入了解它们的高级 API。

我建议您做一个简短的练习，让您从头开始训练您的第一个可微模型：

1. 加载一个玩具数据集：例如，使用 scikit-learn 库中 datasets 模块提供的数据集之一。[7]

2. 构建一个线性模型（根据数据集选择回归或分类模型）。请思考如何使代码尽可能模块化：正如我们将看到的，您至少需要两个函数，一个用于初始化模型参数，另一个用于计算模型的预测。

3. 使用梯度下降法训练模型。现在您可以手动计算梯度：试着想象一下如何使这部分模块化。即，如果您想动态地添加或删除模型中的偏置项，如何更改梯度的计算？

4. 绘制损失函数和独立测试集上的准确率图。如果您了解标准机器学习，可以将结果与其他监督学习模型（例如决策树或 $k$-NN）进行比较，始终使用 scikit-learn。

[7] https://scikit-learn.org/stable/datasets/toy_dataset.html

# 5 全连接模型

**本章导读**

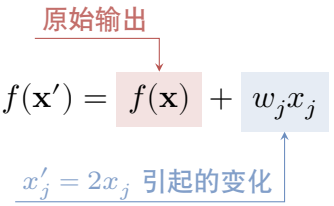
在本章中，我们将展示如何通过组合一系列所谓的全连接层（fully-connected layer）来构建可微模型。由于历史原因，这些模型也被称为多层感知机（multilayer perceptron，MLP）。MLP 是通过交替堆叠线性块（4章）和非线性函数（通常称为*激活函数*，activation function）构建的。

## 5.1 线性模型的局限

在本质上，线性模型存在局限性，因为根据定义，它们无法建模特征之间的非线性关系。例如，考虑两个输入向量 $\mathbf{x}$ 和 $\mathbf{x}'$，它们除了索引为 $j$ 的单个特征之外，其他部分完全相同。

$$x_i' = \begin{cases} x_i & \text{若 } i \neq j \\ 2x_i & \text{否则} \end{cases}$$

例如，这可以代表银行的两个客户，他们唯一的不同是收入，其中 $\mathbf{x}'$ 的收入是 $\mathbf{x}$ 的两倍。如果 $f$ 是一个线性模型（且无偏置项），则有：




因此，输入变化的唯一结果是由 $w_j$ 决定的一个微小的线性输出变化。假设我们正在对用户进行评分，我们可能希望建立诸如“收入 *1500* 为低收入，如果年龄 *<30* 则例外”的关系模型。[1] 综上所述，线性模型显然无法实现这一点。

典型的例子是 XOR 数据集，这是一个二值数据集，每个特征只能取值于 $\{0,1\}$。因此，整个数据集只有 4 种可能的组合：

$$f([0,0]) = 0\ ,\ f([0,1]) = 1\ ,\ f([1,0]) = 1\ ,\ f([1,1]) = 0$$

当且仅当两个输入中只有一个为正时，输出为正。尽管看起来简单，但这个问题是**非线性可分**（non-linearly separable）的，无法通过线性模型 100% 准确解决——见图F.5.1的可视化示例。

## 5.2 复合与隐藏层




在编程中，一种强大的思想是分解（decomposition），即将一个问题递归地分解为其组成部分，直到每个部分都可以用简单、可管理的操作来表示。在我们的情况下，可以通过假设模型 $f$ 实际上是两个可训练操作的组合来实现类似的效果：

$$f(\mathbf{x}) = (f_2 \circ f_1)(\mathbf{x})$$

[1] 无论如何，你大概不该使用机器学习来执行信用评分。

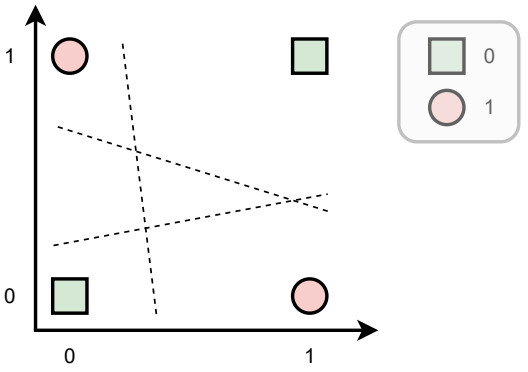



**图 F.5.1:** *XOR* 数据集示意图：绿色方块表示一类的值，红色圆点表示另一类的值。任何线性模型都无法完美地将它们分开（将所有方块放在决策边界的一侧，而所有圆点放在另一侧）。我们称这个数据集为非**线性可分**（*linearly separable*）的。

其中 $f_2 \circ f_1$ 是两个函数的复合：$(f_2 \circ f_1)(\mathbf{x}) = f_2(f_1(\mathbf{x}))$，假设每个函数都实例化自己的一组可训练参数。我们可以继续细分计算过程：

$$f(\mathbf{x}) = (f_l \circ f_{l-1} \circ \cdots \circ f_2 \circ f_1)(\mathbf{x})$$

其中，我们将总共复合 $l$ 个函数。请注意，只要每个 $f_i$ 不改变其输入数据的 "类型"，我们就可以任意多次地将这些变换串联起来，并且每个变换都会引入它自己的一组可训练参数。

例如，在我们的情况下，输入 $\mathbf{x}$ 是一个向量，因此任意向量到向量的操作（例如，矩阵乘法 $f_i(\mathbf{x}) = \mathbf{W}\mathbf{x}$）都可以无限次地组合。然而，还有一些问题需要注意。假

如我们将两个不同的线性投影串联起来：

$$\mathbf{h} = f_1(\mathbf{x}) = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1 \tag{E.5.1}$$
$$y = f_2(\mathbf{h}) = \mathbf{w}_2^\top\mathbf{h} + b_2 \tag{E.5.2}$$

很容易证明，这两个线性投影会“坍缩”为一个单一的线性投影：

$$y = \underbrace{(\mathbf{w}_2^\top\mathbf{W}_1)}_{\triangleq\,\mathbf{A}}\mathbf{x} + \underbrace{(\mathbf{w}_2^\top\mathbf{b}_1 + b_2)}_{\triangleq\,\mathbf{c}} = \mathbf{A}\mathbf{x} + \mathbf{c}$$

**全连接**（fully-connected，FC）模型，因为历史原因也被称为**多层感知机**（MLP），这种模型的思想是，在线性投影之间插入一个简单的逐元素非线性函数 $\phi : \mathbb{R} \to \mathbb{R}$，来避免这种“坍缩”现象的发生：

逐元素非线性

$$\mathbf{h} = f_1(\mathbf{x}) = \phi\left(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1\right) \tag{E.5.3}$$

$$y = f_2(\mathbf{h}) = \mathbf{w}_2^\top\mathbf{h} + b_2 \tag{E.5.4}$$

第二个块可以是线性的，如公式(E.5.4)所示，也可以根据任务需求包装在另一个非线性函数中（例如，分类任务中的 softmax 函数）。函数 $\phi$ 可以是任意非线性函数，例如多项式、平方根或 sigmoid 函数 $\sigma$。正如我们将在下一章中看到的，选择 $\phi$ 对模型的梯度及其优化过程有着重要影响，其挑战在于选择一个“足够非线性”的 $\phi$，以防止坍缩现象，同时确保其导数尽可能接近恒等式。一个较好的默认选择是**修正线性单元**（rectified linear unit，ReLU）。




**定义 D.5.1 (修正线性单元)**

**修正线性单元**（*ReLU*）的逐元素定义为：

$$\mathrm{ReLU}(s) = \max(0, s) \tag{E.5.5}$$

下一章将对 ReLU 进行更多讨论。随着激活函数 $\phi$ 的加入，现在我们可以将任意多的变换链接在一起：

$$y = \mathbf{w}_l^\top \phi\left(\mathbf{W}_{l-1}\left(\phi\left(\mathbf{W}_{l-2}\phi\left(\cdots\right) + \mathbf{b}_{l-2}\right)\right) + \mathbf{b}_{l-1}\right) \tag{E.5.6}$$

在本章的其余部分，我们将重点分析这类模型的训练和逼近性质。然而，首先我们需要简要讨论一下命名约定。

## 神经网络术语

正如我们已经提到的，神经网络有悠久的历史以及大量的术语，在此对这些术语进行简要总结。每个 $f_i$ 被称为模型的一**层**（layer），其中 $f_l$ 是**输出层**（output layer），$f_i, i = 1, \ldots, l-1$ 是**隐藏层**（hidden layer），而 $\mathbf{x}$ 则是**输入层**（input layer，此处稍微滥用了符号表示法）。基于这些术语，我们可以用批处理的形式重新定义**全连接层**（fully-connected layer），具体如下：

**定义 D.5.2 (全连接层)**




对于一批 $n$ 个大小为 $c$ 的向量，用矩阵 $\mathbf{X} \sim (n, c)$ 表示，一个**全连接**（*FC*）层定义为：

$$\mathrm{FC}(\mathbf{X}) = \phi\left(\mathbf{X}\mathbf{W} + \mathbf{b}\right) \tag{E.5.7}$$

该层的参数包括权重矩阵 $\mathbf{W} \sim (c, c')$ 和偏置向量 $\mathbf{b} \sim (c')$，参数总数为 $(c+1)c'$（假设 $\phi$ 没有额外的参数）。其超参数包括宽度 $c'$ 和非线性函数 $\phi$。

输出 $f_i(\mathbf{x})$ 是该层的**激活值**（activation），有时会区分**前激活**（pre-activation）和**后激活**（post-activation），即非

```
class FullyConnectedLayer(nn.Module):
  def __init__(self, c: int, cprime: int):
    super().__init__()
    # 初始化参数
    self.W = nn.Parameter(
                torch.randn(c, cprime))
    self.b = nn.Parameter(
                torch.randn(1, cprime))

  def forward(self, x):
    return relu(x @ self.W + self.b)
```

**框 C.5.1:** *在 PyTorch 中将公式(E.5.7)中的 FC 层实现为一个对象。我们需要一种特殊的语法来区分可训练的参数（例如 $\mathbf{W}$）和其他不可训练的张量：这在 PyTorch 中通过将张量包装在`Parameter`对象中实现。PyTorch 在`torch.nn`中也提供一系列现成的层，包括 FC 层（通过`torch.nn.Linear`实现）。*

线性激活函数作用之前和之后的结果。非线性函数 $\phi$ 被叫做**激活函数**（activation function）。$f_i$ 的每个输出被称为一个**神经元**（neuron）。尽管这些术语在某些程度上已经显得过时，但它们仍然被广泛使用，我们也将在需要时使用这些术语。

每一层的尺寸（输出的形状）是一个超参数，用户可以自行选择。它只影响下一层的输入形状，被称为该层的**宽度**（width）。对于大量层数，超参数的数量会线性增长，其选择成为一个组合问题。在第9章讨论包含几十层（甚至上百层）的模型的设计时，我们将再次回顾这一点。

层的概念在常见框架中也非常普遍。如公式(E.5.7)中的层可以定义为一个具有两个主要函数的对象：一个是初始化函数，用于根据所选的超参数随机初始化模型的所有参数；另一个是调用函数，用于提供该层的输出。见框C.5.1中的示例。一个模型可以通过将这些层的实例依次链接起来定义。例如，在 PyTorch 中，这可以通过`Sequential`对象实现：

```
model = nn.Sequential(
    FullyConnectedLayer(3, 5),
    FullyConnectedLayer(5, 4)
)
```

请注意，从输入输出特征的角度来看，框C.5.1中定义的层与上述定义的模型并没有本质区别。我们可以将`model`用作更大模型中的一层。这种可组合性（compositionality）是可微模型的一个重要特性。

### 5.2.1 MLP 的逼近特性




训练 MLP 的过程与前文讨论的线性模型训练类似。例如，对于回归任务，我们可以最小化均方误差（mean-squared error）：

$$\min_{\{\mathbf{W}_k,\mathbf{b}_k\}_{k=1}^{l}} \frac{1}{n}\sum_i \left(y_i - f(\mathbf{x}_i)\right)^2$$

其中，最小化过程目前是对模型的所有参数同时进行的。在下一章中，我们将介绍在这种情况下计算梯度的通用方法。

目前我们注意到，相较于线性模型，MLP 主要的区别在于，添加隐藏层会使整个优化问题变为非凸，其优化结果依赖于模型初始化，可能存在多个局部最优解。这一点在历史上具有重要意义，因为其他监督学习方法（例如支持向量机 [HSS08]）虽然能提供非线性模型，但优化问题仍然为凸。[2]

从理论角度，我们可以问：添加隐藏层的意义是什么？换句话说，如果线性模型只能解决线性可分的任务，添加

[2]可微模型具有优秀泛化能力的原因是一个有趣且尚未解决的研究问题，第9章将回顾这个问题。现有的解释包括了从（随机）梯度下降法的隐式偏差 [PPVF21]到模型自身架构的内在属性 [AJB+17, TNHA24]。

隐藏层后，模型可以逼近的函数类别是什么？事实证明，只需要添加单个隐藏层就足以实现**通用逼近**（universal approximation）能力。这个重要的结果由 G.Cybenko 于 1989 年首次证明 [Cyb89]。

> **定理 5.1 (MLP 的通用逼近性)** *给定一个连续函数 $g : \mathbb{R}^d \to \mathbb{R}$，我们总是可以找到一个形式为(E.5.3)-(E.5.4)的模型 $f(\mathbf{x})$（具有单隐藏层的 MLP），且使用 sigmoid 激活函数，使得对于任意 $\varepsilon > 0$：*
>
> $$|f(\mathbf{x}) - g(\mathbf{x})| \leq \varepsilon \ , \ \forall \mathbf{x}$$
>
> *上述结论在一个紧致域（compact domain）内成立。换句话说，单隐藏层 MLP 在连续函数空间中是“稠密的（dense)”。*

这一定理的优美之处不应掩盖其纯理论构造，它使用了模型隐藏层的宽度可以无限增长这一事实。因此，对于任何不满足前述不等式的输入 $\mathbf{x}$，我们总是可以通过添加新单元来减少逼近误差（详见附录B）。实际上，可以构造一些函数类，使所需的隐藏层神经元数量随输入特征的维度呈指数增长 [Ben09]。[3]

诸如 [Hor91]的许多其他研究者逐步完善了这一结果，将其推广到使用几乎任何可能的激活函数（包括 ReLU）的模型。此外，对于具有有限宽度但可能拥有无限深度的模型，也可以证明其具有通用逼近能力 [LPW+17]。另一个独立的研究方向关注过参数化（overparameterize）模型的逼近能力，即参数数量超过训练数据的模型。在这种情况下，可以在许多有趣的场景下证明训练可以达到

[3] 问题之一是奇偶性（parity）问题，它与 XOR 任务密切相关https://blog.wtf.sg/posts/2023-02-03-the-new-xor-problem/。

全局最优 [DZPS19, AZLL19]（通俗地说，如果参数数量足够大，模型可以在每个训练样本上实现损失的最小化，达到优化问题的全局最小值）。有关 Cybenko 定理的一维可视化，请参阅附录B。

可微模型的逼近和学习能力是广阔的研究领域，已经有大量专著对其深入探讨，而我们在此仅提及一些重要结果。在本书的后续部分中，我们将重点关注模型本身的高效设计，其行为往往比这些定理所揭示的更加复杂，且难以控制和设计。

## 5.3 随机优化

为了优化模型，我们可以在对应的经验风险最小化问题上执行梯度下降。然而，当 $n$（数据集的大小）变得非常大时，这可能难以实现。我们将在下一章中看到，计算损失梯度所需的时间与样例数量成线性关系。当 $n$ 达到 $10^4$ 或更高时，计算变得不可行或非常缓慢，特别是对于大模型（暂时不考虑内存问题）。

幸运的是，这种形式的问题可以通过一种优雅的近似方法来解决，即使用数据的子集来计算下降方向。为此，假设在梯度下降的第 $t$ 次迭代中，从数据集 $\mathcal{S}_n$ 中随机采样一个包含 $r$ 个样本的子集 $\mathcal{B}_t \subset \mathcal{S}_n$，其中 $r \ll n$。我们称之为一个**小批量**（mini-batch）。可以通过仅考虑小批量数据来计算近似损失：

$$\widetilde{L}_t = \underbrace{\frac{1}{r}\sum_{(x_i,y_i)\in\mathcal{B}_t} l(y_i, f(x_i))}_{\text{小批量}} \approx \underbrace{\frac{1}{n}\sum_{(x_i,y_i)\in\mathcal{S}_n} l(y_i, f(x_i))}_{\text{完整数据集}} \tag{E.5.8}$$

如果假设小批量中的样本是从数据集中独立同分布采样的，则 $\widetilde{L}_t$ 是完整损失的蒙特卡洛近似，其梯度也是如

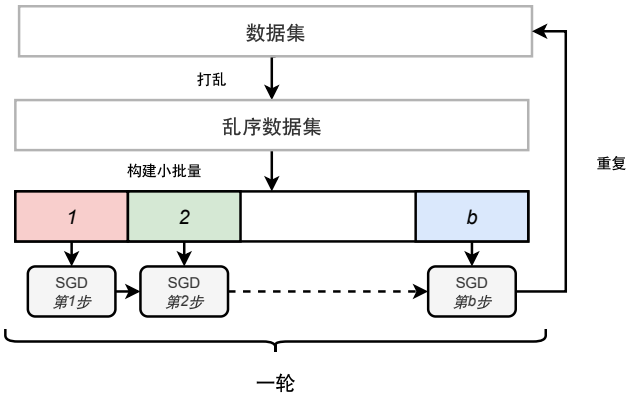



**图 F.5.2:** 构建小批量序列：打乱数据集后，随机优化从第 1 个小批量开始，它由数据集的前 $r$ 个元素组成。然后依次处理到第 $b$ 个小批量（其中 $b = \frac{n}{r}$，假设数据集大小可以被 $r$ 整除）。完成一轮后，训练在第 $b+1$ 个小批量上继续，它包含再次打乱的数据集的前 $r$ 个元素。第二轮在小批量 $2b$ 结束，依此类推。

此。然而，其计算复杂度仅随小批量大小 $r$ 增长，$r$ 是用户可控的。大致来说，小批量的维度 $r$ 较低时，迭代速度更快，但梯度方差更高；而较高的 $r$ 导致迭代速度更慢，更精确。对于大模型而言，内存通常是最大的瓶颈，因此小批量的大小 $r$ 可以选择为填满每次迭代中可用的硬件资源。

**随机梯度下降**（stochastic gradient descent，SGD）的一个例子是在小批量数据上使用梯度下降法。由于上述性质，可以证明 SGD 能收敛到一个期望最小值。因此，SGD 是训练可微模型时的首选优化策略。

剩下的最后一个问题是如何选择小批量数据。对于大型数据集，随机采样元素可能代价较高，尤其是需要将数据从 GPU 内存中来回移动时。一种更易于优化的折中方案如下：

```
# 两个张量组成的数据集
dataset = torch.utils.data.TensorDataset(
    torch.randn(1000, 3),
    torch.randn(1000, 1))

# DataLoader 提供了打乱和小批量功能
from torch.utils.data import DataLoader
dataloader = DataLoader(dataset,
                shuffle=True,
                batch_size=32)

for xb, yb in dataloader:
  # 在小批量上迭代（一轮）
  # xb has shape (32, 3)
  # yb has shape (32, 1)
```

**框 C.5.2:** 使用 *PyTorch* 的 *DataLoader* 构建小批量序列：所有框架都提供了类似工具。

1. 首先打乱（shuffle）数据集。

2. 然后，把原始数据集分成包含 $r$ 个连续元素的小批量，并依次处理每个小批量。假设数据集大小为 $n = rb$，则会被分成 $b$ 个小批量，执行 $b$ 次 SGD 步骤。如果代码在 GPU 上运行，这一步包括将小批量数据传输到 GPU 内存中计算。

3. 完成所有按上述方式构造的小批量后，返回第 1 步并迭代。

上述流程的一个完整循环被称为一**轮**（epoch），**轮数**是一个十分常见的超参数（例如，对于包含 1000 个样本的数据集，和每批 20 个元素的小批量，“训练 *5* 轮”表示进行 250 次迭代）。代价高昂的打乱操作仅在每轮开始时执行一次，在一轮内，小批量数据可以快速预取并由框架优化处理。示意图见F.5.2。大多数框架都提供了将数据集组织为可单独索引的元素的方式，以及用于构

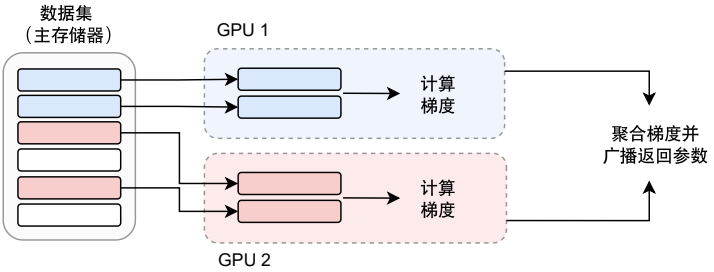



**图 F.5.3:** 分布式随机优化的一种简单形式：在每台可用的机器或 *GPU* 上处理一个小批量（通过在每台设备上复制权重），然后将对应的梯度进行求和或求平均，最后将结果广播回各设备（由于梯度运算的线性特性，这种操作是可行的）。该方法需要在机器或 *GPU* 之间引入同步机制。

建小批量序列的独立接口。例如，在 PyTorch 中，这分别由`Dataset`和`DataLoader`实现——参考框C.5.2。

这种设置还可以实现一种跨 GPU 或跨机器的简单并行形式。假设每台机器的容量足够大，可以存储模型参数的完整副本，那么我们可以让不同的机器并行处理不同的小批量，将它们的本地贡献求和，进行最终更新，之后将更新后的参数广播回每台机器。这种方法在 PyTorch 被称为**数据并行**（data parallel）设置，[4]在图F.5.3中可视化。更复杂的并行形式，例如**张量并行**（tensor parallelism），也是可行的，但本书未涉及这些内容。

## 5.4 激活函数

在本章的最后，我们简要概述激活函数的选择。如上一节所述，在理论上，几乎任何逐元素非线性函数都是有效的。然而，并非所有选择都能带来良好的性能。例如，考虑一个简单的多项式函数，对于用户定义的某个正整

[4] https://pytorch.org/tutorials/intermediate/ddp_tutorial.html

数 $p$:

$$\phi(s) = s^p$$

对于较大的 $p$，该函数在正负两侧都会迅速增长，并在网络的层间叠加。这会导致模型难以训练，并出现数值不稳定的问题。

在历史上，神经网络是被作为生物神经元的近似模型引入的（因此得名人工神经网络）。在这种意义上，点积 $\mathbf{w}^\top \mathbf{x}$ 中的权重 $\mathbf{w}^\top$ 是对突触 (synapse) 的简单建模，偏置 $b$ 表示一个阈值，当输入的累积和超过该阈值时，神经元被“激活”:

$$s = \mathbf{w}^\top \mathbf{x} - b \ , \ \phi(s) = \mathbb{I}_{s \geq 0}$$

其中 $\mathbb{I}_b$ 是一个指示函数，当 $b$ 为真时为 1，否则为 0。由于这种激活函数是不可微的，可以使用 sigmoid 函数 $\sigma(s)$ 作为它的软逼近 (soft-approximation)。事实上，我们可以定义一个带有可调斜率 $a$ 的广义 sigmoid 函数 $\sigma_a(s) = \sigma(as)$:

$$\lim_{a \to \infty} \sigma_a(s) = \mathbb{I}_{s \geq 0}$$

另一个常见的变体是双曲正切函数 (hyperbolic tangent, tanh) 函数，它是 sigmoid 函数在 $[-1, +1]$ 的缩放版本:

$$\tanh(s) = 2\sigma(s) - 1$$

由 AlexNet[KSH12]在 2021 年推广的现代神经网络，则使用了公式(E.5.5)中的 ReLU 函数。下一章将讨论

ReLU 函数对于类似 sigmoid 函数的优势。在此需要指出，ReLU 具有一些反直觉的性质。例如，它们在 0 点不可微，并且由于所有负输入都会被设置为 0，具有高度稀疏性。第二个特性可能导致所谓的“神经元死亡”，即某些单元对于所有输入都输出常数 0。这种问题可以通过 ReLU 的一个简单变体来解决，它叫做**带泄露的 ReLU**（Leaky ReLU）：

$$\text{LeakyReLU}(s) = \begin{cases} s & \text{若 } s \geq 0 \\ \alpha s & \text{否则} \end{cases} \tag{E.5.9}$$

对于非常小的 $\alpha$，例如 $\alpha = 0.01$，我们还可以为每个单元训练一个不同的 $\alpha$ （因为该函数关于 $\alpha$ 是可微的）。在这种情况下，这种激活函数被称为**带参数的 ReLU**（parametric ReLU，PReLU）[HZRS15]。可训练的激活函数通常是一种简单的方式，通过引入少量参数来增加模型的灵活性——对于 PReLU，每个神经元仅需一个额外参数。

ReLU 的完全可微变体也是可行的，例如 **softplus** 函数：

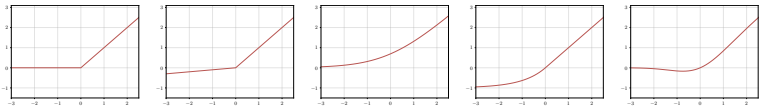
$$\text{softplus}(s) = \log(1 + \exp(s)) \tag{E.5.10}$$

softplus 函数不经过原点，且其值始终大于 0。另一种变体是**指数线性单元**（exponential linear unit，ELU），它保留了经过原点的性质，同时将下界切换为 $-1$：

$$\text{ELU}(s) = \begin{cases} s & \text{若 } s \geq 0 \\ \exp(s) - 1 & \text{否则} \end{cases} \tag{E.5.11}$$

通过注意 ReLU 和指示函数的相似性，可以定义另一类变体。可以将 ReLU 重写为：

$$\text{ReLU}(s) = s \cdot \mathbb{I}_{s \geq 0}$$



*ReLU* *LeakyReLU* *softplus* *ELU* *GELU*

**图 F.5.4:** *ReLU* 和四种变体的可视化对比：*LeakyReLU* (E.5.9)，*softplus* (E.5.10)，*ELU* (E.5.11)和 *GELU*。为了更好的可视化效果，*LeakyReLU* 中 $\alpha = 0.1$，但在实践中 $\alpha$ 可以更接近 0（例如 0.01）。

因此，ReLU 在负象限上等价于指示函数，而在正象限上用 $s$ 代替了 1。我们可以通过将指示函数替换为一个加权因子 $\beta(s)$ 来对其进行推广：

$$\text{GeneralizedReLU}(s) = s \cdot \beta(s)$$

将 $\beta(s)$ 选为高斯累积分布函数，我们可以得到**高斯误差线性单元** (Gaussian error linear unit，GELU) [HG16]；而如果 $\beta(s) = \sigma(s)$ 可以得到 **sigmoid 线性单元** (sigmoid linear unit，SiLU) [HG16]，也被称为 **Swish** 函数 [RZL17]。图F.5.4中绘制了一些激活函数的示例。除了某些细节（例如在负象限中的单调性）外，它们总体上是相对相似的。通常，仅通过更换激活函数很难显著提升模型的性能。

通过向激活函数添加可训练参数，可以生成多种可训练变体。例如，的一个常见可训练变体具有四个参数 $\{a, b, c, d\}$，定义为：

$$\text{Trainable-Swish}(s) = \sigma(as + b)(cs + d) \qquad \text{(E.5.12)}$$

我们还可以考虑设计**非参数**(non-parametric)激活函数，即那些没有固定数量可训练参数的激活函数。例如，考虑一组由整数 $i$ 索引的（不可训练）通用标量函数 $\phi_i$。我们可以通过 $n$ 个这样的基函数的线性组合来构造一

个灵活的激活函数：

$$\phi(s) = \sum_{i=1}^{n} \alpha_i \phi_i(s) \tag{E.5.13}$$

其中 $n$ 是一个超参数，系数 $\alpha_i$ 通过梯度下降法训练得到。它们可以对所有函数统一设置，也可以针对每一层和/或每个神经元单独设置。基于 $\phi_i$ 的选择，可以得到不同类别的激活函数：如果每个 $\phi_i$ 是 ReLU，我们将得到**自适应分段线性**（adaptive piecewise linear，APL）[AHSB14]；如果 $\phi_i$ 是更通用的核函数，则得到**核激活函数**（kernel activation function，KAF）[MZBG18, SVVTU19]。更通用的模型可以通过考虑具有多个输入和多个输出的函数来构造 [LCX+23]。相关综述请参阅 [ADIP21]。

一般来说，对于“什么是最佳激活函数”这个问题，并没有统一的答案，因为最佳选择取决于具体的任务、数据集以及网络架构。ReLU 是一种常见的选择，原因在于它表现良好、代码高度优化，且计算开销较低。重要的是要考虑基本的计算权衡，在给定资源预算的情况下，更复杂的激活函数可能会导致网络的宽度或深度受限，从而可能影响整个网络架构的性能。因此，具有大量可训练参数的激活函数在实际应用中较少见。

### 设计变体

并非所有的层都适合线性投影和逐元素非线性这种框架，这里介绍三种常见的变体。首先，**门控线性单元**（gated linear unit）[DFAG17]结合了(E.5.12)的结构与乘法（阿达玛）交互：

$$f(\mathbf{x}) = \sigma\left(\mathbf{W}_1\mathbf{x}\right) \odot \left(\mathbf{W}_2\mathbf{x}\right) \tag{E.5.14}$$

其中 $\mathbf{W}_1$ 和 $\mathbf{W}_2$ 被训练。另一种变体 SwiGLU 将(E.5.14)中的 sigmoid 函数替换为 Swish 函数 [Sha20]。事实上，门控 MLP 是现代大语言模型中的常见选择，见第11章。

其次，在 **maxout** 网络 [GWFM+13]中，每个单元输出 $k$（超参数）个不同投影的最大值。最后，近期还有研究提出用形如公式(E.5.13)的可训练非线性矩阵 $W_{ij} \to \phi_{ij}(x_j)$ 代替线性映射 $\mathbf{W}$，称为 **Kolmogorov-Arnold 网络**（KAN，[LWV+24]）：

$$h_i = \sum_j \phi_{ij}(x_j)$$

## 从理论到实践



本章介绍了训练可微模型的通用框架必须满足的两个关键要求：

1. 处理大型数据集的方法：需要将其打乱、划分为小批量，并在 GPU 之间来回传输。在 PyTorch 中，这些功能大部分通过`Dataset`和`DataLoader`接口实现，如框C.5.2所示。

2. 通过基本模块（即*层*，layer）的组合来构建模型的机制。在 PyTorch 中，层由 `torch.nn`模块实现，并且可以通过`Sequential`接口或子类化`Module`来组合，如框C.5.1所示。

现在建议您尝试复现 PyTorch 官方文档中的快速指南之一。[5] 这些内容应该是相对容易理解的，除了梯度计算机制（将在下一章介绍）。这也是一个研究 Hugging Face 数据集的好时机，它结合了大量数据集资源库，并提供了由 Apache Arrow 支持的，与框架无关的接口，来处理和缓存数据。[6]

JAX 本身不提供高级工具。您可以使用任何现有工具来加载数据，包括 PyTorch 的 DataLoader 和 Hugging Face 的 datasets。对于构建模型，最简单的方法是使用外部库。由于 JAX 是一个纯函数式的框架，因此无法实现类似框C.5.1中的面向对象抽象。我个人建议使用 Equinox[KG21]，该库将 JAX 的基本数据结构（`pytree`）与可调用节点结合，提供了与类相似的使用体验。

[5] https://pytorch.org/tutorials/beginner/basics/quickstart_tutorial.html

[6] https://huggingface.co/docs/datasets/en/quickstart

# 6 | 自动微分

**本章导读**

上一章强调了需要一种高效、自动的过程来为任意运算序列计算梯度。在本章中，我们将介绍这样一种方法，在神经网络领域被称为**反向传播**（back-propagation），而在计算机科学领域则被称为**反向模式自动微分**（reverse-mode automatic differentiation）。对这种方法的分析能够提供多方面的见解，包括模型选择以及优化模型的内存需求。

## 6.1 问题设定

我们将讨论高效计算通用计算图（computational graphs）梯度的问题，例如优化全连接模型的标量损失函数时诱导产生的，这种问题叫做**自动微分**（automatic differentiation，AD）[BPRS18]。可以将计算图视为程序运行过程中产生的一系列原子操作（称为**原语**）。为简明起见，我们主要以顺序计算图为例，但所有内容都可以轻松扩展到无环（acyclic）图，甚至更通用的计算图 [GW08, BR24]。

这个问题看起来微不足道，因为雅可比矩阵的链式法则（见第2.2节，(E.2.23)）告诉我们，复合函数的梯度只是相

应雅可比矩阵的乘积。然而，本章的关键难点是如何有效地实现这个过程，而最终形成的算法（**反向模式 AD** 或**反向传播**）是神经网络以及可微编程领域的基石 [GW08, BR24]。理解该算法对于理解大多数用于实现和训练此类程序的框架（如 TensorFlow、PyTorch 或 JAX）的设计理念（以及它们的差异）至关重要。关于该算法的简史可参考 [Gri12]。

为了设定问题，假设我们有一组**原语**：

$$\mathbf{y} = f_i(\mathbf{x}, \mathbf{w}_i)$$

每个原语表示一个对输入向量 $\mathbf{x} \sim (c_i)$ 的运算，由向量 $\mathbf{w}_i \sim (p_i)$ 参数化（例如，线性投影中的权重），并给出一个输出向量 $\mathbf{y} \sim (c_i')$。

我们对原语的定义存在很大的灵活性。它既可以表示基本的线性代数运算（例如矩阵乘法），也可以表示第5章所述的层（例如带有激活函数的全连接层），甚至还能表示更大规模的功能模块或完整模型。这种递归可组合性是编程的核心特性之一，可以扩展到我们的情形。

对每个原语，我们仅假设知道如何计算它关于两个输入参数的偏导数，将其称为该操作的**输入雅可比矩阵**（input Jacobian）和**权重雅可比矩阵**（weight Jacobian）：

$$\begin{aligned} &\textbf{输入雅可比矩阵：} && \partial_{\mathbf{x}} [f(\mathbf{x}, \mathbf{w})] \sim (c', c) \\ &\textbf{权重雅可比矩阵：} && \partial_{\mathbf{w}} [f(\mathbf{x}, \mathbf{w})] \sim (c', p) \end{aligned}$$

由于我们将分析限制在可微模型范围内，所以这些假设是合理的。对于具有一个或多个不可微点的连续原语（如 ReLU），可以通过使用**次梯度**（subgradient）使其适配此框架（见第6.4.4节）。对于诸如采样或阈值化的不可微操作，也可以通过寻找其梯度的松弛形式或等效估计来将其包含在内 [NCN+23]。我们将在下一卷中对后者进

行详细讨论。

## 符号体系和高阶雅可比矩阵




为了便于阅读，我们仅考虑向量，因为所有最终的梯度都是矩阵。然而，在实际中，现有原语可能具有更高阶的输入、权重或输出。例如，考虑一个作用于小批量输入的基本全连接层：

$$f(\mathbf{X}, \mathbf{W}) = \mathbf{X}\mathbf{W} + \mathbf{b}$$

在这种情况下，输入 $\mathbf{X}$ 的形状为 $(n, c)$，权重的形状为 $(c, c')$，偏置的形状为 $(c')$（其中 $c'$ 为超参数），输出的形状为 $(n, c')$。因此，输入雅可比矩阵的形状为 $(n, c', n, c)$，权重雅可比矩阵的形状为 $(n, c', c, c')$，两者都是 4 阶。

在我们的符号体系中，可以考虑等效的扁平向量 $\mathbf{x} = \text{vect}(\mathbf{X})$ 和 $\mathbf{w} = [\text{vect}(\mathbf{W}); \mathbf{b}]$，得到的“扁平”雅可比矩阵分别具有形状 $(nc', nc)$ 和 $(nc', cc')$。这在下文中至关重要，因为每当我们提到“输入大小 $c$”时，指的是“所有输入形状的乘积”，这包括最终的小批量维度。这也表明了，尽管我们知道如何计算这些雅可比矩阵，但由于其巨大的维度，我们大概不希望在内存中完整地显式构造（materialize）它们。

最后需要说明的是，我们的符号体系与这些原语在函数式编程库（例如 JAX）的实现方法一致。在面向对象的框架（例如 TensorFlow 和 PyTorch）中，层通常被实现为对象（见上一章的框C.5.1），其中参数是对象的属性，而函数调用则被替换为对象的方法。面向对象的风格在某些实践中简化了操作，例如在输入形状未知的情况下延迟初始化所有参数（**惰性初始化**，lazy initialization）。但它也增加了一层抽象性，使我们需要认真考虑如何将符号体系转换为可运行的代码。正如我们将要看到的，这些差异会反映在两种框架中 AD 的实现方式上。

### 6.1.1 问题陈述

了解所有细节之后，我们现在准备正式描述自动微分。考虑一个 $l$ 次原语调用的序列，接着进行最终求和：

$$\begin{aligned}\mathbf{h}_1 &= f_1(\mathbf{x}, \mathbf{w}_1)\\ \mathbf{h}_2 &= f_2(\mathbf{h}_1, \mathbf{w}_2)\\ &\vdots\\ \mathbf{h}_l &= f_l(\mathbf{h}_{l-1}, \mathbf{w}_l)\\ y &= \sum \mathbf{h}_l\end{aligned}$$

这被称为程序的**计算轨迹**（evaluation trace）。大致来说，前 $l-1$ 次操作可以表示一个可微模型的多个层，第 $l$ 次操作可以是每个输入的损失（例如交叉熵损失），最终的操作是对小批量的损失求和。因此，程序的输出始终是一个标量，因为需要一个标量来进行数值优化。我们将上述程序简写为 $F(\mathbf{x})$。




**定义 D.6.1 (自动微分)**

给定一个由可微原语序列组成的程序 $F(\mathbf{x})$，**自动微分**指的是在已知计算图结构以及所有输入和权重雅可比矩阵的情况下，同时且高效地计算该程序的所有权重雅可比矩阵的任务：

$$AD(F(\mathbf{x})) = \left\{\partial_{\mathbf{w}_i} y\right\}_{i=1}^{l}$$

正如我们将看到的，自动微分主要分为两大类，**前向模式**（forward-mode）和**反向模式**（backward-mode），它们对应于操作的不同组合顺序。我们将发现，反向模式

(在神经网络文献中称为**反向传播**) 在我们的环境中明显效率更高。虽然我们关注的是简化的场景，但将我们的推导扩展到原语的无环图（如前文所述）和不同层之间共享参数（**权重共享**，weight sharing）的情况相对容易。在第13章中，我们将看到一个权重共享的示例。

### 6.1.2 数值微分与符号微分




在介绍前向模式自动微分前，我们先来讨论自动微分与其他函数微分算法的区别。首先，我们可以直接应用梯度的定义（见第2.2节）来获得梯度的数值近似值。该过程被称为**数值微分** (numerical differentiation)。然而，在一个朴素实现中，每个标量值求导需要 2 次函数调用。因此，除了用于实现的数值验证之外，数值微分通常不适合实际使用。

然后，考虑以下简单函数：

$$f(x) = a\sin(x) + bx\sin(x)$$

我们可以让符号计算引擎预先计算出导数的完整符号表达式。这被称为**符号微分** (symbolic differentiation)，在 Python 中的实现示例如框C.6.1所示。

在实际的实现中，中间值 $h = \sin(x)$ 只被计算一次，并存储在一个中间变量中，该变量也可以在梯度轨迹的相应计算过程中复用（导数中的 $\cos(x)$ 项适用类似的推理）。然而，这种优化并不像看起来那么简单：找到一个能避免任何不必要计算的，最优的雅可比矩阵实现方式（**最优雅可比矩阵累积**，optimal Jacobian accumulation）是一个 NP 完全（NP-complete）问题。不过，我们将看到，可以利用程序的结构，设计出一种高效的自动微分实现，比上述符号方法显著更高效（实际上，这等价于一种允许存在子序列的符号方法 [Lau19]）。

```python
import sympy as sp
x, a, b = sp.symbols('x a b')
y = a*sp.sin(x) + b*x*sp.sin(x)
sp.diff(y, x)
# [输出]: acos(x)+bxcos(x)+bsin(x)
```

**框 C.6.1:** *Python 中使用 SymPy 的符号微分。*

## 6.2 前向模式微分

首先回顾雅可比矩阵的链式法则。设有两个原语函数（primitive function）的组合：

$$\mathbf{h} = f_1(\mathbf{x}) \ , \ \mathbf{y} = f_2(\mathbf{h})$$

基于它们的梯度，可得：

$$\partial_{\mathbf{x}} \mathbf{y} = \partial_{\mathbf{h}} \mathbf{y} \cdot \partial_{\mathbf{x}} \mathbf{h}$$

如果 $\mathbf{x}$，$\mathbf{h}$ 和 $\mathbf{y}$ 的维度分别为 $a$，$b$ 和 $c$，那么之前的雅可比矩阵计算需要将一个 $c \times b$ 的矩阵（绿色部分）和一个 $b \times a$ 矩阵（红色）。可以这样理解这个法则：如果已经计算了 $f_1$ 及其雅可比矩阵（红色部分），只要应用 $f_2$，我们就可以通过与对应的雅可比矩阵（绿色部分）相乘来“更新”梯度。

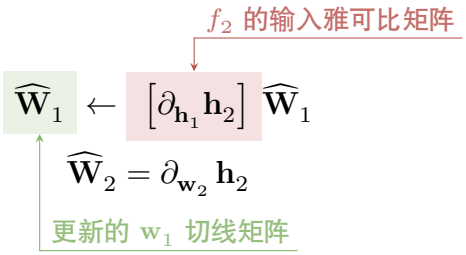
我们可以立即应用这一发现，来获得称为**前向模式自动微分**（forward-mode automatic differentiation，F-AD）的工作算法。其核心思想是：每次应用一个原语函数时，初始化其对应的权重雅可比矩阵（在上下文中称为**切线**），同时更新所有先前的切线矩阵。为了更直观地描述该算法的主要步骤，下面通过一个简单的例子来说明。

考虑程序中的第一条指令 $\mathbf{h}_1 = f_1(\mathbf{x}, \mathbf{w}_1)$。由于此时尚

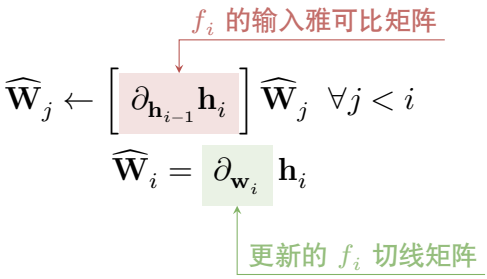
未存储任何信息，我们将 $\mathbf{w}_1$ 的切线矩阵初始化为其对应的权重雅可比矩阵。

$$\widehat{\mathbf{W}}_1 = \partial_{\mathbf{w}_1} \mathbf{h}_1$$

接下来执行第二条指令 $\mathbf{h}_2 = f_2(\mathbf{h}_1, \mathbf{w}_2)$ 更新先前的切线矩阵，同时初始化第二个切线矩阵：




该更新需要该原语的输入雅可比矩阵，而第二项则需要其权重雅可比矩阵。抽象地看，考虑第 $i$ 个原语 $\mathbf{h}_i = f_i(\mathbf{h}_{i-1}, \mathbf{w}_i)$。初始化 $\mathbf{w}_i$ 的切线矩阵，同时更新所有先前的矩阵：




在第一行中存在 $i-1$ 次更新操作（对于已经存储在内存中的每个切线矩阵，各执行一次）其中红色项——第 $i$ 次操作的输入雅可比矩阵——被所有先前的切线共享。程序的最后一步是求和操作，其对应梯度为算法的输出结果：[1]

[1]为了与符号体系表示保持一致，公式(E.6.1)的输出是一个行向

$$\nabla_{\mathbf{w}_i} y = \mathbf{1}^\top \widehat{\mathbf{W}}_i \quad \forall i \tag{E.6.1}$$

好了！现在让我们更详细地分析这一算法。首先，我们列出的所有操作都可以与原始程序的执行无缝交错(interleave)。这意味着，该算法的空间复杂度大致与我们所计算梯度的程序本身的空间复杂度成正比。

从反面来看，算法的核心操作（$\widehat{\mathbf{W}}_i$ 的更新）需要两个矩阵相乘。两个矩阵形状通常为 $(c_i', c_i)$ 和 $(c_i, p_j)$，其中 $c_i, c_i'$ 是输入/输出的形状，$p_j$ 是 $\mathbf{w}_j$ 的形状。这种操作的计算代价非常高：例如，假设输入和输出形状均为 $(n, d)$，其中 $n$ 为小批量维度，$d$ 表示输入/输出特征。那么，矩阵乘法的复杂度为：$\mathcal{O}(n^2 d^2 p_j)$，在小批量大小和特征维度上都是二次方。这种复杂度很容易变得不可接受，尤其是对于图像等高维输入。

注意到算法的最后一步是一个更简单的矩阵-向量乘法，我们可以在此获得更好的权衡，这种简化是因为输出是一个标量。这一点将在下一节中进行更详细的探讨。

## 6.3 反向模式微分




接下来，展开第 $i$ 个权重矩阵对应的单个梯度项：

$$\nabla_{\mathbf{w}_i} y = \mathbf{1}^\top \left[\partial_{\mathbf{h}_{l-1}} \mathbf{h}_l\right] \cdots \left[\partial_{\mathbf{h}_i} \mathbf{h}_{i+1}\right] \left[\partial_{\mathbf{w}_i} \mathbf{h}_i\right] \tag{E.6.2}$$

请记住，抛开符号体系，公式(E.6.2)只是一系列可能很长的矩阵乘法，包括了一个常数项（由全为 **1** 的向量表示），一系列输入雅可比矩阵（红色部分），以及相应权重矩阵的权重雅可比矩阵（绿色部分）。让我们为红色部

量，而我们将梯度定义为列向量。我们暂时忽略这一细微差异，直到本章稍后定义向量-雅可比乘积时再处理这个问题。

分定义一个简写形式：

$$\tilde{\mathbf{h}}_i = \mathbf{1}^\top \prod_{j=i+1}^{l} \partial_{\mathbf{h}_{j-1}} \mathbf{h}_j \tag{E.6.3}$$

由于矩阵乘法满足结合律，，我们可以按照任意顺序计算公式(E.6.2)。在 F-AD 中，我们从右向左计算，因为这对应于原语函数执行的顺序。然而，我们可以从以下两个有趣的方面来改进这一过程：

1. 公式(E.6.2)最左边的项是向量与矩阵的乘积（这是输出中有标量项的结果），它在计算上比两个矩阵的乘积更高效。其输出也是一个向量。

2. 公式(E.6.3)中的项（从第 $i$ 层到第 $l$ 层所有输入雅可比矩阵的积）可以从最后一项开始递归计算，以相反的顺序迭代与输入雅可比矩阵相乘。

结合上述观察，可以提出另一种自动微分的方法，我们称为**反向模式自动微分**（reverse-mode automatic differentiation），其概述如下。

1. 与 F-AD 不同，，我们首先执行整个待微分的程序，并存储所有的中间输出。

2. 初始化向量 $\tilde{\mathbf{h}} = \mathbf{1}^\top$，其对应公式(E.6.2)中最左边的项。

3. 反向移动。即，对于范围在 $l, l-1, l-2, \ldots, 1$ 的索引 $i$，我们首先计算相对于第 $i$ 个权重矩阵的梯度：

   $$\partial_{\mathbf{w}_i} y = \tilde{\mathbf{h}} \left[\partial_{\mathbf{w}_i} \mathbf{h}_i\right]$$

   这是我们需要的第 $i$ 个梯度。然后，更新“反向传播”的输入雅可比矩阵，公式为：

   $$\tilde{\mathbf{h}} \leftarrow \tilde{\mathbf{h}} \left[\partial_{\mathbf{h}_{i-1}} \mathbf{h}_i\right]$$

步骤 (1)-(3) 描述了一个与原始程序大致对称的程序，我们称之为**对偶**（dual）或**反向**（reverse）程序。其中的项 $\tilde{\mathbf{h}}$ 被称为**伴随**（adjoint），它们（依次）存储了输出相对于程序中变量 $\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_l$ 的所有梯度。[2]

在神经网络术语中，有时会称原始程序（**主程序**，primal program）为**前向传播**（forward pass，不要与前向模式混淆），而反向程序是**反向传播**（backward pass）。与不同的是，在 R-AD 中，整个主程序必须先执行完毕，才能运行反向程序，还需要专门的机制存储所有中间输出，以“展开”计算图。不同的框架对此实现方式不同，接下来将对此进行概述。

从计算角度，R-AD 远比 F-AD 更高效。具体来说，R-AD 步骤 (3) 中的两个操作都是向量-矩阵乘积，计算复杂度仅随所有形状参数线性变化。需要考虑的权衡是，执行 R-AD 需要大量内存空间，因为它必须存储主程序的所有中间值。为了解决这一权衡问题，可以采用一些特定的技术，例如**梯度检查点**（gradient checkpointing），它以计算量增加为代价降低了内存需求。这是通过仅存储少量的中间输出值（称为**检查点**，checkpoint），并在反向传播时重新计算其余的中间值来实现的。可视化示例见图F.6.1。

## 6.4 实践注意事项

### 6.4.1 向量-雅可比乘积

在 R-AD 算法的步骤 (3) 中，可以发现一个有趣的现象：所需的唯一操作是一个行向量 $\mathbf{v}$ 和一个雅可比矩阵 $f$（输入雅可比矩阵或权重雅可比矩阵）相乘。我们将这些操作称为 $f$ 的**向量-雅可比乘积**（vector-Jacobian

[2] 将其与 F-AD 对比，其中切线表示 $\mathbf{h}_i$ 变量相对于权重的梯度。

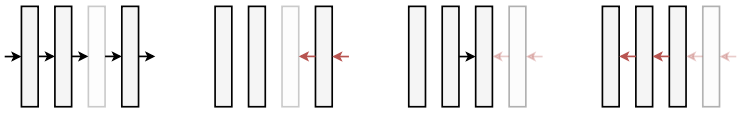



**图 F.6.1: 梯度检查点**的示例。*(a)* 执行了一次前向传播，但仅存储了第 *1*、*2* 和第 *4* 个模块的输出（**检查点**）。*(b)* 反向传播（红色箭头）第三个模块停止，它的激活值不可用。*(c)* 为了重新生成第 *3* 个模块的激活值，从最近的检查点重新执行前向传播。*(d)* 完成反向传播。与标准反向传播相比，这种方法需要 *1.25* 倍的计算量。通常，存储的检查点越少，反向传播的计算成本越高。

products，VJP）。[3] 在下一个定义中，我们将通过对向量添加转置，来恢复维度一致性。

**定义 D.6.2 (向量-雅可比乘积（VJP）)**




给定函数 $\mathbf{y} = f(\mathbf{x})$，其中 $\mathbf{x} \sim (c)$，$\mathbf{y} \sim (c')$，其 *VJP* 是另一个函数，定义为：

$$\text{vjp}_f(\mathbf{v}) = \mathbf{v}^\top \partial f(\mathbf{x}) \tag{E.6.4}$$

其中 $\mathbf{v} \sim (c')$。若 $f$ 有多个参数 $f(\mathbf{x}_1, \dots, \mathbf{x}_n)$，可定义 $n$ 个独立 *VJP*，表示为 $\text{vjp}_{f,\mathbf{x}_1}(\mathbf{v}), \dots, \text{vjp}_{f,\mathbf{x}_n}(\mathbf{v})$。

特别地，在我们的例子中，可以定义两种类型的 VJP，

[3]相反，F-AD 可以用 VJP 的转置来表述，这种操作被称为**雅可比矩阵-向量乘积**（Jacobian-vector product，JVP 对于一维输出，JVP 等价于第2.2节中公式(E.2.20)定义的方向导数。以此类推，VJP 表示与函数输出的无穷小变化相关的线性映射的应用，见[BR24]。

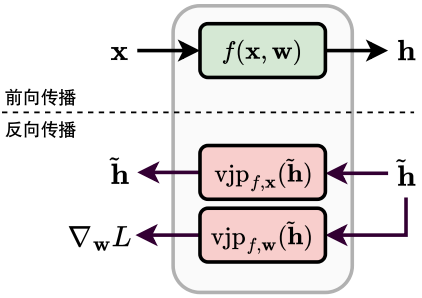



**图 F.6.2:** 为了执行 *R-AD*，必须用两个 *VJP* 运算来扩充原语，以便执行反向传播。两个运算分别对应输入 *VJP*(E.6.5)和权重 *VJP*(E.6.6)。对每个操作调用一次就足以通过原语执行反向传播，对应(E.6.7)-(E.6.8)。

分别对应原语关于输入和权重参数的偏导数：

$$\mathrm{vjp}_{f,\mathbf{x}}(\mathbf{v}) = \mathbf{v}^\top \partial_{\mathbf{x}} f(\mathbf{x}, \mathbf{w}) \tag{E.6.5}$$

$$\mathrm{vjp}_{f,\mathbf{w}}(\mathbf{v}) = \mathbf{v}^\top \partial_{\mathbf{w}} f(\mathbf{x}, \mathbf{w}) \tag{E.6.6}$$

现在可以将 R-AD 算法步骤 (3) 中的两个操作重写为对带有伴随值的原语函数的两次 VJP 调用（为方便阅读，忽略索引 $i$），分别对应伴随值乘以权重 VJP，以及伴随值乘以输入 VJP。

$$\partial_{\mathbf{w}} y = \mathrm{vjp}_{f,\mathbf{w}}\left(\tilde{\mathbf{h}}\right) \tag{E.6.7}$$

$$\tilde{\mathbf{h}} \leftarrow \mathrm{vjp}_{f,\mathbf{h}}\left(\tilde{\mathbf{h}}\right) \tag{E.6.8}$$

因此，我们可以首先选择一组原语操作，然后用对应的 VJP 扩充它们，从而实现一个完整的自动微分系统，无需在内存中显式构造雅可比矩阵。图F.6.2展示了该操作的示意图。

事实上，可以通过使用基向量 $\mathbf{e}_1, \ldots, \mathbf{e}_n$ 重复调用 VJP 来恢复雅可比矩阵的计算，逐行生成雅可比矩阵。例如，

对于输入雅可比矩阵，有：

$$\partial_{\mathbf{x}} f(\mathbf{x}, \mathbf{w}) = \begin{bmatrix} \text{vjp}_{f,\mathbf{x}}(\mathbf{e}_1) \\ \text{vjp}_{f,\mathbf{x}}(\mathbf{e}_2) \\ \vdots \\ \text{vjp}_{f,\mathbf{x}}(\mathbf{e}_n) \end{bmatrix}$$

为了理解这种重新表述的便利性，我们来看一个全连接层的 VJP，它由线性投影和（逐元素）非线性组成。首先，考虑一个没有偏置的简单线性投影：

$$f(\mathbf{x}, \mathbf{W}) = \mathbf{W}\mathbf{x}$$

此处输入雅可比矩阵只是 $\mathbf{W}$，但权重雅可比矩阵是 3 阶张量（见第2.2节）。相比之下，输入 VJP 没有特殊结构：

$$\text{vjp}_{f,\mathbf{x}}(\mathbf{v}) = \mathbf{v}^\top \mathbf{W}^\top = [\mathbf{W}\mathbf{v}]^\top \tag{E.6.9}$$

相反，权重 VJP 是一个简单的外积，完全避免了 3 阶张量：

$$\text{vjp}_{f,\mathbf{w}}(\mathbf{v}) = \mathbf{v}\mathbf{x}^\top \tag{E.6.10}$$

**计算 VJP**

为了计算公式(E.6.10)，可以写成 $y = \mathbf{v}^\top \mathbf{W}\mathbf{x} = \sum_i \sum_j W_{ij} v_i x_j$，由此可以直接得到 $\frac{\partial y}{\partial W_{ij}} = v_i x_j$，这是外积的逐元素定义。

因此，每次在前向传播中应用线性投影时，都会用其权重的转置来修改反向传播的梯度，并且执行外积来计算 $\mathbf{W}$ 的梯度。

现在，考虑一个没有可训练参数的逐元素激活函数，例如 ReLU：

$$f(\mathbf{x}, \{\}) = \phi(\mathbf{x})$$

由于没有可训练的参数，我们只需要考虑输入 VJP。梯度是一个对角矩阵，其对角线元素为 $\phi$ 的导数：

$$[\partial_{\mathbf{x}}\phi(\mathbf{x})]_{ii} = \phi'(x_i)$$

输入 VJP 是一个对角矩阵与一个向量的积，相当于阿达玛积（即，缩放操作）：

$$\text{vjp}_{\mathbf{x}}(f, \mathbf{v}) = \mathbf{v} \odot \phi'(\mathbf{x}) \tag{E.6.11}$$

有趣的是，在这种情况下，我们也可以直接计算 VJP，无需显式构造完整的对角矩阵。

### 6.4.2 实现 R-AD 系统

实现 R-AD 系统的方法有多种，范围从 Wengert 列表（如 TensorFlow 的实现方式）到源到源代码转换（source-to-source code transformation）不一而足 [GW08]。在本节中，我们将简要讨论一些现有框架中常见的实现方法。

首先，按照 JAX 等框架中 “一切皆为函数” 的风格，将原语描述为具有两个参数的函数 $f(\mathbf{x}, \mathbf{w})$。考虑一个函数 $f(\mathbf{x})$，其中输入为 $c$ 维，输出为 $c'$ 维。从这个角度来看，VJP 可以实现为一个更高阶函数，其具有以下函数签名（signature）：

$$(\mathbb{R}^c \to \mathbb{R}^{c'}) \to \mathbb{R}^c \to (\mathbb{R}^{c'} \to \mathbb{R}^c) \tag{E.6.12}$$

即，给定一个函数 $f$ 和输入 $\mathbf{x}'$，VJP 返回另一个函数，该函数可以应用于一个 $c'$ 维向量 $\mathbf{v}$，返回 $\mathbf{v}^\top \partial f(\mathbf{x}')$。类似地，一维函数的梯度可以实现为另一个高阶函数，具有以下函数签名：

$$(\mathbb{R}^c \to \mathbb{R}) \to (\mathbb{R}^c \to \mathbb{R}^c) \tag{E.6.13}$$

它以函数 $f(\mathbf{x})$ 作为输入，并返回另一个计算 $\nabla f(\mathbf{x})$ 的

```
# 原始函数（平方和公式）
def f(x: Float[Array, "c"]):
  return (x**2).sum()

grad_f = func.grad(f)
print(grad_f(torch.randn(10)).shape)
# [输出]: torch.Size([10])
```

**框 C.6.2:** 梯度计算作为高阶函数。*torch.func*接口模仿了*JAX* 的 *API*。在实践中，可以对函数进行追踪（例如使用*torch.compile*）生成优化的计算图。

函数。在 JAX 中，这些思想分别通过函数jax.grad和jax.jvp实现，PyTorch 的torch.func模块也复制了这种实现——见框C.6.2中的示例[4]

正如前文提到的，在实践中，模型是以对象的组合形式实现的，这些对象的参数被封装为属性（见框C.5.1）。一种可能的做法是“纯化”对象，将其转换为纯函数，例如：[5]

```
# 提取参数
params = dict(model.named_parameters())
# 调用模型的前向函数
y = torch.func.functional_call(
                model, params, x
                )
```

更一般地，像 PyTorch 这样的框架增强了相应技术来直接处理这种情况，而无需引入中间操作。例如，在 PyTorch 中，张量对象被附加有关生成它们的操作的信息（图F.6.3，左）。每当对一个标量值请求backward()调

[4]诸如计算黑塞矩阵等许多运算，都可以根据其函数签名，巧妙地组合 JVP 和 VJP 来实现：https://jax.readthedocs.io/en/latest/notebooks/autodiff_cookbook.html。

[5]https://sjmielke.com/jax-purify.htm

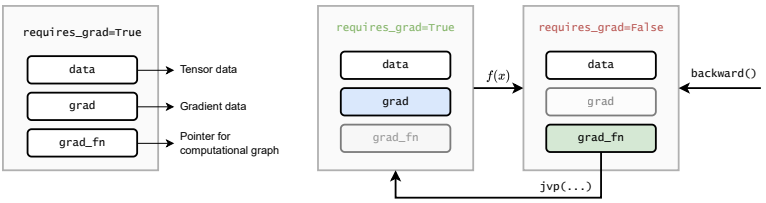



**图 F.6.3:** *左图：在 PyTorch 中，一个张量被增强了关于其梯度（初始化时为空）以及创建它的操作的信息。右图：在反向传播过程中，`grad_fn`属性被用于反向遍历计算图，并且当`requires_grad`属性被显式设置为`True`时，梯度会被存储在该张量的`grad`属性内部（以避免内存的不必要消耗）。*

用时，这些附加的信息会被用来反向遍历计算图，并将相应的梯度存储在需要它们的张量内部（图F.6.3，右）。

这只是对这些系统在实践中的实现方法的高层次概述，我们省略了许多细节，对此建议读者参考官方文档。[6]

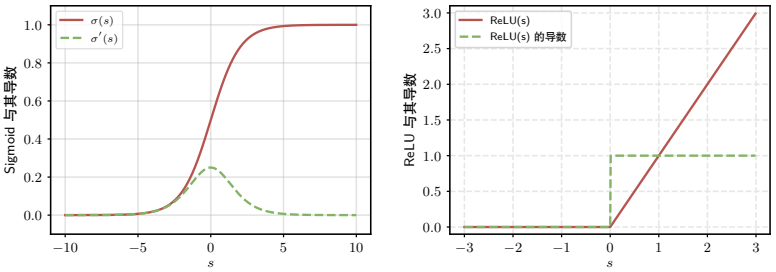
### 6.4.3 选择激活函数

恰好，我们现在可以解释为什么 ReLU 是激活函数的良好选择。仔细观察 (E.6.11)式可以发现，每次在模型中添加一个激活函数时，反向传播中的伴随值都会乘以一个因子 $\phi'(\mathbf{x})$。对于多层模型，这可能会导致两种病态行为：

1. 如果 $\phi'(\cdot) < 1$ 处处成立，则存在梯度随着层数以指数速度迅速缩小至 0 的风险。这被称为**梯度消失**（vanishing gradient）问题。

2. 相反，如果 $\phi'(\cdot) > 1$ 处处成立，则会出现相反的问题，梯度会随着层数以指数形式趋于无穷大。这被称为**梯度爆炸**（exploding gradient）问题。

[6]我强烈建议尝试从头开始实现 R-AD 系统：网上可以找到许多实现教程，例如：https://github.com/karpathy/micrograd.




**(a)** *sigmoid* **(b)** *ReLU*

**图 F.6.4:** *(a)sigmoid* 函数（红色）与其导数（绿色）。*(b)ReLU* 函数（红色）与其导数（绿色）。

这些在实践中是严重的问题，因为库表示浮点数的精度有限（通常为 32 位或更低）。这意味着当增加层数时，下溢或溢出可能会迅速出现。

### 线性的非线性模型

令人惊讶的是，以浮点精度实现的线性层堆叠并非完全线性，因为在机器精度级别上存在微小的非连续性！这通常不构成问题，但可以利用它来训练完全线性的深度神经网络。[a]

[a] https://openai.com/research/nonlinear-computation-in-deep-linear-networks

作为梯度消失如何产生的一个示例，考虑 sigmoid 函数 $\sigma(s)$。前文已经提到，由于它是阶跃函数的一种软逼近，因此它曾是常用的激活函数。我们也知道 $\sigma'(s) = \sigma(s)(1 - \sigma(s))$。与 $\sigma(s) \in [0, 1]$ 的事实相结合，可得：

$$\sigma'(s) \in [0, 0.25]$$

因此，sigmoid 函数是梯度消失问题的主要受害者：见图F.6.4a。

设计一种既不会出现梯度消失也不会出现梯度爆炸的激活函数并非易事，因为唯一满足 $\phi'(s) = 1$ 处处成立的函数是恒等函数。因此，我们需要一个“足够线性”以避免梯度问题，但又“足够非线性”以分离线性层的函数。最终 ReLU 成为一个不错的候选者，因为：

$$\partial_s \mathrm{ReLU}(s) = \begin{cases} 0 & s < 0 \\ 1 & s > 0 \end{cases}$$

梯度要么被置零，从而在计算中引入稀疏性，要么乘以 1，以避免缩放问题——见图F.6.4b。

作为补充说明，无论我们是否用 ReLU 层的输出替换其输入，ReLU 的梯度都是相同的（因为其只屏蔽负值，而保持正值不变）。因此，使用 ReLU 作为激活函数的另一个好处是，在执行 R-AD 时，我们可以通过在前向传播中覆盖层的输入来节省少量内存，而不影响自动微分过程的正确性：例如，在 PyTorch 中，通过设置`in_place`参数来实现。[7]

### 6.4.4 次可微性和自动微分




有一个我们一直避免讨论的小细节：ReLU 在 0 处不可微，，这使整个网络变得非光滑。这种情况下会发生什么？“实用主义”的回答是，通过从随机（非零）初始化开始使用随机梯度下降进行最小化，，最终恰好落在 $s = 0$ 的概率几乎为零，而对于任意 $|\varepsilon| > 0$，$\mathrm{ReLU}(\varepsilon)$ 的梯度都有定义。

为了给出更具技术性的回答，我们可以引入函数**次梯度**（subgradient）的概念。

[7] https://pytorch.org/docs/stable/generated/torch.nn.ReLU.html

**定义 D.6.3 (次梯度)**

给定一个凸函数 $f(x)$，在 $x$ 处的一个次梯度是一个点 $z$，使得对于所有 $y$：

$$f(y) \geq f(x) + z(y - x)$$

请注意其与凸性定义的相似之处：次梯度是与 $f(x)$ "相切" 的一条直线的斜率，使得整个 $f$ 以该直线为下界。如果 $f$ 在 $x$ 处可微，则只有一条这样的直线存在，为 $f$ 在 $x$ 处的导数。在非光滑点（non-smooth point）处，，存在多个次梯度，它们构成一个集合，称为 $f$ 在 $x$ 处的**次微分**（subdifferential）：

$$\partial_x f(x) = \{z \,|\, z为f(x)的一个次梯度\}$$

有了这个定义，我们可以用次微分替换 0 处的梯度，来完成对 ReLU 梯度的分析：

$$\partial_s \mathrm{ReLU}(s) = \begin{cases} \{0\} & s < 0 \\ \{1\} & s > 0 \\ [0,1] & s = 0 \end{cases}$$

因此，$[0,1]$ 中的任何值都是 0 处的有效次梯度，实践中的大多数实现倾向于 $\mathrm{ReLU}'(0) = 0$。在迭代下降过程的每一步选择次梯度的过程称为**次梯度下降**（subgradient descent）。

事实上，情况甚至更复杂，因为对于非凸函数，次梯度不一定是定义明确的。在这种情况下，可以采用一些广义方法，将之前的定义放宽到 $x$ 的局部邻域，例如克拉克次微分。[8]次可微性也可能在自动微分中造成问题，其

[8] https://en.wikipedia.org/wiki/Clarke_generalized_derivative

中同一函数的不同实现可能会提供不同（可能无效）的次梯度，并且对于正式的证明，必须考虑更精细的链式法则概念 [KL18, BP20]。[9]

[9]考虑复现自 [BP20]的示例：定义两个函数，$\mathrm{ReLU}_2(s) = \mathrm{ReLU}(-s)+s$ 和 $\mathrm{ReLU}_3(s) = 0.5(\mathrm{ReLU}(s)+\mathrm{ReLU}_2(s))$。它们都等价于 ReLU，但在 PyTorch 中，在 0 处进行反向传播时，ReLU 返回 0.0，$\mathrm{ReLU}_2$ 返回 1.0，$\mathrm{ReLU}_3$ 返回 0.5。

## 从理论到实践



如果您完成了第5章的练习，您就已经看到了 R-AD 在 PyTorch 和 JAX 中的应用，而本章（尤其是6.4.2节）应该已经阐明了它们的实现。

尝试重新实现一个类似 PyTorch 的简单的 R-AD 系统是一个好主意。例如，如果专注于标量值，`micrograd` 库[10]就是一个非常好的教学实现。我们唯一没有涵盖的细节是，一旦扩展到一般的无环图，在反向传播之前，计算图变量的排序至关重要，以避免创建错误的反向传播路径。在 micrograd 中，这是通过对变量进行简便的拓扑排序来实现的。

另一个有趣的主意是尝试在 PyTorch 中实现一个新的原语（就本章而言），这需要指定其前向传播和 VJP。[11]相应例子可以参考第5.4节中的任一可训练函数。这是一项教学练习，可以通过子类化`nn.Module`并让 PyTorch 的自动微分引擎计算反向传播，来实现等价的功能。

以下所有步骤也可以在 JAX 中复现：

- 使用 `autodidax` 实现一个教学版本的 JAX。[12]
- 通过实现相应的 VJP，来编写一个新的原语。[13]
- 阅读 **JAX 自动微分**手册[14]，以了解自动微分引擎的高级用例，例如高阶导数、黑塞矩阵等。

[10] https://github.com/karpathy/micrograd

[11] https://pytorch.org/docs/master/notes/extending.html

[12] https://jax.readthedocs.io/en/latest/autodidax.html

[13] https://jax.readthedocs.io/en/latest/notebooks/Custom_derivative_rules_for_Python_code.html

[14] https://jax.readthedocs.io/en/latest/notebooks/autodiff_cookbook.html

# Part II
# 陌生的土地



“越变越希奇了，
越变越古怪了！”
（阿丽思诧异得
连话都说不好了）

**—第二章，眼泪池**

# 7 | 卷积层

**本章导读**

本章将介绍我们的第二个核心层——**卷积层**（convolutional layer）。它旨在通过利用我们称为*局部性*（locality）和*参数共享*（parameter sharing）的两个关键思想来处理图像（或更广义地说，任何类型的序列数据）。

全连接层在历史有重要意义，但从实践角度看，其重要性有所降低：在非结构化数据（我们称为**表格**数据，tabular data，因为它容易以表格形式表示）上，MLP 的性能通常不如其他方案，例如随机森林（random forest）或经过良好调优的支持向量机 [GOV22]。然而，一旦我们考虑其他类型的数据（这些数据具有一些可以在层和模型设计中加以利用的结构），情况就并非如此了。

本章将考虑图像领域，而在后续章节中，我们还将考虑时间序列、音频、图和视频的应用。在以上情况下，输入都具有序列结构（时间上的、空间上的或其他类型的），可以利用这种结构来设计性能良好、易于组合且参数高效的层。

# 7.1 走进卷积层

## 7.1.1 全连接层还不够

图像可以通过张量 $X \sim (h, w, c)$ 描述，其中 $h$ 为图像的高度，$w$ 为图像宽度，$c$ 为通道数（对于黑白图片，$c$ 为 1，对于彩色图片为 3，对于高光谱图像，可能更高）。因此，一小批量图像通常为 4 阶，具有一个额外的前导批次维度 $(b, h, w, c)$。这三个维度并不相同，因为 $h$ 和 $w$ 表示像素（pixel）的空间排列。由于以 RGB 或 GBR 格式存储图像只是一个惯例问题，通道 $c$ 并没有特定的顺序。

**符号、通道和特征**

我们使用与表格数据特征相同的符号（$c$），因为它将在模型设计中扮演类似的角色，即，我们可以将认为，每个像素由一组通用的 $c$ 特征描述，特征由模型的各层并行更新。因此，卷积层将返回一个通用张量 $(h, w, c')$，其中每个 $hw$ 像素的嵌入大小为 $c'$。

为了使用全连接层，我们需要“扁平化”（向量化）图像：

$$\mathbf{h} = \phi(\mathbf{W} \cdot \underbrace{\text{vect}(X)}_{\text{扁平化图像}}) \tag{E.7.1}$$

其中 $\text{vect}(x)$ 等价与 PyTorch 的`x.reshape(-1)`。对于一个通用的 $n$ 阶张量 $x \sim (i_1, i_2, \ldots, i_n)$，该操作返回一个等价的张量 $\mathbf{x} \sim \left(\prod_{j=1}^{n} i_j\right)$。

显然，这并非一种优雅的方法，但仍有必要强调其一些缺点。首先，我们丢失了前一节中一个非常重要的性质，即**可组合性**（composability）：我们的输入为图像，而输

出是向量，这意味着我们无法将其中两层连接起来。可以通过将输出向量重塑（reshape）为图像来弥补这一点：

$$H = \text{unvect}(\phi(\mathbf{W} \cdot \text{vect}(X))) \tag{E.7.2}$$

假设该层不改变像素数量，而 unvect 操作将输出重塑为 $(h, w, c')$ 形状的张量，其中 $c'$ 是一个超参数。

这直接导致了第二个问题，该层具有大量的参数。例如，考虑一幅 (1024, 1024) 的 RGB 图像，若保持输出维度相同，则会导致参数量为 $(1024 * 1024 * 3)^2$（或更一般地，$(hw)^2cc'$ 个参数），其数量级为 $10^{13}$！可以将前一层解释如下：对于每个像素，输出中的每个通道都是输入图像中所有像素的 所有通道的加权组合。正如我们即将看到的，通过约束此计算，可以获得更高效的解决方案。

**关于重塑的更多内容**

为了扁平化（或更广义地，重塑）一个张量，我们需要确定处理值的顺序。在实践中，这由张量在内存中的存储方式决定：在大多数框架中，张量数据以连续的内存块按顺序存储，这被称为**跨步布局**（strided layout）。考虑以下示例：

```
torch.randn(32, 32, 3).stride()     # (96, 3, 1)
```

跨距是指在内存中沿某个轴移动一个位置所需的步数，即张量的最后一个维度是连续存储的，而要在第一维中移动一个位置，需要 96 $(32 * 3)$ 步。这被称为**行优先**（row-major）顺序，或在图像分析中称为**光栅**（raster）顺序。[a] 每个重塑操作都沿着这种跨距表示进行。

[a] https://en.wikipedia.org/wiki/Raster_scan

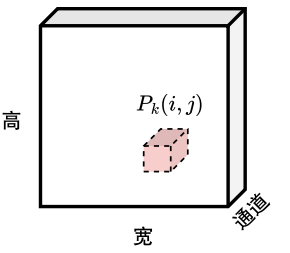



**图 F.7.1:** 给定张量 $(h, w, c)$ 和最大距离 $k$，**图像块**$P_k(i,j)$（红色）是一个形状为 $(2k+1, 2k+1, c)$ 的张量，收集了所有与位于 $(i,j)$ 的像素距离不超过 $k$ 的像素。

为使后续内容更易于可视化，考虑一个 1 维序列（我们将在后续更深入地探讨 1 维序列；目前，可以将其视为“具有单通道的 *4* 个像素”）：

$$\mathbf{x} = [x_1, x_2, x_3, x_4]$$

在这种情况下，不需要任何重塑操作，并且前一层（其中 $c' = 1$）可以写为：

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \end{bmatrix} = \begin{bmatrix} W_{11} & W_{12} & W_{13} & W_{14} \\ W_{21} & W_{22} & W_{23} & W_{24} \\ W_{31} & W_{32} & W_{33} & W_{34} \\ W_{41} & W_{42} & W_{43} & W_{44} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

### 7.1.2 局部层

像素的空间排列在像素之间引入了一种度量（距离）。虽然存在许多有效的“距离”概念，但我们发现使用以下定义更为方便，该定义将像素 $(i,j)$ 和 $(i',j')$ 之间的距离定义为两个轴上的最大距离：

$$d((i,j),(i',j')) = \max(|i-i'|, |j-j'|) \qquad \text{(E.7.3)}$$

如何在层的定义中利用这一思想？理想情况下，可以设想一个像素对另一个像素的影响随着它们之间距离增加

而成反比递减。将这一思想推向极致，我们可以假设对于大于某个阈值的距离，影响实际上为零。为了形式化这一发现，我们引入**图像块**（patch）的概念。




**定义 D.7.1 (图像块)**

给定图像 $X$，则**图像块** $P_k(i,j)$ 定义为以 $(i,j)$ 为中心，且包括所有与中心距离小于等于 $k$ 的像素的子图像：

$$P_k(i,j) = [X]_{i-k:i+k,j-k:j+k,:}$$

其中，距离的定义如公式(E.7.3)所示。图*F.7.1*中以可视化方式展示。

此定义仅对与图像边界距离至少为 $k$ 的像素有效：我们暂且忽略这一点，将在后续内容中讨论。每个图像块的形状为 $(s,s,c)$，其中 $s = 2k+1$，因为我们考虑了中心像素以及沿每个方向的 $k$ 个像素。出于将在后文讲解的缘由，我们将 $s$ 称为**滤波器尺寸**（filter size）或**核尺寸**(kernel size)。

考虑一个通用层 $H = f(X)$，其输入是形状为 $(h,w,c)$ 的张量，返回一个形状为 $(h,w,c')$ 的张量。如果给定像素的输出仅依赖于一个预先设定尺寸的图像块，则称该层是**局部的**（local）。

**定义 D.7.2 (局部层)**

给定输入图像 $X \sim (h,w,c)$，如果存在满足以下条件的 $k$，则 $f(X) \sim (h,w,c')$ 是**局部的**：

$$[f(X)]_{ij} = f(P_k(i,j))$$

此条件必须对图像中的所有像素均成立。

通过将所有属于每个像素影响区域（**感受野**）之外的像素的权重设为 0，可以将层(E.7.1)转换为一个局部层：

扁平化的图像块（形状为 $s^2c$）

$$H_{ij} = \phi\left(\mathbf{W}_{ij} \cdot \text{vect}(P_k(i,j))\right)$$

位置相关的权重矩阵

我们称此类层为**局部连接**（locally-connected）层。请注意，对于每个输出像素，我们都有一个不同的权重矩阵 $\mathbf{W}_{ij} \sim (c', ssc)$，产生了 $hw(s^2cc')$ 个参数。相比之下，在初始层中有 $(hw)^2cc'$ 个参数，参数量减少了 $\frac{s^2}{hw}$ 倍。

考虑前述简单示例，假设 $k = 1$ （因此 $s = 3$），我们可以将所得运算写成：

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \end{bmatrix} = \begin{bmatrix} W_{12} & W_{13} & 0 & 0 \\ W_{21} & W_{22} & W_{23} & 0 \\ 0 & W_{31} & W_{32} & W_{33} \\ 0 & 0 & W_{41} & W_{42} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

我们的运算对于 $x_1$ 和 $x_4$ 没有定义，在这种情况下，我们考虑通过移除与未定义运算对应的权重，来“缩短”滤波器。等价地，可以将其视为在必要时于边界添加 0：

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \end{bmatrix} = \begin{bmatrix} W_{11} & W_{12} & W_{13} & 0 & 0 & 0 \\ 0 & W_{21} & W_{22} & W_{23} & 0 & 0 \\ 0 & 0 & W_{31} & W_{32} & W_{33} & 0 \\ 0 & 0 & 0 & W_{41} & W_{42} & W_{43} \end{bmatrix} \begin{bmatrix} 0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ 0 \end{bmatrix}$$

这种技术被称为**零填充**（zero-padding）。在一幅图像中，对于尺寸为 $2k+1$ 的核，需要在每一侧精确地填充 $k$ 行和 $k$ 列的 0，以确保运算对每个像素都有效。否则，输出无法在边界附近计算，并且输出张量的形状将 $(h-2k, w-2k, c')$。这两种方法在大多数框架中都是有效的选择。

**图像块定义的补充说明**

使用图像块的概念来定义卷积有些非常规，但我发现它极大地简化了符号表示。我将在后文提供一个更传统的、面向信号处理的定义。这两个定义是等价的，可以互换使用。面向图像块的定义要求核尺寸为**奇数**，不允许**偶数**核尺寸，但偶数核尺寸在实践中并不常见。

### 7.1.3 平移等变性和卷积层




在局部连接层中，两个相同的图像块可能由于其位置不同而产生不同的输出：例如，像素 $(5, 2)$ 上的某些内容将与像素 $(39, 81)$ 上的相同内容以不同方式进行处理，因为两个矩阵 $\mathbf{W}_{5,2}$ 和 $\mathbf{W}_{39,81}$ 是不同的。然而，在大多数情况下，我们可以假设此信息无关紧要：通俗地说，一匹马无论出现在图像中的什么位置，它还是马。我们可以使用**平移等变性**（translation equivariance）这一性质来形式化该概念。

**定义 D.7.3 (平移等变性)**

如果输入发生平移，且输出发生等价平移，则称层 $H = f(X)$ 是**平移等变**的：

相同图像块

$$P_k(i,j) = P_k(i',j') \quad \text{意味着} \quad f(P_k(i,j)) = f(P_k(i',j'))$$

相同输出

为了理解这一命名，可将上述定义阐释如下：若图像中的某一物体从位置 $(i,j)$ 平移至 $(i',j')$，则原先在 $(i,j)$ 得到的输出 $f(P_k(i,j))$ 将相应地出现在 $f(P_k(i',j'))$ 处。换言之，网络层的激活值随着输入进行等量的（拉丁文 *èqui*，意为相等）平移运动。后续将对等变性和不变性进行更严格的定义。

实现平移等变性的一种简单方法是**权重共享**（weight sharing），即让每个位置共享同一组权重：

$$H_{ij} = \phi(\ \mathbf{W}\ \cdot \mathrm{vect}(P_k(i,j)))$$

独立于 $(i,j)$ 的权重矩阵

这被称为**卷积层**（convolutional layer），其显著优势在于参数效率极高我们仅需一个形状为 $(c', ssc)$ 的权重矩阵 $\mathbf{W}$，且该矩阵规模与原始图像的分辨率无关（再次与参数量为 $hw(s^2c'c)$ 的局部连接层比较：参数数量减少了 $\frac{1}{hw}$ 倍）。以偏置向量 $\mathbf{b} \sim (c')$ 的形式添加 $c'$ 个额外参数，可以得到带有偏置项的变体。鉴于其重要性，下文将重述该层的完整定义。

```python
from torch.nn import functional as F
x = torch.randn(16, 3, 32, 32)
w = torch.randn(64, 3, 5, 5)
F.conv2d(x, w, padding='same').shape
# [输出]: torch.Size([16, 64, 32, 32])
```

**框 C.7.1:** *PyTorch* 中的卷积运算。请注意，在默认情况下通道维度是批次维度之后的第一个参数。卷积核矩阵以 $(c', c, k, k)$ 张量的形式组织。填充方式可指定为整数或字符串（*'same'* 表示输出张量须与输入张量具有相同的维度，*'valid'* 则表示不进行填充）。

**定义 D.7.4 (卷积层)**




给定图像 $X \sim (h, w, c)$ 与核尺寸 $s = 2k + 1$，**卷积层** $H = \text{Conv2D}(X)$ 的逐元素定义如下：

$$H_{ij} = \mathbf{W} \cdot \text{vect}(P_k(i, j)) + \mathbf{b} \tag{E.7.4}$$

可训练参数为 $\mathbf{W} \sim (c', ssc)$ 和 $\mathbf{b} \sim (c')$。超参数为 $k$，$c'$，以及（最终）是否应用零填充。在前一种情况下，输出形状为 $(h, w, c')$；在后一种情况下，输出形状为 $(h - 2k, w - 2k, c')$。

请参阅框C.7.1中的示例代码。等价的面向对象实现可在`torch.nn.Conv2D`模块中找到。相较而言，我们此前的示例可进行如下完善：

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \end{bmatrix} = \begin{bmatrix} W_2 & W_3 & 0 & 0 \\ W_1 & W_2 & W_3 & 0 \\ 0 & W_1 & W_2 & W_3 \\ 0 & 0 & W_1 & W_2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \tag{E.7.5}$$

其中，目前只有三个权重 $\mathbf{W} = [W_1, W_2, W_3]^\top$（零填充版本与之前等价，为简洁起见，此处省略）。此权重矩阵具有特殊的结构，即沿任何对角线的元素都是常数（例如，主对角线只由 $W_2$ 构成）。我们称这些矩阵为**托普利兹矩阵**（Toeplitz matrices），[1]，它们是在现代硬件上正确实现卷积层的基础。托普利兹矩阵是**结构化**密集矩阵的一个示例 [QPF+24]。公式(E.7.5)还应阐明，与全连接运算相比，卷积仍为一种线性运算，但其权重矩阵受到更强的约束。

## 卷积及其相关术语




我们的术语（主要）来自信号处理领域，可以通过以更标准的形式重写卷积层的输出来理解这一点。为此，首先将权重矩阵重新排列成一个等价的权重张量 $W$，形状为 $(s, s, c, c')$，类似于框C.7.1中的 PyTorch 实现。为便于后续讨论，定义如下函数，该函数将区间 $[1, \dots, 2k+1]$ 内的整数 $i'$ 转换到 $[i-k, \dots, i+k]$：

$$t(i) = i - k - 1 \tag{E.7.6}$$

其中，$k$ 在 $t(\bullet)$ 的参数中被隐式包含。接下来，将使用跨轴的显式求和重写该层的输出：

$$H_{ijz} = \sum_{i'=1}^{2k+1} \sum_{j'=1}^{2k+1} \sum_{d=1}^{c} [W]_{i',j',z,d} [X]_{i'+t(i),j'+t(j),d} \tag{E.7.7}$$

请仔细检查索引：对于给定像素 $(i, j)$ 和输出通道 $z$（一个从 1 到 $c'$ 的自由索引），在空间维度上，$W$ 的索引必须沿着 $1, 2, \dots, 2k+1$ 进行，而 $X$ 的索引必须沿着 $i-k, i-k+1, \dots, i+k-1, i+k$ 进行。索引 $d$ 则遍历输入通道。

[1] https://en.wikipedia.org/wiki/Toeplitz_matrix

从信号处理的角度来看，公式(E.7.7)对应于通过一组**有限冲激响应**（finite impulse response）滤波器 [Unc15]对输入信号 $X$ 进行的滤波操作，该操作通过离散卷积实现（除符号变化外）。每个滤波器对应于权重矩阵的一个切片 $W_{:,:,:,i}$。在标准信号处理中，这些滤波器可以手动设计以对图像执行特定的操作。例如，一个用于检测脊线的 $3 \times 3$ 可以被写为：[2]

$$W = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

与传统方法不同，在卷积层中，此类滤波器可进行随机初始化，并通过梯度下降法进行训练。下一节将探讨基于卷积层构建**卷积模型**的设计。卷积层的一个有趣之处在于：其输出结果保持某种“空间一致性”，并可将其可视化。我们将输出的一个切片 $H_{:,:,i}$ 定义为该层的**激活图**（activation map），其代表了特定滤波器在各输入区域的“激活”强度。关于激活图的深入分析将在下一卷中展开。

## 7.2 卷积模型

### 7.2.1 设计卷积“块”

掌握了卷积层的定义后，我们现在转向构建**卷积模型**（convolutional model），此类模型也称为**卷积神经网络**（convolutional neural network，CNN）。以图像分类问题为例讨论，但我们阐述的许多内容可以推广到其他应用场景。首先，我们对**感受野**（receptive field）的概念进行形式化定义。

[2] https://en.wikipedia.org/wiki/Kernel_(image_processing)

**定义 D.7.5 (感受野)**

以 $X$ 表示图像，并以 $H = g(X)$ 表示卷积模型的通用中间输出，例如，应用一个或多个卷积层后的结果。像素 $(i, j)$ 的**感受野**$R(i, j)$ 是 $X$ 中对其计算有贡献的子集：

$$[g(X)]_{ij} = g(R(i, j)), \quad R(i, j) \subseteq X$$

对于单个卷积层，一个像素的感受野等于一个图像块：$R(i, j) = P_k(i, j)$。而，不难证明，对于两个具有相同卷积核尺寸的连续卷积层，其最终得到的感受野为 $R(i, j) = P_{2k}(i, j)$，而对于三个卷积层，则为 $P_{3k}(i, j)$，以此类推。因此，感受野随着卷积层数量增加呈线性增长。这正是我们提出局部性概念的动机：即使单层卷积的感受野受限于卷积核尺寸，通过堆叠足够多的卷积层，可以获得全局感受野。

现在考虑两个串行连接的卷积层：

$$H = \text{Conv}(\text{Conv}(X))$$

由于卷积是线性运算（见前一节所述），因此，连续进行两次卷积运算等价于使用更大的核尺寸进行一次卷积运算（如上文所述）。类似于处理全连接层的方法，通过在卷积层之间引入激活函数，可以避免“坍缩”现象的发生：

$$H = (\phi \circ \text{Conv} \circ ... \circ \phi \circ \text{Conv})(X) \tag{E.7.8}$$

为了推进我们的设计，我们注意到，在公式(E.7.8)中，通道维度会受到每个卷积层的影响而发生改变，而空间维度将保持相同的形状（如果不使用零填充，则会略微缩小）。然而，对于诸如图像分类等任务而言，在实际应用中，空间维度的降低可能是有益的。

考虑一匹马在两幅不同图像的不同区域中出现的例子。卷积层的平移等变性确保了在第一幅图像的区域 1 中检测到的任何特征，均会在第二幅图像的相应区域 2 中被检测到。然而，若目标为“马匹分类”，则最终需要激活一个或多个神经元，以响应图像中“马”的存在，且不受其在图像中具体位置的影响：仅考虑平移变换时，此性质被称为**平移不变性**（translation invariance）。

许多在空间维度上执行缩减的操作都自然地具备平移不变性，例如：

$$H' = \sum_{i,j} H_{ij} \quad \text{or} \quad H' = \max_{i,j}(H_{ij})$$

在 CNN 中，这被称为**全局池化**（global pooling）。然而，它会破坏图像中存在的所有空间信息。我们可以通过部分缩减获得稍微更有效的解决方案，这被称为**最大池化**（max-pooling）。

**定义 D.7.6 (最大池化层)**




给定一个张量 $X \sim (h, w, c)$，一个最大池化层，记作 $MaxPool(X) \sim (\frac{h}{2}, \frac{w}{2}, c)$，其逐元素定义如下：

$$[\text{MaxPool}(X)]_{ijc} = \max\left([X]_{2i-1:2i,2j-1:2j,c}\right)$$

$2 \times 2$ 个图像块

因此，取输入的 $2 \times 2$ 窗口，并独立计算每个通道的最大值（这可以很容易地推广到更大的窗口）。最大池化有效地将空间分辨率减半，同时保持通道数不变，示例见图 F.7.2。

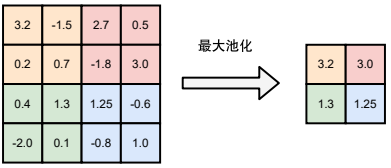



**图 F.7.2:** 对一个 *(4,4,1)* 图像进行 *2x2* 最大池化的可视化。对于多通道情况，该操作将独立应用于每个通道。

通过堆叠数个卷积层，并辅以一次最大池化操作，即可构建一个卷积“块”（见图F.7.3）：

$$\text{ConvBlock}(X) = (\text{MaxPool} \circ \phi \circ \text{Conv} \circ ... \circ \phi \circ \text{Conv})(X)$$

以此类推，我们通过堆叠多个此类模块来定义更复杂的网络：

$$H = (\text{ConvBlock} \circ \text{ConvBlock} \circ ... \circ \text{ConvBlock})(X) \tag{E.7.9}$$

这种设计方案具有大量的超参数，例如各层的输出通道数、各层的卷积核尺寸等。为有效缩减设计过程中的搜索空间，通常需要进行一定的简化假设。例如，VGG 设计 [SLJ+15]设计推广了一种策略：在各层中保持滤波器尺寸不变（如 $k = 3$），同时保持每个模块内部通道数不变，并在相邻模块之间将通道数翻倍。

降低维度的另一种方法是对卷积层的输出进行下采样：此操作称为卷积的**步幅**（stride）。例如，步幅为 1 的卷积为标准的卷积运算；步幅为 2 的卷积每隔 2 个像素计算一个输出；步幅为 3 的卷积每隔 3 个像素计算一个输出，以此类推。较大的步幅值可与最大池化相结合，具体取决于模型的整体架构设计。

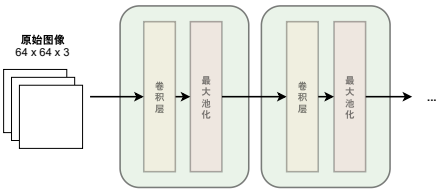



**图 F.7.3:** 将“层”抽象为“块”来简化可微模型的设计。

**不变性和等变性**

非正式地，如果 $T$ 是某个集合（例如，所有可能的平移）上 $x$ 的一种变换，若 $f(Tx) = Tf(x)$，则称函数 $f$ 具有等变性；若 $f(Tx) = f(x)$，则函数具有不变性。所有变换构成的空间组成一个群[BBL+17]，与特定变换相对应的矩阵被称为该群的一个**表示**（representation）。卷积层通过其内在设计，（大致）具备对平移变换的等变性，然而，针对更为广义的对称性，可以采用其他策略，例如对群元素进行平均化处理（帧平均，frame averaging，[PABH+21]）。其他类型层的等变性将在第10章和第12章中进行阐述。

### 7.2.2 设计完整模型

现在，我们可以完成模型的设计。通过如公式(E.7.9)的方法堆叠多个卷积块，输出 $H$ 的形状为 $(h', w', c')$，其中 $w'$ 和 $h'$ 取决于最大池化操作的次数（或卷积层的步幅），而 $c'$ 仅取决于序列中最后一个卷积层的超参数。请注意，每个元素 $H_{ij}$ 将对应于原始图像中的一个“宏区域”，例如，当 $h', w' = 2$ 时，$H_{11}$ 对应于原始图像的“左上”象限。我们可以在分类操作之前执行一次全局池化操作，来消除这种空间依赖性。

完整的模型可以分解为三个主要组成部分：一系列卷积

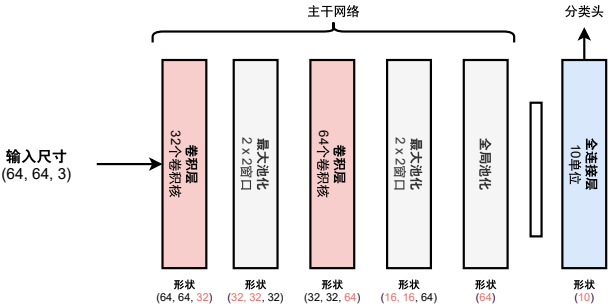



**图 F.7.4:** 用于图像分类的简单 *CNN* 的完整设计示例（假设有 *10* 个输出类别）。每层的底部展示了输出维度。全局池化操作可由扁平化操作替代。分类器之前的最后一个（**隐性**）表示在微调大规模预训练模型时非常有用——它是第*3.1.1*节意义上的图像**嵌入**。

块、全局平均池化以及用于分类的最终模块。

$$H = (\text{ConvBlock} \circ \ldots \circ \text{ConvBlock})(X) \tag{E.7.10}$$

$$\mathbf{h} = \frac{1}{h'w'} \sum_{i,j} H_{ij} \tag{E.7.11}$$

$$y = \text{MLP}(\mathbf{h}) \tag{E.7.12}$$

其中，$\text{MLP}(\mathbf{h})$ 表示一个通用的全连接层序列（也可以使用扁平化操作来代替全局池化）。这是一个典型的 CNN 示例。完整示例可参考图F.7.4。

该设计有如下几个值得注意的性质：

1. 它可以像第4章和第5章中的模型一样训练。例如，对于分类任务，我们可以将输出包装在一个 softmax 函数中，并通过最小化交叉熵来训练所有参数。第6章中描述的反向传播的相同规则也适用于此处。

2. 得益于全局池化操作，该模型并不依赖于特定的

输入分辨率。然而，为简化小批量处理流程（关于变长输入的更多细节将在下一章阐述），通常需要在训练和推理阶段固定输入分辨率。

3. (E.7.11)可以被视为一个“特征提取”模块，而(E.7.12)可以被视为“分类模块”。当我们在下一卷中讨论迁移学习时，这种解释将非常有用。我们将特征提取模块称为模型的**主干网络**（backbone），将分类模块称为模型的**头部**（head）。

## 典型的卷积类型

本章的结尾，我们将提及两种在实践中常见的卷积层实例。

首先，考虑 $k = 0$ 的卷积层，即所谓的 $1 \times 1$ 卷积。这相当于通过其通道的加权和来更新每个像素的嵌入，而忽略所有其他像素：

$$H_{ijz} = \sum_{t=1}^{c} W_{zt} X_{ijt}$$

它是一种很有用的操作，例如，用于修改通道维度（我们将在第9 章讨论残差连接时看到一个示例）。这种情况下，参数可以紧凑地表示为一个矩阵 $\mathbf{W} \sim (c', c)$。这等价于独立地应用于每个像素的全连接层。

然后，考虑一种与 $1 \times 1$ “正交”的变体，它在一个小的邻域内组合像素，但只使用一个通道的信息，忽略其余通道的信息。

$$H_{ijc} = \sum_{i'=1}^{2k+1} \sum_{j'=1}^{2k+1} W_{i',j',c} X_{i'+t(i),j'+t(j),c}$$

其中，$t(\bullet)$ 是(E.7.6)中定义的偏移量。在这种情况下，存在一个尺寸为 $(s, s, c)$ 的 3 阶权重矩阵，并且每个输出

通道 $H_{:,:,c}$ 的更新仅依赖于相应的输入通道 $X_{:,:,c}$。此操作被称为**深度卷积**（depthwise convolution），并且可以将其推广至考虑若干通道构成一组的情况，，在这种情况下，它被称为**分组卷积**（groupwise convolution，深度卷积是分组大小为 1 时的特例）。

我们还可以将这两种思想结合起来，构建一种由交替的 $1 \times 1$ 卷积（用于通道混合）和深度卷积（用于像素混合）构成的卷积块。这种方法称为**深度可分离**（depthwise separable）卷积，常见于面向低功耗设备的 CNN 设计中 [HZC+17]。请注意，在这种情况下，单个块的参数数量（与标准卷积相比）从 $sscc'$ 降低到了 $ssc + cc'$。我们将在第10章进一步介绍这些在不同轴上交替处理输入信息的分解方法对于其他类型架构（例如 transformer）的重要性。

## 从理论到实践



本章中介绍的所有层（卷积层、最大池化层）都在`torch.nn`模块中实现。torchvision 库提供了数据集和加载图像的函数，同时也提供执行图像变换的接口，这将在下一章中非常有用。[3]

在继续之前，我建议您跟随并复现网上众多基于 torchvision 的图像分类的教程中的一种，此时理解这些教程应较为容易。[4]常用的简单数据集包括 MNIST（数字分类）和 CIFAR-10（通用图像分类）。结合使用 torchvision 的加载器与 Equinox 库中的层，可以在 JAX 框架下复现相同的教程，例如：`https://docs.kidger.site/equinox/examples/mnist/`。

[3] `https://pytorch.org/vision/stable/transforms.html`

[4] 官方文档中的示例：`https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html`

从头开始实现卷积也是一项有趣的练习，其复杂度取决于抽象层次。一种可行的实现方式是使用 PyTorch 的 `fold/unfold` 操作，来从图像中提取图像块。[5]由于预先构建的卷积核在计算速度上具有显著优势，因此，从头实现卷积是一项教学性质的练习。

如果您具备一定的信号处理背景，可能会了解卷积运算可以通过转换到频域进行乘法运算来实现。对于常用的较小尺寸的卷积核而言，此方法并不具备实用价值；然而，对于尺寸非常大的卷积核（亦称长卷积），该方法可能更为有效，例如：

https://github.com/fkodom/fft-conv-pytorch

PyTorch 也提供了可微的快速傅里叶变换，您可以将其用作进一步开发的起点。

[5]见示例：https://github.com/loeweX/Custom-ConvLayers-Pytorch

# 8 | 图像之外的卷积

**本章导读**

卷积模型作为一种强大的基准模型（baseline model），其应用范围已远远超越图像分类领域，广泛渗透于众多应用场景。本章将系统介绍卷积层的几类扩展应用，包括针对一维与三维数据的卷积运算、文本建模方法以及自回归生成技术。我们将引入掩码、词元化等关键概念，这些概念对于理解本书后续内容以及现代 LLM 至关重要。

## 8.1 面向一维与三维数据的卷积

### 8.1.1 超越图像：时间序列、音视频与文本

在上一章中，我们重点讨论了图像数据处理。然而，许多其他类型的数据也具备类似特征：即存在一个或多个表示时间或空间关系的“有序”维度，以及一个表示特征的通道维度（对应图像场景中的通道概念）。考虑如下示例：

1. **时间序列**是对一个或多个过程的测量值集合（例如股票价格、传感器读数、能量流动）。可以将时间序列表示为矩阵 $\mathbf{X} \sim (t, c)$，其中 $t$ 代表时间序

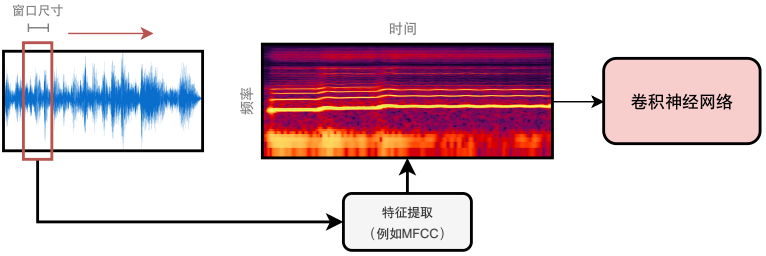



**图 F.8.1:** 音频既可用一维序列（左）表示，也可在时频域中表示为二维图像（中）。在第二种表示形式下，可应用上一章所述的标准卷积模型进行处理。

列长度，$\mathbf{X}_i \sim (c)$ 则表示在时刻 $t$ 采集的 $c$ 个测量值（例如，脑电图扫描的 $c$ 个传感器读数，或 $c$ 支股票价格）。每个时间点相当于一个像素点，每个测量值则等价于一个通道。

2. **音频**文件（如语音、音乐）同样可以表示为矩阵 $\mathbf{X} \sim (t, c)$，其中 $t$ 代表音频信号长度，$c$ 为录音通道数（1 表示单声道，2 表示立体声，依此类推）。

> **频率分析**
>
> 音频数据同样可通过频率分析（例如，在短时间窗口上提取 MFCC 系数）转换为类图像形式，所得的时频图代表信号中频率成分随时间的演变过程，示例见图F.8.1。经此预处理后，即可使用标准卷积模型对其进行处理。

3. **视频**可以用 4 阶张量 $X \sim (t, h, w, c)$ 表示，其中 $t$ 是视频的帧数，每帧对应一幅形状为 $(h, w, c)$ 的图像。另一个例子是医学中的容积扫描（volumetric scan），此时 $t$ 代表容积深度。

时间序列、音频信号及视频均可用**采样率**(sampling rate)

来描述。采样率表示单位时间内采集的样本数量，其单位通常为每秒样本数（samples per second）或赫兹（Hz）。例如：常规脑电图设备采样率为 240 Hz，代表每秒采集 240 个样本。假如每分钟检查一次股票价格，其采样率相当于 1/60 Hz。相比之下，为保证高保真度，音频信号需要极高采样率：例如，音乐信号采样率可达 $44.1e^3$ Hz（即 44.1 kHz）。一般的视频采集**帧率**（frame rates）为 24 帧每秒（fps）左右，以确保人眼观看的流畅性。

图像分辨率、音频采样率与视频帧率在决定信号采集精度方面起相似作用。对于图像，可以先验地假定一个固定分辨率（例如 $1024 \times 1024$ 像素）。这种处理在除极低分辨率外的大多数情况下均能保持一致性。相比之下，音频与视频的时长可能因输入而异（例如，30 秒的歌曲与 5 分钟的歌曲），它们无法被调整为统一维度，[1]这意味着数据集将由**变长**（variable-length）数据构成。此外，音频信号的数据量可能非常庞大：在 44.1 kHz 的采样率下，一段 3 分钟的音频将包含约 800 万个样本。

还需指出，上述示例中的维度可大致归类为“空间维度”（如图像）或“时间维度”（如音频）。图像通常沿空间轴对称（例如，多数情况下，沿宽度方向翻转的图像仍为有效图像）。然而，时间维度本质上是非对称的：在时间轴上倒放的音频样本通常无意义，而时间序列的逆序则表示该序列从未来向过去演化。在模型设计中，我们既可利用该特性（**因果性**，causality），也可关注预测信号的未来值——此任务称为**预测**（forecasting）。

最后，以文本句子“*the cat is on the table*”（猫在桌子上）为例。存在多种方式可将此句子拆解为若干片段。例如，可以按音节划分为：[“*the*”，“*cat*”，“*i*”，“*s*”，“*on*”，“*the*”，“*ta*”，*ble*”]。这是序列的另一个例子，区别在于序列的每个元素为类别值（音节），而非数值编码。因此，需要某种方法将这些类别值编码为模型

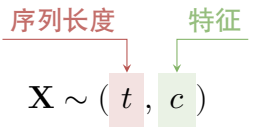
[1]从相同时长和分辨率的意义上来说。

可处理的特征：将文本序列拆解为基本单元的过程称为**词元化**（tokenization），而将每个词元（token）转化为向量的过程则称为词元**嵌入**（embedding）。

后续章节将依次讨论这些核心要素（变长输入、因果性、预测、词元化和嵌入），以研究如何构建卷积模型应对这些挑战。我们将介绍的部分技术（如掩码，masking）具有普适性，同样适用于其他类型模型（如 transformer）。而另一些技术（如膨胀卷积，dilated convolutions）则为卷积模型特有。

### 8.1.2 一维与三维卷积层

接下来，将探讨一维信号（如时间序列、音频）的卷积运算的定义方法，并将其扩展至三维信号（如视频）。请注意，此处维度仅表示进行卷积运算所沿的空间或时间维度数量，不包含通道维度。回顾前文可知，在一维情形下，输入 $\mathbf{X}$ 可表示为单个矩阵：




现沿用第7章的推导过程。给定卷积核尺寸 $s = 2k + 1$，定义 $P_k(i) \sim (s, c)$ 为输入矩阵 $\mathbf{X}$ 中所有与第 $i$ 行距离不超过 $k$ 的行组成的子集（暂不考虑边界元素，此类情形可通过零填充处理）。一维卷积层 $\mathbf{H} = \text{Conv1D}(\mathbf{X})$ 的输出为矩阵 $\mathbf{H} \sim (t, c')$，其中 $c'$ 是定义输出维度的超参数。该层逐行计算公式为：

$$[\text{Conv1D}(X)]_i = \phi(\mathbf{W} \cdot \text{vect}(P_k(i)) + \mathbf{b}) \qquad \text{(E.8.1)}$$

其中 $\mathbf{W} \sim (c', sc)$ 和 $\mathbf{b} \sim (c')$ 为可训练参数。与二维情形类似，该层具有局部性（需适当调整局部性定义，以适应序列数据），并且对序列平移具有等变性。

在二维情形中，也曾讨论过一种替代表示法，即显式地对所有索引求和：

$$H_{ijz} = \sum_{i'=1}^{2k+1} \sum_{j'=1}^{2k+1} \sum_{d=1}^{c} [W]_{i',j',z,d} [X]_{i'+t(i),j'+t(j),d} \quad \text{(E.8.2)}$$

其中 $t(i) = i + k - 1$，如公式(E.7.6)所示。请注意，使用 $t$ 对 $i'$ 和 $j'$ 使用了不同的索引方式：对于权重 $W$，$i'$ 和 $j'$ 索引范围是 1 到 $2k+1$；对于输入 $X$，索引范围是 $i-k$ 到 $i+k$。公式(E.8.1)的等价形式可通过移除一个求和索引得到：

$$H_{iz} = \sum_{i'=1}^{2k+1} \sum_{d=1}^{c} [W]_{i',z,d} [X]_{i'+t(i),d} \quad \text{(E.8.3)}$$

其中，参数 $W \sim (s, c', c)$ 被重组为一个 3 阶张量。与此相对，三维卷积的变体可通过在第三维度上增加一个索引为 $p$ 的求和运算得到：

$$H_{pijz} = \sum_{p'=1}^{2k+1} \sum_{i'=1}^{2k+1} \sum_{j'=1}^{2k+1}, \sum_{d=1}^{c} [W]_{p',i',j',z,d} [X]_{p'+t(p),i'+t(i),j'+t(j),d}$$

为简化表述，假设各维度上的卷积核尺寸一致。通过类似推导，可得三维卷积的向量化形式，并进一步推导出一维及三维最大池化操作的对应变体。

## 8.2 一维与三维卷积模型

本节将重点讨论一维卷积模型的设计方案，特别关注变长输入及文本序列的处理方法。本节阐述的多项理念对于所有可微模型均具有通用性。

### 8.2.1 变长输入处理机制

假设存在两个音频文件（或时间序列、文本），对应输入矩阵分别为 $\mathbf{X}_1 \sim (t_1, c)$ 和 $\mathbf{X}_2 \sim (t_2, c)$。两个输入具有相同通道数 $c$（例如传感器数量），但序列长度 $t_1$ 和 $t_2$ 不同。根据7.1中的讨论，卷积运算原则上能够处理此类**变长**输入。实际上，若用 $g$ 表示由一维卷积和最大池化操作组成的通用复合函数，对应于模型的特征提取模块，该模块的输出为两个矩阵：

$$\mathbf{H}_1 = g(\mathbf{X}_1)\,,\ \mathbf{H}_2 = g(\mathbf{X}_2)$$

两个矩阵有相同的列数，但行数不同（具体取决于对输入应用最大池化或步进卷积次数）。经过全局平均池化（global average pooling）后，序列长度依赖性将消失：

$$\mathbf{h}_1 = \sum_i \mathbf{H}_{1i}\,,\ \mathbf{h}_2 = \sum_i \mathbf{H}_{2i}$$

此时可基于向量 $\mathbf{h}_1$ 和 $\mathbf{h}_2$ 进行最终分类。然而，在实际操作层面，由于无法从不同维度的矩阵构建小批量（mini-batch）样本，运算难以实现向量化。该问题可通过零填充（zero-padding）技术，将小批量样本统一填充至最大序列长度。例如，不失一般性地假设 $t_1 > t_2$，可构建填充后的小批量样本：

```python
# 变长序列
# (长度分别为 3, 5, 2)
X1, X2, X3 = torch.randn(3, 8),
             torch.randn(5, 8),
             torch.randn(2, 8)

# 填充到单个小批量中
X = torch.nn.utils.rnn.pad_sequence(
                    [X1, X2, X3],
                    batch_first=True)
print(X.shape)
# [输出]: torch.Size([3, 5, 8])
```

**框 C.8.1:** 由三个可变长度序列（通道数 $c = 8$）构成的填充小批量示例. 当使用数据加载器（*DataLoader*）时，可通过重写其默认的整理函数（*collate_fn*）来实现填充操作，该函数定义了加载器如何将单个样本拼接成批次。

$$X = \text{stack}\left(\mathbf{X}_1, \begin{bmatrix}\mathbf{X}_2 \\ \mathbf{0}\end{bmatrix}\right)$$

其中 stack 在一个新的前导维度上运行，得到最终张量 $X$ 形状为 $(2, t_1, c)$。此方法可推广至任意小批量场景，只需以批次中最长序列为基准进行填充。对卷积运算而言，这种方法与常规零填充无异（例如，音频中的零填充相当于在末尾添加静音段）。具体示例见框C.8.1。

另一种解决方案是构建掩码矩阵（masking matrix）来标识小批量张量中的有效和无效索引：

$$\mathbf{M} = \begin{bmatrix} & \mathbf{1}_{t_1} \\ \mathbf{1}_{t_2} & \mathbf{0}_{t_1 - t_2} \end{bmatrix}$$

其中下标表示向量的大小。这些掩码矩阵有助于避免在输入张量的无效（填充）部分上执行运算。

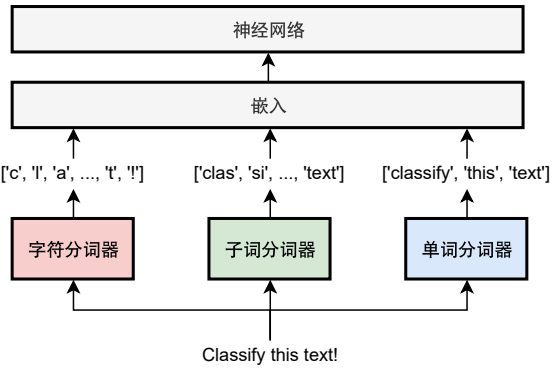



**图 F.8.2:** 从文本输入开始，可以使用多种类型的分词器。符号会被嵌入为向量，并由通用的一维模型进行处理。

### 8.2.2 面向文本数据的卷积神经网络

本节将探讨文本数据处理问题。如前文所述，处理文本的第一步是**词元化**（tokenization），即将文本字符串拆解为已知符号序列（此语境下称为**词元**，tokens）。常见分词器（tokenizer）可分为三类：

1. **字符分词器**（Character tokenizer）：以单个字符为基本符号单位。
2. **单词分词器**（Word tokenizer）：以每个（合法的）单词为基本符号单位。
3. **子词分词器**（Subword tokenizer）：介于字符分词器和单词分词器之间，每个符号长度通常大于单个字符，但小于完整单词。

图F.8.2直观展示了这三种词元化策略。无论采用哪种方法，都需要预先定义包含所有合法词元的**词典**（dictionary，或**词表**，vocabulary）。例如，字符分词器可以采用所有 ASCII 字符作为词表。在实际应用中，通常先设定预期词典规模，然后在文本中寻找最高频词元填充词表，其余符号则归入特殊的“词表外

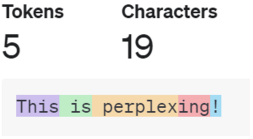



**图 F.8.3:** 对句子应用 *tiktoken* 分词器的示例。

(out-of-vocabulary，OOV)”词元。子词分词器为此有多种专用算法，例如字节对编码（byte-pair encoding, BPE）算法 [SKF+99]。[2]

鉴于大规模文本数据的多样性，预训练子词分词器已成为主流选择。以 OpenAI 的开源子词分词器为例，[3]该模型包含约 10 万个子词（截至本书撰写时）。例如，如图F.8.3所示，使用 tiktoken 分词器编码句子“*This is perplexing!*”（“这真令人困惑!”）。部分词元对应完整单词（如“This”），部分对应单词片段（如“perplex”），其余则对应标点符号。该序列可等价表示为整数序列：

$$[2028, 374, 74252, 287, 0] \tag{E.8.4}$$

其中每个整数取值范围为 0 至词表大小（本例中约为 10 万），并且它在该词表中唯一地标识了一个词元。实际应用中，我们还可以向序列添加特殊词元，如句首(beginning of the sentence，有时表示为 [BOS]）词元、OOV 词元等。其中 [BOS] 词元将在下节发挥重要作用。

需要警惕的是，使用超大型词表对子词进行词元化时，可能产生反直觉现象：例如，常见数字 52 拥有独立词元，而 2512 可能被拆分为“251”与“2”两个词元。因此，可视化词元化过程对于模型行为调试至关重要。[4]鉴

[2]本节重点讨论可微模型，因此省略了诸多文本预处理操作（如停用词剔除、标点去除、词干提取等）。随着模型规模的不断扩大，这类操作已逐渐减少应用。

[3]https://github.com/openai/tiktoken

[4]面向对数值敏感的应用，可采用专门的数值分词器 [GPE+23]。

于词元化环节的重要性，该领域研究十分活跃，例如近期提出的字节级分词器 [PPR+24]和端到端可训练分词器 [HWG25]。

完成词元化后，需将词元**嵌入**为向量，才能用作 CNN 的输入。简单的独热编码策略效果欠佳，因为大词表会导致生成向量极度稀疏。此时有两种替代方案：其一是使用预训练嵌入网络，这将在后续介绍 transformer 时详细讨论；为建立直观认知，我们重点阐释第二种方案：将嵌入模块与网络其他部分一同训练。

假设固定嵌入维度 $e$ 为超参数，由于词典尺寸 $n$ 也为固定值，可初始化嵌入矩阵 $\mathbf{E} \sim (n, e)$。通过查表（look-up）操作将每个整数替换为 $\mathbf{E}$ 中的对应行，用 $x$ 表示 ID 序列，可得：

嵌入矩阵中的第 $x_1$ 行

$$\text{LookUp}(x) = \mathbf{X} = \begin{bmatrix} \mathbf{E}_{x_1} \\ \mathbf{E}_{x_2} \\ \vdots \\ \mathbf{E}_{x_m} \end{bmatrix}$$

最终得到的输入矩阵 $\mathbf{X}$ 形状为 $(m, e)$，其中 $m$ 为序列长度。此时，可以应用通用的一维卷积模型（例如，对文本序列进行分类）：

$$\hat{y} = \text{CNN}(\mathbf{X})$$

该模型可根据任务需求进行常规训练，不同的是，梯度下降将同时作用于模型参数和嵌入矩阵 $\mathbf{E}$。图F.8.4直观展示了这一点，框C.8.2展示了模型定义示例。

该方法的强大之处在于，在许多情况下，我们发现最终得到的嵌入支持与向量一致的代数运算。例如通过欧氏

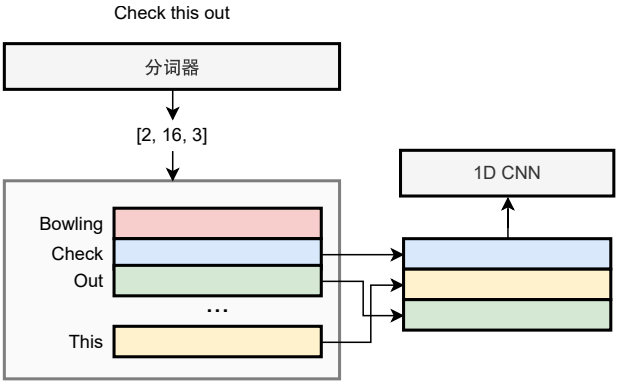



**图 F.8.4:** 通过查表操作将词元 *ID* 序列转换为对应嵌入矩阵的示意图：输入为列表形式，输出为矩阵形式。框内所示的嵌入可通过梯度下降算法与所有其他参数共同训练。在本例中，假设词表尺寸 $n = 16$。

空间最近邻搜索最接近的嵌入，可发现”语义相似”的词汇或句子。这种思想使可微模型在需要文档检索与搜索的诸多领域发挥着核心作用。

**可微模型与嵌入**

再次强调，嵌入的核心思想具有高度普适性：任何将对象转化为具备代数特征的向量的过程，都可称为嵌入。例如，一个已训练的卷积神经网络的主干网络经过全局池化后的输出，可以被视为输入图像的一种高层嵌入。并且，它可以通过与其他所有嵌入进行对比，来检索“相似”的图像。

### 8.2.3 长序列处理技术

前文所述的许多序列可能极长。在这种情况下，卷积层的局部性 (locality) 可能成为限制因素，因为需要线性增加

```
class TextCNN(nn.Module):
  def __init__(self, n, e):
    super().__init__()
    self.emb = nn.Embedding(n, e)
    self.conv1 = nn.Conv1d(e, 32, 5,
                           padding='same')
    self.conv2 = nn.Conv1d(32, 64, 5,
                           padding='same')
    self.head = nn.Linear(64, 10)

  def forward(self, x):       # (*, m)
    x = self.emb(x)           # (*, m, e)
    x = x.transpose(1, 2)     # (*, e, m)
    x = relu(self.conv1(x))   # (*, 32, m)
    x = max_pool1d(x, 2)      # (*, 32, m/2)
    x = relu(self.conv2(x))   # (*, 64, m/2)
    x = x.mean(2)             # (*, 64)
    return self.head(x)       # (*, 10)
```

**框 C.8.2:** 具有可训练嵌入层的一维卷积神经网络。其中 $n$ 为词典尺寸，$e$ 为嵌入维度。我们使用两个卷积层，其输出通道数分别为 32 和 64。前向传播中每一步操作的输出形状均在注释中标明。

网络层数才能获得更大的感受野。后续章节将介绍诸如 transformer 的其他模型架构如何解决这一问题。本节我们仍聚焦于卷积模型，重点介绍一种由 WaveNet 语音生成模型推广的有趣方案：**膨胀**卷积（dilated convolution，亦称**空洞**卷积，atrous，源自法语 *à trous*）[ODZ+16]。

该技术引入了称为**膨胀率**（dilation rate）的超参数，标准卷积的膨胀率为 1。膨胀率为 2 时，卷积核在序列中每隔一个元素进行一次采样；类似地，膨胀率为 4 时，则每隔三个元素选取一次，以此类推。如图F.8.5所示，通过堆叠指数级增长膨胀率的卷积层，可以在不增加参数量的前提下（因为卷积核邻域元素数量保持不变），使模型的感受野随网络层数增加呈指数级扩张。

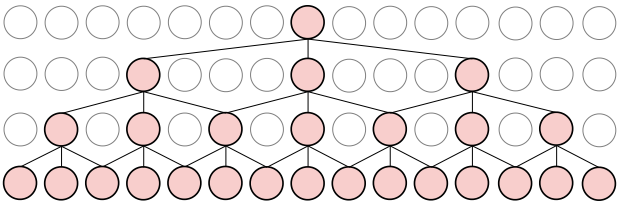


**图 F.8.5:** 膨胀率递增的卷积层。用于卷积的元素为红色，其他元素为灰色。图中展示了单个输出元素的感受野。

## 8.3 预测与因果模型

### 8.3.1 序列预测

序列分析的一个重要方向是构建预测未来元素的模型，例如能源价格、湍流运动、呼叫中心话务量等。预测词元也是大语言模型和近期其他突破性成果的基础构建模块。广义而言，当前神经网络研究热点很大程度上围绕以下问题展开：通过在大规模文本语料库上进行下一词元预测，模型能够推断出多少信息？这种范式能否复现到其他模态（如视频）和动态系统中 [WFD+23]？在统计学和时间序列分析中，严格意义上的序列下一元素预测被称为**预测**（forecasting）。为与当前文献保持一致，后续我们将统一使用**词元**（token）指代序列中的每个元素，无论它是嵌入的文本词元还是广义的向量化输入。

预测之所以具有重要意义，是因为仅需一组序列数据即可训练预测模型，无需额外标注标签。在现代术语中，这也被称为**自监督学习**（self-supervised learning）任务，因为目标值可以从输入中自动提取。

**平稳性与预测**

与文本处理类似，预测真实世界时间序列时需面对诸多问题（例如可能存在的非平稳性、趋势性与周期性等），此处我们不作讨论。[a]在实践中，音频、文本及许多常见序列均可视为平稳序列，无需特殊预处理。类似文本数据，对于超大型预测数据集及相应的大模型而言，预处理的影响趋于减弱 [AST+24]。

[a] https://filippomb.github.io/python-time-series-handbook/

具体实现时，假设用户定义的固定长度 $t$，从数据集中提取所有可能的长度为 $t$ 的子序列（例如，$t = 12$ 时，可提取连续 12 个元素构成的窗口，或 12 个词元组成的句子等）。在大语言模型中，输入序列的长度被称为模型的**上下文**（context）。将每个子序列与一个目标值相关联，该目标值就是该序列自身的下一个元素，由此，我们可以构建一组数据对 $(\mathbf{X}, \mathbf{y})$，其中 $\mathbf{X} \sim (t, c)$，$y \sim (c)$。预测模型在该数据集上以监督学习方式进行训练：

$$f(\mathbf{X}) \approx \mathbf{y}$$

需要注意的是，一个标准一维卷积模型就可用作预测模型，使用均方误差（连续时间序列）或交叉熵（离散序列，如文本）作为损失函数训练。虽然模型被训练用于单步预测，但可以通过**自回归**（autoregressive）方法实现多步预测，其中模型根据自身的输出来进行预测（回归，regressing）。假设通过 $\hat{\mathbf{y}} = f(\mathbf{X})$ 获得单步预测结果，将预测值添加到输入序列末尾（同时移除首元素以保持序列长度为 $t$）：

**离散序列预测**

对于连续时间序列而言，这一问题较为直观。对于一个含有离散值的时间序列，函数 $f$ 将返回可能取值（即所有可能词元）上的概率分布向量，通过取 $\arg\max$（即选择最高概率对应的词元）来获得预测值 $\hat{\mathbf{y}}$。此外，也可以根据预测概率分布对词元按比例抽样：详见第8.4.1节。

$t-1$ 个输入元素的窗口

$$\mathbf{X}' = \begin{bmatrix} \mathbf{X}_{2:t} \\ \hat{\mathbf{y}} \end{bmatrix} \tag{E.8.5}$$

在时间 $t+1$ 的预测值

通过循环执行 $f(\mathbf{X}')$ 可逐步生成后续序列值。该过程始终以先进先出（First-In, First-Out, FIFO）的方式更新缓冲输入。这种方法虽然强大，但它要求预先固定输入序列长度，限制了其应用范围。为解决这一局限，只需对模型进行微小调整。

### 8.3.2 因果模型




假设我们仅能获取长度为 4 的短序列，并构造矩阵 $\mathbf{X} \sim (4, c)$，但已基于更长序列 $t = 6$ 训练了预测模型。为使该模型能处理较短序列，我们可在序列起始处填充两个零向量 $\mathbf{0}$，但除非对模型运算进行掩码处理，否则这些零值将被视为实际时间序列值。幸运的是，**因果**（causal）模型提供了一种更简洁优雅的解决方案。

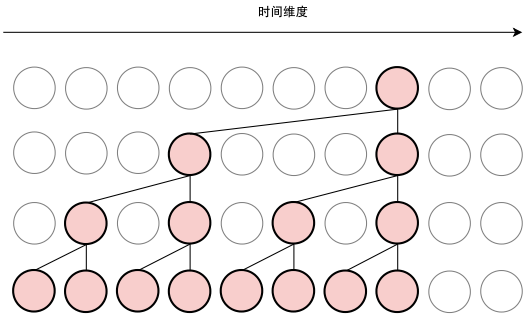



**图 F.8.6:** 一维因果卷积层结构示意图。该层（原始）卷积核大小为 3，并使用指数增长的膨胀率。图中已移除被置零的连接，并展示了单个输出元素的感受野范围。




**定义 D.8.1 (因果层)**

若层 $\mathbf{H} = f(\mathbf{X})$ 满足 $\mathbf{H}_i = f(\mathbf{X}_{:i})$，即序列中第 $i$ 个元素对应的输出值仅依赖于其“过去”的元素，那么该层是**因果** $(causal)$ 的。

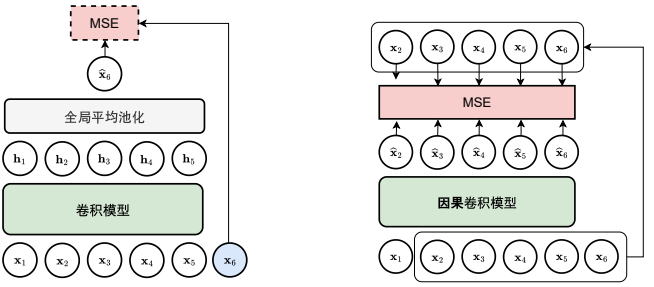
由纯因果层构成的模型天然具有因果性。例如，卷积核大小为 1 的卷积层具有因果性，因为每个元素仅基于其自身进行处理。然而，卷积核大小为 3 的卷积层则不具备因果性，因其同时考虑左右相邻元素。通过将对应非因果连接的权重进行部分零掩码，可将任意卷积改造成因果版本：

$$\mathbf{h}_i = \phi\left(\left[\mathbf{W} \odot \mathbf{M}\right] \operatorname{vect}(P_k(i)) + \mathbf{b}\right)$$

掩蔽权重矩阵

如果权重对应输入中满足 $j > i$ 的元素，则 $M_{ij} = 0$，否则为 1。因果一维卷积可与膨胀卷积核结合，构建音频自回归模型，如 WaveNet 模型 [ODZ+16]，示意图见F.8.6。

在单通道情形下，掩码机制更易理解，此时 $\mathbf{M}$ 简化为




**(a)** 非因果模型 **(b)** 因果模型

**图 F.8.7:** 两种模型的对比示意图。*(a)* 非因果预测模型（仅针对整个输入序列生成末端单步预测）；*(b)* 因果模型（对序列中每个输入元素均预测对应输出元素）。

下三角二元矩阵。该掩码操作将参数量从 $(2k+1)cc'$ 有效降低至 $(k+1)cc'$。

通过堆叠多个因果卷积层，可得到因果一维模型变体。假设将其应用于输入序列，且模型无最大池化操作，则输出序列与输入序列等长：

$$\widehat{\mathbf{Y}} = f_{\text{causal}}(\mathbf{X})$$

并且，输出中任一元素仅取决于同位置及之前的输入元素。因此，可以通过预测 输入序列每个元素对应的值，来定义更精细的预测模型。具体而言，考虑一个矩阵输出：

$$\mathbf{Y} = \begin{bmatrix} \mathbf{X}_{2:t} \\ \mathbf{y} \end{bmatrix}$$

这与(E.8.5)中的平移输入类似，区别在于此处使用真实值作为序列末元素。可以通过最小化所有元素的损失（如

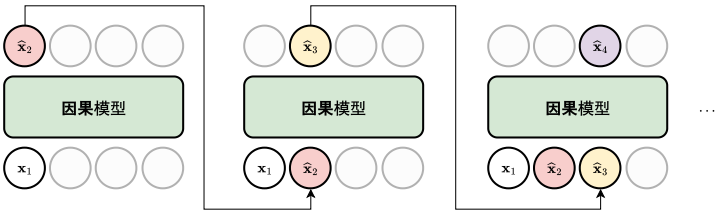



**图 F.8.8:** 因果卷积神经网络推理示意图，通过自回归方式逐步生成序列。未使用的输入词元以灰色显示，生成的词元用不同颜色区分。

均方误差）来训练该模型：

$$l(\widehat{\mathbf{Y}}, \mathbf{Y}) = \|\widehat{\mathbf{Y}} - \mathbf{Y}\|^2 = \sum_{i=1}^{t} \|\widehat{\mathbf{Y}}_i - \mathbf{Y}_i\|^2 \tag{E.8.6}$$

预测 $\mathbf{X}_{i+1}$ 时的损失

这样，模型会同时基于第一个元素预测第二个元素，基于前两个元素预测第三个元素，以此类推。对于单个输入窗口，可得到 $t$ 个独立损失项，显著增强了梯度传播。两种方法的对比见图F.8.7：图F.8.7a展示预测序列下一元素的非因果卷积模型，图F.8.7b展示基于(E.8.6)训练的因果模型。

更重要的是，现在可以通过自回归方式使用该模型，处理长度最大为 $t$ 的任意序列。通过示例可直观理解：假设 $t = 4$，并且我们已观测到两个值 $\mathbf{x}_1$ 和 $\mathbf{x}_2$。首次调用模型时，对序列进行零填充以生成第三个词元：

$$\begin{bmatrix} - \\ \widehat{\mathbf{x}}_3 \\ - \\ - \end{bmatrix} = f\left(\begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix}\right)$$

我们仅取用第二个输出值，忽略其他值（实际上，第三、

四输出因零填充而无效)。将 $\hat{\mathbf{x}}_3$ 添加到序列后，继续自回归调用，此处预测值用彩色标注：

$$\begin{bmatrix} - \\ - \\ \hat{\mathbf{x}}_4 \\ - \end{bmatrix} = f\left(\begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \hat{\mathbf{x}}_3 \\ \mathbf{0} \end{bmatrix}\right), \quad \begin{bmatrix} - \\ - \\ - \\ \hat{\mathbf{x}}_5 \end{bmatrix} = f\left(\begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \hat{\mathbf{x}}_3 \\ \hat{\mathbf{x}}_4 \end{bmatrix}\right), \quad \begin{bmatrix} - \\ - \\ - \\ \hat{\mathbf{x}}_6 \end{bmatrix} = f\left(\begin{bmatrix} \mathbf{x}_2 \\ \hat{\mathbf{x}}_3 \\ \hat{\mathbf{x}}_4 \\ \hat{\mathbf{x}}_5 \end{bmatrix}\right) \ldots$$

在最后一步中，为了维持输入尺寸约束，我们移除一个原始输入，过程如图F.8.8所示。需要注意，模型仅基于真实值训练，不依赖自身预测，该方法被称为**教师强制**(teacher forcing)。教师强制的一种变体是：在训练过程中，随着模型精度提升，逐步将小批量中的部分真实值替换为模型预测值。

因果自回归模型在处理文本序列（此时只有一个通道，即词元的索引）中尤为关键。我们可从代表序列开始的 [BOS] 词元出发，从头生成文本语句；或通过将用户提示附加至 [BOS] 词元后实现条件生成。类似原理可应用音频模型，用以生成语音或音乐等 [ODZ+16]。

## 8.4 生成模型

### 8.4.1 概率论框架




自回归模型是**生成模型**（generative model）的一个简单例子。[5] 我们将在下一卷中深入探讨其他类型的生成式模型。此处重点阐述一些针对自回归算法的见解。我们考虑单通道离散值序列（如文本）。基于文本词元的自回归模型是 LLM 的基础，也可作为多模态架构的基础组件（详见第11章）。

生成式模型更适宜在概率框架下表述，因此我们首先采用概率形式重述前文讨论。用 $\mathcal{X}$ 表示所有可能序列构成的空间（例如所有文本词元的可能组合）。一般而言，其中许多序列是无效的，例如英语中的序列 [“*tt*”，“*tt*”]。然而，即使是极罕见的序列，也可能在超大规模语料库中出现至少一两次（想象一个角色大喊 “Scotttt!” 的场景）。

考虑所有可能序列 $x \in \mathcal{X}$ 上的概率分布 $p(x)$，可以推广这一概念。在文本领域，这也被称为**语言模型**（language model）。生成式建模的核心任务就是学习从该分布中高效采样的方法：[6]

$$x \sim p(x)$$

为建立与前述内容的联系，根据概率乘积法则，我们可

[5] 回顾第3章，我们假设监督学习的数据对 $(x, y)$ 来自某个未知概率分布 $p(x, y)$。根据概率乘法法则，该分布可等价分解为 $p(y \mid x)p(x)$ 或 $p(x \mid y)p(y)$。任何类似 $p(x)$ 或 $p(x \mid y)$ 的模型都被认为是**生成式**的，因其可用于生成新的输入样本。相对地，如前文所述，只对 $p(y \mid x)$ 进行逼近的模型被称为**判别式**的。

[6] 本节中 $\sim$ 表示概率分布采样，而非张量形状。

将 $p(x)$ 重写为：

$$p(x) = \prod_i p(x_i \mid x_{:i}) \tag{E.8.7}$$

其中每个值 $x_i$ 的条件概率均依赖于其之前所有值。若假设模型输入长度足以容纳所有可能序列，则可使用因果预测模型来参数化(E.8.7)中的概率分布：

$$p(x_i \mid x_{:i}) = \text{Categorical}(x_i \mid f(x_{:i}))$$

此处所有时间步都使用一个单一的、共享的模型。在该模型上进行最大似然估计，等价于最小化预测概率的交叉熵损失，如第4.2.2节所述。

### 8.4.2 自回归模型中的采样

一般而言，从概率分布中采样并非易事。对于自回归模型，我们可以利用公式(E.8.7)中的乘积分解，设计一种简单的迭代策略：

1. 采样 $x_1 \sim p(x_1)$。这等价于在空集 $p(x_1 \mid \{\})$ 条件下采样。在实践中，我们总是以一个初始固定词元（如 [BOS]）作为条件，确保输入永不为空。

2. 将步骤 (1) 中采样的值作为输入，再次运行网络以采样 $x_2 \sim p(x_2 \mid x_1)$ 如图F.8.8所示。

3. 采样 $x_3 \sim p(x_3 \mid x_1, x_2)$.

4. 持续该过程，直至达到期望序列长度，或出现句子结束词元。

在前文中，我们通过始终采样最高概率元素，隐式实现了这一过程：

$$x_i = \arg\max_i f(x_{:i})$$

然而，我们也可以根据模型 $f$ 预测的概率分布进行采样，来将其泛化。回顾第4.2.1节，softmax 通过引进额外的温度参数进行泛化。在推理过程中调整该参数，可以在始终选择 argmax 值（温度极低）与几乎均匀的词元分布（温度极高）之间平滑过渡。

在概率建模领域，模型的此种采样方式称为**祖先采样**（ancestral sampling）；而在语言建模领域，我们有时称之为**贪心解码**（greedy decoding）。“贪心”一词的使用及此简要讨论足以揭示该方法的潜在缺陷：尽管 $p(x)$ 的乘积分解是精确的，但贪婪解码并不能保证得到对应于高 $p(x)$ 值的样本。

要理解这一点，需注意到，$f$ 提供的是单个词元的估计，而序列概率由许多此类项的乘积构成。因此，在序列开头采样一个具有高（局部）概率的词元，未必能得到具有高（全局）概率的句子序列。可以设想，第一个词元的选择可能使解码阶段陷入一条低概率路径，从而直观理解此问题。

针对该问题，一个常见缓解策略是**束搜索**（beam search，或**束编码**，beam decoding）。在束搜索中，第一步采样 $k$ 个不同元素（称为束，其中 $k$ 为用户定义参数）。第二步，对 $k$ 束中的每一个，都采样 $k$ 个可能的后续标记。在 $k^2$ 个候选对中，仅保留根据其乘积概率 $p(x_1)p(x_2 \mid x_1)$（或其对数概率）排名前 $k$ 的序列。以此方式迭代进行，直至序列结束。

由此观之，从自回归模型中采样最可能序列是一个组合搜索（combinatorial search）问题（可想象为一棵树，其中每个词元节点处都扩展出所有可能的下一节点，以此类推）。从编程角度，束搜索是在该树上进行**广度优先**（breadth-first）搜索的一个实例。在某种意义上，集束搜索是以更昂贵的推理阶段为代价，换取了简单的训练过程。为此还存在许多其他技术，包括引导解码过程，以满足外部奖励函数的可能性 [WBF+24]。

### 8.4.3 条件建模

如前文所述，我们通常更关注如何根据已知序列（如用户提问或交互内容）生成后续文本，而非完全从零开始生成序列。这可以通过建立条件概率分布 $p(x \mid c)$ 来实现，其中 $c$ 代表条件参数（如用户提示）。前文论述几乎可直接推广至此情形：例如，概率乘积分解可表示为：

$$p(x \mid c) = \prod_i p(x_i \mid x_{:i}, c)$$

我们基于先前的输入和用户的上下文信息来进行条件设定。采样与解码过程也以类似方式扩展。为实现条件生成，我们通过神经网络 $f(x, c)$ 对条件概率 $p(x_i \mid x_{:i}, c)$ 进行参数化：

$$p(x_i \mid x_{:i}, c) \approx \text{Categorical}(x_i \mid f(x_{:i}, c))$$

因此，条件生成与无条件情况的主要区别在于，需要具有双输入参数的函数 $f(x, c)$，并且在其第一个参数 $x$ 上满足因果性。使用自回归模型时，如果 $x$ 和 $c$ 均为文本，可以将 $c$ 视为输入序列的一部分，处理拼接的单输入 $x' = [c \| x]$。例如，对于用户提示词 "*The capital of France*"（法国的首都），采用单词分词器可得：[7]。

$$f_{\text{causal}}([\text{The}, \text{capital}, \text{of}, \text{France}]) = \text{is}$$
$$f_{\text{causal}}([\text{The}, \text{capital}, \text{of}, \text{France}, \text{is}]) = \text{Paris}$$

由此可见，我们可以使用单一模型同时处理无条件建模

[7]此处为简化说明，忽略用于终止自回归生成的序列结束（end-of-sequence，EOS）词元

与条件建模任务。[8] 在下一卷中，我们将探讨需要更复杂策略的条件生成模型，并在第11章讨论仅解码器架构的 transformer 模型时，进一步拓展该主题。

[8]第11章将阐述，几乎任何数据类型均可转换为词元序列。假设需要根据图像提示生成文本序列（例如**图像描述**），若文本与图像都被转换为具有相同嵌入维度的词元，则可通过拼接两种输入类型（在此语境中称为**模态**，modality）词元来应用自回归模型，此时将图像词元视为条件集合 $c$。

## 从理论到实践



处理文本数据比图像分类更为复杂，因为它涉及词元化、数据格式化、特殊字符处理以及变长序列等诸多细节。PyTorch 提供了专用文本库 `torchtext`，但截至本文撰写时，其文档完善度不及主库，且依赖另一个处于测试阶段的库（`torchdata`）来处理数据流水线。因此本文暂不展开介绍，建议读者自行探索。

在此领域，Hugging Face Datasets 堪称功能最全面的工具，提供海量数据集与预训练分词器，还能直接导出至 PyTorch。[9]在进行后续练习前，建议读者熟悉该库的基本操作。

1. 选择一个文本分类数据集，例如经典的 IMDB 数据集。[10]

2. 进行词元化处理，获得格式为 $(x, y)$ 的数据集，其中 $x$ 为如(E.8.4)所示的整数列表，$y$ 为文本标签。

3. 构建并训练类似于C.8.2的一维 CNN 模型。可通过调整模型设计观察对最终准确率的影响。

PyTorch 未提供直接实现一维因果卷积的快捷方式，因此，我们把自回归实验留待后续介绍 transformer 时展开。[11] 虽然训练自定义分词器是极佳的练习，但其超出本书范畴。作为入门，读者可参考以下极简 BPE 实现方法：https://github.com/karpathy/minbpe。

[9]教程可参考：https://huggingface.co/docs/datasets/use_dataset。

[10]https://huggingface.co/datasets/stanfordnlp/imdb

[11]如果读者想尝试，可以通过合理的填充来模拟因果卷积；参考此处课程讲义 10.2 讲: https://fleuret.org/dlc/。该课程整套流媒体讲座质量上乘。

# 9 | 扩展模型规模

**本章导读**

本章将重点探讨如何设计具有数十层（甚至数百层）的可微模型。如前文所述，卷积模型的感受野会随网络层数增加而线性增长，这促使了此类深度架构的设计。实现深层模型的关键在于通过多种技术手段稳定训练过程，包括数据增强与隐藏状态归一化等方法。

## 9.1 ImageNet 挑战赛

让我们再次聚焦图像分类任务——无论实践价值与历史意义，该任务在神经网络发展史上都具有重要地位。事实上，在 2012-2018 年间神经网络研究的复兴，很大程度上与 **ImageNet 大规模视觉识别挑战赛**[1]（ImageNet Large Scale Visual Recognition Challenge，简称 ILSVRC）密切相关。该年度竞赛在 2010 至 2017 年间举办，旨在评估图像分类领域的前沿模型。赛事使用 ImageNet 数据集的子集，包含约 100 万张图像，涵盖约 1000 个类别。

[1] https://image-net.org/challenges/LSVRC/

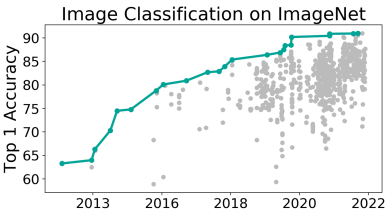



**图 F.9.1:** *ImageNet 数据集上的 Top-1 准确率。转载自 Papers With Code。*

早期赛事的结果颇具启示意义：2010 年[2] 与 2011 年[3]的冠军方案均采用线性核方法，结合专业图像描述符与核函数，其 top-5% 错误率分别为 28%（2010 年）和 26%（2011 年）。尽管取得了一些有潜力的成果，[4] 但基于梯度下降训练的卷积模型在计算机视觉领域仍属小众研究方向。随后，2012 年的冠军模型（AlexNet，[KSH12]）取得了 15.3% 的 top-5% 错误率，较所有（非神经网络）竞争对手降低超过 10% 。

这一突破引发了计算机视觉领域的“哥白尼式革命”（恕我们借用哥白尼之名），短短数年内，几乎所有参赛方案都转向卷积模型，整体准确率以空前速度提升至 95% 以上（直接导致该赛事于 2017 年终止），如图F.9.1所示。在五年间，基于梯度下降的卷积模型不仅成为计算机视觉的主导范式，更拓展至目标检测、语义分割、深度估计等众多子领域。

AlexNet 模型相对简单，由 5 个卷积层和 3 个全连接层组成，总计约 6000 万参数，而图F.9.1中的最新模型已需数百个网络层。这印证了缩放定律（第1章）所述：达到数据集决定的饱和点之前，增加网络层数与计算资源投入能与模型准确率形成比例关系。然而，扩展卷积模型并非易事，面临优化缓慢、梯度问题与数值稳定性等

[2] https://image-net.org/challenges/LSVRC/2010/

[3] https://image-net.org/challenges/LSVRC/2011/

[4] https://people.idsia.ch/~juergen/computer-vision-contests-won-by-gpu-cnns.html

多种挑战。因此，在 2012-2017 年间，研究者开发出大量技术，来稳定超大型模型的训练。

在本章中，我们将概述其中的一些技术。我们重点选取至今仍具基础性价值的方法，它们甚至也适用于其他架构（例如 transformer）。首先回顾机器学习中的三种著名训练优化技术：**权重正则化**（weight regularization）、**数据增强**（data augmentation）与**早停**（early stopping）。随后按时间顺序阐释 2012-2017 年间最具影响力的三种技术：**Dropout**（丢弃法）、**批量归一化**（batch normalization）及**残差连接**（residual connection）。针对每种方法，我们将介绍其基础算法，以及实践验证有效的改进版本（例如层归一化等变体）。

## 9.2 数据与训练策略

### 9.2.1 权重正则化

提升训练效果的一种方法是对不合理的结果施加惩罚（例如存在个别极大权重值的情况）。假设参数向量 $\mathbf{w}$ 表示模型全部参数，$L(\mathbf{w}, \mathcal{S}_n)$ 表示数据集上的损失函数（如平均交叉熵）。可以通过定义**正则化项**（regularization term）$R(\mathbf{w})$ 来形式化上述思想，该项基于我们的偏好对结果进行评估，并通过将正则化项添加到原始损失函数中来对损失施加惩罚：

$$L_{\text{reg}} = L(\mathbf{w}, \mathcal{S}_n) + \lambda R(\mathbf{w})$$

其中，假设 $R(\mathbf{w})$ 取值越大，对应的解越差，并且 $\lambda \geq 0$ 是用于权衡两项的标量。当 $\lambda = 0$ 时，正则化项没有影响；当 $\lambda \to \infty$，我们仅根据先验知识选择最佳函数。

这也可以解释为：基于权重的先验分布 $p(\mathbf{w})$ 与数据上的标准似然函数，执行最大后验（而非最大似然）推断（见3.3）：

$$\mathbf{w}^* = \arg\max_{\mathbf{w}} \{\log p(\mathcal{S}_n \mid \mathbf{w}) + \log p(\mathbf{w})\} \tag{E.9.1}$$

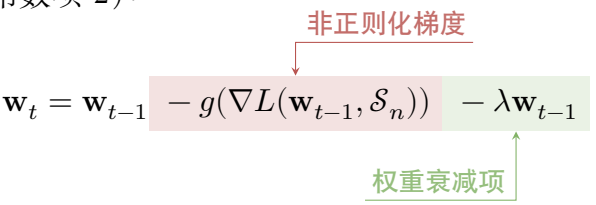
其中，引入正则化项对应一个非均匀先验分布 $p(\mathbf{w})$。我们在4.1.5中见过正则化的一个例子，即权重的 $\ell_2$ 范数：

$$R(\mathbf{w}) = \|\mathbf{w}\|^2 = \sum_i w_i^2$$

对于相同的未正则化损失，惩罚 $\ell_2$ 范数会倾向于权重幅值较小的解，这意味着当输入发生微小偏差时，输出的变化会更为“更不剧烈”。[5]现在考虑正则化对梯度项的影响：

$$\nabla L_{\text{reg}} = \nabla L(\mathbf{w}, \mathcal{S}_n) + 2\lambda \mathbf{w}$$

以这种形式表述时，它有时被称为**权重衰减**（weight decay），因为若没有第一项，其净效果是以一个小比例因子 $\lambda$（若 $\nabla L(\mathbf{w}, \mathcal{S}_n) = 0$，则权重会随迭代次数增加，呈指数级衰减至 0）。对于（随机）梯度下降（而言，$\ell_2$ 正则化与权重衰减等价。然而，对于其他类型优化算法（例如基于动量的随机梯度下降或 Adam），通常对梯度进行后处理。若用 $g(\nabla L(\mathbf{w}, \mathcal{S}_n))$ 表示（未正则化）损失函数的后处理梯度，则可将广义权重衰减公式写为（忽略常数项 2）：




[5]对于式(E.9.1)，$\ell_2$ 正则化项等价于对权重选择一个协方差矩阵为对角阵 $\sigma^2\mathbf{I}$ 的高斯先验。

这与纯 $\ell_2$ 正则化不同。在纯 $\ell_2$ 正则化中，正则化项的梯度位于 $g(\bullet)$ 内部。这对 Adam 等算法尤为重要，因为对它们而言，权重衰减形式（称为 **AdamW**[LH19]）往往效果更好。

我们还可以考虑其他类型的正则化项。例如，$\ell_1$ 范数:

$$R(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_i |x_i|$$

能倾向于稀疏（sparse）解，即包含大量零值的解（并且它对应于在权重施加拉普拉斯先验）。这一方法可以推广到群稀疏变体，对神经元施加结构化稀疏性约束[SCHU17].[6] 与其他机器学习模型相比，稀疏 $\ell_1$ 惩罚较少见，因为它不能很好地兼容优化问题的强非凸性以及梯度下降算法 [ZW23]。不过，可以通过重参数化（re-parameterize）优化问题来缓解该问题，代价是更大的内存占用。具体而言，[ZW23]展示了，我们可以用两个形状相同的向量 $\mathbf{a}$ 和 $\mathbf{b}$ 来替代 $\mathbf{w}$，并满足:

$$\mathbf{w} = \mathbf{a} \odot \mathbf{b}\ ,\ \|\mathbf{w}\|_1 \approx \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 \tag{E.9.2}$$

其中 $\approx$ 表示，在极其普遍的条件下，可以证明这两个问题是几乎等价的 [ZW23]。

通过考虑凸损失函数 $L(\cdot,\cdot)$（例如最小二乘损失），我们可以从几何角度直观理解正则化为何（以及如何）发挥作用。此时，正则化问题可重写为如下显式约束形式:

$$\begin{array}{ll} \arg\min & L(\mathbf{w}, \mathcal{S}_n) \\ \text{subject to} & R(\mathbf{w}) \leq \mu \end{array} \tag{E.9.3}$$

[6] 训练稀疏模型是一个宏大的主题，且与高效硬件执行密切相关。关于凸模型背景下的稀疏惩罚综述，请参阅 [BJMO12]；对于一般可微模型中的稀疏性和剪枝的综述见 [HABN+21]。

其中 $\mu$ 与 $\lambda$ 成正比，通过引入拉格朗日乘子重写(E.9.3)，可得无约束形式。在此情形下，$\ell_2$ 正则化对应于将解限制在以原点为圆心的圆（或球）内；而 $\ell_1$ 正则化则对应于将解限制在以原点为中心的正多面体内部（或其顶点上），其中稀疏解位于与坐标轴相交的顶点处。

### 9.2.2 早停机制

从优化角度来看，最小化函数 $L(\mathbf{w})$ 的任务是尽快地找到一个驻点，即找到满足 $\nabla L(\mathbf{w}_t) \approx 0$ 的点 $\mathbf{w}_t$：

$$\|L(\mathbf{w}_t) - L(\mathbf{w}_{t-1})\|^2 \leq \varepsilon$$

其中 $\varepsilon > 0$ 为容差。然而，这并不一定符合我们优化模型时的目标。特别是在数据量较少的情况下，训练时间过长可能导致过拟合。一般而言，只要能提升泛化能力的手段都是有益的，而不论其对 $L(\bullet)$ 的值或下降方向产生何种具体影响（例如权重衰减）。

**早停**（Early stopping）是区分纯粹优化与学习的一个简单例子。假设有一个独立于训练集和测试集的小型监督数据集，称为**验证集**（validation dataset）。在每轮（epoch）结束时，我们监测验证集上的某项指标（例如准确率或 F1 分数）。假设第 $t$ 轮的指标得分为 $a_t$。早停的核心思想是观察该指标是否持续提升：如果指标不再提升，说明模型可能进入过拟合状态，此时应停止训练。由于随机波动可能导致准确率出现小幅震荡，为了提高判断的稳健性，我们考察包含 $k$ 轮的窗口（即**耐心值**，patience）：

$$\text{如果} a_t \leq a_i, \quad \underbrace{\forall i = t-1, t-2, \ldots, t-k}_{\text{等待 } k \text{ 轮}} \rightarrow \text{停止训练}$$

当耐心值超参数 $k$ 更大时，算法会等待更久，但对潜在的波动更稳健。若具有模型权重保存机制（即**检查点**），checkpoint），还可以将权重回滚至指标最后一次提升时的状态，即对应第 $t-k$ 轮的权重。

早停机制可被视为**模型选择**（model selection）的一种简单形式，即根据特定指标来确定最佳的轮数。与模型本身的优化过程不同，在此过程中，我们可以针对任意感兴趣的指标（例如 F1 分数）进行优化，即便该指标不可导。

值得注意的是，对于大型过参数化模型，早停并不总是有益。因为训练轮数与验证误差之间的关系可能呈现非单调性，包含多次上升与下降的阶段（这种现象被称为**多次下降**（multiple descents）[RM22]），且损失值可能在长时间停滞后突然大幅下降 [PBE+22]。因此，早停主要适用于在小数据集上进行优化。

### 9.2.3 数据增强




通常，提升模型性能最有效的方法是增加可用数据的数量。然而，数据标注既耗时又昂贵；而人工生成数据（例如借助大语言模型）需要定制化的流水线才能有效运作 [PRCB24]。

在许多情况下，可以通过对现有数据应用预先设定的（保持语义不变的）变换，从而虚拟地增加可用数据量，以此在一定程度上缓解上述问题。举一个简单的例子，考虑向量输入 $\mathbf{x}$ 以及通过添加高斯噪声产生变换：

$$\mathbf{x}' = \mathbf{x} + \varepsilon,\ \varepsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$$

这相当于在以 $\mathbf{x}$ 为中心的一个球体内生成了近乎无限量的数据。此外，这些数据无需存储在磁盘上，且该过程可以在每次选取新的小批量数据时，实时应用变换来

模拟。事实上，研究表明，采用这种方式训练能增强模型的稳健性，且该方法与 $\ell_2$ 正则化存在关联 [Bis95]。然而，向量数据是非结构化数据，若添加方差过大的噪声，可能产生无效的数据点。

对于图像，我们有更好的方法，通常存在大量既能改变图像外观，又能保持其语义不变的变换。例如缩放、旋转、亮度调整、对比度变化等。假设 $T(x; c)$ 为此类变换（例如旋转），由参数 $c$（例如旋转角度）控制。大多数变换都将原始图像作为一种特例包含在内（例如，旋转角度 $c = 0$ 时）。**数据增强**（data augmentation）是在训练中，根据上述一种或多种变换对图像进行处理的过程：

$$x' = T(x; c),\ c \sim p(c) \tag{E.9.4}$$

其中 $p(c)$ 表示所有有效参数分布（例如，$-20°$ 到 $+20°$ 之间的旋转角度）。在训练期间，数据集中的每个样本在每轮中被采样一次，且每次采样均可应用不同的变换(E.9.4)，从而生成（虚拟）无限的独特数据点流。

数据增强在图像（或音频、视频等类似数据）处理中非常普遍，但它涉及诸多设计选择：包括哪些变换操作、考虑哪些参数，以及如何组合这些变换。一种名为 **RandAugment** [CZSL20]的简单策略考虑了广泛的变换操作，并针对每个小批量数据随机抽取少量操作（例如 2 或 3 个），以相同强度依次应用。不过，用户仍需验证这些变换是否合理（例如，在文本识别任务中，水平翻转可能会导致图像失效）。从实际应用的角度看，数据增强既可以作为数据加载组件的一部分（见框C.9.1），也可以作为模型的一部分来实现。

数据增强流水线和方法可能比简单变换更复杂。即使对于更复杂的类型，其直观思想仍然是：只要模型能在复杂场景下完成任务（例如在各种光照条件下识别物体），那么它在真实且条件温和的场景中理应表现更好。此外，

```python
# 图像张量 (b, c, h, w)
img = torch.randint(0, 256,
            size=(32, 3, 256, 256))

# 数据增强流水线
from torchvision.transforms import v2
transforms = v2.Compose([
    v2.RandomHorizontalFlip(p=0.5),
    v2.RandomRotation(10),
])

# 应用数据增强流水线:
# 每次函数调用都会
# 从相同的输入张量
# 返回不同的小批量数据
img = transforms(img)
```

**框 C.9.1:** 包含两个依次执行的变换操作的数据增强流水线，示例来自 *torchvision*库。在 *PyTorch* 中，增强操作可以传递给数据加载器，可以独立使用。在 *TensorFlow* 和 *Keras* 等其他框架中，数据增强也可以原生地作为模型内部的层集成。

数据增强能避免重复使用相同输入，从而防止过拟合。

作为更复杂方法的示例，我们介绍针对向量的 **mixup** [ZCDLP17]，及其面向图像的扩展版 **cutmix** [YHO+19]。对于前者，假设我们对两个样本 $(\mathbf{x}_1, y_1)$ 和 $(\mathbf{x}_2, y_2)$ 采样，mixup 的思想是创建一个新的虚拟样本，该虚拟样本由二者的凸组合构成：

$$\mathbf{x} = \lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2 \tag{E.9.5}$$

$$y = \lambda y_1 + (1 - \lambda) y_2 \tag{E.9.6}$$

其中 $\lambda$ 在区间 $[0, 1]$ 内随机选取。该过程应促使模型在这两个样本之间产生简单的（线性）输出，避免输出发

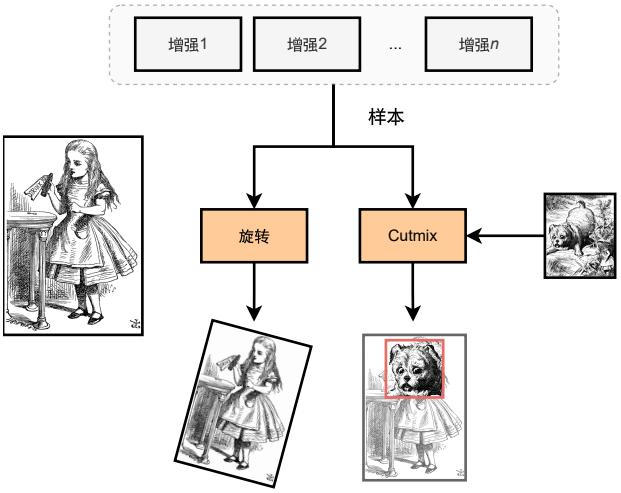



**图 F.9.2:** 数据增强的总体概览。对于每个小批量，从基础集合中随机采样一组数据增强操作，并将其应用于该小批量的图像。此处展示旋转和 *cutmix* 的示例。原图由约翰 · 坦尼尔绘制，转载自 *Wikimedia*。

生突变。从几何角度来看，对于两个相近的点，我们可以将(E.9.6)视为沿着连接这两点的直线在数据流形上缓慢移动的过程，即随着 $\lambda$ 从 0 变化到 1 的过程。

mixup 可能不适用于图像，因为对两幅图像进行逐像素线性插值会产生模糊的图像。通过 **cutmix**，我们改为从第一幅图像中截取一个固定形状（例如 $32 \times 32$）的图像块。假设 $\mathbf{M}$ 为一个与图像形状相同的二值掩码，其中图像块内像素对应值为 1，图像块外像素对应值为 0。在 cutmix 中，我们将两张图像 $x_1$ 和 $x_2$ 结合，具体做法是将第一张图像中的一个区域“拼接”到第二张图像上：

$$x = \mathbf{M} \odot x_1 + (1 - \mathbf{M}) \odot x_2$$

同时，标签仍与之前一样，用随机系数 $\lambda$ 进行线性插值。

同时使用旋转和 cutmix 进行数据增强的示例见图F.9.2。

## 9.3 Dropout 与归一化

上一节中描述的策略非常通用，因为它们涉及对优化算法或数据集本身作修改，并且可以广泛应用于多种算法。

相反，我们现在重点关注三种在 2012 到 2016 年间，尤其在 ImageNet 挑战赛的背景下，流行起来的思想。这三种方法都专门针对可微模型，因为它们可以作为额外层或连接实现，从而简化超深模型的训练。我们将按大致的时间顺序列出这些方法。正如我们在后续章节中将看到的，这些方法不仅适用于卷积模型，而且在更广泛的场景中依然重要。

### 9.3.1 通过 Dropout 实现正则化

在讨论数据增强时，我们提到过一种观点：数据增强迫使网络在更困难的条件下进行学习，从而提升其在简单环境下的准确率和稳健性。**Dropout** [SHK+14]将这一思想扩展到模型的内部嵌入：通过在训练期间人为地向模型的中间输出引入噪声，可以改善最终解。

噪声类型有多种选择：例如，在循环模型相关文献中，在激活值中加入少量高斯噪声是一种流行方案。顾名思义，Dropout 的核心思想是在计算过程中随机移除某些单元（神经元），从而降低对任何单一内部特征的依赖，并（期望）训练出具有良好冗余性的稳健层。

我们以全连接层为例来定义 Dropout，这是其最常见的应用场景。

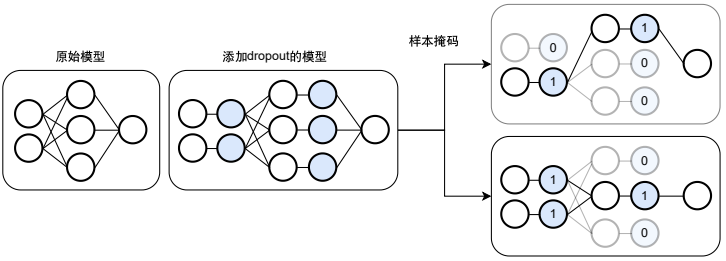



**图 F.9.3:** *Dropout* 的示意图：以基础模型为起点，在每个关注的层之后添加额外单元（图中以蓝色标示）。在训练阶段，每个 *Dropout* 单元被随机赋予一个二值（*0* 或 *1*），从而掩码部分前层输出。因此，每次进行前向传播时，我们实际上是从指数多个有活跃隐藏单元子集的可能模型中选择一个。*Dropout* 也可以应用于输入层，即随机移除部分输入特征。




**定义 D.9.1 (Dropout 层)**

假设 $\mathbf{X} \sim (n, c)$ 表示模型内部激活值的一个小批量（例如，某个中间全连接层的输出），其中包含 $n$ 个样本，$c$ 个特征。在 *Dropout* 层中，我们首先采样一个尺寸相同的二值矩阵 $\mathbf{M} \sim Binary(n, c)$，其元素从以概率 $p$ 为参数的伯努利分布中抽取（其中 $p \in [0, 1]$ 为用户指定的超参数）[a]

$$M_{ij} \sim Bern(p) \tag{E.9.7}$$

对输入进行掩码操作，得到该层的输出：

$$\text{Dropout}(\mathbf{X}) = \mathbf{M} \odot \mathbf{X}$$

该层仅包含一个超参数 $p$，没有可训练参数。

[a] $\text{Bern}(p)$ 中的样本以概率 $p$ 取值 1 ，以概率 $1-p$ 取值 0。

```
model = nn.Sequential(
    nn.Dropout(0.3),
    nn.Linear(2, 3), nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(3, 1)
)
```

**框 C.9.2:** 图*F.9.3*中的模型在 *PyTorch* 中实现为四层序列。在训练中，由于存在两个 *Dropout* 层，模型输出具有随机性。

我们称 $1-p$ 为**丢弃概率** (drop probability)。此，对于小批量中的任意元素，会有随机数量的单元(约占 $(1-p)\%$)被置零，从而有效地移除它们。

如图F.9.3所示，其中添加的 Dropout 单元以蓝色标示。掩码的采样过程属于该层前向传播的一部分：对于两次不同的前向传播，由于被掩码的元素不同，输出结果也会不同，如图F.9.3右侧所示。

如图所示，我们可以将 Dropout 实现为一个层，并将其插入到每个我们希望施加 Dropout 的层之后。例如，考虑图F.9.3所示的两层全连接模型：

$$y = (\mathrm{FC} \circ \mathrm{FC})(\mathbf{x})$$

在输入端以及第一层的输出端添 Dropout 正则化后，会得到一个包含四层的新模型：

$$y = (\mathrm{FC} \circ \mathrm{Dropout} \circ \mathrm{FC} \circ \mathrm{Dropout})(\mathbf{x})$$

PyTorch 具体实现如框C.9.2所示。

尽管 Dropout 能提升模型性能，但由于 Dropout 层内部涉及不同掩码的采样，输出 $y$ 变成了随机变量，这不是

我们在训练完成后想要的。例如，对网络进行两次前向传播可能会产生两个不同的输出，且某些采样结果（如包含大量零值的掩码）可能并非最优。因此，我们需要一种策略，用确定性运算来替代前向传播过程。

假设模型包含 $m$ 个 Dropout 层。其中 $\mathbf{M}_i$ 表示第 $i$ 个 Dropout 层的掩码，$p(\mathbf{M}_1, \dots, \mathbf{M}_m) = \prod_{i=1}^{m} p(\mathbf{M}_i)$ 表示这些掩码的联合概率分布，$f(\mathbf{x}; \mathbf{M})$ 表示在选定一组掩码 $\mathbf{M} \sim p(\mathbf{M})$ 后，模型产生的确定性输出。一种方案是在推理阶段用期望值代替 Dropout 效应：

$$f(\mathbf{x}) = \begin{cases} f(\mathbf{x}; \mathbf{M}), \ \ \mathbf{M} \sim p(\mathbf{M}) & \text{[训练]} \\ \mathbb{E}_{p(\mathbf{M})} \left[ f(\mathbf{x}; \mathbf{M}) \right] & \text{[推理]} \end{cases}$$

我们可以通过蒙特卡洛采样（见附录A）来近似该期望值，即重复采样掩码并取平均值：

$$\mathbf{E}_{p(\mathbf{M})} \left[ f(\mathbf{x}; \mathbf{M}) \right] \approx \frac{1}{k} \sum_{i=1}^{k} f(\mathbf{x}; \mathbf{Z}_i), \ \ \mathbf{Z}_i \sim p(\mathbf{M})$$

这本质上是 $k$ 次前向传播结果的平均值。这被称为**蒙特卡洛 Dropout**（Monte Carlo Dropout）[GG16]。其输出仍然是随机的，但通过恰当选择 $k$ 的值，可以将方差控制在一定范围内。此外，不同前向传播过程的输出还能提供对预测不确定性的度量。

然而，执行多次前向传播可能计算成本过高。一种更简单、更常用的方法是逐层换随机变量，这是一种合理的近似。在这种情况下，期望值可以写成闭式：

$$\mathbb{E}_{p(\mathbf{M})} \left[ \text{Dropout}(\mathbf{X}) \right] = p\mathbf{X}$$

```
x = torch.randn((16, 2))

# 使用 Dropout 训练
model.train()
y = model(x)

# 使用 Dropout 推理
model.eval()
y = model(x)

# 用于推理的蒙特卡洛 Dropout
k = 10
model.train()
y = model(x[:, None, :].repeat(1, k, 1))
                       .mean(1)
```

**框 C.9.3:** 将框*C.9.2*中的模型应用于包含 16 样本的小批量数据。对于像 *Dropout* 这样的层，框架需要一种机制来区分前向传播是在训练阶段还是推理阶段执行的。在 *PyTorch* 中，这是通过调用模型的*train*和*eval*方法实现的。这些方法会设置所有层内部的*train*标志。我们还展示了蒙特卡洛 *Dropout* 的向量化实现。

即输出按常数因子 $p$(掩码中采样到 1 的概率)进行缩放。由此产生了一种更简单的形式，**反向 Dropout**（inverted dropout），它在训练期间就考虑了这种修正：

$$\text{Dropout}(\mathbf{X}) = \begin{cases} \dfrac{\mathbf{M} \odot \mathbf{X}}{p} & \text{[训练]} \\ \mathbf{X} & \text{[推理]} \end{cases}$$

在这种情况下，Dropout 层在推理阶段不起作用，可以直接移除。这是大多数框架首选的实现方式。相关比较请参阅框C.9.3。

如前文所述，Dropout（通常采用较低的丢弃概率 $1-p$，即保留概率 $p = 0.8$ 或 $p = 0.9$）在全连接层中很常见。

它也常用于注意力图（将在下一章介绍）。但在卷积层中较少使用，因为丢弃输入张量的单个元素会导致过于无序的稀疏模式。人们设计了一些考虑图像特定结构的 Dropout 变体：例如，**空间 Dropout**（spatial dropout）[TGJ+15]丢弃张量的整个通道，而 **Cutout** [DT17]丢弃单个通道中的空间图像块。

还有其他替代方案。例如 **DropConnect** [WZZ+13]丢弃全连接层中的单个权重：

$$\text{DropConnect}(\mathbf{x}) = (\mathbf{M} \odot \mathbf{W})\mathbf{x} + \mathbf{b}$$

在推理时，DropConnect 也可以通过矩匹配进行高效近似 [WZZ+13]。然而，这些方法在实践中较少使用，通常更倾向于下文介绍的技术。

### 9.3.2 批量归一化与层归一化




在处理表格数据时，我们尚未讨论的一种常见预处理操作是**归一化**（normalization），即确保所有特征（输入矩阵的所有列）具有相似的数值范围和统计特性。例如，我们可以对数据预处理，将所有列的值压缩到 $[0, 1]$ 范围内（称为**最小-最大归一化**，min-max normalization），或确保每一列的均值为 0 且方差为 1（称为**标准缩放**，**正态缩放**或 **z-score 缩放**）。

**批量归一化**（batch normalization，BN，[IS15]）将这些思想应用到了模型的中间嵌入上。这并非平凡工作，因为在每次梯度下降更新后，一个单元的统计特性（例如其均值）都会产生变化。因此，若要计算某个单元的均值，理论上需要在每次迭代时，对整个训练数据集进行前向传播，但这在实际操作中不可行。顾名思义，BN 的核心思想是仅利用小批量内部的数据来近似这些统计量。

再次考虑任意全连接层的输出 $\mathbf{X} \sim (n, c)$，其中 $n$ 是小批量的大小。稍后我们将探讨如何将这些思想推广到图

像及其他类型数据上。在 BN 中，我们仅基于当前小批量数据，对每个特征（即 $\mathbf{X}$ 的每一列）进行归一化处理，使其均值为 0 且方差为 1。为此，我们首先计算按列的经验均值 $\mu \sim (c)$ 和方差 $\sigma^2 \sim (c)$：

$$\text{第}j\text{列的均值:} \qquad \mu_j = \frac{1}{n}\sum_i X_{ij} \tag{E.9.8}$$

$$\text{第}j\text{列的方差:} \qquad \sigma_j^2 = \frac{1}{n}\sum_i (X_{ij} - \mu_j)^2 \tag{E.9.9}$$

随后，我们对各列进行归一化：

将列均值设为 0

$$\mathbf{X}' = \frac{\mathbf{X} - \mu}{\sqrt{\sigma^2 + \varepsilon}}$$

将列方差设为 1

此处遵循标准的广播规则（即 $\mu$ 和 $\sigma^2$ 沿第一个维度进行广播），其 $\varepsilon > 0$ 是一个极小的正数，用于避免除以零的情况。它与针对表格数据的归一化处理有所不同，后者通常在训练前对整个数据集执行一次该操作，而在批量归一化中，该操作必须在每次前向传播过程中，针对每个小批量重新计算。

选择零均值和单位方差只是一种惯例，未必是最优选择。为了将其推广，可以让优化算法来自行选择最佳参数，这仅需增加少量的参数开销。考虑两个可学习参数 $\alpha \sim (c)$ 和 $\beta \sim (c)$（初始值可分别设为 1 和 0），执行如下操作：

$$\mathbf{X}'' = \alpha\mathbf{X}' + \beta$$

其中采用与上文相似的广播规则。变换后的矩阵，其第

$i$ 列的均值为 $\beta_i$，方差为 $\alpha_i$。BN 层由这两种操作的组合定义。




**定义 D.9.2 (批量归一化层)**

给定输入矩阵 $\mathbf{X} \sim (n, c)$，*a* **批量归一化** (*BN*) 层应用如下归一化操作：

$$\mathrm{BN}(\mathbf{X}) = \alpha \left( \frac{\mathbf{X} - \mu}{\sqrt{\sigma^2 + \varepsilon}} \right) + \beta$$

其中 $\mu$ 和 $\sigma^2$ 根据公式 (E.9.8) (E.9.9)计算得出，而 $\alpha \sim (c)$ 和 $\beta \sim (c)$ 为可学习参数。该层不包含超参数。在推理阶段，$\mu$ 和 $\sigma^2$ 的值固定，具体方式见下文。

该层仅有 $2c$ 个学习参数，并且实践证明，在各个模块中插入该层可以极大地简化复杂模型的训练过程。特别地，通常将 BN 置于模型的线性组件与非线性组件之间：

$$\mathbf{H} = (\mathrm{ReLU} \circ \mathrm{BN} \circ \mathrm{Linear})(\mathbf{X})$$

在 ReLU 之前对数据进行中心化处理，有助于更有效地利用其负（稀疏）象限。此外，这种配置使得线性层中的偏置项变得多余（因为它会与 $\beta$ 参数合并），从而可将其移除。最后，这种双重线性运算能够被大多数框架中的标准编译器轻松优化。

批量归一化成效显著，引发了大量关于其背后原理的研究 [BGSW18]。其最初的推导基于一个被称为**内部协变量偏移** (internal covariate shift) 的问题，即从单层网络的角度来看，由于前层权重的变化，该层接收到的输入数据的统计特性会在优化过程中发生改变。然而，现

有文献普遍认为，BN 的主要优势体现在优化过程本身，即提高了训练稳定性并允许使用更高的学习率，这归功于其对梯度产生的缩放（scaling）和中心化（centering）效应 [BGSW18]。[7]

将 BN 推广到表格数据以外的场景非常简单。例如，考虑一个图像嵌入小批量数据 $X \sim (n, h, w, c)$。我们可以通过将前三个维度视为整体，对每个通道应用 BN；即计算通道维度的均值如下：

$$\mu_z = \frac{1}{nhw} \sum_{i,j,k} X_{ijkz}$$

通道 $z$ 的均值（所有像素）

**推理阶段的批量归一化**

BN 使得对某个输入的预测结果依赖于其所在的小批量数据，这在推理阶段是不合理的（换言之，将同一张图像从一个小批量移动到另一个小批量，会导致其预测结果变化）。然而，我们可以利用模型参数在训练结束后保持不变这一特性，将均值和方差固定为预设值。为此，有两种常见的做法：

1. 训练结束后，在整个训练集上再次进行前向传播，计算基于该数据集的经验均值和方差 [WJ21]。

2. 更常见的方法是，维护一组滚动统计量，在训练期间，每次模型前向传播后对其进行更新，并在训练结束后使用它们。为简化起见，仅以均值为例，假设我们初始化另一个向量 $\hat{\mu} = \mathbf{0}$ 对应于“均值的滚动平均”。按照公式(E.9.8)计算 $\mu$ 后，我们利用

[7] 另见 https://iclr-blog-track.github.io/2022/03/25/unnormalized-resnets/，这是关于该领域文献（以及开发**无归一化层**（normalizer-free）模型的相关文献）的精彩入门介绍

指数移动平均更新滚动均值：

$$\hat{\mu} \leftarrow \lambda\hat{\mu} + (1-\lambda)\mu$$

其中 $\lambda$ 设为一个较小值，例如 $\lambda = 0.01$。假设训练过程收敛，滚动均值也将收敛到选项 (1) 所述平均值的近似值。因此，训练结束后，我们可以将 $\mu$ 替换为（预先计算好的）$\hat{\mu}$ 来使用 BN，方差的处理方式同理。[8]

### 批量归一化的变体

尽管批量归一化在实践中表现良好，但它也存在一些显著缺陷。我们之前已经提到过它对小批量数据的依赖，这会带来其他问题：例如，当小批量规模较小时，训练过程中均值 $\mu$ 的方差会变得很大；若小批量规模极小，甚至可能导致无法进行训练。此外，在分布式训练场景下（即每个 GPU 仅处理小批量数据的一部分），训练过程也可能更困难。最后，在训练结束后用其他值替代 $\mu$ 会导致训练与推理阶段之间出现不匹配现象。

为了解决这些问题，人们提出了一些 BN 的变体。一种常见的思路是保留该层的整体结构，但改变执行归一化操作的轴向。例如，**层归一化**（Layer Normalization）[BKH16] 针对矩阵的行（即针对每个输入样本独立地）计算经验均值和方差：

[8] $\hat{\mu}$ 是层张量的一个例子：它属于层状态的一部分，在训练过程中进行更新，但梯度下降并不需要它。在 PyTorch 中，这类张量被称为 *缓冲区*（buffers）。

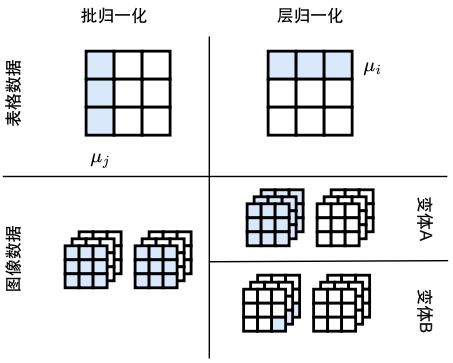



**图 F.9.4:** 表格数据和图像数据的 *BN* 与 *LN* 对比。蓝色区域表示计算均值和方差时所覆盖的数据范围。对于 *LN*，我们有两种变体，正文中对此有更详细的讨论。

$$\text{第 } i \text{ 行的均值:} \quad \mu_i = \frac{1}{c}\sum_j X_{ji} \tag{E.9.10}$$

$$\text{第 } i \text{ 行的方差:} \quad \sigma_i^2 = \frac{1}{c}\sum_j (X_{ji} - \mu_i)^2 \tag{E.9.11}$$

考虑图F.9.4，其中展示了表格数据和类图像数据 BN 与 LN 的对比。具体而言，图中用蓝色标示了用于计算单个均值和方差的所有样本。对于 LN，我们可以同时在 $h, w, c$ 维度上计算统计量（变体 A），也可以对每个空间位置单独计算（变体 B）。后者在 transformer 模型中很常见，我们将在下一章讨论。还存在其他变体，例如**组归一化** (group normalization) 将操作限制在通道的一个子集上；当子集仅包含单通道时，被称为**实例归一化** (instance normalization)。[9]

在 BN 中，计算 (E.9.8)和(E.9.9) 所示统计量时所沿的轴，与应用可学习参数时所沿的轴是相同的。而在 LN 中，这两者是解耦的。例如，考虑一个应用于维度为 $(b, 3, 32, 32)$ 的小批量数据的 PyTorch LN 层：

[9] 见https://iclr-blog-track.github.io/2022/03/25/unnormalized-resnets/，其中有图F.9.4的一个更直观的变体。

```
nn.LayerNorm(normalized_shape=[3, 32, 32])
```

这对应于图F.9.4中的变体 A。在这种情况下，$\alpha$ 和 $\beta$ 的形状将与计算归一化所沿的轴相同，即 $\alpha, \beta \sim (3, 32, 32)$，总共包含 $2 \times 3 \times 32 \times 32 = 6144$ 个可学习参数。针对不同框架和模型，必须核查 LN 和 BN 的具体实现方式。

最后，我们介绍层归一化的另一个常见变体，称为**均方根**归一化（RMSNorm）[ZS19]。它通过去除均值中心化和平移操作来简化 LN；对于单个输入向量 $\mathbf{x} \sim (c)$，其公式可表示为：

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{c}\sum_i x_i^2}} \odot \alpha \tag{E.9.12}$$

当 $\beta = 0$ 且数据已经零中心化时，LN 与 RMSNorm 是等价的。

## 9.4 残差连接

### 9.4.1 残差连接与残差网络




结合上一节介绍的各种技术，能够显著增加模型的层数，但只能增加到某个上界。考虑三个通用的层 $f_1$、$f_2$ 和 $f_3$，以及两个模型 $g_1$、$g_2$，其中 $g_1$ 是 $g_2$ 的子集：

$$g_1(x) = (f_3 \circ f_1)(x)$$
$$g_2(x) = (f_3 \circ f_2 \circ f_1)(x)$$

直观上，根据通用逼近定理，中间部分 $f_2$ 应当总是能逼近恒等函数（即 $f_2(x) \approx x$），在这种情况下，$g_2(x) \approx g_1(x)$。因此，理论上总存在一组参数设置，使得第二个（更深）模型的表现至少不差于第一个（较浅）模型。然

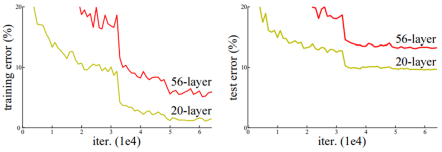



Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

**图 F.9.5:** 更大模型的训练误差并不总是呈单调下降趋势，尽管它们表示了更大的函数类。引自 *[HZRS16]*.

而，实际情况并非如此，如图 F.9.5 所示。

我们可以通过引导网络中的模块趋向于恒等函数来解决这一问题。这可以通过引入所谓的**残差（跳跃）连接** [HZRS16]来简单实现，将模块 $f(x)$ 重写为：

$$r(x) = f(x) + x$$

因此，我们利用该模块来建模相对于恒等映射的偏差（即 $f(x) = r(x) - x$），而不是建模相对于零函数的偏差。仅凭这一小技巧，就有助于训练多达数百层的模型。我们将 $f(x)$ 称为**残差路径**（residual path），$r(x)$ 称为**残差块**（residual block），并将由残差块构成的卷积模型称为**残差网络**（residual network，简称 Resnet）。

残差连接与残差路径上的批量归一化结合使用效果良好，这种组合能促使模型在训练初期更倾向于表现为恒等映射 [DS20]。然而，只有当 $f(x)$ 的输入和输出维度相同时，才能直接添加残差连接；否则，需要在残差连接中加入重缩放操作。例如，若 $x$ 为图像且 $f(x)$ 改变了通道数，我们可以引入一个 $1 \times 1$ 卷积：

$$r(x) = f(x) + \text{Conv2D}_{1\times1}(x)$$

残差块的优势也可以从其反向传播过程来理解。考虑残差块的向量-雅可比乘积（VJP）：

$$\text{vjp}_r(\mathbf{v}) = \text{vjp}_f(\mathbf{v}) + \mathbf{v}^\top \mathbf{I} = \underbrace{\text{vjp}_f(\mathbf{v})}_{f \text{ 的 VJP}} + \underbrace{\mathbf{v}^\top}_{\text{跳跃连接的 VJP}}$$

因此，在前向传播中，输入 $x$ 经跳跃连接无改动地通过；而在反向传播中，未被修改的反向传播梯度 $\mathbf{v}$ 被加到了原始 VJP 上，这有助于缓解梯度不稳定性问题。.

## 残差块的设计

如何设计模块 $f(x)$？考虑之前介绍过的批量归一化模块：

$$h = \underbrace{(\text{ReLU} \circ \text{BN} \circ \text{Conv2D})(x)}_{=f(x)} + x$$

由于 ReLU 的输出始终为非负值，因此满足 $h \geq x$（逐元素比较）。这意味着，堆叠这种形式的残差块只会增大输入张量的值，或将其置零。正因如此，[HZRS16]中提出的原始设计通过移除最后一个激活函数来构建残差块堆叠。例如，对于两个残差块，其设计如下：

$$h = (\text{BN} \circ \text{Conv2D} \circ \text{ReLU} \circ \text{BN} \circ \text{Conv2D})(x) + x$$

在一系列此类残差块之前，通常会放置一个不含残差连接的小型组件用于降低图像维度，该组件有时被称为**入口层**（stem）。多年来，针对该残差块的超参数具体选择也发生了显著变化。

原版 ResNet 模块在第一步操作中压缩了通道数，随后进行标准的 $3 \times 3$ 卷积，最后再将通道数升维。近年来，诸如 ResNeXt 模块 [LMW+22]（见图F.9.6右侧）这类**瓶颈**（bottleneck）层变得日益流行。为了增大卷积的感受

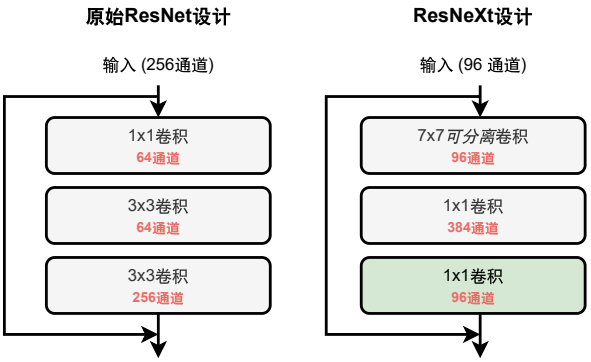



**图 F.9.6:** 原版 *ResNet* 模块 *[HZRS16]*，与较新的 *ResNeXt[LMW+22]*模块. 如图所示，设计思路已从早期的通道降维转变为后期的压缩（**瓶颈**结构）。图中未展示的其他细节包括从 *BN* 切换到 *LN*，以及使用 *GELU* 激活函数。改编自 *[LMW+22]*。

野，初始层被替换为深度卷积（depthwise convolution）。为了充分利用参数量的减少，通道数会先按给定倍数因子（例如 3×、4×）增加，再通过最后的 $1 \times 1$ 卷积缩减。

### 9.4.2 残差连接的额外视角




本章最后，我们将讨论关于残差连接使用的两个有趣视角，这两个视角在当前研究中均已被深入探讨。首先，考虑一个由两个残差块组成的网络：

$$h_1 = f_1(x) + x \tag{E.9.13}$$
$$h_2 = f_2(h_1) + h_1 \tag{E.9.14}$$

如果展开计算过程：

$$h_2 = f_2(f_1(x) + x) + f_1(x) + x$$

这对应网络中多条路径之和，其中输入要么不变，要么仅经过单次变换（$f_1$ 或 $f_2$），要么经过它们的组合变换。

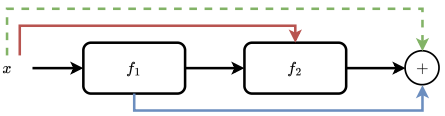



**图 F.9.7:** 残差路径：黑色、红色和蓝色路径是显式实现的；绿色路径仅为隐式的。

显然，此类路径的数量随残差块数量的增加呈指数级增长。因此，深度残差模型可以被视为通过权重共享实现的、大量较小模型的组合（集成）。这一观点已得到验证，例如，研究表明 ResNet 对其组件的微小删减或修改具有鲁棒性 [VWB16]。图 F.9.7展示了这一点。

然后，考虑以下关于代表时间的连续参数 $t$ 的微分方程：

$$\partial_t x_t = f(x, t)$$

我们利用以 $x$ 和 $t$（标量）为自变量的神经网络，来参数化某个函数的时间导数。这被称为**常微分方程**（ordinary differential equation，ODE）。求解 ODE 时的一个常见问题是从已知的初始值 $x_0$ 积分至指定的时刻 $T$：

$$x_T = x_0 + \int_{t=0}^{T} f(x, t)dt$$

用于计算 $x_T$ 的欧拉法[10]选取一个较小步长 $h$，并通过迭代计算一阶离散化形式来实现：

$$x_t = x_{t-1} + hf(x_{t-1}, t)$$

若将 $h$ 合并到 $f$ 中，这便对应于一种受限形式的残差模型：其中所有残差块共享相同权重，每一层对应一个离散的时间点，而 $x_T$ 为网络的输出。基于这种观点，我们可以直接处理原始的连续时间方程，并利用现代 ODE 求解器对其积分来计算输出。这种方法被称为**神经 ODE**（neural ODE）[CRBD18]。我们可以推导出反

[10] https://en.wikipedia.org/wiki/Euler_method

向传播算法的连续时间变体，其形式也是一个 ODE 问题。在下一卷中，我们将探讨神经 ODE 与一类被称为**归一化流**（normalizing flows）的生成模型之间的有趣联系。[PNR+21].

## 从理论到实践



本章讨论的所有层（如批量归一化、Dropout 等）都已在 PyTorch、Equinox 以及几乎所有其他框架中实现。至于我们描述的其他技术，情况则因框架而异：例如，权重衰减（weight decay）已在所有 PyTorch 优化器中原生实现；数据增强（data augmentation）功能可以在 `torchvision`（及其他相应库）中找到对应的变换；而早停则必须手动实现。[11]

1. 在进入下一章之前，建议尝试仅使用标准的线性代数运算来实现 Dropout 或批量归一化层，并将结果与内置层进行比较。

2. 在第7章中，你应该已经实现了一个用于图像分类的简单卷积模型。尝试逐步增加模型规模，并根据需要添加归一化、Dropout 或残差连接。

3. 选一种标准架构，例如 ResNet[HZRS16]或 ResNeXt[LMW+22]。尝试根据原始论文的建议实现整个模型。在消费级 GPU 上使用类似 ImageNet 的数据集进行训练可能具有挑战性——如果你无法使用高性能硬件或云端 GPU 资源，可以继续使用较简单的数据集（如 CIFAR-10）进行练习。

[11]在 PyTorch 中，一种常见的替代方案是使用 PyTorch Lightning 等外部库来管理训练过程。针对训练流程的修改（如早停）通常已预先实现为回调（callback）函数。

4. 你可能已经意识到，在较小的数据集上从头训练超大型模型（例如 ResNet-50）实际上是不可行的。一种解决方案是利用在线资源库中的权重（例如在 ImageNet 上训练过的模型权重）来初始化模型，并通过修改最后一层（对应于分类头）来**微调**模型。读到这里，这应该已不再困难——我建议使用 torchvision 或 Hugging Face Hub 上可用的众多预训练模型之一。[12]我们将在下一卷中更深入地探讨微调技术。

[12]示例教程见：https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html.

# Part III
# 钻进兔子洞



“如果我能看到
正常点儿的事情
就好了。”

**—爱丽丝梦游仙境，**
**1951 年电影**

# 10 | transformer 模型

**本章导读**

卷积模型是强有力的基准模型，特别是在局部关系占主导地位的图像和序列任务中。然而，在处理超长序列，或序列元素间的非局部依赖关系时，它们存在局限性。本章将介绍另一类旨在克服这些挑战的模型——transformer。

## 10.1 长卷积与非局部模型

继上一章讨论的 2012-2016 年间的关键发展之后，可微模型设计领域的下一个重要突破出现在 2016-2017 年，即 **transformer** [VSP+17]架构的普及。该架构专为高效处理自然语言处理中的长距离依赖关系而设计。凭借其强大的缩放特性，该架构随后被推广应用到图像、时间序列和图数据等多种数据类型中；如今，由于其在海量数据训练下表现出的优异扩展性能，它已成为众多领域中的最先进（state-of-the-art）模型 [KMH+20, BPA+24]。

正如我们将看到的，transformer 的一个有趣方面是数据类型（通过使用适当的分词器）与架构之间的解耦，后者在很大程度上与具体数据类型无关。这开辟了一些有趣的方向，例如简单的多模态架构和迁移学习策略。我们首

先引出 transformer 的核心组件，称为**多头注意力**（multi-head attention，MHA）层。我们将把关于 [VSP+17]中原始 transformer 模型的讨论留待下一章进行。

**历史背景**

从历史发展的时间顺序来看，本章的编排顺序有些特殊：在 2015 年，文本处理任务中 CNN 的主流替代方案是**循环神经网络**（recurrent neural network，RNN）。MHA 最初作为一种独立组件引入 RNN[BCB15]，后来才成为 transformer 模型的核心组件。我们将在第13章探讨 RNN 及其现代变体——线性化 RNN。近年来 RNN 已成为语言建模领域中 transformer 的有力竞争者。

### 10.1.1 处理长程与稀疏依赖关系

考虑以下两个句子：

> “*The cat is on the table*”

以及一个更长的句子：

> “*The cat, who belongs to my mother, is on the table*” .

为了能被可微模型处理，必须对句子进行词元化，并将词元嵌入为向量（第 8 章）。从语义角度看，属于红色单词（cat）和绿色单词（table）的词元在两个句子中具有相似的依赖关系。然而，它们的相对偏移量在两种情况下不同，且它们的距离可以变得任意大。因此，文本中的依赖既可以是**长程的**，也可以是**依赖于输入的**。

假设 $\mathbf{X} \sim (n, e)$ 表示一个由 $n$ 个词元组成的句子，其中每个词元被嵌入为 $e$ 维向量，$\mathbf{x}_i$ 为第 $i$ 个词元。我们可

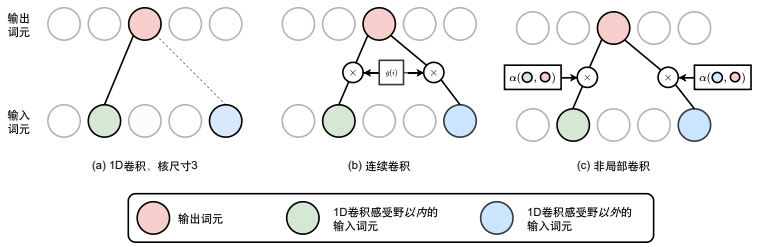



**图 F.10.1:** 一维序列中不同类型卷积的比较。图中展示了一个输出词元（红色）如何与另外两个词元发生交互：一个位于卷积感受野内（绿色），另一个位于感受野外（蓝色）。*(a)* 在标准卷积中，蓝色词元因位于滤波器感受野之外而被忽略。*(b)* 在连续卷积（*continuous convolution*）中，两个词元均被考虑，相应的权重矩阵分别由 $g(-1)$ 和 $g(2)$ 给出。*(c)* 在非局部（*non-local*）情况下，权重矩阵取决于词元自身之间的成对比较。

以将针对词元 $i$、核大小为 $k$ 的一维卷积重写如下：

$$\mathbf{h}_i = \sum_{j=1}^{2k+1} \mathbf{W}_j \mathbf{x}_{i+k+1-j} \tag{E.10.1}$$

感受野内的每个词元都用固定的权重矩阵 $\mathbf{W}_i$ 处理，该矩阵仅依赖于特定的偏移量 $i$。若要在层内建模长距离依赖关系，就需要扩大该层的感受野，这将导致参数数量随感受野大小呈线性增长。

解决该问题的一种方法如下：与其显式学习矩阵 $\mathbf{W}_1, \mathbf{W}_2, \ldots$，不如通过定义一个独立的神经模块 $g(i) : \mathbb{R} \to \mathbb{R}^{e \times e}$ 来隐式地定义它们；该模块根据相对偏移量 $i$ 输出所有权重矩阵。因此，我们将(E.10.1)重写为：

$$\mathbf{h}_i = \sum_{j=1}^{n} g(i-j)\mathbf{x}_j$$

对所有词元求和

这种方法称为**长卷积**（long convolution），因为该卷积覆盖了整个输入矩阵 $\mathbf{X}$。它也被称为**连续卷积**（continuous convolution）[RKG+22]，因为我们可以使用 $g(\bullet)$ 来参数化中间位置或可变分辨率 [RKG+22]。在这种情况下，参数量仅取决于 $g$ 的参数，而与序列长度 $n$ 无关。定义 $g$ 并非平凡，因为它需要输出整个权重矩阵。我们可以简单地还原出标准卷积：

$$g(i,j) = \begin{cases} \mathbf{W}_{i-j} & \text{若 } |i-j| \leq k \\ 0 & \text{否则} \end{cases} \tag{E.10.2}$$

这在一定程度上解决了长程依赖问题，但未能解决依赖关系随输入变化的问题，因为赋予某个词元的权重仅取决于其相对于索引 $i$ 的相对偏移量。然而，该公式提供了一种简单的解决方案：让训练后的函数 $g$ 依赖于词元的内容而非其位置：

$$\mathbf{h}_i = \sum_{j=1}^{n} g(\mathbf{x}_i, \mathbf{x}_j)\mathbf{x}_j \tag{E.10.3}$$

在计算机视觉领域，这些模型也称为**非局部**（non-local）网络 [WGGH18]。我们在图 F.10.1 中展示了标准卷积、连续卷积和非局部卷积的对比。

### 10.1.2 注意力层




MHA 层是式(E.10.3)的一种简化。首先，处理输出为矩阵的函数较困难，因此我们将该层限制为使用标量权重。具体而言，衡量词元间相似度的一种简单方法是计算它们的内积（**点积**）：

$$g(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$$

正如后文将看到的，这种方法能带来一种针对整个序列、易于并行化的算法。在下文中，我们考虑点积的归一化形式：

$$g(\mathbf{x}_i, \mathbf{x}_j) = \frac{1}{\sqrt{e}} \mathbf{x}_i^\top \mathbf{x}_j$$

其动机如下：假设 $\mathbf{x}_i \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$，则 $\mathbf{x}_i^\top \mathbf{x}_j$ 中每个元素的方差为 $\sigma^4$，因此其元素幅度很容易变得非常大。引入缩放因子可以确保点积的方差保持在 $\sigma^2$ 的有界范围内。

由于我们需要对数量可变的词元（记为 $n$ 个）进行求和，因此引入归一化操作（例如 softmax）会很有帮助：[1]

$$\mathbf{h}_i = \sum_{j=1}^{n} \text{softmax}_j(g(\mathbf{x}_i, \mathbf{x}_j))\mathbf{x}_j \tag{E.10.4}$$

在此语境下，我们将 $g(\bullet, \bullet)$ 称为**注意力评分函数**（attention scoring function），将 softmax 的输出称为**注意力分数**（attention score）。由于 softmax 的归一化性

[1] (E.10.4) 中的 $\text{softmax}_j$ 符号表示我们针对集合 $\{g(\mathbf{x}_i, \mathbf{x}_j)\}_{j=1}^{n}$ 应用 softmax 归一化，且该操作对于每个 $i$ 都是独立进行的。这一点在下文描述的向量化形式中更容易看出。

质，我们可以想象每个词元 $i$ 拥有一定量的“注意力”可分配给其他词元：若增加分配给某个词元的份额，则分配给其他词元的注意力必然会减少，这是由 softmax 分母项的性质决定的。

若采用“点积注意力”，则 $g$ 不包含可训练参数。注意力层的设计思路是通过在计算上述公式前，向输入添加可训练的投影变换，从而引入可训练参数。为此，我们定义三个可训练矩阵：$\mathbf{W}_k \sim (e,k)$、$\mathbf{W}_v \sim (e,v)$ 和 $\mathbf{W}_q \sim (e,k)$，其中 $k$ 和 $v$ 为超参数。每个词元使用这三个矩阵进行投影，总共得到 $3n$ 个词元：

$$\text{键词元：} \quad \mathbf{k}_i = \mathbf{W}_k^\top \mathbf{x}_i \tag{E.10.5}$$

$$\text{值词元：} \quad \mathbf{v}_i = \mathbf{W}_v^\top \mathbf{x}_i \tag{E.10.6}$$

$$\text{查询词元：} \quad \mathbf{q}_i = \mathbf{W}_q^\top \mathbf{x}_i \tag{E.10.7}$$

这些经过处理的词元分别称为**键**（key）、**值**（value）和**查询**（query）（你可以暂时忽略术语的选择；我们将在本节末尾回到这一点）。**自注意力**（self-attention，SA）层通过将三个投影 (E.10.5)-(E.10.6)-(E.10.7) 与式(E.10.4)相结合得到：

$$\mathbf{h}_i = \sum_{j=1}^{n} \text{softmax}_j(g(\mathbf{q}_i, \mathbf{k}_j))\mathbf{v_j}$$

因此，我们将词元 $i$ 的查询与所有可能的键进行比较，来计算其更新后的表示，并利用归一化后的权重来组合相应的值词元。请注意，键和查询的维度必须相同，而值的维度则可以不同。

如果我们使用点积，就可以为所有词元简洁地重写 SA 层的操作。为此，我们将所有可能的键、查询和值堆叠

起来，定义三个矩阵：

$$\mathbf{K} = \mathbf{X}\mathbf{W}_k \tag{E.10.8}$$
$$\mathbf{V} = \mathbf{X}\mathbf{W}_v \tag{E.10.9}$$
$$\mathbf{Q} = \mathbf{X}\mathbf{W}_q \tag{E.10.10}$$

三个派生矩阵 $\mathbf{K}$、$\mathbf{V}$ 和 $\mathbf{Q}$ 形状分别为 $(n, k)$、$(n, v)$ 和 $(n, k)$。顺便一提，我们也可以通过一次矩阵乘法实现它们，并将输出结果分为三部分：

$$[\mathbf{K} \,\|\, \mathbf{V} \,\|\, \mathbf{Q}] = \mathbf{X}\left[\mathbf{W}_k \,\|\, \mathbf{W}_v \,\|\, \mathbf{W}_q\right]$$

其中 $\|$ 表示拼接。

此时，SA 层可表示为：

$$\mathrm{SA}(\mathbf{X}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{k}}\right)\mathbf{V}$$

其中假设 softmax 操作按行应用。我们也可以将投影显式写出，如下所示。

**定义 D.10.1 (自注意力层)**




对于输入 $\mathbf{X} \sim (n, e)$，**自注意力** (*self-attention*，*SA*) 层定义为：

$$\mathrm{SA}(\mathbf{X}) = \mathrm{softmax}\left(\frac{\mathbf{X}\mathbf{W}_q\mathbf{W}_k^\top\mathbf{X}^\top}{\sqrt{k}}\right)\mathbf{X}\mathbf{W}_v \tag{E.10.11}$$

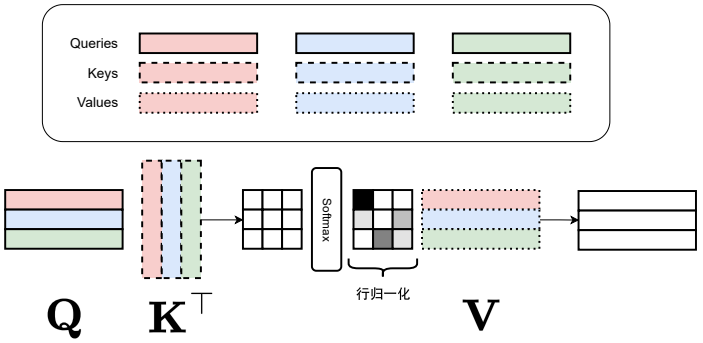



**图 F.10.2:** *SA 层主要运算的可视化（不包括投影）。*

> 可训练参数为 $\mathbf{W}_q \sim (e,k)$、$\mathbf{W}_k \sim (e,k)$ 和 $\mathbf{W}_v \sim (e,v)$，其中 $k$ 和 $v$ 为超参数。因此，该层共有 $2ke+ve$ 个可训练参数，且数量与 $n$ 无关。

我们在图F.10.2中直观地展示了该层的运算过程。

### 10.1.3 多头注意力

前述层也被称为**单头**（single-head）注意力操作。它能以极高的灵活性建模词元之间的成对依赖关系。然而，在某些情况下，我们需要考虑多组依赖关系：再次以“*the cat, which belongs to my mother, is on the table*”（属于我母亲的那只猫在桌子上）为例，“*cat*”与“*table*”之间的依赖关系，与“*cat*”和“*mother*”之间的依赖关系是不同的；我们可能希望该层能分别对这些关系建模。[2]

多头注意力层通过并行执行多个注意力操作来实现这一点，每个操作各自拥有一组可训练参数，随后用某种池化操作来聚合结果。为此，我们定义一个新的超参数 $h$，称为该层的**头数**（number of heads）。我们针对词元实例化 $h$ 个独立的投影，总共涉及 $3hn$ 个词元（每个头对

[2] 当然，一切都取决于那只猫。

应 $3n$ 个)：

$$\mathbf{K}_e = \mathbf{X}\mathbf{W}_{k,e} \tag{E.10.12}$$
$$\mathbf{V}_e = \mathbf{X}\mathbf{W}_{v,e} \tag{E.10.13}$$
$$\mathbf{Q}_e = \mathbf{X}\mathbf{W}_{q,e} \tag{E.10.14}$$

其中 $\mathbf{W}_{k,e}$ 表示第 $e$ 个头的键投影，其他变量定义方式与之类似。**多头注意力** (multi-head attention, MHA) 层执行 $h$ 次独立的自注意力操作，堆叠所得的输出嵌入，最后将其投影至目标维度：

单个 SA 层

$$\mathrm{MHA}(\mathbf{X}) = \left[\mathrm{SA}_1(\mathbf{X}) \parallel ... \parallel \mathrm{SA}_h(\mathbf{X})\right] \mathbf{W}_o \tag{E.10.15}$$

输出投影

其中：

$$\mathrm{SA}_i(\mathbf{X}) = \mathrm{softmax}\left(\frac{\mathbf{Q}_i\mathbf{K}_i^\top}{\sqrt{k}}\right)\mathbf{V}_i$$

每次 SA 操作返回一个形状为 $(n, v)$ 的矩阵。这 $h$ 个矩阵沿第二维度拼接，得到一个 $(n, hv)$ 矩阵，随后用矩阵 $\mathbf{W}_o \sim (hv, o)$ 对其进行投影；其中 $o$ 是另一个超参数，允许灵活选择输出维度。

**头与回路**

我们稍后将看到，MHA 层总是与残差连接（见第9.4节）结合使用。在这种情况下，我们可以将其针对第 $i$ 个词元的输出写为：

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \underbrace{\sum_e}_{\text{对注意力头求和}} \underbrace{\sum_j}_{\text{对词元求和}} \alpha_e(\mathbf{x}_i, \mathbf{x}_j) \mathbf{W}_e^\top \mathbf{x}_j \quad \text{(E.10.16)}$$

其中 $\alpha_e(\mathbf{x}_i, \mathbf{x}_j)$ 是注意力头 $e$ 中词元 $i$ 与 $j$ 之间的注意力分数，而 $\mathbf{W}_e$ 组合了第 $e$ 个注意力头的值投影与式(E.10.15)中输出投影的第 $e$ 个块。词元嵌入有时被称为模型的**残差流**（residual stream）。[a]因此，可以将注意力头理解为从残差流中“读取”信息（通过 $\mathbf{W}_e$ 投影，并通过注意力分数进行选择），再线性地将信息“写回”该流中。

[a]这一概念在**机制可解释性**（mechanistic interpretability）领域得到了普及；该领域试图通过逆向工程分析各层行为，以寻找被称为回路（circuits）的可解释组件：https://transformer-circuits.pub。差流的线性特性对该类分析至关重要。

## 术语解释




为了理解为什么这三类词元分别称为查询、键和值，我们将 SA 层与标准 Python 字典进行类比，如框C.10.1中所示。

形式上，字典是一组形如（键，值）的对，其中键充当唯一标识符，用于检索相应的值。例如，在框C.10.1中第

```
d = dict()
d["Alice"] = 2
d["Alice"]     # 返回 2
d["Alce"]      # 报错
```

**框 C.10.1:** *Python* 中的字典：只有当查询的键与字典中的键完全匹配时，才会返回对应的值；否则会报错。

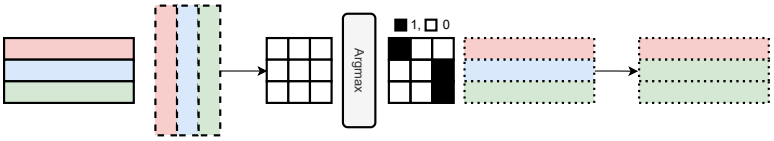



**图 F.10.3:** 使用“硬”注意力的 *SA* 等同于向量值字典。

三行和第四行，我们用两个不同的字符串（“*Alice*”和“*Alce*”）查询字典：字典将查询字符串与其中存储的所有键进行比较，如果找到完美匹配，则返回对应值，否则报错。

若给定一种在键与查询之间度量相似性的方式，我们可以考虑标准字典的一种变体：该变体总是返回与字典中最接近的键相对应的值。如果键、查询和值均为向量，那么只要将 softmax 操作替换为对词元的 argmax 操作，这种字典变体就等价于我们的 SA 层，如图F.10.3所示。

这种注意力的“硬”变体难以实现，因为 argmax 操作的梯度几乎处处为零（我们将在下一卷中讨论离散采样，以及用离散松弛来近似 argmax 操作）。因此，我们可以将 SA 层解释为一种软逼近：其中每个词元都根据相应的键/查询相似度，用所有值的加权组合来更新。

## 10.2 位置嵌入

在掌握了 MHA 层之后，我们来探讨完整 transformer 模型的设计；该模型还需要另一个组件，即位置嵌入

(positional embeddings)。

### 10.2.1 置换等变性

考虑当词元的顺序被重新排列（置换，permutation）时，MHA 层的输出会如何变化，这是一件有趣的事。为了将其形式化，我们引入**置换矩阵**（permutation matrix）的概念。

> **定义 D.10.2 (置换矩阵)**
>
> 大小 $n$ 的**置换矩阵**是一个二值方阵 $\mathbf{P} \sim \mathit{Binary}(n, n)$，其每一行和每一列都恰好包含一个 *1*：
>
> $$\mathbf{1}^\top \mathbf{P} = \mathbf{1}, \;\; \mathbf{P}\mathbf{1} = \mathbf{1}$$
>
> 如果我们取消矩阵元素必须为二值的要求，只约束元素非负，就得到**双随机**（*doubly stochastic*）矩阵的集合（即行和与列和均为 *1* 的矩阵）。

应用置换矩阵的效果是重新排列矩阵的对应行或列。例如，考虑如下排列：

$$\mathbf{P} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

观察各行可知，应用置换矩阵后，第二和第三个元素被交换了：

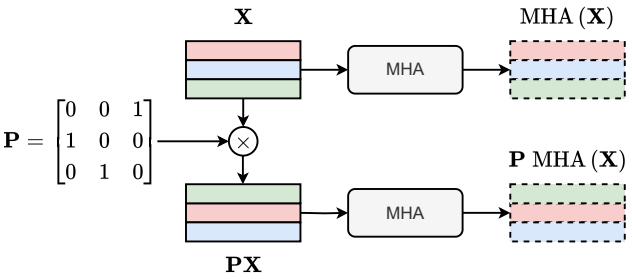



**图 F.10.4:** 置换词元顺序后，*MHA* 层的输出显然就是原始输出的置换结果。

$$\mathbf{P}\begin{bmatrix}\mathbf{x}_1\\ \mathbf{x}_2\\ \mathbf{x}_3\end{bmatrix} = \begin{bmatrix}\mathbf{x}_1\\ \mathbf{x}_3\\ \mathbf{x}_2\end{bmatrix}$$

有趣的是，将置换矩阵应用于 MHA 层的输入时，其唯一效果是以等价方式重新排列该层的输出：

$$\mathrm{MHA}(\mathbf{PX}) = \mathbf{P} \cdot \mathrm{MHA}(\mathbf{X})$$

这一点很容易证明。我们重点讨论单头注意力的情况，因为多头注意力的推导过程与之类似。首先，softmax 操作会对矩阵各列的元素进行归一化，因此它在行和列的维度上都具有置换等变性：

$$\mathrm{softmax}(\mathbf{PXP}^\top) = \mathbf{P}\,[\mathrm{softmax}(\mathbf{X})]\,\mathbf{P}^\top$$

由此，我们可以直接推导出 SA 的置换等变性：

$$\mathrm{SA}(\mathbf{PX}) = \mathrm{softmax}\left(\mathbf{P}\frac{\mathbf{X}\mathbf{W}_q\mathbf{W}_k^\top\mathbf{X}^\top}{\sqrt{k}}\mathbf{P}^\top\right)\mathbf{P}\mathbf{X}\mathbf{W}_v \tag{E.10.17}$$

$$= \mathbf{P}\cdot\mathrm{softmax}\left(\frac{\mathbf{X}\mathbf{W}_q\mathbf{W}_k^\top\mathbf{X}^\top}{\sqrt{k}}\right)\mathbf{X}\mathbf{W}_v = \mathbf{P}\cdot\mathrm{SA}(\mathbf{X}) \tag{E.10.18}$$

其中利用了任意置换矩阵 $\mathbf{P}$ 均满足 $\mathbf{P}^\top\mathbf{P} = \mathbf{I}$ 这一性质。这也可以通过逐词元地推理 SA 层操作来理解：输出结果是各项的加权和，其中每个元素的权重由成对比较决定。因此，对于给定的词元，该操作是**置换不变的**（permutation invariant）；而对于整个输入矩阵而言，该操作则是**置换等变的**（permutation equivariant）。

平移等变性是卷积层的可取性质，但置换等变性是不可取的（至少在此场景下），因为它丢弃了输入序列中有价值的顺序信息。例如，处理一个词元顺序被反转的文本，其唯一效果是反转该层的输出，尽管反转后的输入序列可能已经无效。从形式上讲，SA 层和 MHA 层属于集合函数，而非序列函数。[3]

transformer 没有修改该层或添加非置换等变的层，而是引入**位置嵌入**（positional embeddings）这一新概念。位置嵌入是仅依赖于词元在序列中位置（**绝对位置嵌入**，absolute positional embeddings）或两个词元之间偏移（**相对位置嵌入**，relative positional embeddings）的辅助词元。我们将在下文分别介绍这两种嵌入。

[3]更严格地说，由于词元可以重复，它们实际上是多重集（multiset）函数。

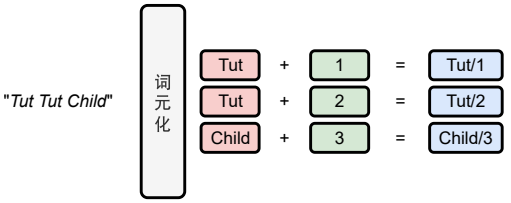



**图 F.10.5:** 将位置嵌入（绿色）加到词元嵌入（红色）上。同一词元在不同位置的输出不同（蓝色）。

### 10.2.2 绝对位置嵌入

输入矩阵 $\mathbf{X} \sim (n, e)$ 中的每个词元代表特定文本片段（例如子词）的内容。假设任意序列的最大长度固定为 $m$ 个词元。为了克服置换等变性，我们引入一组额外的**位置嵌入**（positional embeddings）$\mathbf{S} \sim (m, e)$，其中向量 $\mathbf{S}_i$ 唯一地编码了“处于位置 $i$”这一概念。因此，输入矩阵与 $\mathbf{S}$ 的前 $n$ 行之和为：

$$\mathbf{X}' = \mathbf{X} + \mathbf{S}_{1:n}$$

所得结果使 $[\mathbf{X}']_i$ 代表“位于位置 $i$ 的词元 $\mathbf{X}_i$”。因为置换位置嵌入没有意义（它们只取决于位置），所以得到的层不再具有置换等变性：

$$\mathrm{MHA}(\mathbf{PX} + \mathbf{S}) \neq \mathbf{P} \cdot \mathrm{MHA}(\mathbf{X} + \mathbf{S})$$

这一思想的可视化见图F.10.5。

如何构建位置嵌入？最简单的策略是将 $\mathbf{S}$ 视为模型参数的一部分，并与其余可训练参数一起训练，这与处理词元嵌入的方式类似。当词元数量相对稳定时，这种策略效果很好；我们将在下一章的计算机视觉相关内容中看到一个具体的例子。

另一个方法是，定义一个确定性函数，将词元位置映射

到一个能唯一标识该位置的向量。有些策略显然效果不佳，例如：

1. 为每个位置关联一个标量 $p = i/m$，该标量随位置线性增加。然而，向词元嵌入中添加单个标量的影响微乎其微。

2. 对位置进行独热编码，转换为长度为 $m$ 的二值向量，但由此产生的向量极其稀疏且维度很高。

原版 transformer 论文 [VSP+17]中提出的一种潜在方案是**正弦嵌入**（sinusoidal embeddings）。要理解这种嵌入，考虑一个正弦函数：

$$y = \sin(x)$$

正弦函数为 $[0, 2\pi]$ 范围内的任意输入 $x$ 分配一个唯一的值。我们也可以改变正弦函数的频率：

$$y = \sin(\omega x)$$

根据频率 $\omega$ 的不同，函数振荡的快慢也会有所不同，并且它能为 $[0, \frac{2\pi}{\omega}]$ 范围内的任意输入分配唯一的值。

这可以类比为（模拟）时钟：秒针以 $\frac{1}{60}$ Hz 的频率（即每分钟转一圈）旋转。因此，在一分钟内，通过观察指针的位置，就能区分每一个“时间点”；但在一般情况下，对于任意两个时间点，只能确定它们各自在 60 秒周期内的位置。在时钟设计中，我们通过增加一个以 $\frac{1}{3600}$ Hz 低频率旋转的指针（即分针）来解决这一问题。因此，通过观察（秒，分）这一坐标对（即时间的“嵌入”表示），我们可以区分一小时内的任意时刻。若再增加一个频率更低的指针（即时针），我们则能区分一天内的任意时刻。该思路可以进一步推广：可以设计频率更低或更高的时钟，以区分月份、年份或毫秒级的时间点。

此处也可应用类似的策略：通过一组 $e$ 个频率递增的正

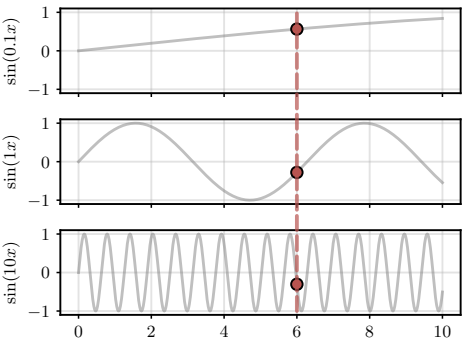



**图 F.10.6:** 图中展示了三个 $\omega$ 值分别为 $\omega = 0.1$, $\omega = 1$ 和 $\omega = 10$, 的正弦函数。位置 $x = 6$ 处的嵌入由相应的函数值（红色圆圈）标出。

弦函数（其中 $e$ 为超参数）来编码并区分每个位置 $i$:

$$\mathbf{S}_i = [\sin(\omega_1 i), \sin(\omega_2 i), \ldots, \sin(\omega_e i)]$$

在实践中，[VSP+17]提出的原始方案仅使用 $e/2$ 个可能的频率，但同时包含正弦和余弦函数:

$$\mathbf{S}_i = \left[\sin(\omega_1 i), \cos(\omega_1 i), \ldots, \sin(\omega_{e/2} i), \cos(\omega_{e/2} i)\right]$$

这种做法的合理性在于：在这种嵌入表示中，两个位置的嵌入向量可以通过简单线性变换（即旋转）来相互转换, 且该变换仅取决于这两个位置间的相对偏移量。[4] 只要所选频率足够大，并以超线性速率递增，任意频率选择都是有效的。[VSP+17] 中采用了一种等比数列形式:

$$\omega_i = \frac{1}{10000^{i/e}}$$

其取值范围从 $\omega_0 = 1$ 到 $\omega_e = \frac{1}{10000}$。其可视化见图F.10.6。

[4]详细计算见https://kazemnejad.com/blog/transformer_architecture_positional_encoding/。

### 10.2.3 相对位置嵌入

可训练位置嵌入和正弦位置嵌入都是**绝对**嵌入的例子，它们编码的是序列中的特定位置。对于非常长的序列，一种已变得常见的替代方案是**相对位置嵌入**。在这种情况下，我们不再向词元添加位置嵌入，而是修改注意力函数，使其依赖于任意两个词元之间的偏移：

$$g(\mathbf{x}_i, \mathbf{x}_j) \rightarrow g(\mathbf{x}_i, \mathbf{x}_j, i-j)$$

这是本章开头介绍的两种思想的结合（见图F.10.1）。需要注意的是，绝对位置嵌入仅在输入端添加一次，而相对位置嵌入则必须在每次使用 MHA 层时添加。举例来说，我们可以添加一个可学习的偏置矩阵 $\mathbf{B} \sim (m, m)$，并将点积运算改写为包含偏移相关偏置的形式：

$$g(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j + B_{ij}$$

一种更简单的变体是**线性偏置注意力**（attention with linear biases，ALiBi）[PSL22]，它在每个注意力头中引入一个可训练标量，并将其与偏移量矩阵相乘。此外，也存在更高级的策略，例如**旋转位置嵌入**（rotary positional embeddings，RoPE）[SAL+24]。

## 10.3 构建 transformer 模型

### 10.3.1 transformer 模块与模型

原则上，模型可以通过堆叠多个 MHA 层来构建（其中 softmax 提供了必要的非线性，以避免多个线性投影坍缩）。然而，经验表明，当 MHA 层与一个独立的全连接模块交替使用时，效果最好；该全连接模块对每个词元独立进行处理。这两种操作可以分别理解为混合词元（MHA）和混合通道（MLP），这与深度可分离卷积

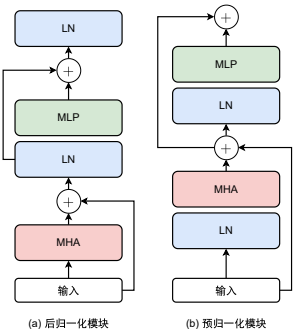



**图 F.10.7:** *预归一化和后归一化 transformer 模块示意图。在后归一化变体中，LN 模块应用于 MHA 或 MLP 之后；在预归一化变体中，LN 模块应用于每一层之前。*

(depthwise-separable convolution) 模型的原理类似。

具体地，对于 MLP 模块，通常采用由两个全连接层组成的瓶颈架构，形式如下：

$$\text{MLP}(\mathbf{x}) = \mathbf{W}_2\phi\left(\mathbf{W}_1\mathbf{x}\right)$$

其中 $\mathbf{x} \sim (e)$ 是一个词元，$\mathbf{W}_1 \sim (p, e)$，此处 $p$ 通常取 $e$ 的整数倍，例如 $p = 3e$ 或 $p = 4e$，而 $\mathbf{W}_2 \sim (e, p)$ 将数据重新投影回原始嵌入维度。通常不使用偏置项，因为增加的隐藏维度已提供了足够的自由度。

为了确保深层模型的高效训练，还需要一些额外的正则化策略。特别地，通常会在 MHA 和 MLP 模块中分别引入两次层归一化步骤和两个残差连接。根据层归一化应用位置不同，我们得到基础 transformer 模块的两种变体，有时分别称为**预归一化** (pre-normalize) 和**后归一化** (post-normalize)。如图 F.10.7 所示。

尽管后归一化版本对应于原版 transformer 模块，但研究发现，预归一化变体通常更稳定且训练速度更快 [XYH+20]。图 F.10.7 中的模块设计本质上是一种经验性选择，文献中已提出并测试了许多变体。我们将在之后的 11.3 节中回顾其中一些变体。

现在我们可以完整描述一个基本的 transformer 模型：

1. 将原始输入序列词元化并嵌入为矩阵 $\mathbf{X} \sim (n, e)$。
2. 若使用绝对位置嵌入，则将其加到输入矩阵上。
3. 应用一个或多个上述形式的模块。
4. 根据具体任务添加最终的输出头。

步骤 (3) 的输出是一组处理后的词元 $\mathbf{H} \sim (n, e)$，其中 $n$ 和 $e$ 均未被 transformer 模型改变（前者是因为模型中没有针对集合的局部池化操作，后者是由于模块中的残差连接）。以分类任务为例，我们可以应用标准的分类头（classification head），即对所有词元进行池化，随后接一个全连接模块：

$$y = \text{softmax}\left(\text{MLP}\left(\frac{1}{n}\sum_i \mathbf{H}_i\right)\right)$$

这部分与相应的 CNN 设计完全相同。然而，transformer 具有很多有趣的性质，这些特性主要源于它将输入视为一个（多重）集合进行处理，且在整个架构中不改变数据的维度。接下来，我们将探讨一个简单的例子。

### 10.3.2 类词元与寄存器词元

尽管到目前为止，我们假设每个词元对应输入序列的一部分，但我们也可以向 transformer 的输入中添加额外的词元。这严格取决于其具体架构：例如，CNN 要求其输入精确有序，而在图像或序列中添加额外词元的方法并不明确。这是一种非常有力的想法，此处我们仅考虑两种具体实现。

首先，我们考虑使用**类词元**（class token）[DBK+21]，这是一个为分类任务添加的额外词元，用于替代上述的全局池化操作。假设我们初始化一个可学习词元 $\mathbf{c} \sim (e)$，并将其添加到输入矩阵中：

$$\mathbf{X} \leftarrow \begin{bmatrix} \mathbf{X} \\ \mathbf{c}^\top \end{bmatrix}$$

新矩阵形状为 $(n+1, e)$。类词元对于小批量中的所有序列都是相同的。在执行上述步骤 (3) 之后，transformer 输出一个矩阵 $\mathbf{H} \sim (n+1, e)$，其中包含了所有词元（包括类词元）的更新后表示。其核心理念是，模型不应仅仅对词元进行池化，而应能够将所有与分类任务相关的信息“压缩”到类词元中，我们可以通过简单地丢弃所有其他词元来重写分类头：[5]

$$y = \text{softmax}\left(\text{MLP}\left(\mathbf{H}_{n+1}\right)\right)$$

即使不被显式使用，额外的可训练词元也可能发挥作用。例如 [DOMB24]表明，添加少量额外词元（在该研究中称为**寄存器**），可以改善注意力图的质量。这是因为这些寄存器为模型提供了一种机制，使其能够利用它们，来“存储”那些不显式依赖于给定位置的辅助信息。

## 从理论到实践



我们将在下一章介绍许多与 transformer 相关的重要概念。因此，在本章中，我建议进行一项不那么常规的练习：将卷积主干（backbone）与类 transformer 结构的头组合——如图F.10.8所示。

在第7章和第9章中开发的卷积模型是应用于单张图像的。然而，有时我们可以获得同一待识别物体的一组图像——例如，在监控系统中，我们可能有一个可疑人员的多张截图。文献中将其称为**多视图**（multi-view）系统，每张图像称为该物体的一个**视图**（view）。多视图模型应当对整组视图给出一个统一的预测结果，同时预

[5] 若用第 10.1.3 节中回路与头的语言，可以等价地表述为，模型必须学会将所有与分类任务相关的信息转移到类词元的残差流中。

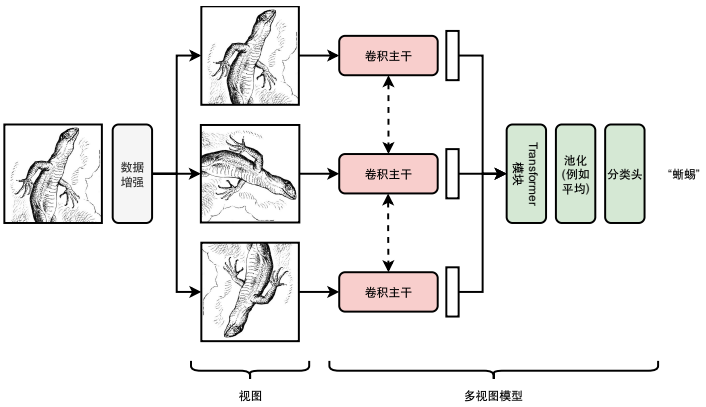



**图 F.10.8:** 本章将实现的多视图模型。图像通过一系列随机数据增强策略处理，得到输入的一组**视图**（灰色）。每个视图由同一个卷积主干处理，以获得固定维度的嵌入（红色）。嵌入集合在最终分类前由 *transformer* 模块处理（绿色）。插图由约翰 · 坦尼尔绘制。

测结果应具有对输入视图顺序的不变性。在本练习中，我们将实现一个简单的多视图模型——如图F.10.8所示。

1. 使用任意图像分类数据集，可以通过对输入（图F.10.8 中的灰色方块）应用固定数量的数据变换，来模拟多视图模型。忽略批次维度，对于形状为 $x \sim (h, w, c)$ 的每张输入图像，将获得形状为 $x' \sim (v, h, w, c)$ 的多视图输入，其中 $v$ 是视图数量。该张量对应一个单一标签 $y$——即原始图像标签。在不同小批量之间，视图数量也可以不同，因为模型的任何部分都不受预设视图数量限制。

2. 多视图模型由三个组件构成。假设 $g(x)$ 为处理单个视图并将其转换为固定维度嵌入的模型——例如，这可以是你之前练习训练过的任意卷积主干网络。完整模型的第一部分（图F.10.8中红色部分）

对所有视图并行应用 $g$，得到 $\mathbf{h}_i = g(x_i) \sim (e)$，其中 $e$ 是一个超参数（主干网络的输出大小）。

3. 拼接各视图的嵌入后，我们得到一个矩阵 $\mathbf{H} \sim (v, e)$。为了使完整模型具有置换不变性，任何应用于 $\mathbf{H}$ 的组件都必须具有置换等变性。[6]在本练习中，请按照10.3.1节所述，实现并应用一个 transformer 模块。你可以使用基础 PyTorch 实现 MHA，也可以尝试使用 `einops` 库的更高级实现。[7] 你还可以将其与 `torch.nn` 中预先实现的版本进行比较。

4. transformer 模块不会改变输入形状。为了完成模型构建，请对各个视图（在此场景下代表词元）求平均，并添加一个最终的分类头。你也可以尝试添加类词元（见10.3.2节）。不难发现，如此构建的模型对于视图顺序具有置换不变性。

[6]对视图求平均是置换不变层最简单的例子。因此，从图F.10.8中移除 MHA 模块也是一种有效的基准。此外，Deep Sets[ZKR+17]描述了所有线性且置换不变层的完整类别。

[7]参考https://einops.rocks/pytorch-examples.html。

# 11 实践中的 transformer

**本章导读**

本章将探讨基础 transformer 模型的几种变体，包括编码器-解码器（encoder-decoder）架构、因果多头注意力（causal MHA）层，以及在图像与音频领域的应用。

## 11.1 编码器-解码器 transformer

在第 10 章描述的模型可用于对给定序列执行回归或分类任务。然而，最早的 transformer[VSP+17] 是一种更复杂的模型，专为**序列到序列**（sequence-to-sequence，seq2seq）任务而设计。在 seq2seq 任务中，输入和输出均为序列，且两者的词元之间不存在简单对应关系。一个典型例子是**机器翻译**（machine translation），其输出是翻译为另一种语言的输入序列。

构建面向 seq2seq 任务的可微模型的一种方案是**编码器-解码器**（encoder-decoder，ED）设计 [SVL14]。ED 模型由两个模块组成：一个是编码器，将输入序列处理并转换为某种表示形式（其维数可能为固定的）；另一个是解

码器，根据编码器的输出，自回归地生成输出序列。我们之前描述的 transformer 模型可用于构建编码器：这类用于分类的 transformer 称为**仅编码器**（encoder-only）transformer。为了构建解码器，我们需要两个额外组件：一种使模型具有因果性（以执行自回归）的方式，以及一种使其计算以另一个独立输入（编码器输出）为条件的机制。

### 11.1.1 因果多头注意力




首先，考虑使 transformer 模块具有因果性的问题。词元之间发生交互的唯一组件是 MHA 模块。因此，只要具有 MHA 的因果变体，就足以使整个模型具备因果性。回忆一下，对于卷积，我们通过适当掩码卷积核中的权重，来设计因果变体。对于 MHA，则可以掩码所有不满足因果性质的词元间交互：

$$\text{Masked-SA}(\mathbf{X}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top \odot \mathbf{M}}{\sqrt{k}}\right)\mathbf{V}$$

关键的是，掩码必须在 softmax 内部进行。考虑如下（错误）变体：

$$\textbf{错误:}\quad \left(\text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{k}}\right) \odot \mathbf{M}\right)\mathbf{V}$$

由于 softmax 存在分母项，无论如何进行掩码，所有词元都会参与每个词元的计算过程。还需要注意，将非因果连接对应的 $M_{ij}$ 设为 0 是无效的，因为 $\exp(0) = 1$。因此，正确的实现是选择一个上三角矩阵，并在其上三角部分置 $-\infty$，因为 $\exp(-\infty) = 0$ 正好符合预期：[1]

[1]为简化记号，这里采用了一个不太严格的约定：任何有限数与 $-\infty$ 相乘都等于 $-\infty$。在实践中，我们会使用替换函数（例如 PyTorch 中的 `masked_fill`）直接覆盖相应的值，而不是执行乘法。

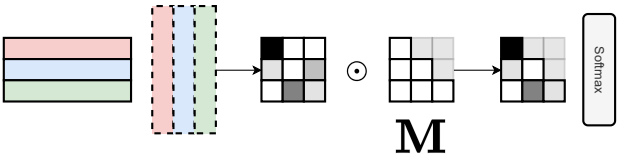



**图 F.11.1:** 使用注意力掩码实现因果注意力的可视化示意图。

$$M_{ij} = \begin{cases} -\infty & \text{若 } i > j \\ 1 & \text{否则} \end{cases}$$

在实际应用中，这些值也可以被设置为一个极大的负数（例如 $-10^9$）。

### 11.1.2 交叉注意力

其次，考虑如何使 MHA 层的输出依赖于另一组独立输入。为此，我们将 MHA 操作重写，显式分离输入矩阵的三次出现：

$$\mathrm{SA}(\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3) = \mathrm{softmax}\left(\frac{\mathbf{X}_1 \mathbf{W}_q \mathbf{W}_k^\top \mathbf{X}_2^\top}{\sqrt{k}}\right) \mathbf{X}_3 \mathbf{W}_v$$

SA 层对应 $\mathbf{X}_1 = \mathbf{X}_2 = \mathbf{X}_3 = \mathbf{X}$ 的情况（巧合的是，这也解释了我们为其所起的名称）。然而，如果键、值和查询分别属于不同的集合，该公式同样适用。一个重要情形是**交叉注意力**（Cross-Attention，CA），其中我们假设键（keys）和值（values）是由第二个矩阵 $\mathbf{Z} \sim (m, e)$ 计算得出：

X 与 Z 之间的交叉注意力

$$\mathrm{CA}(\mathbf{X},\mathbf{Z}) = \mathrm{softmax}\left(\frac{\mathbf{X}\mathbf{W}_q\mathbf{W}_k^\top\mathbf{Z}^\top}{\sqrt{k}}\right)\mathbf{Z}\mathbf{W}_v \quad \text{(E.11.1)}$$

使得 $\mathrm{CA}(\mathbf{X},\mathbf{Z}) = \mathrm{SA}(\mathbf{X},\mathbf{Z},\mathbf{Z})$。其含义是，$\mathbf{X}$ 的嵌入是根据它们与由 $\mathbf{Z}$ 提供的一组外部（键、值）对的相似度进行更新：我们称 $\mathbf{X}$ 在 $\mathbf{Z}$ 上进行*交叉注意力*（cross-attending）操作。请注意，这种形式与将两组输入词元拼接后，再对注意力矩阵进行适当掩码处理的方法非常相似。

## 与前馈层对比




考虑公式(E.11.1)中交叉注意力操作的一种简化变体，其中我们显式参数化了键矩阵和值矩阵：[2]

$$\mathrm{NeuralMemory}(\mathbf{X}) = \mathrm{softmax}\left(\frac{\mathbf{X}\mathbf{W}_q\mathbf{K}}{\sqrt{k}}\right)\mathbf{V} \quad \text{(E.11.2)}$$

该层现在由查询投影矩阵 $\mathbf{W}_q$ 以及两个矩阵 $\mathbf{K}$ 和 $\mathbf{V}$ 参数化。公式 (E.11.2) 被称为**记忆层**（memory layer）[SWF+15]，其含义是，模型利用键矩阵和值矩阵的行来存储感兴趣的模式，并通过类似注意力的操作动态检索这些模式。如果我们进一步简化该层，令 $\mathbf{W}_q = \mathbf{I}$，忽略 $\sqrt{k}$ 的归一化，并将 softmax 替换为通用的激活函数 $\phi$，则得到一个两层 MLP：

$$\mathrm{MLP}(\mathbf{X}) = \phi\left(\mathbf{X}\mathbf{K}\right)\mathbf{V} \quad \text{(E.11.3)}$$

[2] 另请参阅第11.2.1节中关于 Perceiver 网络的讨论。

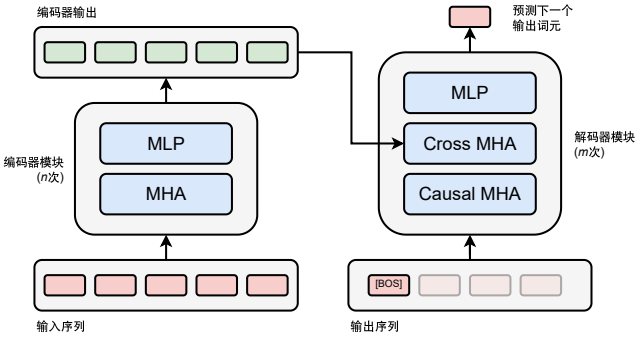



**图 F.11.2:** 编码器-解码器架构，改编自 *[VSP+17]*。解码器中的填充词元以灰色显示。

因此，transformer 网络中的 MLP 可以视为注意力操作的近似形式，其中的键和值均可训练。对训练数据中距离最近的词元可视化分析，会显示出人类可理解的模式 [GSBL20]。

### 11.1.3 编码器-解码器 transformer

掌握这两个组件后，我们就可以讨论原始的 transformer 模型了，如图 F.11.2 所示。[3] 首先，输入序列 $\mathbf{X}$ 由一个标准 transformer 模型（称为**编码器**，encoder）处理，生成更新后的嵌入序列 $\mathbf{H}$。接下来，输出序列由另一个 transformer 模型（称为**解码器**，decoder）以自回归方式进行预测。与编码器不同，解码器的每个模块包含三个组件：

1. 一个掩码版本的 MHA 层（以确保能进行自回归）。
2. 一个交叉注意力层，其中查询由输入序列嵌入 $\mathbf{H}$ 提供。

[3] 一个较真的说明：严格来说，在 [VSP+17]中，Transformer（首字母大写）是一个专有名词。而在本书中，我使用 transformer（首字母小写）来指代任何主要由注意力层构成的模型。

```
def self_attention(Q: Float[Array, "n k"],
                   K: Float[Array, "n k"],
                   V: Float[Array, "n v"]
                  ) -> Float[Array, "n v"]:
^^Ireturn nn.softmax(Q @ K.T) @ V
```

**框 C.11.1:** *SA 层的简单实现，以查询、键和值矩阵为参数。*

3. 一个标准的逐词元 MLP。

**仅解码器**模型也是可行的，在这种情况下，解码器的第二个模块被移除，仅使用掩码 MHA 和 MLP。正如后文所述，大多数现代大语言模型都是基于仅解码器模型构建的，这些模型经过训练，以自回归方式生成文本词元 [RWC+19]。事实上，随着人们认识到许多 seq2seq 任务可以通过拼接输入序列与生成的输出序列，直接用仅解码器模型解决，编码器-解码器模型已变得不再普遍，如第 8.4.3 节所述。

## 11.2 计算方面的考虑

### 11.2.1 时间复杂度与线性时间 transformer

MHA 的性能优势并非没有代价：由于每个词元都必须关注所有其他词元，其复杂度高于简单的卷积操作。为了理解这一点，我们从内存和时间两个角度考察其复杂度。我们用 SA 层的朴素实现作为参考，如框 C.11.1 所示。

首先看时间复杂度。softmax 内部的操作复杂度为 $\mathcal{O}(n^2k)$，因为它需要计算 $n^2$ 次点积（每对词元进行一次计算）。相比之下，一维卷积层的复杂度仅随序列长度线性增长。理论上，复杂度的二次增长对于超长序列（例如大语言模型中常见的情况）可能会造成问题。

该问题促使了多种加速自回归生成的策略（例如推测解码 [LKM23]），同时也催生了线性或亚二次复杂度的 transformer 变体。例如，我们可以用一个交叉注意力层来替代 SA 层，该层使用一组可训练的词元 $\mathbf{Z}$，其中词元数量可作为超参数由用户设定。Perceiver 架构 [JGB+21] 推广了这一策略，旨在将原始词元集合蒸馏为更小的潜在瓶颈（latent bottlenecks）。设计线性化 transformer 有多种替代策略：我们在11.3节和13章讨论了其中的几种变体。

重要的是，如框C.11.1中所示的实现方式，已被证明在现代硬件上严重受限于内存带宽 [DFE+22]，这意味着其计算成本主要由内存和 I/O 操作主导。因此，线性时间复杂度注意力变体在理论上的优势，并不直接转化为硬件上的实际加速效果。考虑潜在的性能下降，与下文介绍的经过深度优化的 MHA 实现相比，这些变体的吸引力相对较低。

### 11.2.2 在线 softmax

在内存方面，框C.11.1 中的实现也有 $n^2$ 的复杂度，因为注意力矩阵 $\mathbf{QK}^\top$ 在计算过程中被完整地显式构造（materialized）了。然而，这种做法并不必要；通过将计算分块（chunk）并仅在最后执行 softmax 归一化，可以将复杂度大幅降低至线性级别 [RS21]。

为了理解这一点，考虑单个查询向量 $\mathbf{q}$，并假设我们将键和值划分为两个块，依次加载到内存中：

$$\mathbf{K} = \begin{bmatrix} \mathbf{K}_1 \\ \mathbf{K}_2 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} \mathbf{V}_1 \\ \mathbf{V}_2 \end{bmatrix} \tag{E.11.4}$$

如果忽略 softmax 中的分母，我们就可以分解 SA 操作，依次计算每个分块的输出：

$$\mathrm{SA}(\mathbf{q},\mathbf{K},\mathbf{V}) = \frac{1}{L_1+L_2}\left[\mathbf{h}_1+\mathbf{h}_2\right] \tag{E.11.5}$$

其中，针对两个分块 $i=1,2$，我们定义了两个辅助量：

$$\mathbf{h}_i = \exp\left(\mathbf{K}_i\mathbf{q}\right)\mathbf{V}_i \tag{E.11.6}$$

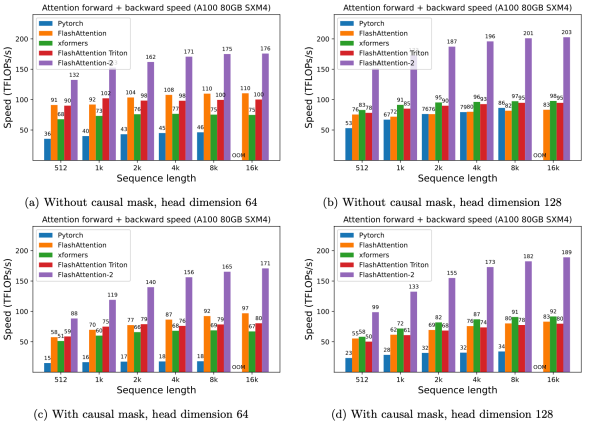
$$L_i = \sum_j \left[\exp\left(\mathbf{K}_i\mathbf{q}\right)\right]_j \tag{E.11.7}$$

请记住，我们是分批将分块加载到内存中的；因此，对于分块 1，我们计算 $\mathbf{h}_1$ 和 $L_1$；随后卸载该分块，转而计算分块 2 的 $\mathbf{h}_2$ 和 $L_2$。值得注意的是，除非我们记录额外的统计量 $L_i$（计算 softmax 操作归一化系数所需），否则该操作无法完全分解。更一般地，对于多个分块 $i=1,\ldots,m$，我们有：

$$\mathrm{SA}(\mathbf{q},\mathbf{K},\mathbf{V}) = \frac{1}{\sum_{i=1}^m L_i}\left[\sum_{i=1}^m \mathbf{h}_i\right] \tag{E.11.8}$$

因此，我们可以设计一种简单的迭代算法：对于加载到内存中的每一组键和值，我们更新并存储公式 (E.11.8)中分子和分母的累加和，仅在最后执行归一化。这种技巧（有时称为在线 softmax）结合了感知 I/O 的实现方式与算子融合（kernel fusion），促成了诸如 FlashAttention-2 等在内存和计算效率上极高的注意力机制实现。[4]注意力机制的分布式实现（例如 RingAttention[LZA23]）也可以通过以下方式设计：将查询分组分配给不同设备，并在设备间轮转键和查询的块。针对特定硬件优化该操作可能会导致一些反直觉现象，例如序列长度增加时速度反而提升——见图 F.11.3。

[4] https://github.com/Dao-AILab/flash-attention




(a) Without causal mask, head dimension 64

(b) Without causal mask, head dimension 128

(c) With causal mask, head dimension 64

(d) With causal mask, head dimension 128

**图 F.11.3:** *NVIDIA A100 GPU 上 FlashAttention 和 FlashAttention-2 的官方基准测试结果，转载自 https://github.com/Dao-AILab/flash-attention.*

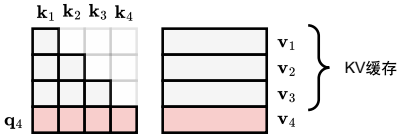



**图 F.11.4:** 在计算新词元的掩码自注意力时，大部分先前计算结果可以复用（灰色部分）。这被称为 ***KV 缓存***。

## 11.2.3 KV 缓存

在仅解码器模型中进行自回归生成时，MHA 的一个重要实现细节涉及 KV 缓存。对于每个待生成的新词元，只需计算注意力矩阵中的新一行以及一个值词元；这意味着之前的键和值可以储存在内存中，如图F.11.4 所示。这种机制被称为 **KV 缓存**（KV cache），它是大多数 MHA 优化实现中的标准配置。

KV 缓存的大小随序列长度线性增长。可以将其与因果卷积层的等价实现比较，后者的内存占用上限由感受野大小决定。在自回归生成中，设计既具表达力又有固定内存开销的层，正是促成第13章内容的动力之一。

### 11.2.4 面向图像和音频的 transformer




transformer 最初是为处理文本开发的，并很快成为语言建模的首选架构。特别是流行的 GPT-2 模型[RWC+19]（及其后续变体），它是一种仅解码器架构，通过预测文本序列中的词元进行预训练。大多数开源 LLM，如 LLaMA[TLI+23]，都遵循类似的架构。相比之下，BERT[DCLT18]是另一类流行的预训练模型，它基于仅编码器架构，通过预测被掩码的词元（即**掩码语言建模**，masked language modeling）进行训练。与 GPT 类模型不同，BERT 类模型不能用于生成文本，只能用于生成文本嵌入或作为微调架构的第一部分。用于语言建模的编码器-解码器模型也存在（例如 T5 家族[RSR+20]），但它们不像其他架构那么流行。[5]

从宏观角度看，transformer 由三个组件构成：词元化/嵌入步骤，将原始输入转换为向量序列；位置嵌入，用于编码原始序列的顺序信息；以及 transformer 模块本身。因此，通过定义合适的词元化过程和位置嵌入，可以设计适用于其他类型数据的 transformer。

首先考虑计算机视觉领域。在像素级别对图像词元化的计算成本过高，因为其复杂度随序列长度呈二次方增长。视觉 Transformer（Vision Transformer，ViT，[DBK+21]）的核心思想是将原始输入分割成互不重叠的固定大小**图像块**，然后将这些图像块展平并投影到预定义维度的嵌入空间中，如图F.11.5 所示。

图F.11.5 中的嵌入步骤可以通过卷积层来实现，其步幅等于卷积核大小。此外，像 einops[6]这样的库扩展了

[5] **扩散语言模型**（diffusion language model，DLM）[YTL+25]是自回归 LLM 的一种新兴替代方案，其训练采用去噪目标，与 BERT 类模型略有相似之处。所有类型的 LLM 在词元预测之外还要经历若干后训练（post-training）步骤（例如指令微调），限于篇幅，本书不作介绍。

[6] http://einops.rocks

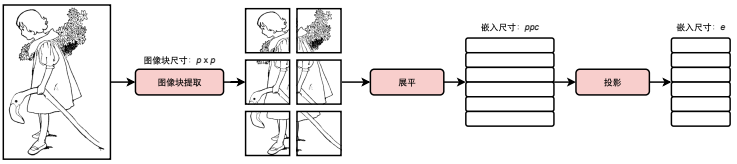



**图 F.11.5:** 图像词元化：将图像分为形状为 $p \times p$ 的不重叠图像块（其中 $p$ 为超参数）。随后，每个图像块被展平，经过线性投影映射到用户定义的嵌入维度 $e$。$c$ 为输入图像通道数。

einsum 操作（见第2.1节），允许将元素分组为预定形状的块。框C.11.2中给出一个相关示例。

原始 ViT 采用可学习的位置嵌入并结合一个额外的类词元来进行图像分类。ViT 也可用于图像生成，即按行优先或列优先的顺序预测图块。在此类应用中，我们可以训练一个独立模块，使用例如**向量量化变分自编码器**（vector-quantized variational autoencoder）[CZJ+22] 将每个图像块转换为离散词元集合，也可以直接处理连续输出 [TEM23]。然而，对于图像生成任务，扩散模型和流匹配等非自回归方法往往更受青睐；我们将在下一卷中介绍这些方法。

通过开发合适的词元化机制和位置嵌入，transformer 模型也被应用于音频，特别是在语音识别领域。在此这种情况下，通常采用小型一维卷积模型（包含池化层）作为词元化模块 [BZMA20, RKX+23]。例如，Wav2Vec[BZMA20]是一种仅编码器模型，其输出使用交叉熵损失的一种扩展进行训练，以将输出嵌入与转录对齐；这种损失函数称为**连接时序分类**（connectionist temporal classification）损失 [GFGS06]。由于具有精确对齐信息的标注数据稀缺，Wav2Vec 模型通常利用大量未标注音频数据，通过掩码语言建模损失的一种变体进行预训练。相比之下，Whisper[RKX+23] 是一种编码器-解码器模型，其解码器通过自回归方式生成转录文

```python
from einops import rearrange
# 一批图像
xb = torch.randn((32, 3, 64, 64))

# 定义操作：与标准 einsum 不同，
# 我们可以使用方括号
# 将输出拆分为多个块
op = 'b c (h ph) (w pw) \
                -> b (h w) (ph pw c)'

# 使用指定图块大小执行操作
patches = rearrange(xb, op, ph=8, pw=8)
print(patches.shape) # [输出]: (32, 64, 192)
```

**框 C.11.2:** *einops* 可通过简单扩展 *einsum* 语法将图像分解为图像块。

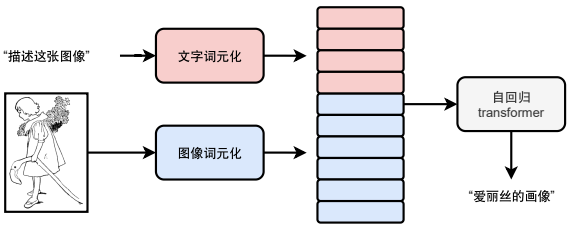



**图 F.11.6:** 同时处理图像和文本的**双模态** *transformer* 示意图：两个分词器的输出被拼接后送入模型。

本。这种架构赋予模型更大的灵活性，并降低了对强标注数据的需求，但代价是转录阶段可能出现“幻觉”（hallucination）现象。**神经音频编解码器**（neural audio codecs）也可被训练用于将音频压缩为离散词元序列 [DCSA23]，这反过来构成了文本转语音等生成式应用的基础 [WCW+23]。

transformer 也可用于时间序列 [AST+24]、图结构（下一章介绍）和其他类型的数据。数据与架构的解耦也是**多模态**（multimodal）变体的基础，这些变体可以接受（或提供）不同模态的数据作为输入（或输出）。这通过

使用合适的分词器对每种模态（图像、音频……）进行词元化，将不同词元拼接成单个序列来实现 [BPA+24]。我们在图F.11.6中给出一个**图像-文本**模型的例子。

## 11.3 transformer 变体

本章最后将讨论基础 transformer 模块的一些有趣变体。首先，为了略微减少计算时间或参数量，人们为超大规模 transformer 设计了几种变体。例如，**并行模块**（parallel blocks）[DDM+23]并行执行 MLP 和 MHA 操作：

$$\mathbf{H} = \mathbf{H} + \text{MLP}(\mathbf{H}) + \text{MHA}(\mathbf{H})$$

以这种方式，MLP 和 MHA 层中的初始与最终线性可以被融合，从而实现更高效计算。再举一例，**多查询 MHA**（multi-query MHA）[Sha19]对每个头共享相同的键和值投影矩阵，仅改变了查询。

更广义地说，我们可以用一种更简单的操作（关于序列长度呈线性复杂度）来替代 MHA 层，同时保持 transformer 模块整体结构，即交替进行词元混合与通道混合，并结合层归一化和残差连接。例如，假设序列长度固定（例如在计算机视觉任务中，图像块数量可以事先设定），此时 MHA 层可以被替代为作用于单个输入通道的 MLP，它对应嵌入的一个维度。这类模型被称为**混合器**（mixer）模型 [THK+21]——如图F.11.7所示。若忽略归一化，这可以表示为在输入矩阵的转置上交替应用 MLP：

$$\mathbf{H} = \text{MLP}(\mathbf{H}) + \mathbf{H} \tag{E.11.9}$$

$$\mathbf{H} = \left[\text{MLP}(\mathbf{H}^\top) + \mathbf{H}^\top\right]^\top \tag{E.11.10}$$

混合器模型也存在其他变体，例如使用一维卷积、傅里叶变换或池化。特别地，在 S2-MLP[YLC+22]模型中，词元混合操作被替换为一个更简单的 MLP，该 MLP 作用于输入数据的平移版本。这类模型被 [YLZ+22]统称为 **MetaFormer**。

门控（乘法）交互也可用于构建该模块。在这种情况下，多个模块并行执行，它们的输出通过阿达玛乘法合并。

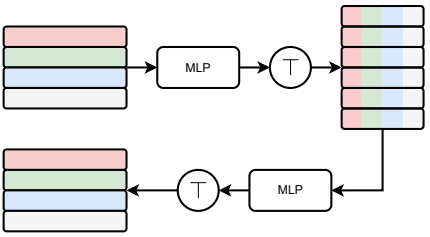



**图 F.11.7:** 混合器模块，由在输入矩阵的行和列上交替应用的 *MLP* 组成

我们可以将通用的门控单元表示为：

$$f(\mathbf{X}) = \phi_1(\mathbf{X}) \odot \phi_2(\mathbf{X}) \tag{E.11.11}$$

其中 $\phi_1$ 和 $\phi_2$ 是可训练模块。例如，若 $\phi_1(\mathbf{X}) = \sigma(\mathbf{XA})$ 且 $\phi_2(\mathbf{X}) = \mathbf{XB}$，我们可以得到如第5.4节所述的**门控线性单元**（gated linear unit，GLU）。

举几个代表性例子，**gMLP** 模型 [LDSL21] 在混合器模型中使用门控单元代替通道混合模块；**LLaMA** 家族模型 [TLI+23]使用类 GLU 单元代替标准 MLP 模块；**门控注意力单元**（gated attention unit，GAU）[HDLL22] 使用了更简单的类注意力模型，其中 $\phi_1$ 对应单头机制，$\phi_2$ 对应线性投影。这些设计在循环模型的一些近期变体中尤为流行，相关内容将在后文第13章中讨论。

为了进一步简化设计，多线性算子网络（MONet）移除了所有激活函数，定义了一种仅由线性投影和逐元素乘法组成的模块 [CCGC24]：

$$\mathbf{H} = \mathbf{E}(\mathbf{AX} \odot \mathbf{BX} + \mathbf{DX})$$

其中 $\mathbf{E}$ 与 transformer 模块中的输出投影类似，$\mathbf{DX}$ 充当残差连接，$\mathbf{B}$ 通过低秩分解实现，以减少参数量 [CCGC24]。为了引入词元混合能力，模型中所有奇数编号模块都实现了词元移位（token-shift）操作。

## 从理论到实践



此时，可以尝试许多有趣的练习——你快要成为设计可微模型的专家了！首先，你可以利用任意图像分类数据集，尝试从零开始实现第11.2.4节中介绍的 ViT，并参考 [DBK+21]设定超参数。在小数据集上从零训练 ViT 颇具挑战性 [LLS21, SKZ+21]，因此，除非你拥有足够的算力来处理百万级规模的数据集，否则要做好可能遇到挫折的心理准备。你也可以尝试更简单的变体，例如第11.3节中介绍的混合器模型。这些练习应该都相对简单。

1. 对于图像词元化，你可以使用框C.11.2中提到的 einops 库，也可以采用其他策略（例如使用大步幅卷积）。对于小尺寸图像，你甚至可以尝试将每个像素当作一个词元。

2. 对于位置嵌入，第10.2节中介绍的所有策略均适用。对于 ViT 而言，最简单的方法是初始化一个包含可训练嵌入矩阵，但我建议尝试使用正弦嵌入和相对位置嵌入练习。

你还可以尝试实现一个小型 GPT 类模型。文本数据词元化涉及许多复杂的细节，我们在此未予详述。不过，minGPT 代码库[7]是一个极佳的教学实现，你可以将其作为入门起点。

[7] https://github.com/karpathy/minGPT

# 12 | 图模型

**本章导读**

本章探讨图结构数据，即由一组（已知）关系连接的集合。图在现实世界中无处不在，涵盖了从蛋白质到交通网络、社交网络以及推荐系统等多种应用场景。我们将介绍专用于处理图数据的层结构，这些结构大致可分为消息传递层（message-passing layers）和图 transformer 架构（graph transformer）。

## 12.1 基于图数据的学习

### 12.1.1 图与图上的特征

到目前为止，我们讨论的数据要么是完全非结构化的（如表示为向量的表格数据），要么是具有简单结构的数据（包括集合、序列以及图像等网格状数据）。然而，许多类型的数据是由其组成部分之间更复杂的依赖关系定义的。例如，分子由原子构成，而原子之间仅通过化学键稀疏连接。各类网络（如社交网络、交通网络、能源网络）由数百万个单元（如人、产品、用户）组成，这些单元仅通过少量连接（如道路、反馈或友谊关系）进行交互。用**图论**（graph theory）的语言来描述这些数据更为自然。本章旨在介绍能处理此类数据的可微模型。

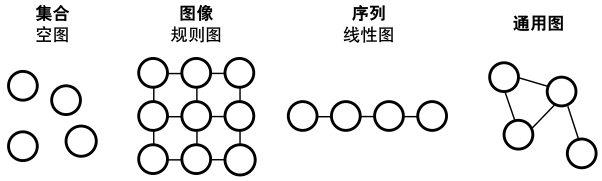



**图 F.12.1:** 图是对多种数据类型的概括：集合可视为空图（*empty graph*，或仅包含自环的图），图像可视为规则图（*regular graph*），而序列则可视为线性图（*linear graph*）。在本章中，我们将探讨更通用的图结构。

最简单的图可以描述为一对集合 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$，其中 $\mathcal{V} = \{1, \dots, n\}$ 是**节点**（node，或称**顶点**，vertex）的集合，而：

$$\mathcal{E} = \Big\{ (i,j) \mid \underbrace{i,j \in \mathcal{N}}_{\text{图中的两个节点}} \Big\}$$

是图中存在的**边**（edge）的集合。在大部分数据集中，节点数量 $n$ 和边数量 $m = |\mathcal{E}|$ 可能因图而异。

图概括了我们已经见过的许多概念：例如，仅包含 $(i,i)$ 形式的**自环**（self-loop）代表对象的集合；而包含所有可能边的图（**全连接图**，fully-connected graph）则与注意力层相关，如下文所示。图像可以表示为图结构[1]：将每个像素对应一个节点，并按照规则的网格结构连接邻近像素——如插图F.12.1所示。[2]

图结构中的连接可以由一种称为**邻接矩阵**（adjacency matrix）的矩阵形式来等价表示。它是一个二值方阵

[1]译者注：为避免歧义或混淆，与“图像”等词同时出现时，“graph”解释性地译为“图结构”。

[2]这种基本设定有许多变体，包括异质图（包含不同类型节点的图）、有向图、符号图等。其中大多数都可以通过下文所述技术的变体来处理。

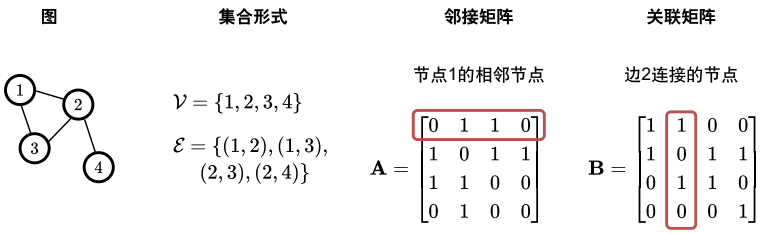



**图 F.12.2:** 可以用三种方式表示图的连接关系：由节点对构成的集合 $\mathcal{E}$（第二列）；大小为 $(n, n)$ 的邻接矩阵（第三列）；或大小为 $(n, |\mathcal{E}|)$ 的关联矩阵（第四列）。

$\mathbf{A} \sim \text{Binary}(n, n)$，定义如下：

$$A_{ij} = \begin{cases} 1 & \text{若 } (i, j) \in \mathcal{E} \\ 0 & \text{否则} \end{cases}$$

在这种形式下，集合由单位矩阵 $\mathbf{A} = \mathbf{I}$ 表示，全连接图由全 1 矩阵表示，图像则由托普利兹矩阵表示。若图中的连接总是双向的，即 $(i, j)$ 和 $(j, i)$ 总是成对出现在边上，则称该图为**无向**(undirected)图，此时满足 $\mathbf{A}^\top = \mathbf{A}$。为简化起见，我们将主要讨论无向图，但这些方法很容易推广到有向情形。值得注意的是，还存在其他的矩阵表示形式，例如关联矩阵 $\mathbf{B} \sim \text{Binary}(n, |\mathcal{E}|)$，其中若节点 $i$ 参与了边 $j$ 的构成，则 $B_{ij} = 1$；由于每条边恰好连接两个节点，因此满足 $\mathbf{B}\mathbf{1}^\top = 2$。示例如图F.12.2所示。

我们假设图包含自环，即 $A_{ii} = 1$。如果邻接矩阵不包含自环，可以通过如下重新赋值来添加它们：

$$\mathbf{A} \leftarrow \mathbf{A} + \mathbf{I}$$

### 12.1.2 图特征

图通常伴随着各种描述其属性的特征。例如，分子中的原子和键可以用表示其类型的类别特征来描述；交通网络中的道路可以具有容量和交通流量属性；社交网络中的两位朋友则可以用彼此相识的年数来描述。

一般来说，这些特征可以分为三类：与每个节点关联的**节点特征**，与每条边关联的**边特征**，以及与整个图关联的**图特征**。我们将从最简单的情形开始，即仅拥有非结构化节点特征，其中每个节点 $i$ 关联一个向量 $\mathbf{x}_i \sim (c)$。整个图可由两个矩阵描述：一个是**特征矩阵** $\mathbf{X} \sim (n, c)$，另一个是邻接矩阵 $\mathbf{A} \sim (n, n)$。

在大多数情况下，节点的顺序无关紧要；也就是说，如果我们考虑一个置换矩阵 $\mathbf{P} \sim \text{Binary}(n, n)$（见第10.2节），那么原图与其置换后的版本在本质上是相同的，即：

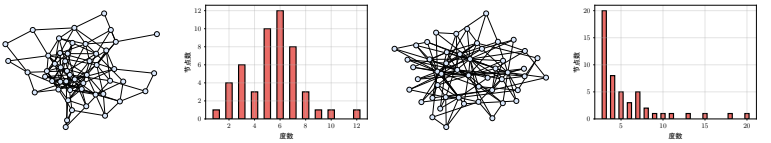
$$(\mathbf{X}, \mathbf{A}) \quad \text{与} \quad (\mathbf{PX}, \mathbf{PAP}^\top) \quad \text{表示相同的图}$$

请注意，置换矩阵的作用是交换 $\mathbf{X}$ 中的行，而在邻接矩阵中，它同时交换行和列。

一些特征也可以直接从图的拓扑结构中提取。例如，我们可以为每个节点关联一个标量值 $d_i$，称为**度**（degree），它描述该节点连接了多少个其他节点：

$$d_i = \sum_j A_{ij}$$

如图F.12.3 所示，图结构中各节点的度分布是该图本身的重要特征。我们可以将这些度值汇总为一个对角矩阵，称为**度矩阵**：



**(a)** 埃尔德什-雷尼 **(b)** 度分布 **(c)** 巴拉巴西-阿尔伯特 **(d)** 度分布

**图 F.12.3:** *(a)* 从伯努利分布中独立抽取每条边，生成的随机图（**埃尔德什-雷尼**模型，*Erdős–Rényi model*）*(b)* 这些图的度分布类似高斯分布。*(c)* 通过依次添加节点生成的随机图；每加入一个新节点，就以与已有节点度数成正比的概率，选择 3 个已有节点建立连接（**优先连接过程**，*preferential attachment process* 或**巴拉巴西-阿尔伯特模型**，*Barabasi-Albert model*）。*(d)* 这些图中少数节点具有大量连接，充当图中的枢纽。

$$\mathbf{D} = \begin{bmatrix} d_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & d_n \end{bmatrix}$$

使用度矩阵，我们可以定义几种类型的加权邻接矩阵。例如，行归一化邻接矩阵定义为：

$$\mathbf{A}' \leftarrow \mathbf{D}^{-1}\mathbf{A}_{ij} \quad \rightarrow \quad A'_{ij} = \frac{1}{d_i} A_{ij}$$

这种归一化满足 $\sum_i A'_{ij} = \mathbf{1}$。我们也可以定义列归一化邻接矩阵为 $\mathbf{A}' = \mathbf{A}\mathbf{D}^{-1}$。两种矩阵都可以被解释为图上的“随机游走”：给定节点 $i$，归一化邻接矩阵的相应行或列表示随机移动到其任一相邻节点的概率分布。一种更通用的对称归一化邻接矩阵定义如下：

$$\mathbf{A}' = \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$$

它由 $A'_{ij} = \frac{A_{ij}}{\sqrt{d_i d_j}}$ 定义，根据所连接两个节点的度，为

每个连接赋予相应权重。即根据所连接的两个节点的度数来赋予每条边相应的权重。邻接矩阵及其加权变体都具有以下性质：只要 $(i,j) \notin \mathcal{E}$，就有 $A_{ij} = 0$。在信号处理术语中，这些被称为**图移位**（graph-shift）矩阵。

**矩阵的稀疏性**

考虑一个包含 6 个节点的图的典型邻接矩阵（作为练习，尝试画出该图）：

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

这个邻接矩阵非常稀疏（许多零元素）。这是一个重要特性，因为针对稀疏矩阵存在专门的实现和技术，与密集矩阵相比，其复杂度更低。[a]

[a] 例如在 JAX 中：https://jax.readthedocs.io/en/latest/jax.experimental.sparse.html。

### 12.1.3 图上的扩散操作

我们所关注的一种基础图操作称为**扩散**（diffusion），它对应于根据图拓扑结构对节点特征进行平滑。为了理解这一点，考虑每个节点上的标量特征，并将它们收集到向量 $\mathbf{x} \sim (n)$ 中，并进行如下特征操作：

$$\mathbf{x}' = \mathbf{A}\mathbf{x}$$

其中 $\mathbf{A}$ 可以为邻接矩阵、其归一化变体或任意加权邻

接矩阵。我们可以逐节点地将该操作重写为：

$$x_i' = \sum_{j \in \mathcal{N}(i)} A_{ij} x_j$$

此处我们定义了 1 跳邻域：

所有以节点 $i$ 为顶点的边

$$\mathcal{N}(i) = \left\{ j \mid (i,j) \in \mathcal{E} \right\}$$

如果将节点特征视为一种物理量，那么通过邻接矩阵进行的投影可被视为一种“扩散”过程，它把该节点原有的物理量替换为节点邻域内物理量的加权平均值。

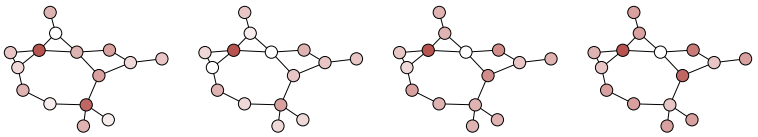
在图分析的背景下，另一个基本矩阵是**拉普拉斯矩阵**(Laplacian matrix)：

$$\mathbf{L} = \mathbf{D} - \mathbf{A}$$

其中度矩阵按 $D_{ii} = \sum_j A_{ij}$ 计算，无论邻接矩阵是否归一化。用拉普拉斯矩阵进行一步扩散可写为：

$$[\mathbf{L}\mathbf{x}]_i = \sum_{(i,j) \in \mathcal{E}} A_{ij}(x_i - x_j) \tag{E.12.1}$$

由此可见，拉普拉斯矩阵与图上的梯度概念密切相关，对它的分析是**谱图论**（spectral graph theory）领域的核心。例如，在公式(E.12.1)中，全 1 向量 $\mathbf{1}$ 始终是拉普拉斯矩阵的一个特征向量，其对应的特征值为零（具体而言，它是最小特征值）。使用拉普拉斯矩阵进行扩散的例子如图F.12.4所示。



**(a)** 初始图 **(b)** *10* 步扩散 **(c)** *20* 步扩散 **(d)** *30* 步扩散

**图 F.12.4:** *(a)* 具有 *15* 个节点的随机图，每个节点上有一个标量特征（用不同颜色表示）。*(b)-(d)* 过拉普拉斯矩阵扩散 *10* 步、*20* 步和 *30* 步后的结果。特征最终收敛至稳定状态。

### 12.1.4 流形正则化

由(E.12.1)我们还可以推导出基于拉普拉斯矩阵的二次型：

$$\mathbf{x}^\top \mathbf{L}\mathbf{x} = \sum_{(i,j)\in\mathcal{E}} A_{ij}(x_i - x_j)^2 \tag{E.12.2}$$

通俗地说，这是一个标量值，用于衡量图上信号的“平滑度”，即图中相连节点对之间信号变化的剧烈程度。为了展示这一概念的简单应用，考虑一个表格分类数据集 $\mathcal{S}_n = \{(\mathbf{x}_i, y_i)\}$。假设我们在该数据集上构建一个图，其中每个节点对应数据集中的一个样本，且邻接矩阵是基于特征间的距离构建的：

$$A_{ij} = \begin{cases} \exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2) & \text{若 } \|\mathbf{x}_i - \mathbf{x}_j\|^2 < \tau \\ 0 & \text{否则} \end{cases} \tag{E.12.3}$$

其中 $\tau$ 是用户定义的超参数。给定一个分类模型 $f(\mathbf{x})$，我们可能希望对其输出施加约束，使其对相似的输入给出相似的输出，其中相似度按公式(E.12.3)定义成正比。为此，我们可以将图特征定义为模型输出：

$$\mathbf{f} = \begin{bmatrix} f(\mathbf{x}_1) \\ \vdots \\ f(\mathbf{x}_n) \end{bmatrix} \sim (n)$$

二次形式(E.12.2)揭示了相似输入在预测结果上的差异：

$$\mathbf{f}^\top \mathbf{L}\mathbf{f} = \sum_{i,j} A_{ij}(f(\mathbf{x}_i) - f(\mathbf{x}_j))^2 \tag{E.12.4}$$

最优模型可以通过正则化优化问题求得，其中正则化项由(E.12.4)给出：

$$f^*(\mathbf{x}) = \arg\min \left\{ \sum_{i=1}^{n} L(y_i, f(\mathbf{x})) + \lambda \ \mathbf{f}^\top \mathbf{L}\mathbf{f} \right\}$$

其中 $L$ 为通用损失函数，$\lambda$ 为标量超参数：

这叫做**流形正则化**（manifold regularization）[BNS06]，它作为一种通用的正则化工具，用于约束模型在图结构上保持平滑；其中，图的邻接关系既可以是预先给定的，也可以由用户根据式(E.12.3)构建。在有少量带标签数据，以及大量来自同一分布的无标签数据的**半监督**（semi-supervised）场景中，这种方法尤为有效，因为式(E.12.4)中的正则化项不依赖于标签信息 [BNS06]。然而，该模型的预测仅依赖单个样本 $\mathbf{x}_i$，且图结构在训练完成后不再被使用。在下一节中，我们将介绍更自然的方法，将这种连接关系嵌入到模型内部。

## 12.2 图卷积层

### 12.2.1 图层的性质

为了设计预测结果依赖于图连接关系的模型，我们可以利用邻接矩阵的信息来增强标准层 $f(\mathbf{X})$，即考虑如下形式的层：

$$\mathbf{H} = f(\mathbf{X}, \mathbf{A})$$

其中与前文一致，$\mathbf{X} \sim (n, c)$（$n$ 为节点数，$c$ 为每个节点特征数），$\mathbf{H} \sim (n, c')$。也就是说，该操作不改变图的连接结构，并为图中每个节点 $i$ 返回更新后的嵌入 $\mathbf{H}_i \sim (c')$。在下文中，$\mathbf{A}$ 可以是邻接矩阵，也可以是具有相同稀疏模式的任意矩阵（图移位矩阵），包括加权邻接矩阵、拉普拉斯矩阵等。

由于对置换图中的节点不应对最终预测有影响，因此该层不应依赖于节点的特定顺序；换言之，对于任意置换矩阵 $\mathbf{P}$，该层的输出应当满足**置换等变性**：

$$f(\mathbf{PX}, \mathbf{PAP}^\top) = \mathbf{P} \cdot f(\mathbf{X}, \mathbf{A})$$

参考图像处理中的做法，我们可以定义图结构层的“局部性”概念。为此，我们首先引入子图的概念。给定全图的一个节点子集 $\mathcal{T} \subseteq \mathcal{V}$，我们将由 $\mathcal{T}$ 诱导的**子图**（subgraph）定义为：

$$\mathcal{G}_\mathcal{T} = (\mathbf{X}_\mathcal{T}, \mathbf{A}_\mathcal{T})$$

其中 $\mathbf{X}_\mathcal{T}$ 是尺寸为 $(|\mathcal{T}|, c)$ 的矩阵，收集 $\mathcal{T}$ 中各节点的特征；而 $\mathbf{A}_\mathcal{T} \sim (|\mathcal{T}|, |\mathcal{T}|)$ 是全图邻接矩阵中对应的分块矩阵。

**定义 D.12.1 (图局部性)** *若对于图中每个节点 $i$，都有 $\mathbf{H}_i = f(\mathbf{X}_{\mathcal{N}(i)}, \mathbf{A}_{\mathcal{N}(i)})$，其中 $\mathcal{N}(i)$ 代表节点 $i$ 的 1 跳邻域，则称该图层 $\mathbf{H} = f(\mathbf{X}, \mathbf{A})$ 具有*局部性。

这类似于在图像处理中，考虑距离为 1 的所有像素点，区别在于：(a) 在图结构的情境下，$\mathcal{N}(i)$ 中的节点没有特定的顺序；(b) $\mathcal{N}(i)$ 的大小会随节点 $i$ 的不同而发生显著变化。因此，我们无法像在图像处理中那样定义卷积，因为卷积的定义需要上述两个性质（参考卷积层中的权重张量）。

在下文中请注意，我们可以将局部性的定义扩展至 1 跳邻域之外。例如 2 跳邻域 $\mathcal{N}^2(i)$ 定义为所有距离不超过 2 的节点：

$$\mathcal{N}^2(i) = \bigcup_{j \in \mathcal{N}(i)} \mathcal{N}(j)$$

其中 $\cup$ 为并集运算符。我们可以扩展局部性的定义，将更高阶的邻域考虑在内，从而设计相当于 $3 \times 3$ 滤波器、$5 \times 5$ 滤波器等结构。

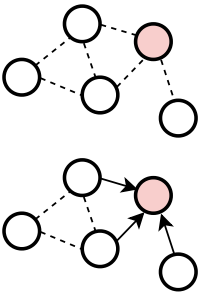
### 12.2.2 图卷积层




为了定义能模拟卷积层的图层，我们需要它具备置换等变性（而非平移等变性）以及局部性。MHA 层天然具备置换等变性，但它不具备局部性，且不显式依赖于邻接矩阵 $\mathbf{A}$。我们将在下一节探讨实现这一目标的可能扩展。目前，让我们先关注一个更简单的全连接层：

$$f(\mathbf{X}, _) = \phi(\mathbf{X}\mathbf{W} + \mathbf{b})$$

其中 $\mathbf{W} \sim (c, c')$ 且 $\mathbf{b} \sim (c')$。该层也天然具备置换等变性，但它不依赖于图的连接关系（即忽略了该信息）。为了构建合适的可微层，我们可以将该层的操作与扩散步骤交替进行。



逐节点更新：

$$\mathbf{x}'_i = \mathbf{W}^\top \mathbf{x}_i + \mathbf{b}$$

聚合：

$$\mathbf{h}_i = \phi\left(\sum_j A_{ij}\mathbf{x}'_j\right)$$

**图 F.12.5:** *GC* 层的两个阶段：每个节点与其他所有节点并行更新其嵌入；输出该节点邻域内所有已更新嵌入的加权平均值。




**定义 D.12.2 (图卷积)**

给定由节点特征矩阵 $\mathbf{X} \sim (n, c)$ 和通用图移位矩阵 $\mathbf{A} \sim (n, n)$（如邻接矩阵、拉普拉斯矩阵等）代表的图，**图卷积**（*graph convolutional*，*GC*）层定义如下 *[KW17]*：

$$f(\mathbf{X}, \mathbf{A}) = \phi(\mathbf{A}(\mathbf{X}\mathbf{W} + \mathbf{b}))$$

其中 $\mathbf{W} \sim (c, c')$ 和 $\mathbf{b} \sim (c')$ 为可训练参数，$c'$ 为超参数；$\phi$ 为标准激活函数（例如 *ReLU*）。

请注意其与标准卷积层的相似之处：我们通过矩阵 $\mathbf{W}$ 执行“通道混合”操作，并通过矩阵 $\mathbf{A}$ 执行“节点混合”操作。区别在于，前者在此情形下不可训练（原因再次由于节点度数各不相同，且该层需要满足置换等变性）。具体可视化见图F.12.5。这种类比还可以借助图信号处理中的概念得到更形式化的证明，但这超出了本书的范围 [BBL+17]。

若忽略偏置项，对于单个节点 $i$，上述过程可重写为：

$$\mathbf{H}_i = \phi\left(\sum_{j\in\mathcal{N}(i)} A_{ij}\mathbf{X}_j\mathbf{W}\right)$$

因此，我们首先同步更新所有节点嵌入（通过右乘 $\mathbf{W}$ 实现）。随后，每个节点计算自身及邻近节点更新后嵌入的加权平均值。由于不同节点的邻域数量可能各不相同，使用邻接矩阵的归一化变体能显著帮助模型训练。该层满足置换等变性，简单证明如下：

$$f(\mathbf{P}\mathbf{X}, \mathbf{P}\mathbf{A}\mathbf{P}^\top) = \phi\left(\mathbf{P}\mathbf{A}\mathbf{P}^\top\mathbf{P}\mathbf{X}\mathbf{W}\right) = \mathbf{P}\cdot f(\mathbf{X},\mathbf{A})$$

### 12.2.3 构建图卷积网络

单个 GC 层具有局部性，但多层堆叠后的网络并非如此。例如，考虑一个两层的 GC 模型：

$$f(\mathbf{X},\mathbf{A}) = \phi(\mathbf{A}\ \underbrace{\phi\left(\mathbf{A}\mathbf{X}\mathbf{W}_1\right)}_{\text{第一个 GC 层}}\ \mathbf{W}_2) \tag{E.12.5}$$

其中有两个可训练权重矩阵 $\mathbf{W}_1$ 和 $\mathbf{W}_2$。与图像处理中的情况类似，我们可以定义“感受野”这一概念。

> **定义 D.12.3 ([图感受野)**
>
> 给定一个通用的图神经网络 $\mathbf{H} = f(\mathbf{X},\mathbf{A})$，节点 $i$ 的**感受野**是指满足 $\mathbf{H}_i = f(\mathbf{X}_{\mathcal{V}(i)}, \mathbf{A}_{\mathcal{V}(i)})$ 的最小节点集合 $\mathcal{V}(i)\subseteq\mathcal{V}$。

对于单 GC 层，其感受野为 $\mathcal{V}(i) = \mathcal{N}(i)$。对于公

式(E.12.5)所示的两层网络，我们需要考虑“邻居的邻居”，其感受野变为 $\mathcal{V}(i) = \mathcal{N}^2(i)$。一般来说，对于堆叠了 $k$ 层的网络，其感受野为 $\mathcal{V}(i) = \mathcal{N}^k(i)$。在图中任意两个节点间移动所需的最少步数，被称为该图的**直径**（diameter）。图的直径决定了使所有节点都能获得全局感受野所需的最少层数。

**多项式 GC 层**

此外，我们还可以扩大单个 GC 层的感受野。例如，如果从邻接矩阵中移除自环，并同时引入邻接矩阵的平方项，就能使该层在 $\mathcal{N}^2(i)$ 而不只是 $\mathcal{N}(i)$ 的范围内具有局部性：

$$\mathbf{H} = \phi\left(\mathbf{X}\mathbf{W}_0 + \mathbf{A}\mathbf{X}\mathbf{W}_1 + \mathbf{A}^2\mathbf{X}\mathbf{W}_2\right)$$

其中有三组参数 $\mathbf{W}_0$、$\mathbf{W}_1$ 和 $\mathbf{W}_2$，分别用于处理自环、直接邻域以及邻域的邻域。这种结构被称为多项式（polynomial）图卷积层。通过加入更高次幂的项，可以获得更大感受野。此外，还可以通过考虑多项式的比值，来设计更复杂的层 [BGLA21]。

我们可以将图卷积层与标准归一化层、残差连接、Dropout 或任何其他具有置换等变性的操作相结合。与图像处理场景不同，在图结构数据上进行池化操作更困难，因为没有直接方法可以对图的连接关系进行下采样。尽管可以通过利用图论工具或额外的可训练组件来定义池化层，但它们在实际应用中并不常见 [GZBA22]。

假设 $\mathbf{H} = f(\mathbf{X}, \mathbf{A})$ 表示由若干层组合的结构，它能为每个节点生成更新后的嵌入表示（且不改变其连接关系）。类比 CNN，我们将其称为**主干**网络。我们可以在这些表示之上添加一个小型头部结构，从而完成通用**图卷积网络**（graph convolutional network，GCN）的设计：

$$y = (g \circ f)(\mathbf{X}, \mathbf{A})$$

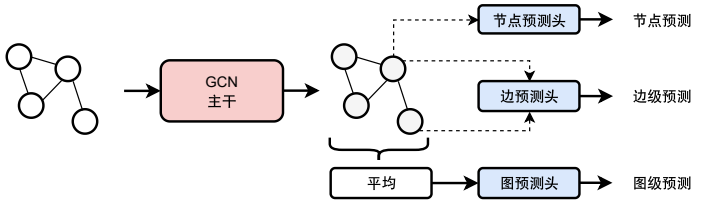



**图 F.12.6:** *(a)* 节点任务需要处理单个节点特征；*(b)* 边任务同时依赖于两个节点；*(c)* 图任务可通过将所有节点表示池化为固定维向量来实现。

头部的设计取决于我们要解决的任务。最常见的任务可归为以下三类基本范畴：节点级任务（例如节点分类）、边级任务（例如边分类）或图级任务（例如图分类）。我们依次简要探讨这三类任务示例，详见图F.12.6。

### 节点分类

首先，假设输入图描述某种社交网络，其中每个用户对应一个节点。对于用户的一个给定子集 $\mathcal{T} \subseteq \mathcal{V}$，我们已知其对应标签 $y_i$（其中 $i \in \mathcal{T}$，例如：该用户是真实用户、机器人账号，还是其他类型的自动化账号）。我们的目标是预测其余所有节点的标签。在这种情况下，我们可以通过处理每个更新后的节点嵌入，来获得逐节点的预测，例如：

$$\hat{y}_i = g(\mathbf{H}_i) = \text{softmax}(\text{MLP}(\mathbf{H}_i))$$

对整个矩阵 $\mathbf{H}$ 执行此操作，可得到所有节点的预测结果；但我们只知道一小部分子集的真实标签。我们可以通过排除训练集之外的节点，来训练图卷积网络（GCN）：

$$\arg\min \frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} \text{CE}(\hat{y}_i, y_i)$$

其中 CE 为交叉熵损失函数。值得注意的是，虽然我们舍弃了训练集之外节点的输出预测，但由于 GCN 内部存在扩散步骤，这些节点的输入特征依然参与了训练。在训练完成后，只需再运行一次 GCN，即可对剩余节点分类。这种仅有部分训练数据带有标签的情形，被称为**半监督**问题。

**边分类**

作为第二个例子，假设我们拥有部分边的标签，即 $\mathcal{T}_E \subseteq \mathcal{E}$。例如，图结构代表交通网络，而我们仅知道其中部分路段的交通流量。此时，我们可以通过添加一个依赖两个相连节点特征的预测头，来实现边级预测。例如，将这两个节点的特征拼接：

$$\widehat{y}_{ij} = g(\mathbf{H}_i, \mathbf{H}_j) = \mathrm{MLP}\left(\left[\mathbf{H}_i \,\|\, \mathbf{H}_j\right]\right)$$

对于二分类任务（例如，预测两个用户之间的亲密度，用 0 与 1 之间的标量表示），我们可以通过计算这两个特征的点积来简化上述过程：

$$\widehat{y}_{ij} = \sigma(\mathbf{H}_i^\top \mathbf{H}_j)$$

与前文类似，我们可以通过最小化已知边上的损失函数来训练网络。

**图分类**

最后，假设我们的目标是对整个图进行分类（或回归分析）。例如，若图结构代表一个分子，我们可能希望预测其某种化学性质，比如它与其他化合物发生反应的活性。我们可以通过对节点表示进行池化（例如求和），并处理由此得到的固定维度的嵌入来实现：

$$y = \text{MLP}\left(\frac{1}{n}\sum_{i=1}^{n}\mathbf{H}_i\right)$$

最后的池化层使得网络对于节点排列顺序具有不变性。在这种情况下，数据集由多个图结构（例如若干个分子）组成，这与标准的图像分类任务类似。而在节点和边的相关任务中，有些数据集可能仅由单个图（例如一个大型社交网络）构成，而另一些数据集则可能包含多个图（例如来自不同城镇、互不相连的道路网络）。这就引出了一个问题：如何高效地构建图的小批量数据。

### 12.2.4 图神经网络的实现




如前文所述，图数据处理的特殊之处是，其中的某些矩阵可能非常稀疏。例如，考虑以下邻接矩阵：

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

这对应于包含三个节点的图，其中节点 1 和节点 3 之间存在一条双向边。我们可以通过仅存储非零值的索引，来更高效地存储该矩阵，例如代码中表示为：

```
A = [[0,2], [2,0]]
```

这被称为**坐标列表**（coordinate list）格式。对于非常稀疏的矩阵，采用此类专用格式不仅能节省存储空间，还能显著提升对稀疏矩阵（或稀疏矩阵与密集矩阵组合）运算时的执行效率。例如。`pytorch-sparse`[3]在 PyTorch 中支持转置和多种矩阵乘法的高效实现。这也反映在各层

[3] https://github.com/rusty1s/pytorch_sparse

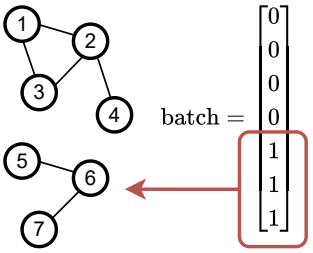



**图 F.12.7:** 小批量中的两个图可以被视为具有不连通分量的单一图。为了区分它们，需要引入一个额外向量，用来存储节点与图 *ID* 之间的映射关系。

的实现上。在 PyTorch Geometric[4]（PyTorch 中最常用的图神经网络库之一）中，各层的前向传播过程以图的特征和连接关系为输入，其中图的连接关系以边坐标列表的形式给出。

稀疏矩阵在小批量数据上还会带来另一个值得关注的结果。假设有 $b$ 个图，记作 $(\mathbf{X}_i, \mathbf{A}_i)_{i=1}^b$。每个图中节点特征的数量 $c$ 相同，但节点数量 $n_i$ 可能不同；因此 $\mathbf{X}_i \sim (n_i, c)$，且 $\mathbf{A}_i \sim \text{Binary}(n_i, n_i)$。为了构建一个小批量，我们可以创建两个 3 阶张量：

$$X \sim (b, n, c) \tag{E.12.6}$$

$$A \sim \text{Binary}(b, n, n) \tag{E.12.7}$$

其中 $n = \max(n_1, \ldots, n_b)$，且两个张量都通过补零操作填充。然而，基于以下观察，我们可以得到一种更优雅的替代方案：在 GC 层中，若两个节点之间没有任何路径（即由一系列边构成的序列），它们便无法相互传递信息。因此，只需将各个图的所有节点汇集到同一张图中，

[4] https://pytorch-geometric.readthedocs.io/en/latest/get_started/introduction.html#learning-methods-on-graphs

可以用单个图来表示整个小批量：

$$\mathbf{X} = \begin{bmatrix} \mathbf{X}_1 \\ \vdots \\ \mathbf{X}_b \end{bmatrix} \tag{E.12.8}$$

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_1 & \dots & \mathbf{0} \\ \vdots & \ddots & \vdots \\ \mathbf{0} & \dots & \mathbf{A}_b \end{bmatrix} \tag{E.12.9}$$

其中 $\mathbf{X} \sim (\sum_i n_i, c)$ 且 $\mathbf{A} \sim \text{Binary}(\sum_i n_i, \sum_i n_i)$。该小批量数据的邻接矩阵呈块对角结构，即对角块之外的所有元素均为零（来自不同图的节点之间没有连接）。虽然这看起来有些浪费空间，但它实际上提高了图的稀疏度，从而更有效地利用稀疏矩阵运算。因此，对于图数据集而言，在许多情况下，处理单个图与处理由多个图组成的小批量数据，在本质上并无区别。

为了追踪每个节点所属的原始图，我们可以在表示中增加一个额外的向量 $\mathbf{b} \sim (\sum_i n_i)$，其中 $b_i$ 是取值范围在 $[0, \dots, b-1]$ 之间的索引，用于标识该节点所属的输入图（共 $b$ 个）——如插图F.12.7所示。在图分类任务中，我们可以利用 $\mathbf{b}$，对属于不同图的节点组分别执行池化操作。假设 $\mathbf{H} \sim (n, c')$ 为 GCN 主干网络的输出，那么：

$$\text{scatter_sum}\,(\mathbf{H}, \mathbf{b}) = \mathbf{Y} \sim (b, c') \tag{E.12.10}$$

这被称为**散射求和**（scattered sum）操作。其中，$\mathbf{Y}_i$ 是 $\mathbf{H}$ 中所有满足 $b_j = i$ 的行之和，如图F.12.8 所示。对于其他类型的池化操作，例如平均池化或最大池化，也可以定义类似的操作。

一个额外的问题是，有时单张图就可能大到无法完整载入内存。在这种情况下，需要从原始图中采样子图来构建小批量数据 [HYL17]。这个任务相对复杂，超出了本章的讨论范围。

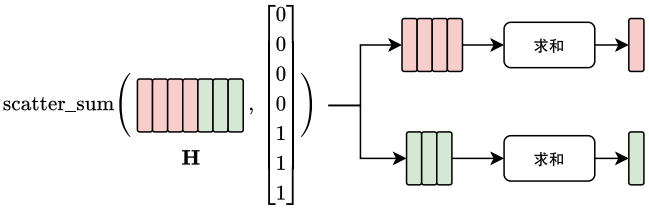



**图 F.12.8:** 图*F.12.7*所示图结构上的散射求和示例。在此示例中，节点 *(1, 2, 3, 4)* 属于图 *1*，而节点 *(5, 6, 7)* 属于图 *2*。经过池化操作后，我们得到了这两个图各自的池化表示。

## 12.3 图卷积层之外

以 GC 层为基础，我们接下来概述几种扩展形式，这些扩展在自适应能力或能处理的图特征方面有所不同。最后我们将讨论**图 transformer** (graph transformer)，这是一种不同的层，其核心思想是将图转化为结构嵌入，并将其与节点特征相加。

### 12.3.1 图注意力层

GC 层存在一个问题：用于汇总邻域贡献的权重是固定的，由邻接矩阵或其归一化变体给出。这等同于假设：除了连接数量的差异外，所有邻近节点的重要性都是相近的。如果图中的节点倾向于与相似的节点连接，则称该图具有**同质性** (homophily)；经验研究表明，同质性是估计图卷积层性能的一个良好指标 [LLLG22]。然而，并非所有图都具有同质性：例如，在社交约会网络中，大部分人倾向于与异性建立连接。因此，在这些场景下，我们需要能自适应地调整节点间权重的方法。

对于足够小的图，我们可以通过梯度下降法，让权重矩阵 $\mathbf{A}$ 中的非零元素在初始值的基础上自适应调整。然而，在这种情况下，可训练参数的数量随节点数呈二次

方增长，并且该方案也不适用于包含多个图的场景。如果假设边仅取决于其连接的两个节点的特征，那么我们可以利用一种类似注意力机制的算子来推广 GC 层：

$$\mathbf{h}_i = \phi\left(\sum_{j\in\mathcal{N}(i)} \operatorname{softmax}(\alpha(\mathbf{x}_i, \mathbf{x}_j))\mathbf{W}^\top \mathbf{x}_j\right)$$

其中 $\alpha$ 为接受两个输入、输出为标量的一般 MLP。对于每个节点，我们将 softmax 函数应用于 $\alpha$ 对其邻域集合 $\mathcal{N}(i)$ 中各节点计算得到的全部输出，从而无论邻域大小如何，都能对权重进行归一化。由于与注意力层的相似性，这类层被称为**图注意力**（graph attention，GAT）层 [VCC+18]。从整个图的视角来看，这与 MHA 层非常相似，区别在于，这里的注意力操作仅限于那些存在边相连的节点对之间。

$\alpha$ 的具体选择相对灵活。在原始的 GAT 公式中，$\alpha$ 并非使用点积，而是对拼接特征应用 MLP：

$$\alpha(\mathbf{x}_i, \mathbf{x}_j) = \operatorname{LeakyReLU}(\mathbf{a}^\top \left[\mathbf{V}\mathbf{x}_i \parallel \mathbf{V}\mathbf{x}_j\right])$$

其中 $\mathbf{V}$ 和 $\mathbf{a}$ 为可训练参数。后期研究发现，这种形式存在一定的限制性，即元素之间的排序并不依赖于中心节点 [BAY22]。一种限制较少的变体，称为 **GATv2**[BAY22]，其定义如下：

$$\alpha(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{a}^\top \operatorname{LeakyReLU}(\mathbf{V}\left[\mathbf{x}_i \parallel \mathbf{x}_j\right])$$

GAT 和 GATv2 如今都是非常流行的基准模型。

### 12.3.2 消息传递神经网络




假设有额外可用的**边特征** $\mathbf{e}_{ij}$；例如，在分子数据集中，我们可能已知每种分子键的独热编码表示。通过适当修改注意力函数，我们可以将 GAT 层推广，使其能纳入这些特征：

$$\alpha(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{a}^\top \text{LeakyReLU}(\mathbf{V} \left[\mathbf{x}_i \parallel \mathbf{x}_j \parallel \mathbf{e}_{ij}\right])$$

通过抽象出基本组件，我们可以进一步推广目前为止介绍的所有层（GC、GAT、GATv2 以及带有边特征的 GAT）。考虑一种非常通用的层公式：

$$\mathbf{h}_i = \psi\left(\mathbf{x}_i, \text{Aggr}\left(\left\{M(\mathbf{x}_i, \mathbf{x}_j, \mathbf{e}_{ij})\right\}_{\mathcal{N}(i)}\right)\right) \quad \text{(E.12.11)}$$

其中：

1. $M$ 构建与节点 $i$ 和节点 $j$ 之间的边相关的特征向量（我们称之为 **消息**）。与 GC 和 GAT 层不同，我们不将消息限制为标量值。

2. Aggr 是通用的置换不变函数（例如求和），用于聚合来自与节点 $i$ 相连的所有节点的消息

3. $\psi$ 是最终模块，它将聚合后的消息与节点特征 $\mathbf{x}_i$ 结合。通过这种方式，即使两个节点具有相同邻域，它们依然能够被区分。

举个例子，在 GC 层中，消息构建为 $M(_, \mathbf{x}_j, _) = A_{ij}\mathbf{W}^\top \mathbf{x}_j$，其中聚合操作为简单求和，且 $\psi(_, \mathbf{x}) = \phi(\mathbf{x})$。形如(E.12.11)的通用层由[GSR+17]提出，称为**消息传递**（message-passing）层。它已成为一种流行的方法，对作用于图的各类层进行归

类，并加以推广 [Vel22]。

现在来考虑几个使用消息传递框架的例子。首先，我们可能希望在消息传递阶段更加突出中心节点。这可以通过修改 $\psi$ 函数实现：

$$\psi(\mathbf{x}, \mathbf{m}) = \phi(\mathbf{V}\mathbf{x} + \mathbf{m})$$

其中 $\mathbf{V}$ 是可训练矩阵（该形式由 [MRF+19] 提出，并在 PyTorch Geometric 库中以 GraphConv[5]层的形式应用）。其次，假设节点具有更复杂的可用特征，例如每个节点对应一个时间序列（例如，分布式传感器数据集）。值得注意，在消息传递框架中，针对节点的操作与消息聚合和处理方式是解耦的。若用 $x_i$ 表示节点 $i$ 上的时间序列，我们只需引入能处理时间序列的层（例如一维卷积层 Conv1d），用它修改消息函数，即可推广 GC 层：

$$h_i = \sum_{j \in \mathcal{N}(i)} A_{ij} \text{Conv1d}(x_i)$$

这是一个**时空** (spatio-temporal) GC 层的例子 [YYZ17]。此外，到目前为止，我们一直假设仅需更新节点特征。然而，通过额外的边更新层，也可以更新边特征：

$$\mathbf{e}_{ij} \leftarrow \text{MLP}(\mathbf{e}_{ij}, \mathbf{h}_i, \mathbf{h}_j)$$

这也可以被视为一次消息传递迭代，其中边聚合来自其邻域（两个相连节点）的消息。基于这一思路，可以将这些层进一步推广，以应用于更广泛的邻域范围及图特征 [BHB+18]。

[5] https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.nn.conv.GraphConv.html

以上仅为对消息传递变体的简要概述。受限于篇幅，许多主题无法在此详述；其中值得指出的是：针对高阶图（即边连接超过两个节点的图）构建消息传递层[CPPM22]以及针对点云数据的消息传递层，在后者中，通常需要满足特定的对称性（如旋转和平移对称性）[SHW21, EHB23]。

### 12.3.3 图 transformer

我们已经学习了两种使用图结构的方法：第一种是添加正则化项，强制网络输出在图上保持平滑；第二种是约束图上的运算，使其遵循图的连接关系。特别地，在 GAT 层中，我们通过适当的掩码处理成对比较，应用了标准的注意力机制。然而，我们在上一章中也看到，transformer 之所以流行，是因为它提供了一种与数据类型无关的架构。那么，能否设计一种**图 transformer**（graph transformer）[MGMR24]呢？

回忆一下，构建 transformer 的两个基本步骤是：输入数据词元化以及位置嵌入定义。图的词元化很简单，例如，我们可以将每个节点视为一个词元；或者（如果边特征可用），在将节点和边映射到共享空间后，将每个节点和每条边分别视为独立词元。此处我们暂时忽略边特征。考虑以节点特征为输入的通用架构：

$$\mathbf{H} = \text{Transformer}(\mathbf{X})$$

这种架构具有置换等变性，却完全没有利用图的连接关系信息。我们可以通过在节点特征中补充一些基于图的特征，例如节点的度，或者到某些预选节点（锚点）的最短路径距离，来部分解决这一问题 [RGD+22, MGMR24]。更一般地，我们可以将图的连接关系映射为所谓的**结构嵌入**（structural embedding）：

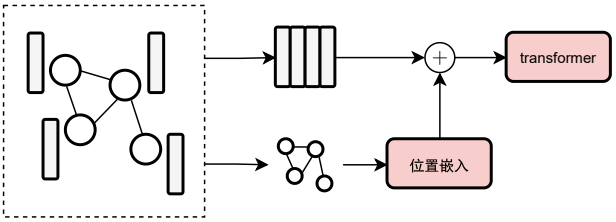



**图 F.12.9:** 图 *transformer* 的基本思想：将图的连接关系编码为一组位置嵌入，将其与节点特征相加。该结果随后由标准 *transformer* 网络处理。

$$\mathbf{H} = \text{Transformer}(\mathbf{X} + \text{Embedding}(\mathbf{A}))$$

在 Embedding($\mathbf{A}$) 中，每一行都给出一个以单个节点为参照、描述图的连接关系的嵌入向量，而不考虑任何特征信息（如F.12.9所示）。幸运的是，将图结构嵌入到向量空间是一个研究广泛的领域。作为示例，我们在此介绍一种基于随机游走的常见嵌入过程 [DLL+22]。回想以下矩阵：

$$\mathbf{R} = \mathbf{A}\mathbf{D}^{-1}$$

可以被解读为“随机游走”，其中 $R_{ij}$ 表示从节点 $i$ 移动到节点 $j$ 的概率。我们可以对该随机游走进行多次迭代，其中迭代次数 $k$ 由用户预先设定：

$$\mathbf{R}, \mathbf{R}^2, \ldots, \mathbf{R}^k$$

随机游走嵌入的构建方式是：收集节点返回自身的所有游走概率，并将它们投影到固定维度的嵌入空间中：

$$\text{Embedding}(\mathbf{A}) = \begin{bmatrix} \text{diag}(\mathbf{R}) \\ \text{diag}(\mathbf{R}^2) \\ \vdots \\ \text{diag}(\mathbf{R}^k) \end{bmatrix} \mathbf{W}$$

在图结构满足特定条件的情况下，可以证明该方法能为每个节点提供唯一表示 [DLL+22]。此外，通过对拉普拉斯矩阵进行特征分解，也可以获得其他类型的嵌入 [LRZ+23]。关于图 transformer 的更详尽介绍，请参阅 [MGMR24]。构建图 transformer 为图领域的类 GPT 基础模型开辟了可能性，同时也使得将基于图的数据作为额外模态引入现有语言模型成为可能 [MCT+]。

## 从理论到实践



由于本章所述的一些问题（例如稀疏性），要高效地处理图数据，需要对基础框架进行扩展。常用的库包括用于 PyTorch 的 PyTorch Geometric，以及 JAX 的 Jraph。两者都提供了大量教程，例如针对小型引文网络中节点分类的教程。[6]

如果你曾在第11章实现了 ViT，我建议你尝试一个有趣的练习，它（在很大程度上）具有教学价值，如图F.12.10所示。假设我们将图像分为若干个图像块，且不添加位置嵌入，而是构建一个邻接矩阵 $\mathbf{A} \sim (p, p)$（其中 $p$ 为图像块数量），其定义如下：

$$A_{ij} = \begin{cases} 1 & \text{若两个图像块在图像中共享边界} \\ 0 & \text{否则} \end{cases} \tag{E.12.12}$$

[6] PyTorch Geometric 中的推荐示例：https://pytorch-geometric.readthedocs.io/en/latest/get_started/introduction.html。

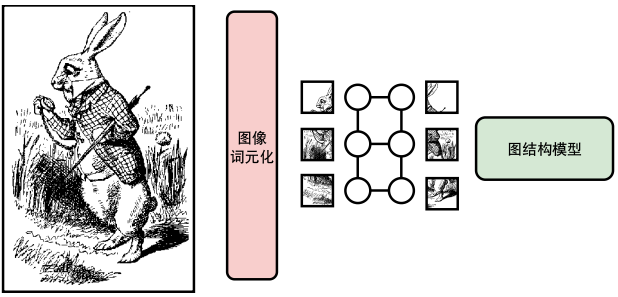



**图 F.12.10:** 用于计算机视觉的 *GNN*：将图像切分为多个图像块，基于这些图像块构建邻接矩阵，并将两者输入到图结构模型中进行传播。

此时，我们得到了一个图结构分类数据集：其中节点特征由图像块嵌入（patch embedding）表示，邻接矩阵则由公式(E.12.12)给出。因此，我们可以适配前文教程中提到的 GNN，来实现图像分类。

# 13 | 循环模型

**本章导读**

transformer 模型处理序列非常有效，但其计算复杂度随序列长度呈二次方增长，这限制了其应用。一种解决方案是用循环层来代替 transformer 层；循环层处理序列中每个元素所需时间是恒定的，与序列总长度无关。在本章中，我们将概述几种循环模型及其特性。该领域在过去两年中发展迅速，因此本章侧重于提供宏观的概览，而在精确度上有所牺牲。关于该领域的最新综述，请参阅 [TCB+24]。

## 13.1 线性化注意力模型

### 13.1.1 点积的替代

为了直观理解循环神经网络（recurrent neural networks，RNN）为何有用，我们先从注意力层的一种推广形式（称为**线性化注意力层**，linearized attention layer）[KVPF20]讲起，该形式可以写为循环结构。我们首先将 SA 层重写为一种抽象形式，用通用的标量值

注意力函数 $\alpha(\bullet,\bullet)$ 代替点积：

$$\mathbf{h}_i = \frac{\sum_{j=1}^{n} \alpha\left(\mathbf{q}_i, \mathbf{k}_j\right) \mathbf{v}_j}{\sum_{j=1}^{n} \alpha\left(\mathbf{q}_i, \mathbf{k}_j\right)} \tag{E.13.1}$$

其中，对于标准自注意力，$\alpha(\mathbf{x},\mathbf{y}) = \exp(\mathbf{x}^\top\mathbf{y})$。如果序列中的元素必须按顺序处理（例如自回归生成任务），公式(E.13.1)不太适用，因为其计算成本会随序列长度增加二次方增长。即使使用了 KV 缓存，内存占用仍随序列长度线性增长。相比之下，卷积层在处理每个元素时，其时间和内存开销固定；但若某个词元位于感受野之外，相关信息就会丢失。我们真正需要的是一种机制：它能将序列中的所有信息压缩为固定大小的输入（我们将其称为**记忆**，memory 或**状态**，state 张量），从而使模型在处理当前输入词元时，结合该记忆所产生的计算开销保持恒定。我们将这类模型称为**循环**模型。

首先注意，任何非负函数 $\alpha$ 都可以作为有效的相似度函数。在机器学习中，这一要求等同于 $\alpha$ 为所谓的**核函数**（kernel function）[HSS08]。许多此类核函数都可以写为广义点积形式：

$$\alpha(\mathbf{x},\mathbf{y}) = \phi(\mathbf{x})^\top\phi(\mathbf{y}) \tag{E.13.2}$$

其中函数 $\phi : \mathbb{R}^c \to \mathbb{R}^e$ 执行特征扩展（feature expansion；这是支持向量机等方法的基石，但详细讨论会超出本书的范围，而且除本节外也不会再用到）。

基于(E.13.2)，可以将(E.13.1)重写为：

$$\mathbf{h}_i = \frac{\sum_{j=1}^{n} \phi(\mathbf{q}_i)^\top\phi(\mathbf{k}_j)\mathbf{v}_j^\top}{\sum_{j=1}^{n} \phi(\mathbf{q}_i)^\top\phi(\mathbf{k}_j)}$$

其中，我们对 $\mathbf{v}_j$ 添加了转置，以保持维度一致。由于 $\phi(\mathbf{q}_i)$ 与 $j$ 无关，我们可以将其移至求和符号外，从而得到：

$$\mathbf{h}_i = \frac{\phi(\mathbf{q}_i)^\top \sum_{j=1}^n \phi(\mathbf{k}_j)\mathbf{v}_j^\top}{\phi(\mathbf{q}_i)^\top \sum_{j=1}^n \phi(\mathbf{k}_j)} \tag{E.13.3}$$

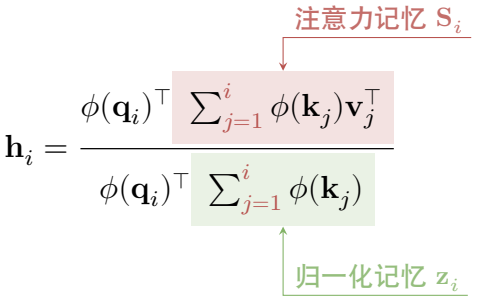
这称为**线性化注意力**（linearized attention）模型[KVPF20]。它对所有词元计算式(E.13.3)的复杂度为 $\mathcal{O}(n(e^2 + ev))$，该复杂度与序列长度呈线性关系；当 $n < e^2$ 时，这种方法具有优势。$\phi$ 的选择非常灵活；例如，文献 [KVPF20]中考虑了二次特征展开；对于短序列，甚至可以使用更简单的 $\phi(\mathbf{x}) = \mathrm{ELU}(\mathbf{x}) + 1$。

**核函数**

以**多项式核函数**（polynomial kernel function）$\alpha(\mathbf{x}, \mathbf{y}) = (1 + \mathbf{x}^\top \mathbf{y})^d$ 为例，若 $\phi(\bullet)$ 显式计算其输入的所有最高至 $d$ 阶的多项式项，那么该核函数可以重写为式(E.13.2)的形式 [HSS08]。一些核函数（如高斯核）对应无限维的特征展开；在这种情况下，仍可通过近似核展开（例如使用随机傅里叶特征 [SW17]）的形式来还原式(E.13.2)。

### 13.1.2 一种循环形式

现在，我们考虑该层的一种因果变体，将线性化注意力模型重写为循环形式。首先，我们修改公式(E.13.3)，将求和范围限定在过去的输入元素上，从而使其具备因果性：




(E.13.4)

这是我们关于**循环层**（recurrent layer）的第一个示例。为了理解这一名称的由来，注意力记忆和归一化记忆可以写成如下递归形式：

$$\mathbf{S}_i = \mathbf{S}_{i-1} + \phi(\mathbf{k}_i)\mathbf{v}_i^\top \tag{E.13.5}$$

$$\mathbf{z}_i = \mathbf{z}_{i-1} + \phi(\mathbf{k}_i) \tag{E.13.6}$$

其中，递归的初始情况由其初始化给出：

$$\mathbf{S}_0 = \mathbf{0} \tag{E.13.7}$$

$$\mathbf{z}_0 = \mathbf{0} \tag{E.13.8}$$

输出由下式给出：

$$\mathbf{h}_i = \frac{\phi(\mathbf{q}_i)^\top \mathbf{S}_i}{\phi(\mathbf{q}_i)^\top \mathbf{z}_i} \tag{E.13.9}$$

公式(E.13.5)-(E.13.9)对自回归场景尤为重要：对于待生成的任意新词元，我们只需更新两个记忆状态（公式(E.13.5)和(E.13.6)），然后使用这些更新后的状态计算

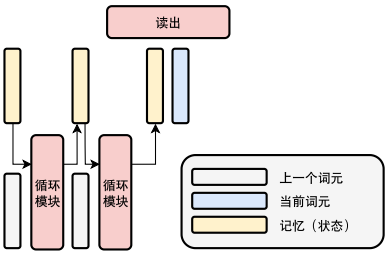



**图 F.13.1:** 循环层概览：过去词元用灰色表示，当前输入词元用蓝色表示，记忆状态用黄色表示。

第 $i$ 个元素的输出。关键的是，生成新词元所需的总计算量是恒定的；此外，由于之前的记忆状态 $\mathbf{S}_{i-1}$ 和 $\mathbf{z}_{i-1}$ 可以被丢弃，因此内存开销也是固定的。我们可以交替使用该层的两种形式：训练时采用向量化变体（以便在 GPU 上高效实现），推理时则采用循环表述形式。

## 13.2 经典循环层

### 13.2.1 一般形式

参考上一节内容，我们来抽象出循环层的关键组成部分。首先，我们需要一个固定大小的**状态**，用于压缩序列中截至第 $i$ 个元素为止的全部有用信息。我们将其记为 $\mathbf{s}_i$，并从现在起，不失一般性地假设它是一个单一向量。其次，我们需要一个**转移函数**（transition function，即循环机制），它根据前一时刻的状态值和当前词元的值来更新状态向量，记作 $f(\mathbf{s}_{i-1}, \mathbf{x}_i)$。第三，我们需要一个所谓的**读出函数**（readout function），用于生成序列中第 $i$ 个元素的输出，记作 $g(\mathbf{s}_i, \mathbf{x}_i)$。可视化参考图F.13.1。

**定义 D.13.1 (循环层)**

给定词元序列 $\mathbf{x}_1, \mathbf{x}_2, \ldots$，一个一般的循环层可写为：



$$\mathbf{s}_i = f(\mathbf{s}_{i-1}, \mathbf{x}_i) \tag{E.13.10}$$
$$\mathbf{h}_i = g(\mathbf{s}_i, \mathbf{x}_i) \tag{E.13.11}$$

其中**状态向量**（*state vector*）$\mathbf{s}_i \sim (e)$ 通常初始化为零向量，即 $\mathbf{s}_0 = \mathbf{0}$。状态向量的维度 $e$ 以及输出向量 $\mathbf{h}_i \sim (o)$ 的维度均为超参数。我们将 $f$ 称为**状态转移**函数（*state transition*），将 $g$ 称为**读出**函数。

在这种形式下，循环层代表一个离散时间、由输入驱动的动态系统，且根据定义，它属于因果层（causal layer）。在控制工程中，这也被称为**状态空间模型**（state-space model）。对于无需考虑因果性的任务，还可以定义**双向层**（bidirectional layers）[SP97]。在双向层中，我们初始化两个循环层（使用各自独立的参数），其中一个从左向右处理序列，另一个则从右向左处理序列。随后，将它们的输出状态拼接，从而得到最终输出。

在更新后的序列 $\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_n$ 上堆叠多个循环层，可以构建循环神经网络（RNN）[PGCB14]。有趣的是，循环层对序列长度没有限制，原则上序列可以任意长。正因如此，可以证明具有无界数值精度或采用可增长架构的 RNN 是图灵完备的 [CS21]。

**隐式层**

如果对单个词元 $\mathbf{x}$ 应用循环层，会发生什么？

$$\mathbf{s}_i = f(\mathbf{s}_{i-1}, \mathbf{x}) \tag{E.13.12}$$

若从已知初始状态 $\mathbf{s}_0$ 开始，多次进行状态转移，这类似一个由多层（每次转移对应一层）组成且共享相同参数的模型。假设我们执行(E.13.12)无限次；如果该动态系统存在一个稳定吸引子（stable attractor），那么输出将由以下不动点方程（fixed-point equation）定义：

$$\mathbf{s} = f(\mathbf{s}, \mathbf{x}) \tag{E.13.13}$$

如果我们采用公式(E.13.13)作为层的定义，就得到了所谓的**隐式层**（implicit layer）[BKK19]。通过使用快速求解器计算不动点方程，并用**隐函数定理**（implicit function theorem）计算反向传播，可以使隐式层的实现切实可行 [BKK19]。此外，通过将各类扩散操作迭代至稳定状态，也可以定义隐式图层（implicit graph layers）[GMS05, SGT+08]。

### 13.2.2 “基本”循环层




历史上，循环层将两个全连接层分别视为转换和读出函数，以此进行实例化：

$$f(\mathbf{s}_{i-1}, \mathbf{x}_i) = \phi(\mathbf{A}\mathbf{s}_{i-1} + \mathbf{B}\mathbf{x}_i) \tag{E.13.14}$$

$$g(\mathbf{s}_i, \mathbf{x}_i) = \mathbf{C}\mathbf{s}_i + \mathbf{D}\mathbf{x}_i \tag{E.13.15}$$

为了简单起见，我们一如既往地忽略偏差项，有四个可训练矩阵 $\mathbf{A} \sim (e, e)$、$\mathbf{B} \sim (e, c)$、$\mathbf{C} \sim (o, e)$ 和 $\mathbf{D} \sim (o, c)$，其中 $c$ 是输入维度（每个词元的大小）。这种形式的层有时通常被称为“循环层”、“基本循环层”（vanilla recurrent layer）或 **Elman** 循环层。当两个矩阵 $\mathbf{A}$ 和 $\mathbf{B}$ 未经训练并且网络只有单层时，这些模型称为**回声**

```
# 输入张量
x = torch.randn(batch_size,
                sequence_length,
                features)

# 状态张量
s = torch.zeros(batch_size,
                state_size)

# 状态更新
state_update = nn.RNNCell(features,
                          state_size)
for i in range(x.shape[1]):
  s = state_update(x[:, i, :], s)
```

**框 C.13.1:** *PyTorch* 中的基本循环。由于循环中的依赖关系，不可能将 *for* 循环与线性代数并行化。在 *PyTorch* 中，状态更新称为**循环单元**（*recurrent cell*），而循环层（例如*torch.nn.RNN*）包装一个单元并执行完整的 *for* 循环。

**状态网络** (echo state network，ESN) 或 **储备池计算机** (reservoir computer) [LJ09]。ESN 可以成为时间序列预测的强大基线，特别是当未训练的矩阵（储备池）以适当方式初始化时 [GBGB21]。

尽管它们具有历史意义，但这类层训练极其低效且困难。要理解这一点，请注意，按照其设计，序列元素之间的计算无法有效并行化，如方框 C.13.1 中所示。因此，我们需要诉诸迭代 (for 循环) 实现，甚至高度定制的 CUDA 实现[1]也比大多数替代序列层 (alternative sequence layer) 慢。

另一个问题源于层计算中涉及的梯度。考虑仅含转换函数的简化情况。我们可以将完整计算展开为：

[1] https://docs.nvidia.com/deeplearning/performance/dl-performance-recurrent/index.html

$$\begin{aligned}\mathbf{s}_1 &= f(\mathbf{s}_0, \mathbf{x}_1)\\ \mathbf{s}_2 &= f(\mathbf{s}_1, \mathbf{x}_2)\\ &\vdots\\ \mathbf{s}_n &= f(\mathbf{s}_{n-1}, \mathbf{x}_n)\end{aligned}$$

这类似一个具有 $n$ 层的模型，只不过参数在各层之间共享（相同）。下文我们将重点关注 $\partial_{\mathbf{A}}\mathbf{s}_n$ （关于 $\mathbf{A}$ 的权重雅可比矩阵），但类似考虑适用于所有梯度。我们定义以下累积乘积：

$$\tilde{\mathbf{s}}_i = \prod_{j=i+1}^{n} \partial_{\mathbf{s}_{j-1}} f(\mathbf{s}_{j-1}, \mathbf{x}_j) \tag{E.13.16}$$

这表示从序列末尾反向传播到元素 $i$ 的转移函数的梯度，如图F.13.2所示。由于权重共享，我们正在寻找的梯度对于序列中每个元素都有一个单独的项，其中涉及这些累积乘积：

$$\partial_{\mathbf{A}}\mathbf{s}_n = \underbrace{\partial_{\mathbf{A}} f(\mathbf{s}_{n-1}, \mathbf{x}_n)}_{\text{元素 } n \text{ 的梯度}} + \sum_{i=1}^{n-1} \underbrace{\tilde{\mathbf{s}}_i \left[\partial_{\mathbf{A}} f(\mathbf{s}_{i-1}, \mathbf{x}_i)\right]}_{\text{元素 } i \text{ 的梯度}} \tag{E.13.17}$$

第一项对应于“标准”权重雅可比矩阵，描述 $\mathbf{A}$ 对序列最后一个元素的影响。求和中的项是附加贡献，序列中每个元素对应一项，并以在序列本身上计算的链式输入雅可比矩阵加权。

以这种形式表示的反向模式自动微分也称为 **通过时间反向传播** (backpropagation through time，BPTT)，它

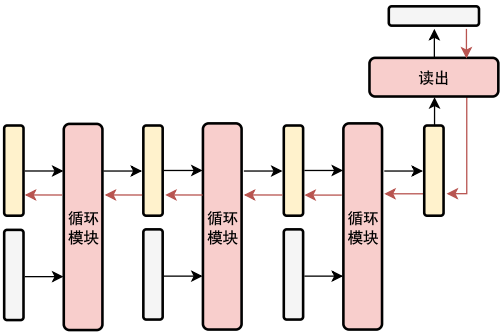



**图 F.13.2:** 循环层的反向传播：伴随值必须通过所有转移步骤传播。随后每个状态为参数的完整梯度贡献一个单独项。

可能是梯度下降过程中不稳定或梯度问题的重要根源。要理解这一点，注意在公式(E.13.17)的内积运算中，每个输入雅可比矩阵都涉及与激活函数 $\phi$ 的导数相乘。关于梯度消失和梯度爆炸问题的最早研究，正是在这一背景下进行的 [Hoc98]。对于长序列而言，只有当转移矩阵的特征值受到适当约束时，该层才能保持稳定 [GM17]。层归一化最初也是为了稳定 RNN 训练而提出的，其方法是对状态序列进行统计计算 [BKH16]。

为了在循环层中部分解决这些不稳定性问题，人们开发了几种技术。例如，可以将公式 (E.13.17)中的求和项截断至特定区间（即**截断 BPTT**，truncated BPTT）；或者，当梯度超过预设的上限时，对其进行阈值限制（即**梯度裁剪**，clipped gradients）。

### 13.2.3 门控循环网络

多年来，为了提升基础循环层的性能，研究者们提出了多种变体。本节将重点介绍其中一类受欢迎的模型，即**门控**循环神经网络（Gated RNN）。RNN 的一个问题是，在每次状态转移时，整个状态都会被覆盖，这一点反映在公式(E.13.17)的部分乘积中。然而，在许多序列中，只有少数转移元素带有重要信息；例如，在音频信

号中，往往存在大量空白区域或无信息区域。在这种情况下，我们或许希望实现转移过程的稀疏化（类似于注意力机制中，大多数注意力权重往往趋近于零的情况），即令 $\tilde{\mathbf{s}}_i$ 中的大部分元素为 1。这可以通过添加专门的门控层来实现。

考虑最简单的门控 RNN 形式，称为**轻量门控循环单元**（Li-GRU，[RBOB18]），它只有一个门控结构。在本文的语境下，门控函数本质上是一个输出值在 $[0,1]$ 范围内的层，可用于对输入进行“掩码”处理。例如，可以通过一个带有 sigmoid 激活函数的全连接层来构建作用于状态的门控：

$$\gamma(\mathbf{s}_{i-1}, \mathbf{x}_i) = \sigma\left(\mathbf{V}\mathbf{s}_{i-1} + \mathbf{U}\mathbf{x}_i\right)$$

其中，$\mathbf{V}$ 和 $\mathbf{U}$ 的形状与 $\mathbf{A}$ 和 $\mathbf{B}$ 相似。我们可以这样理解：若 $\gamma_i \approx 0$，则状态的第 $i$ 个特征应保持不变；若 $\gamma_i \approx 1$，则应将更新后的值作为输出进行传递。因此，通过对新值和旧值进行适当掩码处理，我们可以将状态转移函数重写为：

$$f(\mathbf{s}_{i-1}, \mathbf{x}_i) = \overbrace{\gamma(\mathbf{s}_{i-1}, \mathbf{x}_i) \odot \phi\left(\mathbf{A}\mathbf{s}_{i-1} + \mathbf{B}\mathbf{x}_i\right)}^{\text{新值}} + \underbrace{\left(1 - \gamma(\mathbf{s}_{i-1}, \mathbf{x}_i)\right) \odot \mathbf{s}_{i-1}}_{\text{旧值}}$$

这可以看作对仅取二值的“真正”门的一种软（可微）逼近，也可以看作原始层输出与跳跃连接输出之间的凸组合。理论上，我们可以在损失函数中加入额外的正则化项，约束门的输出尽可能接近 0 或 1，来控制这种逼近的质量。

通过在此设计中增加额外的门，可以衍生出其他类型门控循环层：原始的**门控循环单元**（gated recurrent unit，

GRU）引入了所谓的“重置门”（reset gate）[CVMG+14]；而**长短期记忆**（LSTM）单元则额外包含了一个“遗忘门”（forget gate）[HS97]。LSTM 是文献中最早提出的门控变体，且长期以来一直是处理序列数据最成功的深度学习架构之一 [Sch15]。正因如此，针对 LSTM 模型的研究至今依然十分活跃 [BPS+24]。

## 13.3 结构化状态空间模型

### 13.3.1 线性循环层

我们现在考虑一类简化的循环层，其中移除了状态转移函数中的中间非线性部分：

$$f(\mathbf{s}_{i-1}, \mathbf{x}_i) = \mathbf{A}\mathbf{s}_{i-1} + \mathbf{B}\mathbf{x}_i \tag{E.13.18}$$

$$g(\mathbf{s}_i, \mathbf{x}_i) = \mathbf{C}\mathbf{s}_i + \mathbf{D}\mathbf{x}_i \tag{E.13.19}$$

写为这种形式后，公式(E.13.18)-(E.13.19)被称为**状态空间模型**（state space models，SSM）。[2] 直观上，由于缺乏非线性成分，SSM 层比标准循环层“表达能力更弱”。不过，通过在输出端添加激活函数，或者将这些层与针对每个词元的 MLP 交替堆叠，可以弥补这一不足 [ODG+23]。

对这类模型的研究兴趣在 2020 年（重新）兴起。文献 [GDE+20] 提出了一种针对公式 (E.13.18) 中矩阵 $\mathbf{A}$ 的理论构造；该方法能够高效地压缩一维输入序列。这一成果被称为 HiPPO（**高阶多项式投影算子**，High-Order Polynomial Projection Operator）矩阵。随后，一系列基于 HiPPO 理论、由多个 SSM 层堆叠而成的神经网络

[2] 容易混淆的是，虽然任何形如(E.13.10)–(E.13.11) 的循环层本质上都是状态空间模型，但在神经网络文献中，SSM 这一术语通常指代其线性变体。有时我们称其为**结构化**（structured）SSM，因为正如后文所述，需要对状态转移矩阵 $\mathbf{A}$ 施加适当的约束，才能使其有效。

应运而生，其中包括 2021 年提出的**序列建模结构化状态空间**（Structured State Space for Sequence Modeling，S4）层 [GGR22]，以及 2022 年提出的简化版 S4 模型（即 S5）[SWL23]。

由于它们都起源于 HiPPO 理论，早期的 SSM 层（直至 S4）通常被视为多个一维模型的堆叠，即为输入的每个通道各配置一个模型，并将其中的转移矩阵初始化为 HiPPO 矩阵。相比之下，S5 入了一种标准的多输入多输出模型（形式见公式(E.13.18)–(E.13.19)），这也是本文所要介绍的模型。具体而言，我们将重点分析其一种简化变体，即**线性循环单元**（Linear Recurrent Unit，LRU）[OSG+23]。

该形式具备若干有趣的特性，这些特性主要源于线性转移函数的结合律。为了理解这一点，我们首先指出该递归过程存在闭式解：

$$\mathbf{s}_i = \sum_{j=1}^{i} \mathbf{A}^{i-j}\mathbf{B}\mathbf{x}_j \tag{E.13.20}$$

我们可以从两个不同的视角来看待该求和式。首先，我们可以将关于输入序列的所有系数聚合成一个三阶张量：

$$K = \text{stack}\left(\mathbf{A}^{n-1}\mathbf{B}, \mathbf{A}^{n-2}\mathbf{B}, \ldots, \mathbf{A}\mathbf{B}, \mathbf{B}\right)$$

可以通过一次一维卷积计算所有输出，其卷积核大小等于序列长度（即长卷积）；该卷积在堆叠成单个矩阵 $\mathbf{X} \sim (n, c)$ 的输入序列与预计算的核 $K$ 之间进行：

$$\mathbf{S} = \text{Conv1D}(\mathbf{X}, K)$$

因此，SSM 层可以被看做是一种卷积 [GJG+21]。如果状

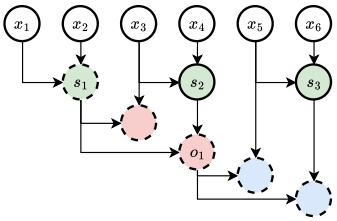



**图 F.13.3:** 针对六个元素序列的并行扫描：同色圆圈代表可并行计算的部分；虚线圆圈为并行扫描输出结果。

态转移矩阵作用于单个通道，则可以通过在频域进行运算来加速计算，例如 FlashConv 的实现便是如此。[3]然而，利用一类被称为**结合（并行）扫描**（associative (parallel) scans，或称**全前缀和**，all-prefix-sums）的算法，可以找到更高效的解决方案。

### 13.3.2 插曲：结合扫描

在探讨其在线性 SSM 中的应用之前，我们先介绍并行扫描（parallel scan）的一般形式。考虑一个元素序列 $(x_1, x_2, \ldots, x_n)$ 以及一个二元运算 $\star$（该运算作用于序列中的任意两个元素），并假设该运算满足结合律。我们希望计算该运算符在序列所有前缀（即从第一个元素开始的各段序列）上的累积运算结果。（为便于阅读，此处使用不同颜色）：

$$x_1,\quad x_1 \star x_2,\quad x_1 \star x_2 \star x_3,\quad \ldots,\quad x_1 \star x_2 \star \cdots \star x_n$$

这可以通过一种迭代算法来简单地实现，即逐个计算这些元素，在每次迭代中加入一个新元素（这对应标准循环层层的计算方式）。然而，利用算子 $\star$ 的结合性，我们可以设计一种高效的并行算法 [Ble90]。其核心思想是：并行计算多个元素对，然后通过递归方式将结果聚合。

[3] https://www.together.ai/blog/h3

举一个简单的例子，考虑由 6 个元素组成的序列 $x_1, x_2, x_3, x_4, x_5, x_6$（关于其应用于 SSM 的深入示例，参考 [SWL23]）。我们将待计算的第 $i$ 个前缀记作 $\hat{x}_i$。整个过程如图F.13.3 所示。首先，我们将相邻数值两两聚合：

$$
\begin{aligned}
s_1 &= x_1 \star x_2 \to \hat{x}_2 \\
s_2 &= x_3 \star x_4 \\
s_3 &= x_5 \star x_6
\end{aligned}
$$

箭头表示算法的输出。接着，我们进行第二层聚合：

$$
\begin{aligned}
s_1 \star x_3 &\to \hat{x}_3 \\
o_1 = s_1 \star s_2 &\to \hat{x}_4
\end{aligned}
$$

最后得到：

$$
\begin{aligned}
o_1 \star x_5 &\to \hat{x}_5 \\
o_1 \star s_3 &\to \hat{x}_6
\end{aligned}
$$

虽然这看起来有些奇怪（我们执行了 7 步，而非 5 步），但如果有 3 个独立线程可用，这三组计算便可轻松实现并行化。一般而言，通过对计算过程的均衡组织，我们能以 $\mathcal{O}(T \log n)$ 复杂度完成并行扫描，其中 $T$ 为二元算子 $\star$ 的计算开销。一个实现示例是 JAX 中的 associative scan 函数。[4]

不难证明，SSM 中的转移函数正是**全前缀和**问题的一个实例。我们将序列中的元素定义为二元组 $x_i = (\mathbf{A}, \mathbf{B}\mathbf{x}_i)$，并将二元算子定义为：

$$
(\mathbf{Z}, \mathbf{z}) \star (\mathbf{V}, \mathbf{v}) = (\mathbf{V}\mathbf{Z}, \mathbf{V}\mathbf{z} + \mathbf{v})
$$

[4] https://jax.readthedocs.io/en/latest/_autosummary/jax.lax.associative_scan.html

基于文献 [SWL23]，该算子 $\star$ 的前缀运算为：

$$x_1 \star x_2 \star \ldots \star x_i = (\mathbf{A}^i, \mathbf{s}_i)$$

因此，通过并行扫描，可以得到 $\mathbf{A}$ 的各次幂（输出第一部分）以及该层在各个时刻的状态（输出第二部分）。该操作的复杂度上限由 $\mathbf{A}^{i-1}\mathbf{A}$ 的计算复杂度决定，即 $\mathcal{O}(n^3)$。为了使整个流程切实可行，可以对 $\mathbf{A}$ 施加约束，从而更高效地计算其各次幂。这正是下一节的主题。

### 13.3.3 对角状态空间模型

使前述思路切实可行的一种常见策略是使用对角转移矩阵（或对角矩阵加一个低秩项 [GGR22]）。在这种情况下，只需分别计算对角线上元素的幂，即可计算出 $\mathbf{A}$ 的各次幂，且计算复杂度仅为线性。此外，如下文所述，使用对角矩阵还能让我们控制转移函数的动态特性，从而避免数值不稳定问题。

特别地，若存在方阵（且可逆）$\mathbf{P}$ 以及对角矩阵 $\Lambda$，使得下式成立，则称方阵 $\mathbf{A}$ 是**可对角化的**（diagonalizable）：

$$\mathbf{A} = \mathbf{P}\Lambda\mathbf{P}^{-1} \tag{E.13.21}$$

从某种意义上说，可对角化矩阵比一般矩阵更“简单”。例如，如果存在这种分解，则容易证明，矩阵的幂也能通过下式高效地计算：

$$\mathbf{A}^i = \mathbf{P}\Lambda^i\mathbf{P}^{-1}$$

假设转移矩阵可对角化，那么我们可以将 SSM 重写为一种等价形式，使其转移矩阵变为对角矩阵。我们首先将式(E.13.21)代入 SSM 的定义中，两边同时乘 $\mathbf{P}^{-1}$：

$$\mathbf{P}^{-1}\mathbf{s}_i = \sum_{j=1}^{i} \Lambda^{i-j}\,\mathbf{PB}\,\mathbf{x}_j$$

新状态向量 $\bar{\mathbf{s}}_i$　　新输入-状态矩阵 $\bar{\mathbf{B}}$

现在用新变量 $\bar{\mathbf{s}}$ 重写读出函数：

新读出矩阵 $\bar{\mathbf{C}}$

$$\mathbf{y}_i = \mathbf{CP}\,\bar{\mathbf{s}}_i + \mathbf{D}\mathbf{x}_i$$

综合以上结果：

$$\bar{\mathbf{s}}_i = \Lambda\bar{\mathbf{s}}_{i-1} + \bar{\mathbf{B}}\mathbf{x}_i \tag{E.13.22}$$

$$\mathbf{y}_i = \bar{\mathbf{C}}\bar{\mathbf{s}}_i + \mathbf{D}\mathbf{x}_i \tag{E.13.23}$$

因此，只要 $\mathbf{A}$ 可对角化，我们总可以将 SSM 重写为具有对角转移矩阵的等效形式。在这种情况下，我们可以直接训练四个矩阵：$\Lambda = \mathrm{diag}(\lambda)$（其中 $\lambda \sim (e)$）、$\bar{\mathbf{B}} \sim (e, c)$、$\bar{\mathbf{C}} \sim (o, e)$ 以及 $\mathbf{D} \sim (o, c)$；其中，对角矩阵 $\Lambda$ 由一个维度为 $e$ 的向量进行参数化。

并非所有矩阵都能被对角化。然而，如果允许矩阵 $P$ 和 $\Lambda$ 包含复数元素，则总能找到一种近似对角化的形式 [OSG+23]。需要谨慎设计对角线上元素的参数化方式，使转移矩阵的特征值的绝对值始终小于 1，从而避免动态过程发散。关于这两点的详细说明以及对由此得到的 LRU 层的完整分析，请参阅文献 [OSG+23]。

## 13.4 其他变体

如何平衡卷积、循环和注意力机制各自的优势，是当前的一个活跃研究课题。在本章结尾，我们列举一些近期文献中提出的循环层（或可被视为循环层的结构）。

### 13.4.1 无注意力 transformer

线性化 transformer 模型（见第13.1.1节）存在一个问题，即其在特征维度 $e$ 上的复杂度呈二次方增长。无注意力机制的 transformer (attention-free transformer, ATF) 作为基础注意力层的一种变体被提出，它在序列长度和特征数量两个维度上均呈现线性复杂度 [ZTS+21]。

其核心思想是用一种更简单的*相乘交互*（multiplicative interaction，即逐元素相乘）来取代键、查询和值之间的点积交互：

$$\mathbf{h}_i = \sigma(\mathbf{q}_i) \odot \frac{\sum_j \exp\left(\mathbf{k}_j\right) \odot \mathbf{v}_j}{\sum_j \exp\left(\mathbf{k}_j\right)} \tag{E.13.24}$$

这与自注意力层类似，区别在于我们将所有点积运算替换为了逐元素（阿达玛）乘法。该方法也受到了线性化注意力层的启发：其中的查询仅被用作全局调制因子；在此模型中，它是通过 sigmoid 函数进行归一化后发挥作用的。我们可以将式(E.13.24)重写为单维度 $z$ 的形式（利用我们仅执行逐元素运算这一事实），来还原标准注意力形式：

$$h_{iz} = \frac{\sigma(q_{iz}) \sum_j \exp(k_{jz})}{\sum_j \exp(k_{jz})} v_{jz}$$

因此，ATF 层可以被重新解读为一种通道维度上的注意力变体。对于每一个通道，它都可以被重写为对序列元素执行的注意力运算。为了增加灵活性，文献 [ZTS+21]提出添加相对嵌入（relative embedding）$\mathbf{W} \sim (m, m)$（其中 $m$ 为序列允许的最大长度）：

$$\mathbf{h}_i = \sigma(\mathbf{q}_i) \odot \frac{\sum_j \exp\left(\mathbf{k}_j + W_{ij}\right) \odot \mathbf{v}_j}{\sum_j \exp\left(\mathbf{k}_j + W_{ij}\right)} \tag{E.13.25}$$

相对嵌入也可以通过低秩分解（low-rank factorization）进行训练，从而减少参数数量。关于这一点，以及基础 ATF 层的其他变体（例如将其与卷积操作结合），请参考文献 [ZTS+21]。此外，通过适当限制求和范围，我们还可以将公式(E.13.24)转换为因果（循环）变体。

### 13.4.2 Receptance Weighted Key Value（RWKV）模型

RWKV 模型 [PAA+23]在 ATF 层的基础上，引入了一些额外的架构改进。在撰写本书时，它是极少数能在超大规模下与 transformer 性能相当的预训练 RNN 之一，因此我们对其进行更详细介绍。首先，它通过引入针对每个偏移量进行缩放的向量 $\mathbf{w} \sim (e)$，简化了相对嵌入的表示：

$$w_{ij} = -(i-j)\mathbf{w}$$

此外，实验表明，针对当前元素引入一个独立的偏移量 $\mathbf{u}$（而非使用 $\mathbf{w}$）是有益的。将其写为因果形式如下：

$$\mathbf{h}_i = \mathbf{W}_o\left(\sigma(\mathbf{q}_i)\odot\frac{\sum_{j=1}^{i-1}\exp\left(\mathbf{k}_j + w_{ij}\right)\odot\mathbf{v}_j + \exp\left(\mathbf{k}_i+\mathbf{u}\right)\odot\mathbf{v}_i}{\sum_{j=1}^{i-1}\exp\left(\mathbf{k}_j + w_{ij}\right) + \exp\left(\mathbf{k}_i+\mathbf{u}\right)}\right)$$

其中，我们用红色标出了与基础 ATF 层的区别。在 [PAA+23] 中，查询被称为 **receptance**，还在末端添加了一个输出投影 $\mathbf{W}_o$。其次，RWKV 模型修改了 transformer 模块中的标准 MLP，将其替换为一个采用不同门控机制的、逐词元处理的模块。对于给定的输入词元 $\mathbf{x}$，该过程可表示为：

$$\mathbf{y} = \sigma(\mathbf{W}_1\mathbf{x})\odot\mathbf{W}_2\max(0,\mathbf{W}_3\mathbf{x})^2 \tag{E.13.26}$$

其中 $\mathbf{W}_1$、$\mathbf{W}_2$ 和 $\mathbf{W}_3$ 为可训练参数。除了最左侧的

门控结构以及平方 ReLU 激活函数的使用外，它与标准 MLP 一致。作为最后一步改进，为了提升性能，第一个模块中的所有三个投影（以及公式(13.4.2)中出现的两处 $\mathbf{x}$）均被替换为 $\mathbf{x}_i$ 与 $\mathbf{x}_{i-1}$ 的凸组合；这一机制被称为*词元移位*。

### 13.4.3 选择性状态空间模型

我们已经接触过三类循环模型：标准循环层（及其门控变体）、线性化注意力层，以及结构化状态空间模型。尽管它们外观各异，但在这几类模型间转换其实很容易。为了说明这一点，让我们考虑一个忽略了分母项的线性化注意力层：

$$\mathbf{S}_i = \mathbf{S}_{i-1} + \phi(\mathbf{k}_i)\mathbf{v}_i^\top \tag{E.13.27}$$

$$\mathbf{h}_i = \phi(\mathbf{q}_i)^\top \mathbf{S}_i \tag{E.13.28}$$

除了状态本身是矩阵形式外，我们可以发现，它具有 SSM 层的形式；区别在于，其中的某些矩阵（例如 $\mathbf{C} = \phi(\mathbf{q}_i)^\top$）并非固定不变，而是取决于具体的输入词元。从动力系统的角度来看，标准 SSM 描述的是时不变（time-invariant）系统，而公式(E.13.27)-(E.13.28)描述的是时变（time-varying）系统。这一发现启发了另一类 SSM 层，其中的矩阵不再被约束为时不变，这类模型被称为**选择性**（selective）SSM。这类模型大多借鉴了注意力层的思想，在进行层内计算之前，先对输入进行多次投影变换。

作为示例，我们在此重点介绍所谓的 Mamba 层 [GD23]。截至本书撰写时，它是少数几种已经扩展到大规模、并在超长上下文和超大参数量下与 transformer 模型性能相当的 SSM 层之一。首先，为了使 SSM 层具备时变性，

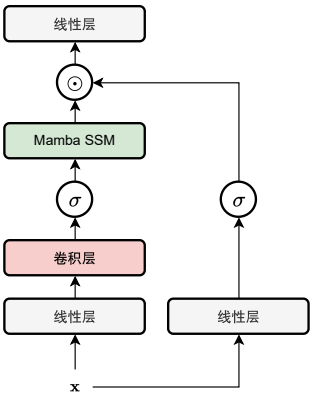



**图 F.13.4:** *Mamba* 模块（图中未显示该模块周边的残差连接及归一化）。$\sigma$ 代表 *sigmoid* 函数。改编自 *[GD23]*.

其一部分矩阵被设为依赖于输入：[5]

$$\mathbf{s}_i = A(\mathbf{x}_i)\mathbf{s}_{i-1} + B(\mathbf{x}_i)\mathbf{x}_i \tag{E.13.29}$$

$$\mathbf{h}_i = C(\mathbf{x}_i)\mathbf{s}_i + \mathbf{D}\mathbf{x}_i \tag{E.13.30}$$

其中 $A(\bullet)$、$B(\bullet)$ 和 $C(\bullet)$ 是对其输入词元的线性投影。为了实现这一点，该层被独立应用于输入的每个通道，且转移矩阵被设定为对角矩阵，从而使 SSM 中的所有矩阵均可用单个数值向量表示。该层无法直接采用简单的并行扫描实现，需要一种定制化的硬件感知实现 [GD23]。可以证明，Mamba SSM 变体以及其他几种 SSM 层，实际上都是门控循环层的退化形式 [GJG+21, GD23]。

为了简化整体架构，Mamba 避免交替排列 MLP 与 SSM，改为采用一种门控架构（类似于11.3 节中提到的门控注意力单元），即利用 MLP 对 SSM 输出进行加权。此外，还引入了深度卷积以增强灵活性——详见图 F.13.4。

[5] 矩阵 $\mathbf{D}$ 可被视为一种简单的残差连接，保持不变。在原始层定义中，参数化方式略微不同，即 $\mathbf{A} = \exp(\Delta\bar{\mathbf{A}})$，其中 $\bar{\mathbf{A}}$ 为可训练参数，$\Delta$ 为依赖于输入的标量值。这并不影响我们的讨论

# 暂时告别

至此，爱丽丝在可微奇境中的首次旅程暂告一段落。这只是一次粗略的游览，重点在于如何设计与组合各种层，以构建现代可微模型（即神经网络）。



许多主题本书只是简要提及，包括如何在实践中使用这些模型：微调、生成式建模、持续学习、多模态、可解释性等。我们也回避了棘手的工程问题：训练和部署大模型是一项浩大的工程，需要分布式训练策略、高效编译器和 DevOps 技术等。[6]此外，大语言模型的兴起开辟了新的应用途径，从提示工程到模型链式调用和智能体行为，甚至无需了解模型的内部机制。

本书配有网站，[7]我希望在其中发布涉及上述部分主题的补充章节；如果时间允许，其中一些或将汇集成新的一卷。

希望你喜欢这段旅程！对本书有任何意见、建议或反馈，欢迎与我联系。

[6]例如，可参考https://jax-ml.github.io/scaling-book/。

[7]https://sscardapane.it/alice-book

# A | 概率论

**本章导读**

机器学习涉及多种不确定性（例如在数据收集阶段），因此概率的应用至关重要。本章将以非正式的方式回顾概率分布及概率密度相关基本概念，这些内容对理解正文有所帮助。本附录介绍了许多概念，但读者应该已经熟悉其中大部分内容。关于机器学习和神经网络背景下概率的更深入阐述，请参考 [Bis06, BB23]。

## A.1 概率的基本法则

以简单的彩票抽奖为例，参与者购买的彩票可能有三种结果：“未中奖”、“小奖”和“大奖”。在每 10 张彩票中，有 1 张大奖，3 张小奖，6 张未中奖。假设彩票供应数量无限，我们可以用一个概率分布来描述这三种事件的相对频率：

$$p(w = \text{‘未中奖’}) = 6/10$$
$$p(w = \text{‘小奖’}) = 3/10$$
$$p(w = \text{‘大奖’}) = 1/10$$

同样地，可以将这三种事件分别对应整数值 $w \in \{1,2,3\}$，并记作 $p(w=1)=6/10$、$p(w=2)=3/10$ 以及 $p(w=3)=1/10$。我们称 $w$ 为**随机变量**（random variable）。在下文中，为了提高可读性，在不引起歧义的情况下，我们用 $p(w)$ 来代替 $p(w=i)$。概率分布中的元素必须为正，且其总和必须等于 1：

$$p(w) \geq 0, \sum_{w} p(w) = 1$$

所有此类向量构成的空间被称为**概率单纯形**。

> 请记住，我们使用 $\mathbf{p} \sim \Delta(n)$ 来表示一个属于概率单纯形的 $n$ 维向量。

假设我们引入第二个随机变量 $r$。它是一个二元变量，用于描述彩票是真（1）还是假（2）。如T.A.1所示，假彩票的利润更高，但出现的概率总体更低。

**表 T.A.1:** 在一种假设的彩票抽奖中，彩票分为真票和假票，基于 100 张彩票的中奖情况统计。

| | $r=1$ (真票) | $r=2$ (假票) |
|---|---|---|
| $w=1$ (未中奖) | 58 | 2 |
| $w=2$ (小奖) | 27 | 3 |
| $w=3$ (大奖) | 2 | 8 |
| 合计 | 87 | 13 |

我们可以利用表中数据来描述**联合概率分布**（joint probability distribution），即两个随机变量同时取特定值的概率：

$$p(r=2, w=3) = 8/100$$

此外，我们还可以定义**条件概率分布**（conditional probability distribution），例如回答“在已知某事件发生的条件下，另一事件发生的概率是多少？”这类问题：

$$p(r = 1 \mid w = 3) = \frac{p(r = 1, w = 3)}{p(w = 3)} = 0.2$$

这被称为概率的**乘法法则**（product rule）。与之前一样，我们可以用随机变量本身来代替其具体取值，从而简化符号表示：

$$p(r, w) = p(r \mid w)p(w) \tag{E.A.1}$$

若 $p(r \mid w) = p(r)$，则有 $p(r, w) = p(r)p(w)$；此时我们称这两个变量是**独立**的。我们可以利用条件概率，将其中一个随机变量**边缘化**（marginalize）：

$$p(w) = \sum_r p(w, r) = \sum_r p(w \mid r)p(r) \tag{E.A.2}$$

这被称为概率的**求和法则**（sum rule）。乘法法则和求和法则是定义概率代数的基本公理。将二者结合，便可得到基础的**贝叶斯规则**（Bayes’ s rule）：

$$p(r \mid w) = \frac{p(w \mid r)p(r)}{p(w)} = \frac{p(w \mid r)p(r)}{\sum_{r'} p(w \mid r')p(r')} \tag{E.A.3}$$

贝叶斯规则使我们能“反转”条件分布。例如，若我们已知中奖彩票在真与假两类中的相对比例，则可以据此计算一张中奖彩票的真假概率（不妨试一试）。

## A.2 实值分布

在实值分布情形，定义 $p(x)$ 更为复杂，因为 $x$ 可以取无穷多个可能值，而按照定义，其中每一个值的概率均为 0。然而，我们可以通过定义概率**累积密度函数**（cumulative density function，CDF）[1]来绕过这一问题：

$$P(x) = \int_0^x p(t)dt$$

并将概率密度函数（probability density function）$p(x)$ 定义为该函数的导数。我们在此略去处理概率密度时涉及的多数细节问题，因为这些问题最好在测度论的框架下严谨处理 [BR07]。我们仅指出，通过将求和替换为积分，乘法法则和求和法则在此情形下仍成立：

$$p(x, y) = p(x \mid y)p(y) \tag{E.A.4}$$

$$p(x) = \int_y p(x \mid y)p(y)dy \tag{E.A.5}$$

请注意，概率密度的取值并不一定必须小于 1。

## A.3 常见分布

前文所述随机变量属于**类别概率分布**（categorical probability distributions），描述变量可在 $k$ 个可能取值中任选其一的情形。我们可以用更紧凑的方式来表示：令 $\mathbf{p} \sim \Delta(k)$ 为概率向量，设 $\mathbf{x} \sim \text{Binary}(k)$ 为观测类别的独热编码，则有：

[1]译者注：更常用的对应术语为**累积分布函数**（cumulative distribution function）

$$p(\mathbf{x}) = \mathrm{Cat}(\mathbf{x}; \mathbf{p}) = \prod_i p_i^{x_i}$$

我们使用分号将分布的输入与参数区分。若 $k = 2$，则该分布也可等价地用单个标量值 $p$ 表示。由此得到的分布称为**伯努利分布**（Bernoulli distribution）：

$$p(x) = \mathrm{Bern}(x; p) = p^x(1-p)^{(1-x)}$$

在连续情形下，我们经常会遇到**高斯分布**（Gaussian distribution），记作 $\mathcal{N}(x; \mu, \sigma^2)$。它描述了一种钟形的概率分布，以 $\mu$（均值）为中心，其离散程度由 $\sigma^2$（即方差）决定：

$$p(x) = \mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right)$$

在均值为零、方差为单位值（即 $\mu = 0, \sigma^2 = 1$）的最简单情形下，它也被称为 **正态分布**（normal distribution）。对于向量 $\mathbf{x} \sim (k)$，通过引入均值向量 $\mu \sim (k)$ 和协方差矩阵 $\Sigma \sim (k, k)$，可以得到高斯分布的多变量形式：

$$p(\mathbf{x}) = \mathcal{N}(\mathbf{x}; \mu, \Sigma) = \\ (2\pi)^{-k/2} \det(\Sigma)^{-1/2} \exp\left((\mathbf{x}-\mu)^\top \Sigma^{-1} (\mathbf{x}-\mu)\right)$$

有两种有趣的情形：一种是协方差矩阵为对角矩阵的高斯分布；另一种更简单，即 **各向同性**（isotropic）高斯分布，其协方差矩阵为对角矩阵且所有对角元素均相等：

$$\Sigma = \sigma^2 \mathbf{I}$$

前者在几何上可视为一个轴对齐的椭球体，而后者则可视为一个轴对齐的球体。

## A.4 矩与期望值

在许多情况下，我们需要用一个或多个值来概括概率分布。有时，有限个数的数值就足够了：例如，对于类别分布，知道参数 $\mathbf{p}$ 即可完全描述该分布；对于高斯分布，只需知道参数 $\mu$ 和 $\sigma^2$。这类数值被称为**充分统计量**（sufficient statistic）。

更一般地，对于任意给定函数 $f(x)$，我们可以定义其**期望值**（expected value）如下：

$$\mathbb{E}_{p(x)}\left[f(x)\right] = \sum_x f(x)p(x) \tag{E.A.6}$$

在实值情况下，只需将求和符号替换为积分符号，即可得到同样的定义。特别值得关注的是，当 $f(x) = x^p$ 时，我们便得到了该分布的（$p$ 阶）**矩**（moment）；当 $p = 1$ 时，该值被称为分布的**均值**（mean）：

$$\mathbb{E}_{p(x)}\left[x\right] = \sum_x xp(x)$$

有时，即使无法直接获取底层的概率分布，我们也需要估计某些期望值。如果能从 $p(x)$ 中采样，便可以使用所谓的**蒙特卡洛估计**（Monte Carlo estimator）：

$$\mathbb{E}_{p(x)}\left[f(x)\right] \approx \frac{1}{n}\sum_{x_i \sim p(x)} f(x_i) \tag{E.A.7}$$

其中 $n$ 决定估计的质量；符号 $x_i \sim p(x)$ 表示从概率分布 $p(x)$ 中采样的操作。对于一阶矩，这回归到了我们熟悉的计算方式，即通过多次测量来求某个量的均值：

$$\mathbb{E}_{p(x)}[x] = \frac{1}{n} \sum_{x_i \sim p(x)} x_i$$

## A.5 分布之间的距离

有时，我们需要衡量概率分布之间的某种距离，以评估两个分布的接近程度。$p(x)$ 与 $q(x)$ 之间的 **Kullback-Leibler 散度**（KL divergence）是一种常用的选择：

$$\mathrm{KL}(p \parallel q) = \int p(x) \log \frac{p(x)}{q(x)} dx$$

KL 散度并非严格意义上的度量（它不具备对称性，也不满足三角不等式）。它的下界为 0，但没有上界。仅当对任意满足 $q(x) = 0$ 的 $x$ 都有 $p(x) = 0$ 都有 $p(x) = 0$ 时（即 $p$ 的支撑集是 $q$ 支撑集的子集），该散度才有定义。当两个分布完全相同时，KL 散度达到最小值 0。由于 KL 散度可以写为期望值形式，因此可以通过蒙特卡洛采样（如公式(E.A.7) 所示）进行估计。

## A.6 最大似然估计




蒙特卡洛采样表明，如果我们能够获取某个概率分布的样本，便可以估计该分布中我们所关注的量。然而，有时我们感兴趣的是估计概率分布本身。假设我们对该分布的函数形式 $f(x; s)$ 有一个猜测，其中 $s$ 为充分统计量（例如高斯分布的均值和方差），并拥有 $n$ 个样本 $x_i \sim p(x)$。我们称这些样本是独立同分布的，因为它们

来自同一个概率分布，且相互独立。由于其独立性，对于任意给定 $s$，其联合分布可以分解为：

$$p(x_1, \dots, x_n) = \prod_{i=1}^{n} f(x_i; s)$$

大规模乘积在计算中不方便处理；但通过对数变换，我们可以将其等价地改写为求和形式：

$$L(s) = \sum_{i=1}^{n} \log(f(x_i; s))$$

寻找能使上述量达到最大值的参数 $s$ 的方法，被称为**最大似然**（maximum likelihood）估计。鉴于其重要性，我们在下文对其进行简要阐述。




**定义 D.A.1 (最大似然)**

给定一个参数化概率分布族 $f(x; s)$，以及一组来自未知分布 $p(x)$ 的 $n$ 个独立同分布样本 $\{x_i\}_{i=1}^{n}$，根据**最大似然**原则，对 $p(x)$ 的最佳近似为：

$$s^* = \arg\max_{s} \sum_{i=1}^{n} \log(f(x_i; s))$$

若 $f$ 可微，我们便可通过梯度下降法最大化该目标函数。这是训练可微模型时遵循的核心方法。本附录的最后，我们将通过标准概率分布下的最大似然估计示例进行说明。此处不再详述具体计算推导过程，相关细节见 [Bis06, BB23]。

**高斯分布的最大似然**

首先考虑参数 $p$ 未知的伯努利分布。在这种情况下，最大似然估计量为：

$$p^* = \frac{\sum_i x_i}{n}$$

即正样本数量占整个数据集总样本数的比例。

**高斯分布的最大似然估计**

对于高斯分布，其对数似然可写为：

$$L(\mu, \sigma^2) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2$$

分别求 $\mu$ 和 $\sigma^2$ 极大值，即可得到计算高斯分布经验均值和方差的常用公式：

$$\mu^* = \frac{1}{n}\sum_i x_i \tag{E.A.8}$$

$$\sigma^{2*} = \frac{1}{n}\sum_i (x_i - \mu^*)^2 \tag{E.A.9}$$

这两个参数可以依次计算得出。由于在方差公式中使用了均值的估计值，因此该估计结果存在微小偏差。通过将归一化项修改为 $\frac{1}{n-1}$，可以修正这一偏差，这被称为贝塞尔校正（Bessel' s correction）。[2] 当 $n$ 较大时，这两种计算方式之间的差异极小。

[2] https://en.wikipedia.org/wiki/Bessel%27s_correction

# B | 通用逼近定理

**本章导读**

虽然对“通用逼近定理”（universal approximation theorem）进行严格的形式化证明超出了本书范围，但建立起关于此类证明如何构建的直观认识是很有帮助的。在本附录中，我们参考并扩展了 M.Nielsen 在 2019 年的一本在线作品（http://neuralnetworksanddeeplearning.com/chap4.html）中提供的直观可视化思想。关于该主题的更深入讨论（以及一些交互式可视化内容），读者可参阅上述资料，尤其是针对多维输入的情况。

我们重点关注 Cybenko[Cyb89]提出的原始逼近定理，该定理针对的是包含一个隐藏层且使用 sigmoid 激活函数的模型。为了便于可视化，我们将分析限定在单输入、单输出的函数上。该推理过程同样可以推广至其他激活函数以及更高维情形。

这一可视化证明的思路相对简单：

1. 第一步，展示如何通过手动设置权重，使一个隐藏层中仅含单个神经元的模型去逼近一个阶跃函数。

2. 接着，展示如何在隐藏层增加另一个神经元，从而能够逼近某种特定函数。这种函数在某个小区间内取常数值，而在其他区域均为零（我们称为区间

函数或“箱形”函数)。

3. 最后，描述一种逼近一般函数的方法：首先将其划分为若干区间并达到所需精度，然后根据需要添加相应数量的神经元，依次逼近每一个区间。若划分为 $m$ 个区间，则最终得到的网络包含 $2m$ 个神经元。对于具有多个输入的一般函数，神经元数量会随维度的增加呈指数级增长，这使得该证明在实际应用中不具备构造性。

## B.1 逼近阶跃函数

首先，让我们考虑隐藏层中的单个神经元。此时，该网络的方程可表示为（此处忽略输出偏置项，因为它对我们的推导没有帮助)：

$$f(x) = a\sigma(wx + s)$$

为了便于可视化，我们对该式进行改写：在偏置项前添加负号，并将乘法因子提取到 $\sigma$ 函数的整个输入部分(两种形式显然等价)：

$$f(x) = a\,\sigma(\,w\,(x - s\,)) \qquad \text{(E.B.1)}$$

幅度　斜率　平移

这与我们在第5.4节中介绍的“可调”型 sigmoid 函数相似。具体而言，在此表达式中，$a$ 控制 sigmoid 函数的振幅，$w$控制其斜率，而 $s$则使函数产生固定量的平移。

图 F.B.1展示了公式(E.B.1)的几种曲线，其中 $a$ 和 $s$ 为固定值，同时改变 $w$。可以看出，随着 $w$ 增大，斜率变得愈发陡峭。若将 $w$ 固定为一个足够大的常数（例如

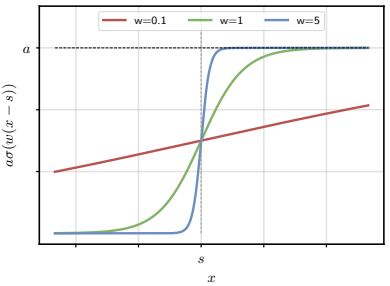
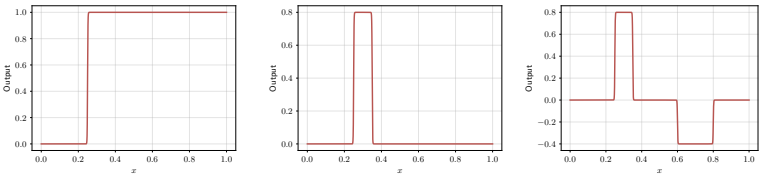



**图 F.B.1:** 隐藏层仅含单个神经元的网络，可被视为一个斜率、中心位置和振幅均可调节的 *sigmoid* 函数。此处展示一个示例：固定幅度和中心位置，仅改变斜率。




**(a)** *1* 个神经元 **(b)** *2* 个神经元 **(c)** *4* 个神经元

**图 F.B.2:** *(a)* 包含一个输入、一个隐藏层神经元和一个输出的神经网络，可以逼近任意阶跃函数（此处 $a = 1$ 且 $s = 0.3$）。*(b)* 包含两个隐藏层神经元和一个输出的神经网络，可以逼近任何在某个小区间内保持常数的函数。*(c)* 使用四个神经元，我们可以逼近任何在两个非零区间上呈分段常数形式的函数。注意，通过设定负的振幅，区间的取值也可以是负数。

$w = 10^4$），该函数便能极好地逼近阶跃函数；此时，我们可以通过参数 $s$ 控制阶跃位置，并通过参数 $a$ 控制其幅度，如图 F.B.2a 所示。

## B.2 逼近常值函数

如果我们加入第二个神经元，使其振幅相反（并进行微小的位移），我们可以逼近一个在小区间内保持恒定的函数（我们称之为“箱形”函数）。设宽度为 $\Delta$，我们可以写出：

$$f(x) = a\sigma\left(w\left(x - \boxed{s - \frac{\Delta}{2}}\right)\right) - a\sigma\left(w\left(x - \boxed{s + \frac{\Delta}{2}}\right)\right) \tag{E.B.2}$$

在 $s - \frac{\Delta}{2}$ 处上升 [下降]　　在 $s + \frac{\Delta}{2}$ 处下降 [上升]

其中，$w$ 是一个较大的常数（例如 $10^4$）。公式(E.B.2)描述的函数（等价于一个包含两个神经元的单隐藏层模型）在 $s - \frac{\Delta}{2}$ 处增加 $a$，在区间 $\left[s - \frac{\Delta}{2}, s + \frac{\Delta}{2}\right]$ 内保持恒定值 $f(x) = a$，随后下降至 0。图F.B.2b 展示了一个示例。

在下文中，为了强调该函数对参数 $a$、$s$ 和 $\Delta$ 的依赖关系，将其记作 $f(x; a, s, \Delta)$。

## B.3 逼近一般函数

由于 $f_{a,s,\Delta}(x)$ 在其对应的区间外实际上为 0，因此定义在互不相交区间上的两个函数互不影响；换言之，我们刚才定义的“箱形”函数具有高度的局部性。因此，通过在隐藏层中增加两个神经元，我们可以定义在两个独立区间上取常数值的函数（如图 F.B.2c所示）：

$$f(x) = f(x; a_1, s_2, \Delta_1) + f(x; a_2, s_2, \Delta_2)$$

证明的其余部分非常简单：只需将待函数在多个小区间上进行分箱。给定任意（连续）函数 $g(x)$，且其定义在某个区间（为简化起见，假设为 $[0, 1]$）上。我们首先将输入域等分为 $m$ 个区间，其中 $m$ 决定逼近的精度（$m$ 越大，逼近效果越好）。因此，第 $i$ 个箱覆盖的范围为：

$$B_i = \left[\frac{i}{m} - \frac{\Delta}{2}, \frac{i}{m} + \frac{\Delta}{2}\right]$$

其中 $\Delta$ 是每个箱长度。对于每个箱，我们计算 $g(x)$ 在该区间内的平均值：

$$g_i = \frac{1}{\Delta} \int_{x \in B_i} g(x) dx$$

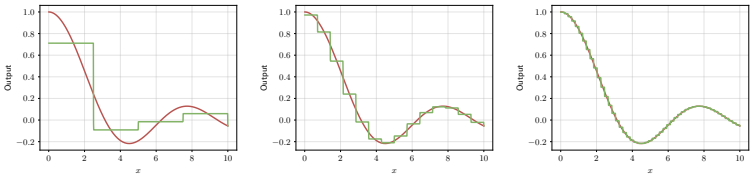
最后，我们构建一个网络，其隐藏层包含 $2m$ 个神经元，即每个箱对应两个神经元。每个箱对应的函数以该箱的中心为基准，取值为 $g_i$：

$$f(x) = \sum_{i=1}^{m} f\left(x; \underbrace{g_i}_{\text{（逼近得到的）常数值}}, \underbrace{\frac{i}{m}}_{\text{第 } i \text{ 个箱中心为 } \frac{i}{m}}, \Delta\right) \tag{E.B.3}$$

如图F.B.3所示，当 $g(x) = \frac{\sin(x)}{x}$ 时，随着分箱数量 $m$ 增加（分别取 $m = 5, 15, 50$），上述逼近方法的示例。显而易见，均方误差（MSE）与 $m$ 成反比；我们只需提高逼近的分辨率，即可将误差减小到任意的程度。

类似的推理也适用于多维输入和不同激活函数。[1]

[1] http://neuralnetworksanddeeplearning.com/chap4.html




**(a)** *5* 个箱 **(b)** *15* 个箱 **(c)** *50* 个箱

**图 F.B.3:** 在区间 $[0, 10]$ 上逼近函数 $g(x) = \frac{\sin(x)}{x}$，其中 *(a)*$m = 5$，*(b)*$m = 15$，*(c)*$m = 50$（分箱数量）。图中红色曲线为原始函数，绿色曲线为逼近函数(E.B.3)。在这三种情况下，均方误差呈指数级下降（分别约为 0.02、0.002 和 0.00016）。

# 参考文献


[AB21] A. N. Angelopoulos and S. Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021. 87

[ADIP21] A. Apicella, F. Donnarumma, F. Isgrò, and R. Prevete. A survey on modern trainable activation functions. *Neural Networks*, 138:14–32, 2021. 104

[AHS23] S. K. Ainsworth, J. Hayase, and S. Srinivasa. Git re-basin: Merging models modulo permutation symmetries. In *ICLR*, 2023. 3

[AHSB14] F. Agostinelli, M. Hoffman, P. Sadowski, and P. Baldi. Learning activation functions to improve deep neural networks. *arXiv preprint arXiv:1412.6830*, 2014. 104

[AJB+17] D. Arpit, S. Jastrzębski, N. Ballas, D. Krueger, E. Bengio, M. S. Kanwal, T. Maharaj, A. Fischer, A. Courville, Y. Bengio, et al. A closer look at memorization in deep networks. In *ICML*, 2017. 95

[AR00] J. A. Anderson and E. Rosenfeld. *Talking nets: An oral history of neural networks*. MIT Press, 2000. 7

[ARAA+16] R. Al-Rfou, G. Alain, A. Almahairi, C. Angermueller, D. Bahdanau, N. Ballas, F. Bastien, J. Bayer, A. Belikov, A. Belopolsky, et al. Theano: A Python framework for fast computation of mathematical expressions. *arXiv preprint arXiv:1605.02688*, pages 1–19, 2016. i, 10

[ASA+23] E. Akyürek, D. Schuurmans, J. Andreas, T. Ma, and D. Zhou. What learning algorithm is in-context learning? investigations with linear models. In *ICLR*, 2023. 2, 50

[AST+24] A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado, H. Shen, O. Shchur, S. S. Rangapuram, S. P. Arango, S. Kapoor, et al. Chronos: Learning the language of time series. *arXiv preprint arXiv:2403.07815*, 2024. 166, 244

[AZLL19] Z. Allen-Zhu, Y. Li, and Y. Liang. Learning and generalization in overparameterized neural networks, going beyond two layers. In *NeurIPS*, 2019. 97

[BAY22] S. Brody, U. Alon, and E. Yahav. How attentive are graph attention networks? In *ICLR*, 2022. 269

[BB23] C. M. Bishop and H. Bishop. *Deep learning: Foundations and concepts*. Springer Nature, 2023. 10, 55, 301, 308

[BBCJ20] M. Biesialska, K. Biesialska, and M. R. Costa-Jussa. Continual lifelong learning in natural language processing: A survey. In *COLING*, 2020. 50

[BBL+17] M. M. Bronstein, J. Bruna, Y. LeCun, A. Szlam, and P. Vandergheynst. Geometric deep learning: going beyond Euclidean data. *IEEE Signal Processing Magazine*, 34(4):18–42, 2017. 147, 260

[BCB15] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. In *ICLR*, 2015. 210

[BCZ+16] T. Bolukbasi, K.-W. Chang, J. Y. Zou, V. Saligrama, and A. T. Kalai. Man is to computer programmer as woman is to homemaker? debiasing word embeddings. In *NeurIPS*, 2016. 49

[Ben09] Y. Bengio. Learning deep architectures for AI. *Foundations and Trends® in Machine Learning*, 2(1):1–127, 2009. i, 6, 96

[BGHN24] J. Blasiok, P. Gopalan, L. Hu, and P. Nakkiran. When does optimizing a proper loss yield calibration? In *NeurIPS*, 2024. 84

[BGLA21] F. M. Bianchi, D. Grattarola, L. Livi, and C. Alippi. Graph neural networks with convolutional ARMA filters. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3496–3507, 2021. 262

[BGMMS21] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In *ACM FAccT*. ACM, 2021. 49

[BGSW18] N. Bjorck, C. P. Gomes, B. Selman, and K. Q. Weinberger. Understanding batch normalization. In *NeurIPS*, 2018. 196, 197

[BHB+18] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018. 271

[Bis95] C. M. Bishop. Training with noise is equivalent to Tikhonov regularization. *Neural Computation*, 7(1):108–116, 1995. 186

[Bis06] C. Bishop. *Pattern recognition and machine learning*. Springer, 2006. 60, 71, 85, 301, 308

[BJMO12] F. Bach, R. Jenatton, J. Mairal, and G. Obozinski. Optimization with sparsity-inducing penalties. *Foundations and Trends® in Machine Learning*, 4(1):1–106, 2012. 183

[BKH16] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. 198, 286

[BKK19] S. Bai, J. Z. Kolter, and V. Koltun. Deep equilibrium models. In *NeurIPS*, 2019. 283

[Ble90] G. E. Blelloch. *Prefix sums and their applications*. School of Computer Science, Carnegie Mellon University Pittsburgh, PA, USA, 1990. 290

[BN24] J. Bernstein and L. Newhouse. Old optimizer, new norm: An anthology. *arXiv preprint arXiv:2409.20325*, 2024. 42

[BNS06] M. Belkin, P. Niyogi, and V. Sindhwani. Manifold regularization: A geometric framework for learning from labeled and unlabeled examples. *Journal of Machine Learning Research*, 7(11), 2006. 50, 257

[BP20] J. Bolte and E. Pauwels. A mathematical model for automatic differentiation in machine learning. In *NeurIPS*, 2020. 128

[BPA+24] F. Bordes, R. Y. Pang, A. Ajay, A. C. Li, A. Bardes, S. Petryk, O. Mañas, Z. Lin, A. Mahmoud, B. Jayaraman, et al. An introduction to vision-language modeling. *arXiv preprint arXiv:2405.17247*, 2024. 209, 245

[BPRS18] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind. Automatic differentiation in machine learning: a survey. *Journal of Marchine Learning Research*, 18:1–43, 2018. 109

[BPS+24] M. Beck, K. Pöppel, M. Spanring, A. Auer, O. Prudnikova, M. Kopp, G. Klambauer, J. Brandstetter, and S. Hochreiter. xLSTM: Extended long short-term memory. *arXiv preprint arXiv:2405.04517*, 2024. 288

[BR07] V. I. Bogachev and M. A. S. Ruas. *Measure theory*. Springer, 2007. 304

[BR24] M. Blondel and V. Roulet. The elements of differentiable programming. *arXiv preprint arXiv:2403.14606*, 2024. 7, 35, 109, 110, 119

[BZMA20] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. In *NeurIPS*, 2020. 243

[CCGC24] Y. Cheng, G. G. Chrysos, M. Georgopoulos, and V. Cevher. Multilinear operator networks. In *ICLR*, 2024. 247

[CMMB22] F. Cinus, M. Minici, C. Monti, and F. Bonchi. The effect of people recommenders on echo chambers and polarization. In *AAAI ICWSM*, 2022. 47

[CPPM22] E. Chien, C. Pan, J. Peng, and O. Milenkovic. You are AllSet: A multiset function framework for hypergraph neural networks. In *ICLR*, 2022. 272

[CRBD18] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. In *NeurIPS*, 2018. 204

[CS21] S. Chung and H. Siegelmann. Turing completeness of bounded-precision recurrent neural networks. In *NeurIPS*, 2021. 282

[CVMG+14] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *EMNLP*. ACL, 2014. 288

[CW82] D. Coppersmith and S. Winograd. On the asymptotic complexity of matrix multiplication. *SIAM Journal on Computing*, 11(3):472–492, 1982. 24

[Cyb89] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989. 96, 311

[CZJ+22] H. Chang, H. Zhang, L. Jiang, C. Liu, and W. T. Freeman. MaskGIT: Masked generative image transformer. In *IEEE/CVF CVPR*, 2022. 243

[CZSL20] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le. RandAugment: Practical automated data augmentation with a reduced search space. In *IEEE/CVF CVPR Workshops*, 2020. 186

[DBK+21] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021. 228, 242, 248

[DCLT18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*. ACL, 2018. 242

[DCSA23] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi. High fidelity neural audio compression. *Transactions on Machine Learning Research*, 2023. 244

[DDM+23] M. Dehghani, J. Djolonga, B. Mustafa, P. Padlewski, J. Heek, J. Gilmer, A. P. Steiner, M. Caron, R. Geirhos, I. Alabdulmohsin, et al. Scaling vision transformers to 22 billion parameters. In *ICML*, 2023. 246

[DFAG17] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier. Language modeling with gated convolutional networks. In *ICML*, 2017. 105

[DFE+22] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022. 239

[DLL+22] V. P. Dwivedi, A. T. Luu, T. Laurent, Y. Bengio, and X. Bresson. Graph neural networks with learnable structural and positional representations. In *ICLR*, 2022. 273, 274

[DOMB24] T. Darcet, M. Oquab, J. Mairal, and P. Bojanowski. Vision transformers need registers. In *ICLR*, 2024. 229

[DS20] S. De and S. Smith. Batch normalization biases residual blocks towards the identity function in deep networks. In *NeurIPS*, 2020. 201

[DT17] T. DeVries and G. W. Taylor. Improved regularization of convolutional neural networks with cutout. *arXiv preprint arXiv:1708.04552*, 2017. 194

[DZPS19] S. S. Du, X. Zhai, B. Poczos, and A. Singh. Gradient descent provably optimizes over-parameterized neural networks. In *ICLR*, 2019. 97

[EHB23] F. Eijkelboom, R. Hesselink, and E. J. Bekkers. E $(n)$ equivariant message passing simplicial networks. In *ICML*, 2023. 272

[FAL17] C. Finn, P. Abbeel, and S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *ICML*, 2017. 50

[Fle23] F. Fleuret. *The Little Book of Deep Learning*. Lulu Press, Inc., 2023. 10

[GBGB21] D. J. Gauthier, E. Bollt, A. Griffith, and W. A. Barbosa. Next generation reservoir computing. *Nature Communications*, 12(1):5564, 2021. 284

[GD23] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023. 296, 297

[GDE+20] A. Gu, T. Dao, S. Ermon, A. Rudra, and C. Ré. Hippo: Recurrent memory with optimal polynomial projections. In *NeurIPS*, 2020. 288

[GFGS06] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *ICML*, 2006. 243

[GG16] Y. Gal and Z. Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *ICML*, 2016. 192

[GGR22] A. Gu, K. Goel, and C. Ré. Efficiently modeling long sequences with structured state spaces. In *ICLR*, 2022. 289, 292

[GJG+21] A. Gu, I. Johnson, K. Goel, K. Saab, T. Dao, A. Rudra, and C. Ré. Combining recurrent, convolutional, and continuous-time models with linear state space layers. In *NeurIPS*, 2021. 289, 297

[GM17] C. Gallicchio and A. Micheli. Echo state property of deep reservoir computing networks. *Cognitive Computation*, 9:337–350, 2017. 286

[GMS05] M. Gori, G. Monfardini, and F. Scarselli. A new model for learning in graph domains. In *IEEE IJCNN*. IEEE, 2005. 283

[GOV22] L. Grinsztajn, E. Oyallon, and G. Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *NeurIPS*, 2022. 133

[GPE+23] S. Golkar, M. Pettee, M. Eickenberg, A. Bietti, M. Cranmer, G. Krawezik, F. Lanusse, M. McCabe, R. Ohana, L. Parker, et al. xVal: A continuous number encoding for large language models. *arXiv preprint arXiv:2310.02989*, 2023. 161

[GPSW17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. In *ICML*, 2017. 87

[Gri12] A. Griewank. Who invented the reverse mode of differentiation? *Documenta Mathematica, Extra Volume ISMP*, 389400, 2012. 110

[GSBL20] M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value memories. In *EMNLP*. ACL, 2020. 237

[GSR+17] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. Neural message passing for quantum chemistry. In *ICML*, 2017. 270

[GW08] A. Griewank and A. Walther. *Evaluating derivatives: principles and techniques of algorithmic differentiation*. SIAM, 2008. 109, 110, 122

[GWFM+13] I. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio. Maxout networks. In *ICML*, 2013. 105

[GZBA22] D. Grattarola, D. Zambon, F. M. Bianchi, and C. Alippi. Understanding pooling in graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 2022. 262

[HABN+21] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241):1–124, 2021. 183

[HBE+24] A. Ho, T. Besiroglu, E. Erdil, D. Owen, R. Rahman, Z. C. Guo, D. Atkinson, N. Thompson, and J. Sevilla. Algorithmic progress in language models. *arXiv preprint arXiv:1710.05941*, 2024. 1, 2

[HDLL22] W. Hua, Z. Dai, H. Liu, and Q. Le. Transformer quality in linear time. In *ICML*, 2022. 247

[HG16] D. Hendrycks and K. Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016. 103

[HHWW14] P. Huang, Y. Huang, W. Wang, and L. Wang. Deep embedding network for clustering. In *ICPR*. IEEE, 2014. 49

[Hoc98] S. Hochreiter. Recurrent neural net learning and vanishing gradient. *International Journal Of Uncertainity, Fuzziness and Knowledge-Based Systems*, 6(2):107–116, 1998. 286

[Hor91] K. Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991. 96

[HR22] M. Hardt and B. Recht. *Patterns, predictions, and actions: Foundations of machine learning*. Princeton University Press, 2022. 10

[HS97] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. 288

[HSS08] T. Hofmann, B. Schölkopf, and A. J. Smola. Kernel methods in machine learning. *The Annals of Statistics*, 36(3):1171–1220, 2008. 95, 278, 279

[HTF09] T. Hastie, R. Tibshirani, and J. H. Friedman. *The elements of statistical learning: data mining, inference, and prediction*. Springer, 2009. 47, 84

[HWG25] S. Hwang, B. Wang, and A. Gu. Dynamic chunking for end-to-end hierarchical sequence modeling. *arXiv preprint arXiv:2507.07955*, 2025. 162

[HYL17] W. Hamilton, Z. Ying, and J. Leskovec. Inductive representation learning on large graphs. In *NeurIPS*, 2017. 267

[HZC+17] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017. 150

[HZRS15] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *IEEE ICCV*, 2015. 102

[HZRS16] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE/CVF CVPR*, 2016. 201, 202, 203, 205

[ICS22] K. Irie, R. Csordás, and J. Schmidhuber. The dual form of neural networks revisited: Connecting test time predictions to training patterns via spotlights of attention. In *ICML*, 2022. 70

[IS15] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015. 194

[JGB+21] A. Jaegle, F. Gimeno, A. Brock, O. Vinyals, A. Zisserman, and J. Carreira. Perceiver: General perception with iterative attention. In *ICML*, 2021. 239

[JK+17] P. Jain, P. Kar, et al. Non-convex optimization for machine learning. *Foundations and Trends® in Machine Learning*, 10(3-4):142–363, 2017. 40

[JLB+22] L. V. Jospin, H. Laga, F. Boussaid, W. Buntine, and M. Bennamoun. Hands-on Bayesian neural networks—a tutorial for deep learning users. *IEEE Computational Intelligence Magazine*, 17(2):29–48, 2022. 58, 60

[KB15] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015. 41

[KG21] P. Kidger and C. Garcia. Equinox: neural networks in JAX via callable PyTrees and filtered transformations. *Differentiable Programming Workshop, NeurIPS*, 2021. 107

[KL18] S. M. Kakade and J. D. Lee. Provably correct automatic sub-differentiation for qualified programs. In *NeurIPS*, 2018. 128

[KMH+20] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. 1, 209

[KPR+17] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. 61

[KSH12] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *NeurIPS*, 2012. 101, 180

[KVPF20] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *ICML*, 2020. 277, 279

[KW17] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017. 260

[Lau19] S. Laue. On the equivalence of automatic and symbolic differentiation. *arXiv preprint arXiv:1904.02990*, 2019. 113

[LBBH98] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 7

[LBH15] Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. 6

[LCX+23] J. Li, Y. Cheng, Z. Xia, Y. Mo, and G. Huang. Generalized activation via multivariate projection. *arXiv preprint arXiv:2309.17194*, 2023. 104

[LDR23] V. Lialin, V. Deshpande, and A. Rumshisky. Scaling down to scale up: A guide to parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.15647*, 2023. 50

[LDSL21] H. Liu, Z. Dai, D. So, and Q. V. Le. Pay attention to MLPs. In *NeurIPS*, 2021. 247

[LH19] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019. 42, 183

[Lim21] L.-H. Lim. Tensors in computations. *Acta Numerica*, 30:555–764, 2021. 8

[LJ09] M. Lukoševičius and H. Jaeger. Reservoir computing approaches to recurrent neural network training. *Computer Science Review*, 3(3):127–149, 2009. 284

[LKM23] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023. 239

[LLLG22] Y. Li, B. Lin, B. Luo, and N. Gui. Graph representation learning beyond node and homophily. *IEEE Transactions on Knowledge and Data Engineering*, 35(5):4880–4893, 2022. 268

[LLS21] S. H. Lee, S. Lee, and B. C. Song. Vision transformer for small-size datasets. *arXiv preprint arXiv:2112.13492*, 2021. 248

[LMW+22] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie. A ConvNet for the 2020s. In *IEEE/CVF CVPR*, 2022. 202, 203, 205

[LPW+17] Z. Lu, H. Pu, F. Wang, Z. Hu, and L. Wang. The expressive power of neural networks: A view from the width. In *NeurIPS*, 2017. 96

[LRZ+23] D. Lim, J. Robinson, L. Zhao, T. Smidt, S. Sra, H. Maron, and S. Jegelka. Sign and basis invariant networks for spectral graph representation learning. In *ICLR*, 2023. 274

[LTM+22] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. A. Raffel. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. In *NeurIPS*, 2022. 3

[LWV+24] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark. KAN: Kolmogorov-Arnold networks. *arXiv preprint arXiv:2404.19756*, 2024. 105

[LZA23] H. Liu, M. Zaharia, and P. Abbeel. Ring attention with blockwise transformers for near-infinite context. In *Foundation Models for Decision Making Workshop, NeurIPS*, 2023. 240

[MCT+] H. Mao, Z. Chen, W. Tang, J. Zhao, Y. Ma, T. Zhao, N. Shah, M. Galkin, and J. Tang. Position: Graph foundation models are already here. In *ICML*. 274

[Met22] C. Metz. *Genius makers: the mavericks who brought AI to Google, Facebook, and the world*. Penguin, 2022. 6

[MGMR24] L. Müller, M. Galkin, C. Morris, and L. Rampášek. Attending to graph transformers. *Transactions on Machine Learning Research*, 2024. 272, 274

[MKS+20] J. Mukhoti, V. Kulharia, A. Sanyal, S. Golodetz, P. Torr, and P. Dokania. Calibrating deep neural networks using focal loss. In *NeurIPS*, 2020. 87

[MRF+19] C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, J. E. Lenssen, G. Rattan, and M. Grohe. Weisfeiler and Leman go neural: Higher-order graph neural networks. In *AAAI Conference on Artificial Intelligence*, volume 33, pages 4602–4609, 2019. 271

[MRT18] M. Mohri, A. Rostamizadeh, and A. Talwalkar. *Foundations of machine learning*. MIT Press, 2018. 53

[MSC+13] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean. Distributed representations of words and phrases and their compositionality. In *NeurIPS*, 2013. 49

[MZBG18] G. Marra, D. Zanca, A. Betti, and M. Gori. Learning neuron non-linearities with kernel-based deep neural networks. *arXiv preprint arXiv:1807.06302*, 2018. 104

[NCN+23] V. Niculae, C. F. Corro, N. Nangia, T. Mihaylova, and A. F. Martins. Discrete latent structure in neural networks. *arXiv preprint arXiv:2301.07473*, 2023. 110

[ODG+23] A. Orvieto, S. De, C. Gulcehre, R. Pascanu, and S. L. Smith. On the universality of linear recurrences followed by nonlinear projections. In *HLD 2023 Workshop, ICML*, 2023. 288

[ODZ+16] A. v. d. Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu. WaveNet: A generative model for raw audio. In *ISCA SSW Workshop*, 2016. 164, 168, 171

[OSG+23] A. Orvieto, S. L. Smith, A. Gu, A. Fernando, C. Gulcehre, R. Pascanu, and S. De. Resurrecting recurrent neural networks for long sequences. In *ICML*, 2023. 289, 293

[PAA+23] B. Peng, E. Alcaide, Q. Anthony, A. Albalak, S. Arcadinho, H. Cao, X. Cheng, M. Chung, M. Grella, K. K. GV, et al. RWKV: Reinventing RNNs for the transformer era. In *EMNLP*. ACL, 2023. 295

[PABH+21] O. Puny, M. Atzmon, H. Ben-Hamu, I. Misra, A. Grover, E. J. Smith, and Y. Lipman. Frame averaging for invariant and equivariant network design. *arXiv preprint arXiv:2110.03336*, 2021. 147

[PBE+22] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. In *1st Mathematical Reasoning in General Artificial Intelligence Workshop, ICLR*, 2022. 2, 185

[PBL20] T. Poggio, A. Banburski, and Q. Liao. Theoretical issues in deep networks. *Proceedings of the National Academy of Sciences*, 117(48):30039–30045, 2020. 53

[PGCB14] R. Pascanu, C. Gulcehre, K. Cho, and Y. Bengio. How to construct deep recurrent neural networks. In *ICLR*, 2014. 282

[PKP+19] G. I. Parisi, R. Kemker, J. L. Part, C. Kanan, and S. Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019. 50

[PNR+21] G. Papamakarios, E. Nalisnick, D. J. Rezende, S. Mohamed, and B. Lakshminarayanan. Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, 22(57):1–64, 2021. 205

[PP08] K. B. Petersen and M. S. Pedersen. *The matrix cookbook*. Technical University of Denmark, 2008. 32

[PPR+24] A. Pagnoni, R. Pasunuru, P. Rodriguez, J. Nguyen, B. Muller, M. Li, C. Zhou, L. Yu, J. Weston, L. Zettlemoyer, et al. Byte latent transformer: Patches scale better than tokens. *arXiv preprint arXiv:2412.09871*, 2024. 162

[PPVF21] S. Pesme, L. Pillaud-Vivien, and N. Flammarion. Implicit bias of SGD for diagonal linear networks: a provable benefit of stochasticity. In *NeurIPS*, 2021. 95

[PRCB24] A. Patel, C. Raffel, and C. Callison-Burch. Datadreamer: A tool for synthetic data generation and reproducible LLM workflows. In *ACL*. ACL, 2024. 185

[Pri23] S. J. Prince. *Understanding Deep Learning*. MIT Press, 2023. 10

[PS+03] T. Poggio, S. Smale, et al. The mathematics of learning: Dealing with data. *Notices of the AMS*, 50(5):537–544, 2003. 53

[PSL22] O. Press, N. A. Smith, and M. Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *ICLR*, 2022. 226

[QPF+24] S. Qiu, A. Potapczynski, M. Finzi, M. Goldblum, and A. G. Wilson. Compute better spent: Replacing dense layers with structured matrices. *arXiv preprint arXiv:2406.06248*, 2024. 142

[RBOB18] M. Ravanelli, P. Brakel, M. Omologo, and Y. Bengio. Light gated recurrent units for speech recognition. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2(2):92–102, 2018. 287

[RGD+22] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini. Recipe for a general, powerful, scalable graph transformer. In *NeurIPS*, 2022. 272

[RHM86] D. E. Rumelhart, G. E. Hinton, and J. L. McClelland. A general framework for parallel distributed processing. In *Parallel Distributed Processing Volume 1*. MIT Press, 1986. 6, 7

[RKG+22] D. W. Romero, D. M. Knigge, A. Gu, E. J. Bekkers, E. Gavves, J. M. Tomczak, and M. Hoogendoorn. Towards a general purpose CNN for long range dependencies in $n$D. *arXiv preprint arXiv:2206.03398*, 2022. 212

[RKX+23] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever. Robust speech recognition via large-scale weak supervision. In *ICML*, 2023. 243

[RM22] J. W. Rocks and P. Mehta. Memorizing without overfitting: Bias, variance, and interpolation in overparameterized models. *Physical Review Research*, 4(1):013201, 2022. 185

[RS21] M. N. Rabe and C. Staats. Self-attention does not need $\mathcal{O}(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021. 239

[RSR+20] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020. 242

[RWC+19] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 2019. 49, 238, 242

[RZL17] P. Ramachandran, B. Zoph, and Q. V. Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017. 103

[SAL+24] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. 226

[Sch15] J. Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015. 288

[SCHU17] S. Scardapane, D. Comminiello, A. Hussain, and A. Uncini. Group sparse regularization for deep neural networks. *Neurocomputing*, 241:81–89, 2017. 183

[SGT+08] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2008. 283

[Sha19] N. Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019. 246

[Sha20] N. Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020. 105

[SHK+14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1):1929–1958, 2014. 189

[SHW21] V. G. Satorras, E. Hoogeboom, and M. Welling. E(n) equivariant graph neural networks. In *ICML*, 2021. 272

[SKF+99] Y. Shibata, T. Kida, S. Fukamachi, M. Takeda, A. Shinohara, T. Shinohara, and S. Arikawa. Byte pair encoding: A text compression scheme that accelerates pattern matching. 1999. 161

[SKZ+21] A. Steiner, A. Kolesnikov, X. Zhai, R. Wightman, J. Uszkoreit, and L. Beyer. How to train your ViT? data, augmentation, and regularization in vision transformers. *Transactions on Machine Learning Researc*, 2021. 248

[SLJ+15] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *IEEE CVPR*, 2015. 146

[SMDH13] I. Sutskever, J. Martens, G. Dahl, and G. Hinton. On the importance of initialization and momentum in deep learning. In *ICML*, 2013. 41

[SP97] M. Schuster and K. K. Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681, 1997. 282

[SSBD14] S. Shalev-Shwartz and S. Ben-David. *Understanding machine learning: From theory to algorithms*. Cambridge University Press, 2014. 53

[Sti81] S. M. Stigler. Gauss and the invention of least squares. *The Annals of Statistics*, pages 465–474, 1981. 7

[SVL14] I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to sequence learning with neural networks. In *NeurIPS*, 2014. 233

[SVVTU19] S. Scardapane, S. Van Vaerenbergh, S. Totaro, and A. Uncini. Kafnets: Kernel-based non-parametric activation functions for neural networks. *Neural Networks*, 110:19–32, 2019. 104

[SW17] S. Scardapane and D. Wang. Randomness in neural networks: an overview. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 7(2):e1200, 2017. 279

[SWF+15] S. Sukhbaatar, J. Weston, R. Fergus, et al. End-to-end memory networks. In *NeurIPS*, 2015. 236

[SWL23] J. T. Smith, A. Warrington, and S. W. Linderman. Simplified state space layers for sequence modeling. In *ICLR*, 2023. 289, 291, 292

[TCB+24] M. Tiezzi, M. Casoni, A. Betti, M. Gori, and S. Melacci. State-space modeling in long sequence processing: A survey on recurrence in the transformer era. *arXiv preprint arXiv:2406.09062*, 2024. 277

[TEM23] M. Tschannen, C. Eastwood, and F. Mentzer. GIVT: Generative infinite-vocabulary transformers. *arXiv preprint arXiv:2312.02116*, 2023. 243

[TGJ+15] J. Tompson, R. Goroshin, A. Jain, Y. LeCun, and C. Bregler. Efficient object localization using convolutional networks. In *IEEE/CVF CVPR*, 2015. 194

[THK+21] I. O. Tolstikhin, N. Houlsby, A. Kolesnikov, L. Beyer, X. Zhai, T. Unterthiner, J. Yung, A. Steiner, D. Keysers, J. Uszkoreit, et al. MLP-Mixer: An all-MLP architecture for vision. In *NeurIPS*, 2021. 246

[TLI+23] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. 2, 49, 242, 247

[TNHA24] D. Teney, A. M. Nicolicioiu, V. Hartmann, and E. Abbasnejad. Neural redshift: Random networks are not random functions. In *IEEE/CVF CVPR*, 2024. 95

[Unc15] A. Uncini. *Fundamentals of adaptive signal processing*. Springer, 2015. 143

[Vap13] V. Vapnik. *The nature of statistical learning theory*. Springer Science & Business Media, 2013. 53

[VCC+18] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio. Graph attention networks. In *ICLR*, 2018. 269

[Vel22] P. Veličković. Message passing all the way up. *arXiv preprint arXiv:2202.11097*, 2022. 271

[VSP+17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017. 209, 210, 224, 225, 233, 237

[VWB16] A. Veit, M. J. Wilber, and S. Belongie. Residual networks behave like ensembles of relatively shallow networks. In *NeurIPS*, 2016. 204

[WBF+24] S. Welleck, A. Bertsch, M. Finlayson, H. Schoelkopf, A. Xie, G. Neubig, I. Kulikov, and Z. Harchaoui. From decoding to meta-generation: Inference-time algorithms for large language models. *arXiv preprint arXiv:2406.16838*, 2024. 174

[WCW+23] C. Wang, S. Chen, Y. Wu, Z. Zhang, L. Zhou, S. Liu, Z. Chen, Y. Liu, H. Wang, J. Li, et al. Neural codec language models are zero-shot text to speech synthesizers. *arXiv preprint arXiv:2301.02111*, 2023. 244

[WFD+23] H. Wang, T. Fu, Y. Du, W. Gao, K. Huang, Z. Liu, P. Chandak, S. Liu, P. Van Katwyk, A. Deac, et al. Scientific discovery in the age of artificial intelligence. *Nature*, 620(7972):47–60, 2023. 1, 165

[WGGH18] X. Wang, R. Girshick, A. Gupta, and K. He. Non-local neural networks. In *IEEE/CVF CVPR*, 2018. 212

[WJ21] Y. Wu and J. Johnson. Rethinking "Batch" in BatchNorm. *arXiv preprint arXiv:2105.07576*, 2021. 197

[WZZ+13] L. Wan, M. Zeiler, S. Zhang, Y. Le Cun, and R. Fergus. Regularization of neural networks using DropConnect. In *ICML*, 2013. 194

[XYH+20] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T. Liu. On layer normalization in the transformer architecture. In *ICML*, 2020. 227

[YCC+24] L. Yuan, Y. Chen, G. Cui, H. Gao, F. Zou, X. Cheng, H. Ji, Z. Liu, and M. Sun. Revisiting out-of-distribution robustness in NLP: Benchmarks, analysis, and llms evaluations. In *NeurIPS*, 2024. 46

[YHO+19] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo. CutMix: Regularization strategy to train strong classifiers with localizable features. In *IEEE/CVF ICCV*, 2019. 187

[YLC+22] T. Yu, X. Li, Y. Cai, M. Sun, and P. Li. S2-MLP: Spatial-shift MLP architecture for vision. In *WACV*, 2022. 246

[YLZ+22] W. Yu, M. Luo, P. Zhou, C. Si, Y. Zhou, X. Wang, J. Feng, and S. Yan. Metaformer is actually what you need for vision. In *IEEE/CVF CVPR*, 2022. 246

[YTL+25] L. Yang, Y. Tian, B. Li, X. Zhang, K. Shen, Y. Tong, and M. Wang. Mmada: Multimodal large diffusion language models. *arXiv preprint arXiv:2505.15809*, 2025. 242

[YYZ17] B. Yu, H. Yin, and Z. Zhu. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. In *IJCAI*, 2017. 271

[ZBH+21] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. Understanding deep learning (still) requires rethinking generalization. *Communications of the ACM*, 64(3):107–115, 2021. 1

[ZCDLP17] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz. mixup: Beyond empirical risk minimization. In *ICLR*, 2017. 187

[ZER+23] A. Zador, S. Escola, B. Richards, B. Ölveczky, Y. Bengio, K. Boahen, M. Botvinick, D. Chklovskii, A. Churchland, C. Clopath, et al. Catalyzing next-generation artificial intelligence through neuroAI. *Nature Communications*, 14(1):1597, 2023. 6

[ZJM+21] J. Zbontar, L. Jing, I. Misra, Y. LeCun, and S. Deny. Barlow twins: Self-supervised learning via redundancy reduction. In *ICML*, 2021. 49

[ZKR+17] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. R. Salakhutdinov, and A. J. Smola. Deep sets. In *NeurIPS*, 2017. 231

[ZLLS23] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola. *Dive into deep learning*. Cambridge University Press, 2023. 10, 41

[ZS19] B. Zhang and R. Sennrich. Root mean square layer normalization. In *NeurIPS*, 2019. 200

[ZTS+21] S. Zhai, W. Talbott, N. Srivastava, C. Huang, H. Goh, R. Zhang, and J. Susskind. An attention free transformer. *arXiv preprint arXiv:2105.14103*, 2021. 294, 295

[ZW23] L. Ziyin and Z. Wang. spred: Solving $l_1$ penalty with SGD. In *ICML*, 2023. 183